

The MediaTek logo consists of the word "MEDIATEK" in white, uppercase, sans-serif font, centered within an orange parallelogram with a slight 3D effect.

MT8676

Application Processor Functional Specification

Version: 0.5
Release date: 2024-8-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Description
0.1	2023-11-28	Draft release.
0.2	2024-2-20	Update 5.12.2 TDM sampling rate description.
0.3	2024-3-19	Update 6.5.2 DSI throughput.
0.5	2024-8-28	Update top of page information

Table of Contents

Version History	2
Table of Contents	3
List of Figures	9
List of Tables	11
Abbreviations	14
1 System Overview	19
1.1 Highlighted Features Integrated in MT8676	20
1.2 Platform Features	21
1.3 Modem Features	23
1.4 Connectivity Features	25
1.5 Multimedia Features	27
2 Product Description	29
2.1 Pin Description	29
2.1.1 Ball Map View	29
2.1.2 Pin Coordinate	30
2.1.3 Detailed Pin Descriptions	43
2.2 Electrical Characteristics	56
2.2.1 Absolute Maximum Ratings	56
2.2.2 Recommended Operating Conditions	58
2.2.3 Storage Condition	61
2.2.4 DC Electrical Characteristics	61
2.2.5 AC Electrical Characteristics and Timing Diagram	64
2.2.6 Clock Squarer	75
2.3 System Configuration	76
2.3.1 Mode Selection	76
2.3.2 Constant Tied Pins	76
2.4 Power-on Sequence	77
2.5 Package Information	78
2.5.1 Package Outlines	78
2.5.2 Ordering Information	78
2.5.3 Top Marking Definition	79
2.5.4 Thermal Operating Specifications	79
2.5.5 Lead-free Packaging	79
2.5.6 MSL	79
3 MCU and Bus Fabric	80
3.1 Microcontroller Unit System (MCUSYS)	80
3.1.1 Introduction	80
3.1.2 Features	80
3.1.3 Block Diagram	81
3.1.4 Interrupt Controller	81
3.1.5 Register Definition	85
3.2 On-Chip Memory Controller	85
3.2.1 Introduction	85

3.2.2	Features	85
3.2.3	Block Diagram	85
3.2.4	Register Definition	87
3.3	External Interrupt Controller (EINTC)	88
3.3.1	Introduction	88
3.3.2	Features	88
3.3.3	Block Diagrams	88
3.3.4	Register Definition	93
3.3.5	Programming Guide	93
3.4	System Interrupt Controller (SYS_CIRQ)	95
3.4.1	Introduction	95
3.4.2	Features	95
3.4.3	Block Diagram	95
3.4.4	Register Definition	96
3.5	Infrastructure System Configuration Module (INFRACFG)	98
3.5.1	Introduction	98
3.5.2	Features	98
3.5.3	Register Definition	98
3.6	External Memory Interface (EMI)	99
3.6.1	Introduction	99
3.6.2	Features	99
3.6.3	Block Diagram	99
3.6.4	Register Definition	100
3.7	DRAM Controller (DRAMC)	101
3.7.1	Overview	101
3.7.2	Features	102
3.7.3	Block Diagram	103
3.7.4	Register Definition	103
3.7.5	Programming Guide	103
3.8	Application Processor DMA (AP_DMA)	105
3.8.1	Introduction	105
3.8.2	Features	105
3.8.3	Block Diagram	106
3.8.4	Register Definition	111
3.8.5	Programming Sequence for Different Types of Channels	112
3.9	Command Queue DMA (CQ_DMA)	115
3.9.1	Introduction	115
3.9.2	Features	115
3.9.3	Block Diagram	115
3.9.4	Special Modes	116
3.9.5	Register Definition	121
3.10	System Companion Processor (SCP)	122
3.10.1	Introduction	122
3.10.2	Features	122
3.10.3	Block Diagram	122

3.10.4 Register Definition	123
3.11 System Level Cache/Buffer (SLC/SLB)	124
3.11.1 Introduction	124
3.11.2 Features	124
3.11.3 Block Diagram	125
3.11.4 Register Definition	125
4 Clock and Power Control	126
4.1 Top Clock Generator (TOPCKGEN)	126
4.1.1 Introduction	126
4.1.2 Features	126
4.1.3 Block Diagram	126
4.1.4 Clock PLL	128
4.1.5 PLL Related Control	128
4.1.6 Clock Gating	129
4.1.7 Frequency Meter	130
4.1.8 Register Definition	132
4.1.9 Programming Guide	132
4.2 Top Reset Generation Unit (TOPRGU)	134
4.2.1 Introduction	134
4.2.2 Features	134
4.2.3 Block Diagram	134
4.2.4 Register Definition	135
4.2.5 Programming Guide	135
4.3 Frequency Hopping Controller (FHCTL)	137
4.3.1 Introduction	137
4.3.2 Features	137
4.3.3 Block Diagram	137
4.3.4 Register Definition	137
4.3.5 Theory of Operation	138
4.3.6 Programming Guide	139
5 Peripherals	141
5.1 Peripheral System Configuration Module (PERICFG)	141
5.1.1 Introduction	141
5.1.2 Features	141
5.1.3 Register Definition	142
5.2 General-Purpose Input/Output (GPIO) Control	143
5.2.1 General Descriptions	143
5.2.2 Register Definition	153
5.3 Keypad Scanner	154
5.3.1 Introduction	154
5.3.2 Features	154
5.3.3 Block Diagram	154
5.3.4 Theory of Operations	154
5.3.5 Programming Guide	156
5.3.6 Register Definition	156

5.4	Universal Asynchronous Receiver/Transmitter	157
5.4.1	Introduction	157
5.4.2	Features	157
5.4.3	Block Diagram	158
5.4.4	Communication Protocol	158
5.4.5	Theory of Operations	159
5.4.6	Register Definition	164
5.4.7	Programming Guide	164
5.5	Super Speed Universal Serial Bus (SSUSB)	168
5.5.1	Introduction	168
5.5.2	Features	168
5.5.3	Host Function Architecture	168
5.5.4	Device Function Architecture	169
5.5.5	Register Definition	170
5.6	Serial Peripheral Interface (SPI) Master	171
5.6.1	Introduction	171
5.6.2	Features	171
5.6.3	Block Diagram	175
5.6.4	Theory of Operations	175
5.6.5	Register Definition	180
5.6.6	Top Programming Outline	180
5.6.7	AC Timing	181
5.7	MMC and SD Controller (MSDC)	183
5.7.1	Introduction	183
5.7.2	Features	183
5.7.3	Block Diagram	184
5.7.4	Register Definition	184
5.7.5	Programming Guide	184
5.8	Inter-IC (I2C)/Improved Inter-IC (I3C) Controller	189
5.8.1	Introduction	189
5.8.2	Features	189
5.8.3	Block Diagram	195
5.8.4	Timing Characteristics	196
5.8.5	Register Definition	198
5.9	Pulse-Width Modulators	199
5.9.1	Introduction	199
5.9.2	Features	199
5.9.3	Block Diagram	199
5.9.4	Theory of Operations	200
5.9.5	Programming Guide	203
5.9.6	Register Definition	209
5.10	General-Purpose Timer (GPT)	210
5.10.1	Introduction	210
5.10.2	Features	210
5.10.3	Block Diagram	210

5.10.4 Theory of Operations	211
5.10.5 Register Definition	211
5.10.6 Programming Guide	211
5.11 Thermal Controller	217
5.11.1 Introduction	217
5.11.2 Features	217
5.11.3 Block Diagram	217
5.11.4 Temperature Measurement Scheme	218
5.11.5 Theory of Operation	218
5.11.6 Thermal Sensor Locations	220
5.11.7 Register Definition	220
5.11.8 Programming Guide	221
5.12 Audio System	222
5.12.1 Introduction	222
5.12.2 Features	222
5.12.3 Block Diagram	223
5.12.4 Register Definition	225
5.13 Digital Signal Processor (DSP)	226
5.13.1 Introduction	226
5.13.2 Features	226
5.13.3 Block Diagram	229
5.13.4 Theory of Operations	230
5.13.5 Register Definition	231
5.13.6 Programming Guide	231
5.14 Universal Flash Storage (UFS)	232
5.14.1 Introduction	232
5.14.2 Features	232
5.14.3 Block Diagram	232
5.14.4 Register Definition	233
5.15 Peripheral Component Interconnect Express (PCIe) Controller	234
5.15.1 Introduction	234
5.15.2 Features	234
5.15.3 Block Diagram	234
5.15.4 PCIe Controller Initial Sequence	236
5.15.5 PCIe Reset Scenarios	238
5.15.6 Memory Transactions	239
5.15.7 I/O Transactions	239
5.15.8 Configuration Transactions	240
5.15.9 Message Transactions	240
5.15.10 Interrupt Handling	240
5.15.11 Error Handling Function	241
5.15.12 PCIe Module Register Definition	241
5.15.13 PCIe Physical Layer Overview	243
5.15.14 Register Definition	244
6 Multimedia	245

6.1	Display Controller	245
6.1.1	Introduction	245
6.1.2	Features	245
6.1.3	Block Diagram	245
6.1.4	Register Definition	246
6.2	Camera Image Signal Processor (ISP)	247
6.2.1	Introduction	247
6.2.2	Features	247
6.2.3	Register Definition	247
6.3	Camera Serial Interface (CSI)	248
6.3.1	Introduction	248
6.3.2	Features	248
6.3.3	Block Diagram	248
6.3.4	Lanes Mapping	249
6.3.5	Electrical Characteristics	249
6.3.6	Register Definition	250
6.4	Display Power Width Module (DISP_PWM) Generator	251
6.4.1	Introduction	251
6.4.2	Features	251
6.4.3	Register Definition	251
6.4.4	Programming Guide	251
6.5	Display Serial Interface (DSI)	252
6.5.1	Introduction	252
6.5.2	Features	252
6.5.3	Register Definition	252
6.6	Video Decoder (VDEC)	253
6.6.1	Introduction	253
6.6.2	Block Diagram	253
6.6.3	Theory of Operation	254
6.6.4	Register Definition	254
6.7	Video Encoder (VENC)	255
6.7.1	Introduction	255
6.7.2	Features	255
6.7.3	Block Diagram	255
6.7.4	Register Definition	256
6.8	MFlexGraphics (MFG)	257
6.8.1	Introduction	257
6.8.2	Features	257
6.8.3	Register Definition	258
Exhibit 1 Terms and Conditions		259

List of Figures

Figure 1-1. Functional block diagram of high-level MT8676	20
Figure 2-1. Ball map view	30
Figure 2-2. LPDDR5X TOP ball map view	錯誤! 尚未定義書籤。
Figure 2-3. LPDDR5X WCK VIX definition	64
Figure 2-4. LPDDR5X single-ended output slew-rate definition	65
Figure 2-5. LPDDR5X differential output slew-rate definition	65
Figure 2-6. LPDDR5X Rx mask	65
Figure 2-7. SPI timing diagram	66
Figure 2-8. I2S master mode timing diagram	67
Figure 2-9. I2C timing diagram of standard mode (100 kHz) and fast mode (400 kHz)	68
Figure 2-10. MSDC device input timing diagram of default speed	69
Figure 2-11. MSDC device output timing diagram of default speed	69
Figure 2-12. MSDC device input timing diagram of high-speed	70
Figure 2-13. MSDC device output timing diagram of high-speed	70
Figure 2-14. MSDC device clock timing diagram of SDR12/SDR25/SDR50/SDR104 modes	71
Figure 2-15. MSDC device input timing diagram of SDR50/SDR104 modes	71
Figure 2-16. MSDC device output timing diagram of fixed data window (SDR12/SDR25/SDR50)	71
Figure 2-17. MSDC device output timing diagram of variable window (SDR104)	72
Figure 2-18. MSDC device clock timing diagram of DDR50 speed mode	72
Figure 2-19. MSDC device input/output timing diagram of DDR50 speed mode	73
Figure 2-20. MSDC device clock timing diagram of HS200	73
Figure 2-21. MSDC device input timing diagram of HS200	74
Figure 2-22. MSDC device output timing diagram of HS200	74
Figure 2-23. Outlines and dimensions of HBPOP 14 mm x 14 mm, 1310-ball, 0.35 mm pitch package	78
Figure 2-24. Top marking of MT8676	79
Figure 3-1. Block diagram of on-chip memory controller	86
Figure 3-2. Security memory protection scheme	87
Figure 3-3. Block diagram of EINTC	88
Figure 3-4. Secure view of external interrupt controller	89
Figure 3-5. System level block diagram of SYS_CIRQ	95
Figure 3-6. Block diagram of SYS_CIRQ	96
Figure 3-7. EMI/DRAM controller top connection	100
Figure 3-8. Block diagram of DRAM controller	103
Figure 3-9. Block diagram of AP_DMA	106
Figure 3-10. UART VFF pointers relationship	107
Figure 3-11. UART Tx VFF data flow diagram	108
Figure 3-12. UART Rx VFF data flow diagram	109
Figure 3-13. I2C sequential random read protocol	110
Figure 3-14. Block diagram of CQDMA	116
Figure 3-15. CQ_DMA address wrapping	118
Figure 3-16. CQ_DMA address fixed mode	120
Figure 3-17. CQ_DMA descriptor mode diagram	120
Figure 3-18. Block diagram of SCP	123
Figure 3-19. EMI/SLC controller top connection	125
Figure 4-1. Block diagram of clock architecture	126
Figure 4-2. Example of clock multiplier	127
Figure 4-3. PLL block diagram (for reference only)	128

Figure 4-4. TOPCKGEN frequency meter structure	130
Figure 4-5. Block diagram of TOPRGU	134
Figure 4-6. Block diagram of FHCTL	137
Figure 4-7. Hopping stage diagram	139
Figure 5-1. GPIO block diagram	143
Figure 5-2. 3*3 keypad matrix (9 keys)	154
Figure 5-3. Signal key scan waveform	155
Figure 5-4. One-key-pressed with debounce mechanism denoted	155
Figure 5-5. Two-keys-pressed sequence	155
Figure 5-6. Block diagram of UART	158
Figure 5-7. UART communication protocol	159
Figure 5-8. UART data transmission with polling the THRE bit status	166
Figure 5-9. UART data reception with polling the DR bit status	166
Figure 5-10. Block diagram of SSUSB	168
Figure 5-11. USB2.0 host architecture	169
Figure 5-12. SSUSB device architecture	170
Figure 5-13. Pin connection between SPI master and SPI slave	171
Figure 5-14. SPI transmission formats	172
Figure 5-15. Operation flow with/without PAUSE mode	173
Figure 5-16. CS_N de-assert mode	173
Figure 5-17. Waveforms of the four communication modes	174
Figure 5-18. Pin connection between the master and the multi-slave for single mode	174
Figure 5-19. Block diagram of the SPI master	175
Figure 5-20. SPI single mode and full duplex mode transaction format	176
Figure 5-21. SPI single mode and half duplex mode Tx transaction format	176
Figure 5-22. SPI single mode and half duplex mode Rx transaction format	176
Figure 5-23. SPI dual mode and half duplex mode Tx transaction format	177
Figure 5-24. SPI dual mode and half duplex mode Rx transaction format	177
Figure 5-25. SPI dual mode and full duplex mode transaction format	177
Figure 5-26. SPI quad mode and half duplex mode Tx transaction format	178
Figure 5-27. SPI quad mode and half duplex mode Rx transaction format	178
Figure 5-28. SPI half duplex output mode transaction format	178
Figure 5-29. SPI half duplex input mode transaction format	179
Figure 5-30. SPI master sampling time	179
Figure 5-31. SPI master timing diagrams	181
Figure 5-32. Block diagram of MSDC	184
Figure 5-33. PIO single block timing	186
Figure 5-34. PIO multiple block timing	186
Figure 5-35. DMA single/multiple block timing	187
Figure 5-36. Tuning flow	188
Figure 5-37. I2C transaction (write)	190
Figure 5-38. I2C transaction (read)	190
Figure 5-39. I2C transaction (read after write)	191
Figure 5-40. I2C high-speed mode	191
Figure 5-41. I3C transaction (write)	192
Figure 5-42. I3C transaction (read)	192
Figure 5-43. I3C transaction (read after write)	193
Figure 5-44. I3C transaction (with 7E or CCC)	193
Figure 5-45. Block diagram of I2C/I3C	195

Figure 5-46. I2C AC timing diagram of HS mode	197
Figure 5-47. Generation procedure of PWM	199
Figure 5-48. Block diagram of PWM	199
Figure 5-49. Old mode	200
Figure 5-50. FIFO mode	201
Figure 5-51. Memory mode	201
Figure 5-52. Memory mode and stop bit position	201
Figure 5-53. Random mode	202
Figure 5-54. 3D_LCM mode	203
Figure 5-55. Breathing light mode	203
Figure 5-56. Block diagram of APXGPT	211
Figure 5-57. Thermal control subsystem	217
Figure 5-58. Block diagram of system temperature measurement scheme	218
Figure 5-59. Interrupt conditions for high/low temperature monitoring	219
Figure 5-60. Finite state machine for high/low temperature monitoring	219
Figure 5-61. Interrupt conditions of high/low OFFSET monitoring	220
Figure 5-62. Finite state machine for high/low OFFSET monitoring	220
Figure 5-63. Block diagram of audio system	224
Figure 5-64. Block diagram of audio DSP	230
Figure 5-65. Block diagram of UFS host controller	233
Figure 5-66. Block diagram of PCIe controller	235
Figure 5-67. Block diagram of PCIe PHY	244
Figure 6-1. Block diagram of display controller	245
Figure 6-2. Block diagram of CSI	248
Figure 6-3. Block diagram of video decoder	253
Figure 6-4. VDEC hardware architecture	254
Figure 6-5. Procedure of VENC	255
Figure 6-6. Block diagram of VENC	256
Figure 6-7. Block diagram and interface of MFG	257

List of Tables

Table 2-1. Pin coordinate	30
Table 2-2. Acronym for pin type	43
Table 2-3. Detailed pin descriptions	43
Table 2-4. Absolute maximum ratings for power supply	56
Table 2-5. Recommended operating conditions for power supply	58
Table 2-6. RTC DC electrical characteristics (DVDD18_IOTM = 1.8V)	61
Table 2-7. SPI, I2S DC electrical characteristics (DVDD18_IOBM = 1.8V)	62
Table 2-8. I2C/I3C DC electrical characteristics (DVDD18_IOBM = 1.8V)	62
Table 2-9. I2C3 DC electrical characteristics (DVDD18_IOBM = 1.8V)	62
Table 2-10. MSDC1 DC electrical characteristics (DVDD28_MSDC1 = 2.9V)	62
Table 2-11. MSDC1 DC electrical characteristics (DVDD28_MSDC1 = 1.8V)	62
Table 2-12. MSDC2 DC electrical characteristics (DVDD18_MSDC2 = 1.8V)	63
Table 2-13. SIM DC electrical characteristics	63
Table 2-14. LPDDR5X AC timing parameter table of external memory interface	66
Table 2-15. SPI AC timing parameters	66
Table 2-16. I2S AC timing parameters	67
Table 2-17. I2C AC timing parameters	68

Table 2-18. MSDC device AC timing parameters of default speed	69
Table 2-19. MSDC device AC timing parameters of high-speed	71
Table 2-20. MSDC device AC timing parameters of SDR12/SDR25/SDR50/SDR104 modes	72
Table 2-21. MSDC device AC timing parameters of DDR50 speed mode	73
Table 2-22. MSDC device AC timing parameters of HS200	74
Table 2-23. SIM AC timing parameters	75
Table 2-24. Clock squarer specifications	76
Table 2-25. Mode selection	76
Table 2-26. Constant tied pins	76
Table 2-27. Thermal operating specifications	79
Table 3-1. Interrupt request list for GIC700	82
Table 3-2. Memory map of on-chip memory controller	85
Table 3-3. External interrupt request signal connection	89
Table 3-4. Definitions of domains	89
Table 3-5. EINT table	89
Table 3-6. DRAM bus signal list (refer to the DRAMC side) (LPDDR5/5X)	101
Table 3-7. DRAM bus command truth table (LPDDR5/5X)	101
Table 3-8. APDMA matrix	106
Table 3-9. AP_DMA and I2C sequential random read settings	110
Table 3-10. Warm/Hard reset features	111
Table 3-11. Example of pause and resume functions	111
Table 3-12. Reference settings for peripheral DMA (I2C)	112
Table 3-13. Reference settings for virtual FIFO DMA Tx (UART_TX)	113
Table 3-14. Reference settings for virtual FIFO DMA Rx (UART_RX)	113
Table 3-15. Reference settings for fixed pattern mode	116
Table 3-16. Reference settings for read address wrapping mode	117
Table 3-17. Reference settings for write address wrapping mode	118
Table 3-18. Reference settings for source address fix mode	119
Table 3-19. Reference settings for destination address fix mode	119
Table 3-20. SCP frequency	123
Table 4-1. PLL related control	129
Table 4-2. Clock gating settings	129
Table 5-1. Summary of configuration registers of GPIOs	143
Table 5-2. Keypad setting flow	156
Table 5-3. Signal descriptions	158
Table 5-4. UART interrupt control bits and interrupt factors	159
Table 5-5. UART interrupt types	160
Table 5-6. UART register map	161
Table 5-7. Bit extend number reference	163
Table 5-8. Suggestion for UART baud rate setting	164
Table 5-9. UART baud rate setting example	164
Table 5-10. UART hardware initialization	165
Table 5-11. SPI master controller interface	171
Table 5-12. SPI master abbreviations and full forms	171
Table 5-13. SPI transaction format summary	176
Table 5-14. SPI master DMA mode guide	180
Table 5-15. SPI master FIFO mode guide	181
Table 5-16. SPI master electrical specifications	182
Table 5-17. MSDC functions and address	184

Table 5-18. I2C AC timing parameters for HS mode	197
Table 5-19. Old mode setting procedures	203
Table 5-20. FIFO mode setting procedures	204
Table 5-21. Memory mode setting procedures	205
Table 5-22. Random mode setting procedures	206
Table 5-23. 3D_LCM mode setting procedures	208
Table 5-24. Breathing light mode setting procedures	209
Table 5-25. Operation mode of GPT	210
Table 5-26. APXGPT ONE-SHOT mode setting flow	212
Table 5-27. APXGPT REPEAT mode setting flow	213
Table 5-28. APXGPT KEEP-GO mode setting flow	214
Table 5-29. APXGPT FREERUN mode setting flow	215
Table 5-30. GPT5 64-bit counter read flow	216
Table 5-31. Temperature sensor monitoring locations	220
Table 5-32. Thermal controller programming guide	221
Table 5-33. ADSP interrupt function	228
Table 5-34. DSP Core0/1 interrupt connection	229
Table 5-35. Debug signals	230
Table 5-36. PCIe module register mapping	241
Table 5-37. PCIe internal register	242
Table 5-38. PCIe configuration space and PCIe extended configuration space	242
Table 6-1. CDPHY_TOP PAD interface	249
Table 6-2. Operation requirements for CSI PHY	249
Table 6-3. C-PHY Rx high speed electrical characteristics	249
Table 6-4. D-PHY Rx high speed electrical characteristics	250
Table 6-5. CD-PHY Rx low power electrical characteristics	250
Table 6-6. Features of VENC	255

Abbreviations

Abbreviation	Full Form
0-9	
3GPP	Third Generation Partnership Project
A	
AE	Auto Exposure
AER	Advanced Error Reporting
AF	Auto Focus
AFBC	Arm Frame Buffer Compression
AFRC	Arm Fixed Rate Compression
AI	Artificial Intelligence
ALP	Alternate Low-Power Mode
AMR	Adaptive Multi-Rate
APB	Advanced Peripheral Bus
APDMA	Application Processor Direct Memory Access
APU	Accelerated Processing Unit
APXGPT	Application Processor of General-Purpose Timer
ASPM	Active State Power Management
ASRC	Asynchronous Sample Rate Converter
AWB	Auto White Balance
AXI	Advanced eXtensible Interface
B	
BPI	Baseband Parallel Interface
BT	Bluetooth
BW	Bandwidth
C	
CA	Carrier Aggregation
CC	Component Carrier
CCC	Common Command Code
CCU	Camera Control Unit
CLK	Clock
CMD	Command
CPC	Continuous Packet Connectivity
CQDMA	Command Queue Direct Memory Access
CRC	Cyclic Redundancy Check
CRS	Cell Reference Signal
CSI	Camera Serial Interface
CTS	Clear to Send
D	
DA	Dynamic Address
DAA	Dynamic Address Assignment
DCM	Dynamic Clock Management
DEV	Device
DMA	Direct Memory Access
DME	Device Management Entity
DMRS	Demodulation Reference Signal

Abbreviation	Full Form
DP	Display Port
DRAM	Dynamic Random-Access Memory
DRAMC	DRAM Controller
DSI	Display Serial Interface
DSP	Digital Signal Processor
DVFS	Dynamic Voltage and Frequency Scaling
E	
EDGE	Enhanced Data Rates for GSM Evolution
EFR	Enhanced Full Rate
EHS	Extra Header Segment
EINT	External Interrupt
EINTC	External Interrupt Controller
EMI	External Memory Interface
EN-DC	EUTRA-NR Dual Connectivity
ETWS	Earthquake and Tsunami Warning System
F	
FDD	Frequency-Division Duplex
FHCTL	Frequency Hopping Controller
FPC	Function Pattern Code
FPS	Frames per Second
FPU	Floating Point Unit
FR	Full Rate
FSM	Finite State Machine
G	
GIC	Genetic Interrupt Controller
GPIO	General-Purpose Input/Output
GPRS	General Packet Radio Service
GPS	Global Positioning System
GPU	Graphic Processing Unit
GSM	Global System for Mobile Communications
H	
HBPOp	High Bandwidth Package on Package
HDR	High Dynamic Range
HEIF	High Efficiency Image File
HEVC	High Efficiency Video Coding
HR	Half Rate
HSDPA	High-Speed Downlink Packet Access
HS-DSCH	High-Speed Downlink Shared Channel
HS-SCCH	High-Speed Shared Control Channel
HSUPA	High-Speed Uplink Packet Access
I	
I2C	Inter-IC
I2S	Inter-IC Sound
I3C	Improved Inter-IC
IIR	Infinite Impulse Response
INFRACFG	Infrastructure System Configuration Module
IOC	Interrupt on Completion

Abbreviation	Full Form
ISP	Image Signal Processor
ISR	Interrupt Service Routine
J	
JTAG	Joint Test Action Group
K	
KP	Keypad
L	
L1PMSS	L1 Power Management Substates
LCM	Liquid Crystal Module
LCR	Line Control Register
LDO	Low Dropout
LFR	Low Frame Rate
LPM	Lower Power Management
LRTE	Latency Reduction and Transport Efficiency
LTE	Long Term Evolution
LTR	Latency Tolerance Reporting
LVTS	Low Voltage Thermal Sensing
M	
MCDI	Multi-Core-Deep-Idle
MCU	Microcontroller Unit
MD	Modem
MFG	MFlexGraphics
MIMO	Multi-Input Multi-Output
MIPI	Mobile Industry Processor Interface
MMU	Memory Management Unit
MPLRU	Modified Pseudo Least Recently Used
MPU	Memory Protection Unit
mSBC	modified Sub-Band Coding
MSDC	MMC and SD controller
MSDC	Mass Storage Device Class
MSI	Message Signaled Interrupt
MVPU	MediaTek Vision Processor Unit
N	
NMI	Non-Maskable Interrupt
NN	Neural Network
NR	New Radio
O	
OBFF	Optimized Buffer Flush/Fill
OTG	On-the-Go
P	
PA	Power Amplifier
PCB	Printed Circuit Board
PCIe	Peripheral Component Interconnect Express; PCI Express
PCM	Pulse Code Modulation
PDCCH	Physical Downlink Control Channel
PDM	Pulse Density Modulation
PERICFG	Peripheral System Configuration Module

Abbreviation	Full Form
PIF	Processor Interface
PIPE	PHY Interface for the PCIe
PLC	Packet Loss Concealment
PMIC	Power Management IC
POR	Power-on Reset
PWM	Pulse-Width Modulation
Q	
QAM	Quadrature Amplitude Modulation
R	
RBR	Rx Buffer Register
RC	Root Complex
RMMI	Reference M-PHY Module Interface
ROM	Read-only Memory
RPU	Region Protection Unit
RTC	Real-Time Clock
RTS	Request to Send
S	
SA	Standalone
SAIC	Single Antenna Interference Cancellation
SBAS	Satellite-Based Augmentation Systems
SBC	Sub-Band Coding
SCL	Serial Clock Line
SCP	System Companion Processor
SDA	Serial Data
SDIO	Secure Digital Input/Output Interface
SDR	Single Data Rate
SIM	Subscriber Identification Module
SINAD	Signal-to-Noise and Distortion Ratio
SLB	System Level Buffer
SLC	System Level Cache
SP FPU	Single Precision Floating Point Unit
SPI	Serial Peripheral Interface
SPMC	System Power Management Controller
SRAM	Static Random Access Memory
SRLTE	Single Radio LTE
SSC	Spread Spectrum Clocking
SSUSB	SuperSpeed Universal Serial Bus
SYS_CIRQ	System Interrupt Controller
T	
TCM	Tightly-Coupled Memory
TDD	Time-Division Duplex
TDM	Time Division Multiplexing
THR	Tx Holding Register
THRE	Tx Holding Register Empty
TOPCKGEN	Top Clock Generator
TOPRGU	Top Reset Generation Unit
TSMCU	Thermal Sensing Micro Circuit Unit

Abbreviation	Full Form
U	
UART	Universal Asynchronous Receiver/Transmitter
UFS	Universal Flash Storage
UFSHCI	Universal Flash Storage Host Controller Interface
UI	Unit interval
USB	Universal Serial Bus
USIM	Universal Subscriber Identity Module
V	
VC	Virtual Channel
VDEC	Video Decoder
VENC	Video Encoder
W	
WCG	Wide Color Gamut
WDT	Watchdog Timer
WLEN	Wireless LEN
X	
xHC	eXtensible Host Controller
xHCI	eXtensible Host Controller Interface

1 System Overview

The MT8676 device is a highly integrated and scalable application automotive processor with rich multimedia features and AI capabilities. The chip integrates 4 Arm® Makalu and 4 Arm® Klein, along with a powerful multi-standard video codec. Furthermore, an extensive set of interfaces is integrated for camera modules, external audio components, UFS/SD cards, and external modules.

The ARM® Makalu application processor offers substantial computing power to support the latest open-source operating systems, as well as In-Vehicle-Infotainment (IVI) and Cockpit Domain Controller (CDC) applications. The ARM® Klein provide adequate computing power for dedicated tasks, such as software-based graphic rendering and real-time operating system in critical execution environments.

In addition, the SMMU with unique performance enhancements for hard real-time masters, integrated in MT8676, provide a straightforward hardware mechanism that minimizes the virtualization overhead of hypervisors. This is particularly beneficial when multiple operating systems coexist for various application domains, thus optimizing overall system performance.

The new Gen 7 APU in MT8676 is capable to address the latest AI trends in achieving maximizing effective performance in AI-multimedia, AI-camera, and AI-voice experiences. The APU Gen 7 also aims to ensure that AI-enhanced technologies to perform sustainably across conditions for extended durations.

The multi-standard video accelerator and an advanced audio subsystem are also integrated to provide advanced multimedia applications and services such as streaming audio and video, a multitude of decoders and encoders.

The high-performance CPU, DSP, and hardware coprocessors are combined to provide a powerful modem subsystem capable of supporting NR Sub6, LTE Cat 18, Category 24 HSDPA downlink and Category 7 HSUPA uplink data rates, as well as Class 12 GPRS, EDGE.

MT8676 also embodies wireless communication devices, including WLAN, Bluetooth and GPS. With four advanced radio technologies by Combo chipsets, MT8676 provides the best and most convenient connectivity solution in the industry.

The chip supports rich automotive camera features, such as around view monitoring, back view monitoring, automotive driving recording, and driver monitoring. These features can be employed via the maximum 16x video stream inputs with MIPI-CSI2, or Ethernet AVB inputs. The rich display interface (MIPI-DSI and DP) allows for support of up to 6 display panels, further enhancing its usability in various automotive applications.

- 4 Arm® Makalu cores
- 4 Arm® Klein cores
- LPDDR5X up to 24 GB (4 channels with 16-bit data bus width)
- Memory clock up to LPDDR5X-7500
- LTE cat-18 4x4 MIMO
- NR Sub6 3CC with 220 MHz BW
- Embedded connectivity system including WLAN/BT/FM/GPS
- Resolution up to WQHD+ (1,600 x 3,360)
- OpenGL ES 3.2/Vulkan 1.2 3D graphic accelerator
- ISP supports 108 MP@30 fps.
- AV1 4K@60 fps decoder
- HEVC 4K@60 fps decoder
- HEVC 4K@60 fps encoder
- Speech codec (FR, HR, EFR, AMR FR, AMR HR and wide-band AMR, and EVS WB)



1.2 Platform Features

• General

- Cockpit, 2 MCU subsystems architecture
- Supports UFS boot
- Supports LPDDR5X

• AP MCU subsystem

- 4 Arm® Makalu cores with each core having 64 KB L1 I-cache, 64 KB L1 D-cache and 512 KB L2 cache
- 4 Arm Klein cores with each core having 32 KB L1 I-cache, 32 KB L1 D-cache. And 2 Klein cores shared 128 KB L2 cache.
- Shared 4 MB L3 cache
- NEON multimedia processing engine with SIMDv2/VFPv4 ISA support
- DVFS technology with adaptive operating voltage from 0.65V to 1.05V for Makalu core, and from 0.55V to 1.05V for both Makalu cores and Klein cores

• APU for CV and NN

- Top performance: Fix8: Up to 1.15 TOPS; Fix16: Up to 0.28 TOPS; FP16: Up to 0.28 TOPS; FP32: Up to 0.07 TOPS
- C, OpenCL and Halide programming language
- Advanced software framework for automatic filing and fusion
- Enhanced data sharing for CV/NN algorithm
- Native transcendental function for special operations
- Vector FPU to support high precision requirement applications
- 512 KB local memory (L1 memory)

• APU to support NN applications with high-performance and high-power efficiency

- Top performance: 8(A) x 4(W) 20 TOPS, 4(A) x 4(W) 20 TOPS, 8(A) x 8(W) 20 TOPS, 16(A) x 8(W) 10 TOPS, 16(A) x 16(W) 5 TOPS FP16/BF16 5 TOPS
- Supports layer based mixed precision
- Simultaneous pipelined hardware function block (CONV/ACT/POOL/EWE/BILINEAR)

- Enhancement of layer fusion to further reduce DRAM/TCM memory bandwidth
- Enhancement of non-CONV OP performance
- Supports Android NN asymmetric quantized data format
- Supports weight decompression to reduce DRAM BW

• APU memory subsystem

- 2nd level data memory: Scenario based allocation policy, up to 3 MB
- eDMA for data movement and data format conversion

• MD MCU subsystem

- High-performance multi-core and multi-thread processor architecture
- High-performance AXI bus interfaces
- General DMA engine and dedicated DMA channels for peripheral data transfer
- Power management for clock gating control

• MD external interfaces

- Dual SIM/USIM interface
- Interface pins with RF and radio-related peripherals (antenna tuner, PA, etc.)

• Security

- Arm® TrustZone® Security

• External memory interface

- LPDDR5X up to 24 GB (4 channels with 16-bit data bus width)
- Memory clock up to LPDDR5X-7500
- Self-refresh/partial self-refresh mode
- Low-power operation
- Programmable slew rate for memory controller's I/O pads
- Dual rank memory device
- Advanced bandwidth arbitration control

• Peripherals

- USB port with USB 3.0 host/device mode or USB2.0 OTG mode
- UFS 4.0 2-lane
- 4 UARTs for debugging and applications
- 8 SPI masters for external devices, one of them supports QSPI.
- 7 I2Cs/7 I3Cs to control peripheral devices, e.g., CMOS image sensor, LCM or FM receiver module
- Max. 4 PWM channels (depending on system configuration and I/O usage)
- GPIOs
- 2 sets of memory card controllers supporting SD/SDIO 3.0 protocols

• Operating conditions

- Core voltage: 0.55~0.8V
- I/O voltage: 1.8V
- LCM interface: 1.8V
- Clock source: 26 MHz, 32.768 kHz

• Package

- Type: MFC VFBGA
- 13.8 mm x 15.2 mm
- Max. height: 0.946 mm
- Ball count: 1,517 balls
- Ball pitch: 0.35 mm

1.3 Modem Features

• NR Sub6

- 3GPP EN-DC option 3/3a/3x
- 3GPP SA option 2
- FDD/TDD up to 5.11 Gbps downlink, 2.5 Gbps uplink
- Supports downlink CA: 5~100 MHz RF bandwidth per component carrier (CC) and up to 3CCs with total 220 MHz BW
- Supports uplink CA: 5~100 MHz RF bandwidth per component carrier (CC) and up to 2CCs with total 200 MHz BW
- Supports SCS 15/30 kHz
- Supports downlink/uplink 256 QAM
- Supports downlink 4x4 MIMO and uplink 2x2 MIMO

• LTE

- FDD/TDD up to 1.2 Gbps downlink, 210 Mbps uplink
- Downlink carrier aggregation (CA) ability; 1.4~20 MHz RF bandwidth per component carrier (CC) and up to 4 CCs
- Uplink intra-band carrier aggregation (CA) ability; 1.4~20 MHz RF bandwidth per component carrier (CC) and up to 2 CCs
- Supports downlink/uplink 256 QAM
- 4 x 2 downlink SU-MIMO per component carrier
- Downlink MU-MIMO per component carrier
- Supports MBMS
- Uplink CoMP ability
- Advanced interference cancellation
 - PDCCH pIRC
 - CRS-based PDSCH
 - DMRS-based Co-UE
- Transmit antenna selection

• 3G UMTS FDD supported features

- 3G modem supports most main features in 3GPP Release 7 and Release 8.
- CPC (DTX in CELL_DCH, UL DRX DL DRX), HS-SCCH-less, HS-DSCH
- Dual cell operation
- MAC-ehs
- 2 DRX (receiver diversity) schemes in URA_PCH and CELL_PCH

- Uplink Cat. 7 (16 QAM), throughput up to 11.5 Mbps
- Downlink Cat. 24 (64 QAM, dual-cell HSDPA), throughput up to 42.2 Mbps
- Fast dormancy
- ETWS
- Network selection enhancement
- Transmit antenna selection

• Radio interface and baseband front-end

- IQ data through high-speed serial interface (DigRF) between RF and baseband chips
- Programmable radio Rx filter with adaptive gain control
- Dedicated Rx filter for FB acquisition
- Baseband parallel interface (BPI) and MIPI RFFE interface with programmable driving strength
- Supports multi-band

• GSM modem and voice codec

- Dial tone generation
- Noise reduction
- Echo suppression
- Advanced side-tone oscillation reduction
- Digital side-tone generator with programmable gain
- 2 programmable acoustic compensation filters
- GSM quad vocoders for adaptive multi-rate (AMR), enhanced full rate (EFR), full rate (FR) and half rate (HR)
- GSM channel coding, equalization and A5/1, A5/2, A5/3, and A5/4 ciphering
- GPRS GEA1, GEA2, GEA3 and GEA4 ciphering
- Programmable GSM/GPRS/EDGE modem
- Packet switched data with CS1/CS2/CS3/CS4 coding schemes
- GSM circuit switch data
- GPRS/EDGE Class 12
- Supports SAIC technology
- VAMOS technology in R9 spec
- Transmit antenna selection
- RF dual-antenna reception

• CDMA2000 modem

- Supports CDMA2000 1xRTT (releases 0)

- Supports maximum 1x data rates of 153.6 kbps for forward and reverse links
- Supports 1x diversity
- Supports single radio LTE (SRLTE)
- Transmit antenna selection

1.4 Connectivity Features

MT8676 includes four wireless connectivity functions:

- WLAN
- Bluetooth
- GPS
- FM receiver

The RF parts of those four blocks are placed on chip MT6637. With four advanced radio technologies integrated on one chip, MT8676 and MT6637 are the best and most convenient connectivity solution in the industry, implementing advanced and sophisticated radio coexistence algorithms and hardware mechanisms. It supports dual antenna sharing among 2.4 GHz Bluetooth, 2.4 GHz/5 GHz/6 GHz WLAN and 1.575 GHz for GPS. The enhanced overall quality is achieved for simultaneous voice, data and audio/video transmission on mobile phones and media tablets. The small footprint with low-power consumption greatly reduces PCB layout resource.

- **Supports integrated Wi-Fi/Bluetooth/GPS**

- Dual antenna for WLAN
- Single/Dual antenna for WLAN/GPS/Bluetooth
- Self-calibration
- Single TCXO and TMS for GPS, BT and WLAN
- Best-in-class current consumption performance
- Intelligent BT/WLAN coexistence scheme that goes beyond PTA signaling (e.g., transmit window and duration that take into account protocol exchange sequence, frequency, etc.)

- **Wi-Fi**

- Dual-band (2.4/5/6 GHz) with 2-stream 802.11 ax/ac/a/b/g/n MAC/BB/RF SoC, 20/40/80/160 MHz bandwidth, MCS0~11 (1,024-QAM)
- 802.11 d/e/h/i/j/k/r/v compliant
- Security: WPA WPA2/WPA3 personal/enterprise, AES-CCMP, WPI-SMS4, GCMP, WPS2.0, WAPI (hardware)
- QoS: WPA WMM, WMM PS

- 802.11n optional features: STBC, A-MPDU, Blk-Ack, RIFS, MCS feedback, 20/40 MHz coexistence (PCO), unscheduled PSMP
- Supports 802.11w protected managed frames
- Supports 802.11ax STBC Tx/Rx, 4T2R beamformee, 2T2R beamformer, MU-MIMO Rx, WoWLAN
- Supports MediaTek proprietary low-power green AP mode for portable hotspot operation
- Auto rate control for optimizing the signal range and performance
- Supports Wi-Fi direct (WFA P2P standard) and Wi-Fi display
- Supports Wi-Fi HotSpot 2.0
- Integrated PA with 2.4G max. power 23 dBm for CCK, and 5G max. power 19 dBm for 54 mbps OFDM
- 2.4G Rx sensitivity is -76.5 dBm for 11n HT20 MCS7; 5G Rx sensitivity is -63 dBm for 11ac VHT80 MCS9
- Supports 32 multicast address filters and TCP/UDP/IP checksum offload
- Per packet Tx power control

- **Bluetooth**

- Supports Bluetooth v5.3 core spec.
- Supports BT/BLE dual mode
- Integrated PA with 14 dBm (class 1) transmit power
- Rx sensitivity @chip-in:
 - GFSK -97 dBm
 - DQPSK -96 dBm
 - 8-DPSK -90 dBm
 - BLE_1M -100 dBm
 - BLE_2M -97 dBm
 - BLE_500K -102 dBm (PER < 30.8%)
 - BLE_125K -106 dBm (PER < 30.8%)
- Supports BT/Wi-Fi/LTE/NR coexistence
- Supports 7 BT links and 16 BLE links
- Supports packet loss concealment (PLC) function for better voice quality
- Supports wideband speech
- Supports mSBC and SBC including mono and stereo
- Supports secure connection with AES128 and ECC256
- Supports adaptive frequency hopping with built-in channel assessment method

• GPS

- Supports L1 GPS/GLONASS/BeiDou/Galileo/QZSS concurrent reception
 - GPS: L1CA, GLONASS: L1OF, BeiDou: B1i, Galileo: E1B/ E1C, QZSS: L1CA
 - GPS/Galileo only (GPS only)
 - GPS/Galileo - GLONASS (G+G)
 - GPS/BeiDou (G+B)
 - GPS/GLONASS/BeiDou (G+G+B)
 - GPS/Galileo/GLONASS (G+G+G)
 - GPS/Galileo/GLONASS/BeiDou (G+G+G+B)
- Supports new BeiDou L1 signal (B1C) reception
- Supports new GPS L1 signal (L1C) reception
- Supports SBAS (satellite-based augmentation systems): WAAS/MSAS/EGNOS/GAGAN
- Supports L5 GPS/BeiDou-III B2A/Galileo-E5a/NavIC concurrent reception
- Best-in-class sensitivity performance
 - -165 dBm tracking sensitivity
 - -163 dBm hot start sensitivity
 - -148 dBm cold start sensitivity
 - -151 dBm warm start sensitivity
- AGPS sensitivity
 - Coarse time assistance is 8 dB design margin over 3 GPP minimum requirement.
 - Fine time assistance is 10 dB design margin over 3 GPP minimum requirement.
- Full A-GPS capability (E911/SUPL/EPO/HotStill)
- Active interference cancellation for up to 12 in-band tones for both L1 and L5 bands
- Supports both TCXO and TMS (thermistor crystal) clock source
- 5 Hz update rate

• FM

- 65~108 MHz with 50 kHz step
- RDS/RBDS
- Digital stereo demodulator
- Simplified digital audio interface (I2S)
- Stereo noise reduction
- Audio sensitivity 2 dBμVemf (SINAD = 26 dB)
- Audio SINAD 60 dB
- Anti-jamming

1.5 Multimedia Features

• Display

- Portrait panel resolution up to WQHD+ 21:9 (1,600 x 3,360), frame rate up to 120 Hz
- DisplayPort 1.4, 4 lanes
- MIPI DSI transmitter with D-PHY 4+4 lanes, or C-PHY 3+3 trios, or C-PHY 3+(2 or 1) trios
- MiraVision™ for picture quality enhancement
- Embedded LCD gamma correction
- True colors
- 8 overlay layers with per-pixel alpha channel and gamma table
- Spatial and temporal dithering
- Color enhancement
- Adaptive contrast enhancement
- Image/video/graphic sharpness enhancement
- Dynamic backlight scaling
- Supports GPU decoder (AFBC)
- Supports hardware wide color gamut (WCG)
- Supports DSC compression
- Supports sub-pixel rendering
- Supports de-mura
- Supports panel overdrive

• Graphics

- Integrated Arm® Mali G615
- Per core capable of processing pixel rate 4M pixels per MHz, bi-linear texture rate 8M texels per MHz
- OpenGL ES 3.2/3.1/3.0/2.0/1.1
- Vulkan 1.2
- OpenCL 2.1

• Image

- Integrated image signal processor
 - Zero shutter delay image capture (up to 108 MP@30 fps, single camera)
 - Multi-camera, [32 MP + 32 MP + 32 MP]@30 fps
- Supports high frame rate sensor (16 + 4M/140 fps, single CAM)
- 6 MIPI CSI-2 high-speed camera serial interfaces; max. 4 data lanes of D-PHY; max. 3 trios of C-PHY
- Electronic image stabilization

- Video stabilization
- Preference color adjustment
- Noise reduction
- Multiple frame noise reduction for image capture
- Multi-scale noise reduction for preview, video record and image capture
- Temporal noise reduction for video recording
- Anti-blooming correction
- Lens shading correction
- Automatic sensor defect pixel correction
- Supports AE/AWB/AF
- Edge enhancement (sharpness)
- Deep learning face detection
- Video face beautification
- Supports video HDR for stagger HDR sensor (16M60fps 2exp, 64X)
- Supports video HDR for 4-cell HDR sensor
- M-stream video HDR
- PD hardware accelerator
- Bayer/YUV frame buffer compression to reduce DRAM BW requirement
- Hardware JPEG encoder
- Supports YUV422/YUV420 color format and EXIF/JFIF format
- Baseline JPEG decoding throughput @YUV420, ≥ 64M pixels
- Dual camera real-time depth

• Video

- HEVC decoder 4K2K@60 fps/160 Mbps, 8/10 bits
- VP9 decoder 4K2K@60 fps/120 Mbps, 8/10 bits
- AV1 decoder 4K2K@60 fps/120 Mbps, 8/10 bits
- H.264 decoder: Constrained baseline 4K2K@60 fps/160 Mbps, 8 bits
- H.264 decoder: Main/high profile 4K2K@60 fps/160 Mbps, 8 bits
- H.264 decoder: High 10 profile 4K2K@60 fps/160 Mbps, 10 bits
- HEIF decoder: Max. resolution 16,383 x 16,383 (grid mode up to 20,656 x 15,496 for 320M pixels capture), 8/10 bits
- H.264 encoder: 10-bit high profile 4K2K@60 fps, 160 Mbps

- HEVC encoder: 10-bit main profile 4K2K@60 fps, 160 Mbps

- **Audio**

- Supported audio content sampling rates: 8 ~ 192 kHz (up to 384 kHz for I2S)
- Supported audio content sample formats: 8-bit/16-bit/24-bit, mono/stereo (up to 32-bit for I2S)
- Supported interfaces: I2S, PCM
- External codec I2S interface supports 16-bit/24-bit/32-bit, mono/stereo, and 8~384 kHz.
- 4-band IIR compensation filter to enhance loudspeaker responses
- Proprietary audio post-processing technologies: BesLoudness (MB-DRC), Android built-in post processing
- Audio encoding: AMR-NB, AMR-WB, AAC, OGG, ADPCM
- Audio decoding: WAV, MP3, MP2, AAC, AMR-NB, AMR-WB, MIDI, Vorbis, APE, AAC-plus v1, AAC-plus v2, FLAC, WMA, ADPCM
- Voice wake-up

- **Speech**

- Speech codec (FR, HR, EFR, AMR FR, AMR HR and wide-band AMR, and EVS)
- CTM
- Noise reduction
- Noise suppression
- Noise cancellation
- Dual-MIC noise cancellation
- Echo cancellation
- Echo suppression
- Dual-MIC voice tracking
- Dual-MIC sound recording without wind noise rejection
- MagiLoudness (enhances the voice clarity based on near-end environment noise)
- MagiClarity (maximizes loudness while controlling the maximum receiver output power; feed-forward receiver protection)
- Compensation filter and digital gain for both uplink and downlink paths

2.1 Pin Description

2.1.1 Ball Map View

Figure 2-1. Ball map view

2.1.2 Pin Coordinate

Table 2-1. Pin coordinate

Ball location	Ball name	Ball location	Ball name	Ball location	Ball name
A1	DUMMY	R2	ANT_SEL0	AJ38	DIGRF_IRQ
A2	DUMMY	R3	ANT_SEL6	AJ39	MD_UCNT
A3	EMI0_REXTDN	R4	ANT_SEL5	AK1	CSI5A_L2N_T1C
A5	EMI0_DMI1	R5	ANT_SEL7	AK2	CSI5A_L2P_T1B
A7	EMI0_DQ11	R6	ANT_SEL10	AK3	CSI5A_L1N_T1A
A9	EMI0_DQ14	R7	ANT_SEL11	AK4	DVSS
A11	EMI0_CA0	R8	DVSS	AK5	DVSS
A13	EMI0_DMIO	R9	DVSS	AK6	DVSS
A15	EMI0_DQ0	R10	DVSS	AK7	CSI4B_L0N_T0B
A17	EMI0_DQ2	R11	DVSS	AK8	DVSS
A18	DVSS	R12	DVDD_SRAM_APU	AK10	DVDD_SRAM_CORE
A19	DVSS	R13	DVSS	AK11	DVDD_MM
A20	X32K_IN	R14	DVSS	AK12	DVDD_MM
A21	DVSS	R18	DVSS	AK13	DVDD_MM
A22	EMI2_DQ10	R19	DVSS	AK14	DVDD_MM
A24	EMI2_DQ8	R20	DVDD_SRAM_CORE	AK15	DVDD_SRAM_MM
A26	EMI2_DMI1	R21	DVDD_SRAM_GPUSTACK	AK16	DVDD_CORE
A28	EMI2_CA3	R22	DVSS	AK17	DVDD_CORE
A30	EMI2_DQ6	R23	DVSS	AK18	DVDD_CORE
A32	EMI2_DQ3	R24	DVSS	AK19	DVDD_CORE
A34	EMI2_DMIO	R25	DVSS	AK20	DVDD_CORE
A36	EMI2_RESET_N	R29	DVSS	AK21	DVDD_CORE
A37	EMI2_REXTDN	R30	DVSS	AK22	DVDD_CORE
A38	DUMMY	R33	DVSS	AK23	DVDD_CORE
A39	DUMMY	R34	DSIO_CK0N_T1C	AK24	AVDD15_MDPLL
B1	DUMMY	R35	DSIO_CK0P_T1B	AK25	DVDD_MODEM
B2	DVSS	R36	DSI1_D1N_T2B	AK26	DVDD_MODEM
B3	DVSS	R37	DSI1_D1P_T2A	AK27	DVDD_MODEM
B4	EMI0_DQ8	R38	DSI1_CK0N_T1C	AK28	DVDD_MODEM
B5	DVSS	R39	DSI1_CK0P_T1B	AK29	DVDD_MODEM
B6	EMI0_DQ9	T1	CAM_RST5	AK30	DVDD_MODEM
B7	DVSS	T2	CAM_RST4	AK31	DVSS

B8	EMI0_DQ12	T3	DVSS	AK33	DVSS
B9	DVSS	T4	DVDD18_IORT3	AK34	DVSS
B10	EMI0_CA2	T5	ANT_SEL4	AK35	DVSS
B11	EMI0_CA1	T6	CAM_CLK5	AK36	DRF_UL_LN1_DP
B13	DVSS	T7	CAM_CLK4	AK37	DRF_UL_LN1_DN
B14	EMI0_DQ1	T8	DVSS	AK38	DVSS
B15	DVSS	T9	DVSS	AK39	DVSS
B16	EMI0_DQ3	T10	DVDD_CORE	AL1	DVSS
B17	DVSS	T11	DVDD_CORE	AL2	CSI5B_L0P_TOA
B18	DVSS	T12	DVDD_APU	AL3	CSI5B_L0N_TOB
B19	SCP_VREQ_VAO	T13	DVDD_APU	AL4	DVSS
B20	DVSS	T14	DVDD_APU	AL5	DVSS
B21	DVSS	T15	DVDD_APU	AL6	CSI4B_L1N
B22	DVSS	T16	DVDD_APU	AL7	CSI4B_L1P_T0C
B23	EMI2_DQ11	T17	DVDD_APU	AL8	DVSS
B24	DVSS	T18	DVDD_APU	AL10	DVSS
B25	EMI2_DQ9	T19	DVDD_APU	AL11	DVSS
B26	DVSS	T20	AVDD12_APUPLL	AL12	DVSS
B28	EMI2_CA5	T21	DVDD_GPUSTACK	AL13	DVSS
B29	EMI2_CA2	T22	DVDD_GPUSTACK	AL14	DVSS
B30	DVSS	T23	DVDD_GPUSTACK	AL15	DVSS
B31	EMI2_DQ4	T24	DVDD_GPUSTACK	AL16	DVSS
B32	DVSS	T25	DVDD_GPUSTACK	AL17	DVSS
B33	EMI2_DQ1	T29	DVDD_GPUSTACK	AL18	DVSS
B34	DVSS	T30	DVDD_GPUSTACK	AL19	DVSS
B35	EMI2_DQ0	T31	DVDD_SRAM_GPUSTACK	AL20	DVSS
B36	DVSS	T32	DVSS	AL21	DVSS
B37	DVSS	T33	DVSS	AL22	DVSS
B38	DVSS	T34	DSIO_D0N_T1A	AL23	DVSS
B39	DUMMY	T35	DSIO_D0P_T0C	AL24	DVSS
C1	WF1_QP	T36	DVSS	AL25	DVSS
C2	WF1_IN	T37	DSI1_D0N_T1A	AL26	DVSS
C3	DVSS	T38	DSI1_D0P_T0C	AL27	DVSS
C4	EMI0_WCK1_C	T39	DVSS	AL28	DVSS
C5	EMI0_RDQS1_T	U1	DVDD18_IORT2	AL29	DVSS
C6	DVSS	U2	CAM_RST0	AL30	DVSS
C7	DVSS	U3	CAM_RST1	AL31	DVDD_MODEM
C8	EMI0_DQ13	U4	I2C_SCL10	AL33	DVSS
C9	EMI0_CA6	U5	I2C_SDA10	AL34	DRF_UL_CP
C10	EMI0_CA5	U6	CAM_CLK1	AL35	DRF_UL_CN

C11	EMI0_CS0	U7	CAM_CLK0	AL36	DVSS
C12	EMI0_CS1	U9	DVSS	AL37	DVSS
C13	EMI0_RDQS0_C	U10	DVSS	AL38	DRF_UL_LN0_DN
C14	DVSS	U11	DVSS	AL39	DRF_UL_LN0_DP
C15	EMI0_DQ4	U12	DVDD_CORE	AM1	CSI5B_L1N
C16	DVSS	U13	DVDD_SRAM_CORE	AM2	CSI5B_L1P_T0C
C17	EMI0_DQ5	U14	DVSS	AM3	DVSS
C18	DVSS	U15	DVSS	AM4	DVSS
C19	WATCHDOG	U16	DVSS	AM5	CSI2A_L0N_T0B
C20	SRCLKENA0	U17	DVSS	AM6	CSI2A_L0P_T0A
C21	DVSS	U18	DVSS	AM7	DVSS
C22	EMI2_DQ13	U19	AVDD15_APUPLL	AM8	DVDD_CORE
C23	DVSS	U20	DVSS	AM10	DVDD_MM
C24	EMI2_DQ12	U21	DVSS	AM11	DVDD_MM
C25	DVSS	U22	DVSS	AM12	DVDD_MM
C26	EMI2_RDQS1_C	U23	DVSS	AM13	DVDD_MM
C27	EMI2_CA4	U24	DVDD_SRAM_GPUSTACK	AM14	DVDD_MM
C28	EMI2_CA6	U25	DVSS	AM15	DVDD_MM
C29	EMI2_CA1	U29	DVSS	AM16	DVDD_MM
C30	EMI2_CA0	U30	DVSS	AM17	DVDD_MM
C31	EMI2_DQ5	U32	DVSS	AM18	DVDD_MM
C32	DVSS	U33	AVDD08_DRV_DSI	AM19	DVDD_MM
C33	DVSS	U34	DVSS	AM20	DVDD_MM
C34	EMI2_RDQS0_T	U35	DSIO_D2N_T0B	AM21	DVDD_MM
C35	EMI2_WCK0_C	U36	DSIO_D2P_T0A	AM22	DVDD_MM
C36	DVSS	U37	DVSS	AM23	DVDD_CORE
C37	UFS_TX1P	U38	DSI1_D2N_T0B	AM24	DVDD_CORE
C38	DVSS	U39	DSI1_D2P_T0A	AM25	DVDD_MODEM
C39	UFS_TX0P	V1	DVDD18_IORT	AM26	DVDD_MODEM
D1	WF1_QN	V2	CAM_I3C_SDA4	AM27	DVDD_MODEM
D2	WF1_IP	V3	CAM_I3C_SCL4	AM28	DVDD_SRAM_MODEM
D3	DVSS	V4	I2C_SCL11	AM29	DVDD_MODEM
D4	EMI0_WCK1_T	V5	I2C_SDA11	AM30	DVDD_MODEM
D5	EMI0_RDQS1_C	V6	DVSS	AM31	DVDD_MODEM
D6	EMI0_DQ10	V7	DVSS	AM33	DVSS
D7	EMI0_DQ15	V8	DVDD_PROC_M	AM34	DVSS
D8	DVSS	V9	DVDD_PROC_M	AM35	DVSS
D9	EMI0_CA4	V13	DVDD_PROC_M	AM36	DRF_DL_LN3_DP
D10	EMI0_CA3	V14	DVSS	AM37	DRF_DL_LN3_DN
D11	EMI0_CK_T	V15	DVDD_PROC_M	AM38	DVSS

D12	EMI0_CK_C	V19	DVDD_PROC_M	AM39	AVDD12_DRF
D13	EMI0_RDQS0_T	V20	DVDD_CORE	AN1	CSI3A_L0P_T0A
D14	EMI0_WCK0_C	V21	DVDD_GPUSTACK	AN2	CSI3A_L0N_T0B
D15	EMI0_WCK0_T	V22	DVDD_GPUSTACK	AN3	CSI2A_L2P_T1B
D16	EMI0_DQ7	V23	DVDD_GPUSTACK	AN4	CSI2A_L2N_T1C
D17	EMI0_DQ6	V24	DVDD_GPUSTACK	AN5	CSI2A_L1N_T1A
D18	DVSS	V25	DVDD_GPUSTACK	AN6	CSI2A_L1P_T0C
D19	DSI_TE	V26	DVDD_GPUSTACK	AN7	AVDD12_CSI0
D20	AUD_CLK_MOSI	V27	DVDD_GPUSTACK	AN8	DVDD18_IORB
D21	DVSS	V28	DVDD_GPUSTACK	AN10	DVSS
D22	EMI2_DQ14	V29	DVDD_GPUSTACK	AN11	DVSS
D23	EMI2_DQ15	V30	DVDD_GPUSTACK	AN12	DVSS
D24	EMI2_WCK1_T	V32	DVSS	AN13	DVSS
D25	EMI2_WCK1_C	V33	DVSS	AN14	DVSS
D26	EMI2_RDQS1_T	V34	AVDD15_DSI	AN15	DVDD_SRAM_MM
D27	EMI2_CK_C	V35	DVSS	AN16	AVDD12_TSFDC_BM
D28	EMI2_CK_T	V36	DVSS	AN17	DVDD_SRAM_CORE
D29	EMI2_CS0	V37	DVSS	AN18	DVSS
D30	EMI2_CS1	V38	DVSS	AN19	DVSS
D31	DVSS	V39	AVDD12_DSI	AN20	DVSS
D32	EMI2_DQ7	W1	DVSS	AN21	DVSS
D33	EMI2_DQ2	W2	DVSS	AN22	DVDD_SRAM_MM
D34	EMI2_RDQS0_C	W3	CAM_I3C_SDA2	AN23	DVSS
D35	EMI2_WCK0_T	W4	CAM_I3C_SCL2	AN24	DVSS
D36	DVSS	W5	DVSS	AN25	DVSS
D37	UFS_TX1N	W6	DVSS	AN26	DVSS
D38	DVSS	W7	CKM_XTAL_CK	AN27	DVSS
D39	UFS_TX0N	W8	DVSS	AN28	DVSS
E1	DVSS	W9	DVDD_PROC_M	AN29	DVSS
E2	DVSS	W13	DVDD_PROC_M	AN30	DVSS
E3	WFO_IP	W14	DVDD_PROC_L	AN31	DVSS
E4	DVSS	W15	DVDD_PROC_M	AN32	DVSS
E5	DVSS	W19	DVDD_PROC_M	AN33	DVSS
E6	DVSS	W20	DVSS	AN34	DRF_DL_LN0_DP
E7	DVSS	W21	DVSS	AN35	DRF_DL_LN0_DN
E8	DVSS	W22	DVSS	AN36	DVSS
E9	DVSS	W23	DVSS	AN37	DVSS
E10	DVSS	W24	DVSS	AN38	DRF_DL_LN1_DN
E11	DVSS	W25	DVSS	AN39	DRF_DL_LN1_DP
E12	DVSS	W26	DVSS	AP1	DVSS

E13	DVSS	W27	DVDD_SRAM_GPUSTACK	AP2	CSI3A_L1P_T0C
E14	DVSS	W28	DVSS	AP3	CSI3A_L1N_T1A
E15	DVSS	W29	DVSS	AP4	DVSS
E16	DVSS	W30	DVSS	AP5	CSI2B_L0N_T0B
E17	DVSS	W31	DVSS	AP6	CSI2B_L0P_T0A
E18	DVSS	W32	SSUSB_TXP_P2	AP7	AVDD15_CSI
E19	LCM_RST	W33	SSUSB_TXN_P2	AP9	DVDD_SRAM_CORE
E20	AUD_SYNC_MOSI	W34	DVSS	AP10	DVDD_MM
E21	DVSS	W35	DVSS	AP11	AVDD075_EMI1
E22	DVSS	W36	SSUSB_RXN_P1	AP12	DVDD_CORE
E23	DVSS	W37	SSUSB_RXP_P1	AP13	DVSS
E24	DVSS	W38	DVSS	AP14	AVDD075_EMI1
E25	DVSS	W39	DVSS	AP15	DVSS
E26	DVSS	Y1	PCIE_LN0_RXP	AP16	AVDD15_EMI1
E27	DVSS	Y2	PCIE_LN0_RXN	AP17	DVDD_MM
E28	DVSS	Y3	DVSS	AP18	DVDD_MM
E29	DVSS	Y4	DVSS	AP19	DVDD_MM
E30	DVSS	Y5	PCIE_LN0_TXN	AP20	DVDD_MM
E31	DVSS	Y6	PCIE_LN0_TXP	AP21	DVDD_MM
E32	DVSS	Y7	DVSS	AP22	DVDD_MM
E33	DVSS	Y8	DVSS	AP23	DVDD_MM
E34	DVSS	Y9	DVDD_PROC_M	AP24	AVDD15_EMI3
E35	DVSS	Y13	DVDD_PROC_M	AP25	DVSS
E36	DVSS	Y14	DVDD_PROC_L	AP26	AVDD075_EMI3
E37	DVSS	Y15	DVDD_PROC_M	AP27	DVSS
E38	DVSS	Y19	DVDD_PROC_M	AP28	DVDD_CORE
E39	DVSS	Y20	DVDD_CORE	AP29	AVDD075_EMI3
F1	WFO_QP	Y21	DVDD_CORE	AP30	DVDD_MODEM
F2	WFO_QN	Y22	DVDD_CORE	AP31	DVDD_MODEM
F3	WFO_IN	Y23	DVDD_CORE	AP32	DVSS
F4	DVSS	Y24	DVDD_CORE	AP33	AVDD15_DRF
F5	DVDD18_IOTM	Y25	DVDD_CORE	AP34	DVSS
F6	SCP_SPI_CK	Y26	DVDD_CORE	AP35	DVSS
F7	SCP_SPI_MO	Y27	DVDD_GPUSTACK	AP36	DRF_DL_LN2_DN
F8	SCP_I3C_SCL2	Y28	DVDD_GPUSTACK	AP37	DRF_DL_LN2_DP
F9	SCP_I3C_SCL0	Y29	DVDD_GPUSTACK	AP38	DVSS
F10	SCP_I3C_SCL3	Y30	DVDD_GPUSTACK	AP39	DVSS
F11	INT_SIM1	Y31	DVSS	AR1	DVSS
F12	SCP_EINT2	Y32	DVSS	AR2	CSI3A_L2P_T1B
F13	SCP_EINT0	Y33	DVSS	AR3	CSI3A_L2N_T1C

F14	DVDD18_IOTM2	Y34	SSUSB_RXN_P2	AR4	DVSS
F15	SPI2_CK	Y35	SSUSB_RXP_P2	AR5	CSI2B_L1N
F16	SPI2_MO	Y36	DVSS	AR6	CSI2B_L1P_T0C
F17	DVDD18_IOTM3	Y37	DVSS	AR7	DVSS
F18	EINT_CTP0	Y38	SSUSB_TXP_P1	AR9	CAM_RST3
F19	GPIO_CTP0_RST	Y39	SSUSB_TXN_P1	AR10	DVSS
F20	AUD_DAT_MOSI0	AA1	DVSS	AR11	DVSS
F21	AUD_DAT_MOSI2	AA2	DVSS	AR12	VDD2H_EMI1
F22	AUD_NLE_MOSI1	AA3	PCIE_CKP	AR13	AVDDQ_EMI1
F23	AUD_DAT_MISO1	AA4	PCIE_CKN	AR14	DVSS
F24	PERIEN_CHG_ENB	AA5	DVSS	AR15	AVDDQ_EMI1
F25	CHG_I2C_SDA5	AA6	DVSS	AR16	AVDD12_EMI1
F26	DMIC_CLK	AA7	AVDD15_PCIE	AR17	DVSS
F27	AUD_I2C_SDA	AA8	DVSS	AR18	DVSS
F28	DVDD18_IOTL	AA9	DVDD_PROC_M	AR19	DVSS
F29	I2C_SCL3	AA10	DVSS	AR20	DVDD18_IOBM3
F30	SPI3_CSB	AA11	DVSS	AR21	DVSS
F31	SPI3_MI	AA12	DVSS	AR22	DVSS
F32	SYSRSTB	AA13	DVDD_PROC_M	AR23	DVDD18_IOBM
F33	SPI4_CK	AA17	DVSS	AR24	AVDD12_EMI3
F34	SPI4_MO	AA18	DVDD_SRAM_PROC_M	AR25	AVDDQ_EMI3
F35	DVSS	AA19	DVDD_PROC_M	AR26	DVSS
F36	UFS_RX0P	AA20	DVSS	AR27	AVDDQ_EMI3
F37	UFS_RX0N	AA21	DVSS	AR28	DVSS
F38	DVSS	AA22	DVDD_SRAM_CORE	AR29	VDD2H_EMI3
F39	DVSS	AA23	DVSS	AR30	DVSS
G2	DVSS	AA24	DVSS	AR31	DVSS
G3	DVSS	AA25	DVSS	AR32	DVDD_SRAM_CORE
G4	BT_IP	AA26	DVSS	AR33	DVSS
G5	DVSS	AA27	DVSS	AR34	DVSS
G6	SCP_SPI_CSB	AA28	DVSS	AR35	DVSS
G7	SCP_SPI_MI	AA29	DVSS	AR36	DVSS
G8	SCP_I3C_SDA2	AA30	DVSS	AR37	DVSS
G9	SCP_I3C_SDA0	AA31	DVDD_SRAM_CORE	AR38	DRF_DL_CP
G10	SCP_I3C_SDA3	AA32	DVSS	AR39	DRF_DL_CN
G11	INT_SIM2	AA33	DVSS	AT1	CSI3B_L0P_T0A
G12	SCP_EINT3	AA34	DVSS	AT2	CSI3B_L0N_T0B
G13	SCP_EINT1	AA35	DVSS	AT3	CSI1A_L0P_T0A
G14	LCM_RST_AUX	AA36	DP_SBU2	AT4	CSI1A_L1P_T0C
G15	DSI_TE_AUX	AA37	DP_SBU1	AT5	CSI1B_L1P_T0C

G16	SPI2_CSB	AA38	DVSS	AT6	CSI1B_L1N
G17	DVSS	AA39	DVSS	AT7	DVSS
G18	EINT_CTP1	AB1	AVDD12_PCIE	AT10	DVSS
G19	GPIO_CTP1_RST	AB3	DVSS	AT11	DVSS
G20	AUD_DAT_MOSI1	AB4	DVSS	AT13	DVSS
G21	AUD_NLE_MOSI0	AB5	SPI0_HOLD	AT14	DVSS
G22	AUD_DAT_MISO0	AB6	SPI0_WP	AT17	DVSS
G23	AUD_DAT_MISO2	AB7	DVDD12_IORM2	AT18	CAM_I3C_SCL9
G24	EINT_CHG_IRQB	AB8	DVSS	AT28	DVSS
G25	CHG_I2C_SCL5	AB9	DVDD_SRAM_PROC_M	AT29	DVSS
G26	DMIC_DAT	AB10	DVDD_PROC_L	AT31	DVSS
G27	AUD_I2C_SCL	AB11	DVDD_PROC_L	AT32	URXD0
G28	I2C_SDA3	AB12	DVDD_PROC_L	AT33	DVDD075_IOBL
G29	SPI3_CK	AB13	DVDD_PROC_L	AT34	DISP_PWM
G30	SPI3_MO	AB17	DVDD_SRAM_PROC_L	AT35	DVSS
G31	NFC_CLK_REQ	AB18	DVSS	AT36	DVSS
G32	TESTMODE	AB19	DVDD_CORE	AT37	DVSS
G33	SPI4_CSB	AB20	DVDD_CORE	AT38	DVSS
G34	SPI4_MI	AB21	DVDD_CORE	AT39	DVSS
G35	DVSS	AB22	DVDD_CORE	AU1	CSI3B_L1P_T0C
G36	UFS_RST_N	AB23	DVDD_CORE	AU2	CSI3B_L1N
G37	DVSS	AB24	DVDD_CORE	AU3	CSI1A_L0N_T0B
G38	UFS_RX1N	AB25	DVDD_CORE	AU4	CSI1A_L1N_T1A
G39	UFS_RX1P	AB26	DVDD_CORE	AU5	CSI1B_L0P_T0A
H1	GL1_Q	AB27	DVDD_CORE	AU6	DVSS
H2	GL1_I	AB28	DVDD_CORE	AU7	DVSS
H3	DVSS	AB29	DVDD_CORE	AU9	DVSS
H4	BT_IN	AB30	DVDD_CORE	AU10	CAM_CLK7
H5	DVSS	AB32	DVSS	AU11	CAM_CLK3
H6	DVSS	AB33	DVSS	AU12	CAM_CLK6
H7	AVDD12_WBG	AB34	DVSS	AU13	I2C_SCL12
H8	DVSS	AB35	DVSS	AU14	CAM_I3C_SCL8
H11	DVSS	AB36	DVSS	AU15	I2C_SCL13
H12	DVSS	AB37	DVSS	AU16	CDM5P5A
H13	DVSS	AB38	DVSS	AU17	CDM3P5A
H14	DVSS	AB39	AVDD12_SSUSB_P1	AU18	CAM_I3C_SDA9
H15	DVSS	AC1	DVDD075_IORM2	AU20	I3C_SCL1
H16	SPI2_MI	AC2	SPI0_CSB	AU21	BPI_D_BUS19
H18	DVSS	AC3	SPI0_CK	AU22	BPI_D_BUS12
H19	DVSS	AC4	SPMI_P_SCL	AU23	BPI_D_BUS10

H20	DVSS	AC5	SPMI_P_SDA	AU24	BPI_D_BUS9
H21	DVSS	AC6	MSDC2_DAT0	AU25	BPI_D_BUS7
H22	DVSS	AC7	MSDC2_DAT1	AU26	BPI_D_BUS5
H23	DVSS	AC8	DVSS	AU27	BPI_D_BUS3
H25	DVSS	AC9	DVDD_SRAM_PROC_L	AU28	BPI_D_BUS1
H26	DVSS	AC10	DVSS	AU29	JTDI
H27	DVSS	AC11	DVDD_PROC_L	AU30	JTMS
H28	DVSS	AC12	DVSS	AU31	JTCK
H29	DVSS	AC13	DVSS	AU32	I3C_SDA0
H30	DVSS	AC17	DVDD_PROC_L	AU33	EINT5
H31	DVSS	AC18	DVSS	AU34	EINT4
H32	DVSS	AC19	DVDD_CORE	AU35	DISP_PWM_1
H33	DVSS	AC20	TN_PLLGP	AU36	PERI_EN0
H34	DVSS	AC21	DVSS	AU37	DVSS
H35	UFS_REFCK_OUT	AC22	DVSS	AU38	DVDD12_IOBL
H36	DVSS	AC23	DVSS	AU39	DVDD18_IOBL
H37	DVSS	AC24	DVSS	AV1	DVSS
H38	DVSS	AC25	DVSS	AV2	CSI0A_L0P_T0A
H39	DVSS	AC26	DVSS	AV3	DVSS
J1	GL5_I	AC27	DVDD_SCPAO	AV4	CSI1A_L2P_T1B
J2	GL5_Q	AC28	DVSS	AV5	CSI1B_L0N_T0B
J3	DVSS	AC29	DVSS	AV6	DVSS
J4	BT_QP	AC30	DVSS	AV7	DVSS
J5	BT_QN	AC32	DVSS	AV8	DVSS
J6	DVSS	AC33	X26M_IN	AV9	CAM_RST6
J7	DVSS	AC34	DVSS	AV10	CAM_RST2
J8	DVSS	AC35	DVSS	AV11	DVSS
J9	DVSS	AC36	USB_DP	AV12	CAM_CLK2
J10	DVSS	AC37	USB_DM	AV13	DVDD18_IOBR
J11	DVSS	AC38	AVDD15_SSUSB_P2	AV14	I2C_SDA12
J12	VDD2H_EMI0	AC39	AVDD12_SSUSB_P2	AV15	CAM_I3C_SDA8
J13	DVSS	AD1	DVDD18_IORM2	AV16	I2C_SDA13
J14	AVDDQ_EMI0	AD2	SPI0_MO	AV17	DVSS
J15	DVSS	AD3	SPI0_MI	AV18	DVSS
J16	AVDDQ_EMI0	AD4	SPMI_M_SCL	AV19	DVDD18_IOBM2
J17	DVSS	AD5	SPMI_M_SDA	AV20	I3C_SDA1
J18	DVSS	AD6	DVSS	AV21	BPI_D_BUS13
J19	DVSS	AD7	MSDC2_DAT2	AV22	BPI_D_BUS11
J20	DVSS	AD9	DVDD_PROC_M	AV24	BPI_D_BUS8
J21	DVSS	AD13	DVDD_PROC_M	AV25	BPI_D_BUS6

J22	DVSS	AD14	DVDD_PROC_L	AV26	BPI_D_BUS4
J23	DVSS	AD15	DVSS	AV27	BPI_D_BUS2
J24	AVDD12_EMI2	AD19	DVDD_PROC_B	AV28	BPI_D_BUS0
J25	AVDDQ_EMI2	AD20	TP_PLLGP	AV29	JTDO
J26	DVSS	AD21	DVDD_CORE	AV30	JTRSTN
J27	AVDDQ_EMI2	AD22	DVDD_CORE	AV31	UTXD0
J28	DVSS	AD23	DVDD_CORE	AV32	I3C_SDA7
J29	VDD2H_EMI2	AD24	DVDD_CORE	AV33	I3C_SCL0
J32	DVSS	AD25	DVDD_VLPAO	AV34	EINT6
J33	DVSS	AD26	DVDD_SCPAO	AV35	PERI_EN3
J34	DVSS	AD27	DVDD_SCPAO	AV36	PERI_EN2
J35	DVSS	AD28	DVDD_SCPAO	AV37	PERI_EN1
J36	DVSS	AD29	DVDD_VLPAO	AV38	MIPI4_D_SCLK
J37	UFS_PLL_CKREF	AD30	DVSS	AV39	MIPI4_D_SDATA
J38	DVSS	AD33	DVSS	AW1	CSI0A_L0N_T0B
J39	DVDD18_VQPS	AD34	AVDD12_CKSQ	AW2	CSI0A_L1P_T0C
K1	DVSS	AD35	AVDD33_USB	AW3	CSI0A_L1N_T1A
K2	DVSS	AD36	DVSS	AW4	CSI1A_L2N_T1C
K3	DVSS	AD37	DVSS	AW5	DVSS
K4	DVSS	AD38	AVDD15_SSUSB_P1	AW6	DVSS
K5	DVSS	AE1	DVDD18_MSDC2	AW7	DVSS
K6	DVSS	AE2	SPI1_CSB	AW8	DVSS
K7	DVSS	AE3	SPI1_CK	AW9	CAM_RST7
K8	XIN_WBG	AE4	EINT_FPS	AW10	DVSS
K9	DVSS	AE5	GPIO_FPS_RST	AW11	DVSS
K10	DVSS	AE6	MSDC2_CMD	AW12	DVSS
K11	DVSS	AE7	MSDC2_DAT3	AW13	DVSS
K12	AVDD075_EMI0	AE8	DVDD_CORE	AW14	DVSS
K13	DVSS	AE9	DVDD_PROC_M	AW15	DVSS
K14	AVDD075_EMI0	AE13	DVDD_PROC_M	AW16	DVSS
K15	AVDD15_EMI0	AE14	DVDD_PROC_L	AW17	DVSS
K16	AVDD12_EMI0	AE15	DVDD_PROC_B	AW18	DVSS
K17	DVDD_APU	AE19	DVDD_PROC_B	AW19	DVSS
K18	DVDD_APU	AE20	AVDD15_APPLL	AW20	AP_GOOD
K19	DVDD_APU	AE21	DVSS	AW21	BPI_D_BUS14
K20	DVDD_CORE	AE22	DVSS	AW22	DVSS
K21	DVDD_GPUSTACK	AE23	DVSS	AW23	DVSS
K22	DVDD_GPUSTACK	AE24	DVSS	AW24	DVSS
K23	DVDD_GPUSTACK	AE25	DVSS	AW25	DVSS
K24	AVDD15_EMI2	AE26	DVSS	AW26	DVSS

K25	DVSS	AE27	DVSS	AW27	DVSS
K26	AVDD075_EMI2	AE28	DVSS	AW28	DVSS
K27	DVSS	AE29	DVSS	AW29	DVSS
K28	DVDD_CORE	AE30	DVSS	AW30	DVSS
K29	AVDD075_EMI2	AE31	DVSS	AW31	DVSS
K30	DVSS	AE32	DVSS	AW32	I3C_SCL7
K31	AVDD12_CKBUF_UFS	AE33	AVDD15_CKSQ	AW33	DVSS
K32	DVSS	AE34	I2S1_BCK	AW34	DVSS
K33	DVSS	AE35	I2S1_LRCK	AW35	DVSS
K34	SIM1_SIO	AE36	I2C_SCL6	AW36	DVSS
K35	SIM1_SRST	AE37	I2C_SDA6	AW37	MIPI2_D_SCLK
K36	SIM1_SCLK	AE38	AVDD15_USB	AW38	MIPI3_D_SCLK
K37	DVSS	AE39	AVDD12_USB	AW39	MIPI3_D_SDATA
K38	DVSS	AF1	DVDD12_MSDC2	AY1	DVSS
K39	DVDD28_SIM1	AF2	SPI1_MO	AY2	CSI0A_L2P_T1B
L1	AVDD15_WBG	AF3	SPI1_MI	AY3	CSI0B_L1P_T0C
L2	CONN_TOP_CLK	AF4	DVDD075_MSDC2	AY4	DVSS
L3	CONN_TOP_DATA	AF5	DVSS	AY5	EMI1_WCK0_T
L4	CONN_BT_CLK	AF6	DVSS	AY6	EMI1_RDQS0_C
L5	CONN_BT_DATA	AF7	MSDC2_CLK	AY7	EMI1_DQ2
L6	DVSS	AF9	DVDD_PROC_M	AY8	EMI1_DQ7
L7	CONN_HRST_B	AF13	DVDD_PROC_M	AY9	DVSS
L8	DVSS	AF14	DVDD_PROC_L	AY10	EMI1_CS1
L9	DVSS	AF15	DVDD_PROC_B	AY11	EMI1_CS0
L10	DVSS	AF19	DVDD_PROC_B	AY12	EMI1_CK_T
L11	DVSS	AF20	AVDD12_APPLL	AY13	EMI1_CK_C
L12	DVSS	AF21	DVDD_CORE	AY14	EMI1_RDQS1_T
L13	DVDD_SRAM_CORE	AF22	DVDD_CORE	AY15	EMI1_WCK1_C
L14	DVDD_CORE	AF23	DVDD_CORE	AY16	EMI1_WCK1_T
L15	DVSS	AF24	DVDD_CORE	AY17	EMI1_DQ15
L16	DVSS	AF25	DVDD_MODEM	AY18	EMI1_DQ14
L17	DVSS	AF26	DVDD_MODEM	AY19	DVSS
L18	DVSS	AF27	DVDD_MODEM	AY20	KPCOLO
L19	DVSS	AF28	DVDD_SRAM_MODEM	AY21	BPI_D_BUS15
L20	DVSS	AF29	DVDD_MODEM	AY22	DVSS
L21	DVSS	AF30	DVDD_MODEM	AY23	EMI3_DQ6
L22	DVSS	AF31	DVDD_MODEM	AY24	EMI3_DQ7
L23	DVSS	AF33	DVDD12_IOLB2	AY25	EMI3_WCK0_T
L24	DVDD_SRAM_GPUSTACK	AF34	I2S1_DI	AY26	EMI3_WCK0_C
L25	DVSS	AF35	I2S1_DO	AY27	EMI3_RDQS0_T

L26	DVSS	AF36	I2S0_LRCK	AY28	EMI3_CK_C
L27	DVSS	AF37	I2S0_BCK	AY29	EMI3_CK_T
L28	DVSS	AF39	DVDD18_IOLB	AY30	EMI3_CA3
L29	DVDD_SRAM_CORE	AG1	DVDD18_IORM	AY31	EMI3_CA4
L31	AVDD12_UFS_TX	AG2	DVSS	AY32	DVSS
L32	DVSS	AG3	DVSS	AY33	EMI3_DQ15
L33	DVSS	AG4	DVSS	AY34	EMI3_DQ10
L34	DVDD15_MSDC1	AG5	CSI4A_LOP_TOA	AY35	EMI3_RDQS1_C
L35	MSDC1_CLK	AG6	CSI4A_LON_TOB	AY36	EMI3_WCK1_T
L36	MSDC1_CMD	AG7	AVDD12_CSI1	AY37	DVSS
L37	MSDC1_DAT0	AG9	DVDD_PROC_M	AY38	MIPI1_D_SCLK
L38	SIM2_SIO	AG10	DVSS	AY39	MIPI2_D_SDATA
L39	DVDD28_SIM2	AG11	DVSS	BA1	CSI0A_L2N_T1C
M1	CONN_WF_CTRL4	AG12	AVDD15_PROC	BA2	CSI0B_LOP_TOA
M2	CONN_WF_CTRL3	AG13	DVDD_PROC_M	BA3	CSI0B_L1N
M3	CONN_WF_CTRL2	AG14	DVSS	BA4	DVSS
M4	DVSS	AG15	DVDD_PROC_B	BA5	EMI1_WCK0_C
M5	CONN_PMIC_EN	AG16	DVDD_PROC_B	BA6	EMI1_RDQS0_T
M6	BPI_WAKEUP_PMIC	AG17	DVSS	BA7	DVSS
M7	CONN_WB_PTA	AG18	DVSS	BA8	DVSS
M8	DVSS	AG19	DVDD_PROC_B	BA9	EMI1_DQ5
M9	DVSS	AG20	DVSS	BA10	EMI1_CA0
M10	DVDD_CORE	AG21	DVSS	BA11	EMI1_CA1
M11	DVDD_CORE	AG22	DVDD_SRAM_CORE	BA12	EMI1_CA6
M12	DVDD_CORE	AG23	DVSS	BA13	EMI1_CA4
M13	DVDD_CORE	AG24	DVSS	BA14	EMI1_RDQS1_C
M14	DVDD_APU	AG25	DVSS	BA15	DVSS
M15	DVDD_APU	AG26	DVSS	BA16	EMI1_DQ12
M16	DVDD_APU	AG27	DVSS	BA17	DVSS
M17	DVDD_SRAM_APU	AG28	DVSS	BA18	EMI1_DQ13
M18	DVDD_APU	AG29	DVSS	BA19	DVSS
M19	DVDD_APU	AG30	DVSS	BA20	KPCOL1
M20	DVDD_CORE	AG31	DVSS	BA21	BPI_D_BUS16
M21	DVDD_GPUSTACK	AG32	DVSS	BA22	DVSS
M25	DVDD_GPUSTACK	AG33	DVSS	BA23	EMI3_DQ5
M26	DVDD_GPUSTACK	AG34	DVSS	BA24	DVSS
M27	DVDD_GPUSTACK	AG35	I2S2_BCK	BA25	EMI3_DQ4
M28	DVDD_GPUSTACK	AG36	I2S2_LRCK	BA26	DVSS
M29	DVDD_CORE	AG37	I2S0_DO	BA27	EMI3_RDQS0_C
M30	AVDD15_UFS	AG38	I2S0_DI	BA28	EMI3_CS1

M31	AVDD12_UFS_RX	AG39	DVDD18_IOLB2	BA29	EMI3_CS0
M32	DVSS	AH1	DVSS	BA30	EMI3_CA5
M33	DVDD28_MSDC1	AH2	CSI5A_L0P_T0A	BA31	EMI3_CA6
M34	MSDC1_DAT1	AH3	CSI5A_L0N_T0B	BA32	EMI3_DQ13
M35	MSDC1_DAT3	AH4	DVSS	BA33	DVSS
M36	MSDC1_DAT2	AH5	DVSS	BA34	DVSS
M37	DVSS	AH6	CSI4A_L1P_T0C	BA35	EMI3_RDQS1_T
M38	SIM2_SCLK	AH7	DVSS	BA36	EMI3_WCK1_C
M39	SIM2_SRST	AH8	DVSS	BA37	DVSS
N1	DVDD12_IORT4	AH9	DVDD_CORE	BA38	MIPI0_D_SDATA
N2	CONN_WF_CTRL1	AH10	DVDD_MM	BA39	MIPI1_D_SDATA
N3	CONN_WF_CTRL0	AH11	DVDD_MM	BB1	DUMMY
N4	GPS_L1_ELNA_EN	AH12	DVDD_MM	BB2	CSI0B_L0N_T0B
N5	GPS_L5_ELNA_EN	AH13	DVDD_MM	BB3	DVSS
N6	DVDD075_IORT4	AH14	DVDD_SRAM_CORE	BB4	DVSS
N7	DVDD18_IORT4	AH15	DVDD_SRAM_PROC_L	BB5	EMI1_DQ0
N8	DVSS	AH16	DVDD_SRAM_PROC_B	BB6	DVSS
N9	DVSS	AH17	DVDD_CORE	BB7	EMI1_DQ1
N10	DVSS	AH18	DVDD_CORE	BB8	DVSS
N11	DVSS	AH19	DVDD_CORE	BB9	EMI1_DQ4
N12	DVSS	AH20	DVDD_CORE	BB10	DVSS
N13	DVSS	AH21	DVDD_CORE	BB11	EMI1_CA2
N14	DVSS	AH22	DVDD_CORE	BB12	EMI1_CA5
N18	DVSS	AH23	DVDD_CORE	BB14	DVSS
N19	DVSS	AH24	DVDD_CORE	BB15	EMI1_DQ9
N20	DVSS	AH25	DVDD_MODEM	BB16	DVSS
N21	DVSS	AH26	DVDD_MODEM	BB17	EMI1_DQ11
N25	DVSS	AH27	DVDD_MODEM	BB18	DVSS
N26	DVSS	AH28	DVDD_MODEM	BB19	DVSS
N27	DVSS	AH29	DVDD_MODEM	BB20	KPROW0
N28	DVSS	AH30	DVDD_MODEM	BB21	BPI_D_BUS17
N29	DVSS	AH31	DVDD_MODEM	BB22	DVSS
N30	DVSS	AH33	DVSS	BB23	DVSS
N32	DVDD075_MSDC1	AH34	DVDD075_IOLB2	BB24	EMI3_DQ3
N33	DVSS	AH35	I2S2_DI	BB25	DVSS
N34	DVSS	AH36	I2S2_DO	BB26	EMI3_DQ1
N35	DVSS	AH37	MIPI_M_SCLK	BB27	DVSS
N36	DVSS	AH38	MIPI_M_SDATA	BB29	EMI3_CA1
N37	DSI1_D3N	AH39	DVSS	BB30	EMI3_CA2
N38	DVSS	AJ1	DVSS	BB31	DVSS

N39	DVSS	AJ2	DVSS	BB32	EMI3_DQ12
P1	ANT_SEL3	AJ3	CSI5A_L1P_T0C	BB33	DVSS
P2	ANT_SEL2	AJ4	CSI4A_L2N_T1C	BB34	EMI3_DQ9
P3	ANT_SEL8	AJ5	CSI4A_L2P_T1B	BB35	DVSS
P4	ANT_SEL9	AJ6	CSI4A_L1N_T1A	BB36	EMI3_DQ8
P5	ANT_SEL13	AJ7	CSI4B_L0P_T0A	BB37	DVSS
P6	ANT_SEL12	AJ8	DVSS	BB38	MIPIO_D_SCLK
P7	ANT_SEL14	AJ10	DVSS	BB39	DUMMY
P8	DVSS	AJ11	DVSS	BC1	DUMMY
P9	DVSS	AJ12	DVSS	BC2	DUMMY
P10	DVDD_CORE	AJ13	DVSS	BC3	EMI1_REXTDN
P11	DVDD_CORE	AJ14	DVSS	BC4	EMI1_RESET_N
P12	DVDD_APU	AJ15	AVDD12_TSFDC_RM	BC6	EMI1_DMIO
P13	DVDD_APU	AJ16	DVSS	BC8	EMI1_DQ3
P14	DVDD_APU	AJ17	DVSS	BC10	EMI1_DQ6
P18	DVDD_APU	AJ18	DVSS	BC12	EMI1_CA3
P19	DVDD_APU	AJ19	DVSS	BC14	EMI1_DMIO
P20	DVDD_CORE	AJ20	DVSS	BC16	EMI1_DQ8
P21	DVDD_GPUSTACK	AJ21	DVSS	BC18	EMI1_DQ10
P25	DVDD_GPUSTACK	AJ22	DVSS	BC19	DVSS
P26	DVDD_GPUSTACK	AJ23	DVSS	BC20	KPROW1
P27	DVDD_GPUSTACK	AJ24	AVDD12_MDPLL	BC21	BPI_D_BUS18
P28	DVDD_GPUSTACK	AJ25	DVDD_SRAM_MODEM	BC22	DVSS
P29	DVDD_GPUSTACK	AJ26	DVSS	BC23	EMI3_DQ2
P30	DVDD_GPUSTACK	AJ27	DVSS	BC25	EMI3_DQ0
P32	DVSS	AJ28	DVSS	BC27	EMI3_DMIO
P33	DSIO_D3N	AJ29	DVSS	BC29	EMI3_CA0
P34	DSIO_D3P_T2C	AJ30	DVSS	BC31	EMI3_DQ14
P35	DSIO_D1N_T2B	AJ31	DVSS	BC33	EMI3_DQ11
P36	DSIO_D1P_T2A	AJ33	DVSS	BC35	EMI3_DMIO
P37	DSIO_D3P_T2C	AJ34	DVSS	BC37	EMI3_REXTDN
P38	DVSS	AJ35	DVSS	BC38	DUMMY
P39	DVSS	AJ36	DVSS	BC39	DUMMY
R1	ANT_SEL1	AJ37	DVSS		

2.1.3 Detailed Pin Descriptions

Table 2-2. Acronym for pin type

Abbreviation	Description
AI	Analog input
AO	Analog output
AIO	Analog bi-direction
DI	Digital input
DO	Digital output
DIO	Digital bi-direction
P	Power
G	Ground

Table 2-3. Detailed pin descriptions

Ball name	Ball location	Type	Description	Power domain
System				
SYSRSTB	F32	DI	System reset input	DVDD18
WATCHDOG	C19	DIO	Watchdog reset output	DVDD18
TESTMODE	G32	DI	Test mode	DVDD18
X26M_IN	AC33	AI	26M clock input	AVDD12_CKSQ
X32K_IN	A20	DIO	32K input	DVDD18
SRCLKENA0	C20	DIO	Output signal; control 26 Hz/buck/LDO normal mode or sleep mode. (High: normal mode; low: sleep mode or low-power mode)	DVDD18
AP_GOOD	AW20	DIO	When bootrom can normally jump into pre-loader execution (i.e., software can take over), bootrom makes AP_GOOD high before jumping.	DVDD18
EINT_CHG_IRQB	G24	DIO	External interrupt input	DVDD18
EINT_CTP0	F18	DIO	External interrupt input	DVDD18
EINT_CTP1	G18	DIO	External interrupt input	DVDD18
EINT_FPS	AE4	DIO	External interrupt input	DVDD18
EINT4	AU34	DIO	External interrupt input	DVDD18
EINT5	AU33	DIO	External interrupt input	DVDD18
EINT6	AV34	DIO	External interrupt input	DVDD18
GPIO_CTP0_RST	F19	DIO	Peripheral control signal	DVDD18
GPIO_CTP1_RST	G19	DIO	Peripheral control signal	DVDD18
GPIO_FPS_RST	AE5	DIO	Peripheral control signal	DVDD18
NFC_CLK_REQ	G31	DIO	Peripheral control signal	DVDD18
PERIEN_CHG_ENB	F24	DIO	Peripheral control signal	DVDD18
PMIC				
SPMI_M_SCL	AD4	DIO	PMIC SPMI control interface	DVDD18
SPMI_M_SDA	AD5	DIO	PMIC SPMI control interface	DVDD18
SPMI_P_SCL	AC4	DIO	PMIC SPMI control interface	DVDD18
SPMI_P_SDA	AC5	DIO	PMIC SPMI control interface	DVDD18
AUD_CLK_MOSI	D20	DIO	PMIC audio input interface	DVDD18

Ball name	Ball location	Type	Description	Power domain
AUD_DAT_MISO0	G22	DIO	PMIC audio input interface	DVDD18
AUD_DAT_MISO1	F23	DIO	PMIC audio input interface	DVDD18
AUD_DAT_MISO2	G23	DIO	PMIC audio input interface	DVDD18
AUD_DAT_MOSI0	F20	DIO	PMIC audio input interface	DVDD18
AUD_DAT_MOSI1	G20	DIO	PMIC audio input interface	DVDD18
AUD_DAT_MOSI2	F21	DIO	PMIC audio input interface	DVDD18
AUD_I2C_SCL	G27	DIO	PMIC audio input interface	DVDD18
AUD_I2C_SDA	F27	DIO	PMIC audio input interface	DVDD18
AUD_NLE_MOSI0	G21	DIO	PMIC audio input interface	DVDD18
AUD_NLE_MOSI1	F22	DIO	PMIC audio input interface	DVDD18
AUD_SYNC_MOSI	E20	DIO	PMIC audio input interface	DVDD18
SIM				
SIM1_SCLK	K36	DIO	SIM1 interface	DVDD28_SIM1
SIM1_SIO	K34	DIO	SIM1 interface	DVDD28_SIM1
SIM1_SRST	K35	DIO	SIM1 interface	DVDD28_SIM1
SIM2_SCLK	M38	DIO	SIM2 interface	DVDD28_SIM2
SIM2_SIO	L38	DIO	SIM2 interface	DVDD28_SIM2
SIM2_SRST	M39	DIO	SIM2 interface	DVDD28_SIM2
INT_SIM1	F11	DIO	SIM1 interrupt	DVDD18
INT_SIM2	G11	DIO	SIM2 interrupt	DVDD18
I2S				
I2S0_BCK	AF37	DIO	I2S interface	DVDD18
I2S0_DI	AG38	DIO	I2S interface	DVDD18
I2S0_DO	AG37	DIO	I2S interface	DVDD18
I2S0_LRCK	AF36	DIO	I2S interface	DVDD18
I2S1_BCK	AF37	DIO	I2S interface	DVDD18
I2S1_DI	AF34	DIO	I2S interface	DVDD18
I2S1_DO	AF35	DIO	I2S interface	DVDD18
I2S1_LRCK	AE35	DIO	I2S interface	DVDD18
I2S2_BCK	AG35	DIO	I2S interface	DVDD18
I2S2_DI	AH35	DIO	I2S interface	DVDD18
I2S2_DO	AH36	DIO	I2S interface	DVDD18
I2S2_LRCK	AG36	DIO	I2S interface	DVDD18
LCD				
DSI_TE	D19	DIO	Parallel display interface tearing effect	DVDD18
DSI_TE_AUX	G15	DIO	Parallel display interface tearing effect	DVDD18
LCM_RST	E19	DIO	Parallel display interface reset signal	DVDD18
LVM_RST_AUX	G14	DIO	Parallel display interface reset signal	DVDD18
PWM				
DISP_PWM	AT34	DIO	Display PWM output	DVDD18
DISP_PWM_1	AU35	DIO	Display PWM1 output	DVDD18
Keypad interface				
KPCOLO	AY20	DIO	Keypad column 0	DVDD18
KPCOL1	BA30	DIO	Keypad column 1	DVDD18
KPROW0	BB20	DIO	Keypad row 0	DVDD18

Ball name	Ball location	Type	Description	Power domain
KPROW1	BC20	DIO	Keypad row 1	DVDD18
SPI				
SPI0_CK	AC3	DIO	SPI0 clock	DVDD18
SPI0_CSB	AC2	DIO	SPI0 chip select	DVDD18
SPI0_HOLD	AB5	DIO	SPI0 hold	DVDD18
SPI0_MI	AD3	DIO	SPI0 data in	DVDD18
SPI0_MO	AD2	DIO	SPI0 data out	DVDD18
SPI0_WP	AB6	DIO	SPI0 write protect	DVDD18
SPI1_CK	AE3	DIO	SPI1 clock	DVDD18
SPI1_CSB	AE2	DIO	SPI1 chip select	DVDD18
SPI1_MI	AF3	DIO	SPI1 data in	DVDD18
SPI1_MO	AF2	DIO	SPI1 data out	DVDD18
SPI2_CK	F15	DIO	SPI2 clock	DVDD18
SPI2_CSB	G16	DIO	SPI2 chip select	DVDD18
SPI2_MI	H16	DIO	SPI2 data in	DVDD18
SPI2_MO	F16	DIO	SPI2 data out	DVDD18
SPI3_CK	G29	DIO	SPI3 clock	DVDD18
SPI3_CSB	F30	DIO	SPI3 chip select	DVDD18
SPI3_MI	F31	DIO	SPI3 data in	DVDD18
SPI3_MO	G30	DIO	SPI3 data out	DVDD18
SPI4_CK	F33	DIO	SPI4 clock	DVDD18
SPI4_CSB	G33	DIO	SPI4 chip select	DVDD18
SPI4_MI	G34	DIO	SPI4 data in	DVDD18
SPI4_MO	F34	DIO	SPI4 data out	DVDD18
UART				
UTXD0	AV31	DIO	UART interface	DVDD18
URXD0	AT32	DIO	UART interface	DVDD18
RF interface				
BPI_D_BUS0	AV28	DIO	BPI_BUS0	DVDD18
BPI_D_BUS1	AW21	DIO	BPI_BUS1	DVDD18
BPI_D_BUS2	AV27	DIO	BPI_BUS2	DVDD18
BPI_D_BUS3	AU27	DIO	BPI_BUS3	DVDD18
BPI_D_BUS4	AV26	DIO	BPI_BUS4	DVDD18
BPI_D_BUS5	AU26	DIO	BPI_BUS5	DVDD18
BPI_D_BUS6	AV25	DIO	BPI_BUS6	DVDD18
BPI_D_BUS7	AU25	DIO	BPI_BUS7	DVDD18
BPI_D_BUS8	AV24	DIO	BPI_BUS8	DVDD18
BPI_D_BUS9	AU24	DIO	BPI_BUS9	DVDD18
BPI_D_BUS10	AU23	DIO	BPI_BUS10	DVDD18
BPI_D_BUS11	AV22	DIO	BPI_BUS11	DVDD18
BPI_D_BUS12	AU22	DIO	BPI_BUS12	DVDD18
BPI_D_BUS13	AV21	DIO	BPI_BUS13	DVDD18
BPI_D_BUS14	AW21	DIO	BPI_BUS14	DVDD18
BPI_D_BUS15	AY21	DIO	BPI_BUS15	DVDD18
BPI_D_BUS16	BA21	DIO	BPI_BUS16	DVDD18

Ball name	Ball location	Type	Description	Power domain
BPI_D_BUS17	BB21	DIO	BPI_BUS17	DVDD18
BPI_D_BUS18	BC21	DIO	BPI_BUS18	DVDD18
BPI_D_BUS19	AU21	DIO	BPI_BUS19	DVDD18
MD_UCNT	AJ39	DIO	RF IC control	DVDD18
DIGRF_IRQ	AJ38	DIO	RF IC control	DVDD18
BPI_WAKEUP_PMIC	M6	DIO	BPI control	DVDD18
RF MIPI interface				
MIPI_M_SCLK	AH37	DIO	RF IC control	DVDD18
MIPI_M_SDAT0	AH38	DIO	RF IC control	DVDD18
MIPI0_D_SCLK	BB38	DIO	RF front-end control	DVDD18
MIPI0_D_SDAT0	BA38	DIO	RF front-end control	DVDD18
MIPI1_D_SCLK	AY38	DIO	RF front-end control	DVDD18
MIPI1_D_SDAT0	BA39	DIO	RF front-end control	DVDD18
MIPI2_D_SCLK	AW37	DIO	RF front-end control	DVDD18
MIPI2_D_SDAT0	AY39	DIO	RF front-end control	DVDD18
MIPI3_D_SCLK	AW38	DIO	RF front-end control	DVDD18
MIPI3_D_SDAT0	AW39	DIO	RF front-end control	DVDD18
MIPI4_D_SCLK	AV38	DIO	RF front-end control	DVDD18
MIPI4_D_SDAT0	AV39	DIO	RF front-end control	DVDD18
DMIC				
DMIC_DAT	G26	DIO	Data pin of digital microphone	DVDD18
DMIC_CLK	F26	DIO	Clock pin of digital microphone	DVDD18
MSDC1				
MSDC1_CLK	L35	DIO	MSDC1 clock output	DVDD28_MSDC1
MSDC1_CMD	L36	DIO	MSDC1 command pin	DVDD28_MSDC1
MSDC1_DAT0	L37	DIO	MSDC1 data 0 pin	DVDD28_MSDC1
MSDC1_DAT1	M34	DIO	MSDC1 data 1 pin	DVDD28_MSDC1
MSDC1_DAT2	M35	DIO	MSDC1 data 2 pin	DVDD28_MSDC1
MSDC1_DAT3	M36	DIO	MSDC1 data 3 pin	DVDD28_MSDC1
MSDC2				
MSDC2_CLK	AF7	DIO	MSDC2 clock output	DVDD18_MSDC2
MSDC2_CMD	AE6	DIO	MSDC2 command pin	DVDD18_MSDC2
MSDC2_DAT0	AC6	DIO	MSDC2 data 0 pin	DVDD18_MSDC2
MSDC2_DAT1	AC7	DIO	MSDC2 data 1 pin	DVDD18_MSDC2
MSDC2_DAT2	AD7	DIO	MSDC2 data 2 pin	DVDD18_MSDC2
MSDC2_DAT3	AE7	DIO	MSDC2 data 3 pin	DVDD18_MSDC2
eFuse				
DVDD18_VQPS	J39	DIO	eFuse blowing power control	DVDD_VQPS
EMI				
EMIO_REXTDN	A3	DIO	Reference pin for impedance calibration of SoC	
EMI1_REXTDN	BC3	DIO	Reference pin for impedance calibration of SoC	
EMI2_REXTDN	A37	DIO	Reference pin for impedance calibration of SoC	

Ball name	Ball location	Type	Description	Power domain
EMI3_REXTDN	BC37	DIO	Reference pin for impedance calibration of SoC	
CAM				
CAM_CLK0	U7	DIO	Master clock to 0th sensor	DVDD18
CAM_CLK1	U6	DIO	Master clock to 1st sensor	DVDD18
CAM_CLK2	AV12	DIO	Master clock to 2nd sensor	DVDD18
CAM_CLK3	AU11	DIO	Master clock to 3rd sensor	DVDD18
CAM_CLK4	T7	DIO	Master clock to 4th sensor	DVDD18
CAM_CLK5	T6	DIO	Master clock to 5th sensor	DVDD18
CAM_CLK6	AU12	DIO	Master clock to 6th sensor	DVDD18
CAM_CLK7	AU10	DIO	Master clock to 7th sensor	DVDD18
CAM_RST0	U2	DIO	Reset control to 0th sensor	DVDD18
CAM_RST1	U3	DIO	Reset control to 1st sensor	DVDD18
CAM_RST2	AV10	DIO	Reset control to 2nd sensor	DVDD18
CAM_RST3	AR9	DIO	Reset control to 3rd sensor	DVDD18
CAM_RST4	T2	DIO	Reset control to 4th sensor	DVDD18
CAM_RST5	T1	DIO	Reset control to 5th sensor	DVDD18
CAM_RST6	AV9	DIO	Reset control to 6th sensor	DVDD18
CAM_RST7	AW9	DIO	Reset control to 7th sensor	DVDD18
I2C				
I2C_SCL3	F29	DIO	I2C3 clock	DVDD18
I2C_SCL6	AE36	DIO	I2C6 clock	DVDD18
I2C_SCL10	U4	DIO	I2C10 clock	DVDD18
I2C_SCL11	V4	DIO	I2C11 clock	DVDD18
I2C_SCL12	AU13	DIO	I2C12 clock	DVDD18
I2C_SCL13	AU15	DIO	I2C13 clock	DVDD18
I2C_SDA3	G28	DIO	I2C3 data	DVDD18
I2C_SDA6	AE37	DIO	I2C6 data	DVDD18
I2C_SDA10	U5	DIO	I2C10 data	DVDD18
I2C_SDA11	V5	DIO	I2C11 data	DVDD18
I2C_SDA12	AV14	DIO	I2C12 data	DVDD18
I2C_SDA13	AV16	DIO	I2C13 data	DVDD18
I3C_SCL0	AV33	DIO	I3C0 clock	DVDD18
I3C_SCL1	AU20	DIO	I3C1 clock	DVDD18
I3C_SCL7	AW32	DIO	I3C7 clock	DVDD18
I3C_SDA0	AU32	DIO	I3C0 data	DVDD18
I3C_SDA1	AV20	DIO	I3C1 data	DVDD18
I3C_SDA7	AV32	DIO	I3C7 data	DVDD18
CAM_I3C_SCL2	W4	DIO	I3C clock	DVDD18
CAM_I3C_SCL4	V3	DIO	I3C clock	DVDD18
CAM_I3C_SCL8	AU14	DIO	I3C clock	DVDD18
CAM_I3C_SCL9	AT18	DIO	I3C clock	DVDD18
CAM_I3C_SDA2	W3	DIO	I3C data	DVDD18
CAM_I3C_SDA4	V2	DIO	I3C data	DVDD18
CAM_I3C_SDA8	AV15	DIO	I3C data	DVDD18
CAM_I3C_SDA9	AU18	DIO	I3C data	DVDD18

Ball name	Ball location	Type	Description	Power domain
CHG_I2C_SCL5	G25	DIO	I2C clock	DVDD18
CHG_I2C_SDA5	F25	DIO	I2C data	DVDD18
CONN				
CONN_HRST_B	L7	DIO	Resets CONNSYS	DVDD18
CONN_TOP_CLK	L2	DIO	CONNSYS top 2-wire interface	DVDD18
CONN_TOP_DATA	L3	DIO	CONNSYS top 2-wire interface	DVDD18
CONN_PMIC_EN	M5	DIO	Enables PMIC	DVDD18
ANT_SEL0	R2	DIO	Antenna select signal	DVDD18
ANT_SEL1	R1	DIO	Antenna select signal	DVDD18
ANT_SEL2	P2	DIO	Antenna select signal	DVDD18
ANT_SEL3	P1	DIO	Antenna select signal	DVDD18
ANT_SEL4	T5	DIO	Antenna select signal	DVDD18
ANT_SEL5	R4	DIO	Antenna select signal	DVDD18
ANT_SEL6	R3	DIO	Antenna select signal	DVDD18
ANT_SEL7	R5	DIO	Antenna select signal	DVDD18
ANT_SEL8	P3	DIO	Antenna select signal	DVDD18
ANT_SEL9	P4	DIO	Antenna select signal	DVDD18
ANT_SEL10	R6	DIO	Antenna select signal	DVDD18
ANT_SEL11	R7	DIO	Antenna select signal	DVDD18
ANT_SEL12	P6	DIO	Antenna select signal	DVDD18
ANT_SEL13	P5	DIO	Antenna select signal	DVDD18
ANT_SEL14	P7	DIO	Antenna select signal	DVDD18
CONN_BT_CLK	L4	DIO	CONNSYS BT interface	DVDD18
CONN_BT_DATA	L5	DIO	CONNSYS BT interface	DVDD18
CONN_WB_PTA	M7	DIO	CONNSYS Wi-Fi/Bluetooth interface	DVDD18
CONN_WF_CTRL0	N3	DIO	CONNSYS Wi-Fi control signal	DVDD18
CONN_WF_CTRL1	N2	DIO	CONNSYS Wi-Fi control signal	DVDD18
CONN_WF_CTRL2	M3	DIO	CONNSYS Wi-Fi control signal	DVDD18
CONN_WF_CTRL3	M2	DIO	CONNSYS Wi-Fi control signal	DVDD18
CONN_WF_CTRL4	M1	DIO	CONNSYS Wi-Fi control signal	DVDD18
GPS_L1_ELNA_EN	N4	DIO	GPS external front-end control signal	DVDD18
GPS_L5_ELNA_EN	N5	DIO	GPS external front-end control signal	DVDD18
SCP				
SCP_EINT0	F13	DIO	System companion processor EINT	DVDD18
SCP_EINT1	G13	DIO	System companion processor EINT	DVDD18
SCP_EINT2	F12	DIO	System companion processor EINT	DVDD18
SCP_EINT3	G12	DIO	System companion processor EINT	DVDD18
SCP_I3C_SCL0	F9	DIO	System companion processor I3C clock	DVDD18
SCP_I3C_SCL2	F8	DIO	System companion processor I3C clock	DVDD18
SCP_I3C_SCL3	F10	DIO	System companion processor I3C clock	DVDD18
SCP_I3C_SDA0	G9	DIO	System companion processor I3C data	DVDD18
SCP_I3C_SDA2	G9	DIO	System companion processor I3C data	DVDD18
SCP_I3C_SDA3	G10	DIO	System companion processor I3C data	DVDD18
SCP_SPI_CK	F6	DIO	System companion processor clock	DVDD18
SCP_SPI_CSB	G6	DIO	System companion processor chip select	DVDD18

Ball name	Ball location	Type	Description	Power domain
SCP_SPI_MI	G7	DIO	System companion processor datain	DVDD18
SCP_SPI_MO	F7	DIO	System companion processor dataout	DVDD18
SCP_VREQ_VAO	B19	DIO	System companion processor voltage request	DVDD18
DigRF				
DRF_DL_CN	AR39	AIO	DRF interface	AVDD15_DRF
DRF_DL_CP	AR38	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN0_DN	AN35	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN0_DP	AN34	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN1_DN	AN38	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN1_DP	AN39	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN2_DN	AP36	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN2_DP	AP37	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN3_DN	AM37	AIO	DRF interface	AVDD15_DRF
DRF_DL_LN3_DP	AM36	AIO	DRF interface	AVDD15_DRF
DRF_UL_CN	AL35	AIO	DRF interface	AVDD15_DRF
DRF_UL_CP	AL34	AIO	DRF interface	AVDD15_DRF
DRF_UL_LN0_DN	AL38	AIO	DRF interface	AVDD15_DRF
DRF_UL_LN0_DP	AL39	AIO	DRF interface	AVDD15_DRF
DRF_UL_LN1_DN	AK37	AIO	DRF interface	AVDD15_DRF
DRF_UL_LN1_DP	AK36	AIO	DRF interface	AVDD15_DRF
MIPI				
DSIO_CK0N_T1C	R34	AIO	DSI interface	AVDD12_DSI
DSIO_CK0P_T1B	R35	AIO	DSI interface	AVDD12_DSI
DSIO_D0N_T1A	T34	AIO	DSI interface	AVDD12_DSI
DSIO_D0P_T0C	T35	AIO	DSI interface	AVDD12_DSI
DSIO_D1N_T2B	P35	AIO	DSI interface	AVDD12_DSI
DSIO_D1P_T2A	P36	AIO	DSI interface	AVDD12_DSI
DSIO_D2N_T0B	U35	AIO	DSI interface	AVDD12_DSI
DSIO_D2P_T0A	U36	AIO	DSI interface	AVDD12_DSI
DSIO_D3N	P33	AIO	DSI interface	AVDD12_DSI
DSIO_D3P_T2C	P34	AIO	DSI interface	AVDD12_DSI
DSI1_CK0N_T1C	R38	AIO	DSI interface	AVDD12_DSI
DSI1_CK0P_T1B	R39	AIO	DSI interface	AVDD12_DSI
DSI1_D0N_T1A	T37	AIO	DSI interface	AVDD12_DSI
DSI1_D0P_T0C	T38	AIO	DSI interface	AVDD12_DSI
DSI1_D1N_T2B	R36	AIO	DSI interface	AVDD12_DSI
DSI1_D1P_T2A	R37	AIO	DSI interface	AVDD12_DSI
DSI1_D2N_T0B	U38	AIO	DSI interface	AVDD12_DSI
DSI1_D2P_T0A	U39	AIO	DSI interface	AVDD12_DSI
DSI1_D3N	N37	AIO	DSI interface	AVDD12_DSI
DSI1_D3P_T2C	P37	AIO	DSI interface	AVDD12_DSI
CSIOA_L0N_T0B	AW1	AIO	CSI interface	AVDD12_CSI
CSIOA_L0P_T0A	AV2	AIO	CSI interface	AVDD12_CSI
CSIOA_L1N_T1A	AW3	AIO	CSI interface	AVDD12_CSI
CSIOA_L1P_T0C	AW2	AIO	CSI interface	AVDD12_CSI

Ball name	Ball location	Type	Description	Power domain
CSI0A_L2N_T1C	BA1	AIO	CSI interface	AVDD12_CSI
CSI0A_L2P_T1B	AY2	AIO	CSI interface	AVDD12_CSI
CSI0B_L0N_T0B	BB2	AIO	CSI interface	AVDD12_CSI
CSI0B_L0P_T0A	BA2	AIO	CSI interface	AVDD12_CSI
CSI0B_L1N	BA3	AIO	CSI interface	AVDD12_CSI
CSI0B_L1P_T0C	AY3	AIO	CSI interface	AVDD12_CSI
CSI1A_L0N_T0B	AU3	AIO	CSI interface	AVDD12_CSI
CSI1A_L0P_T0A	AT3	AIO	CSI interface	AVDD12_CSI
CSI1A_L1N_T1A	AU4	AIO	CSI interface	AVDD12_CSI
CSI1A_L1P_T0C	AT4	AIO	CSI interface	AVDD12_CSI
CSI1A_L2N_T1C	AW4	AIO	CSI interface	AVDD12_CSI
CSI1A_L2P_T1B	AV4	AIO	CSI interface	AVDD12_CSI
CSI1B_L0N_T0B	AV5	AIO	CSI interface	AVDD12_CSI
CSI1B_L0P_T0A	AU5	AIO	CSI interface	AVDD12_CSI
CSI1B_L1N	AT6	AIO	CSI interface	AVDD12_CSI
CSI1B_L1P_T0C	AT5	AIO	CSI interface	AVDD12_CSI
CSI2A_L0N_T0B	AM5	AIO	CSI interface	AVDD12_CSI
CSI2A_L0P_T0A	AM6	AIO	CSI interface	AVDD12_CSI
CSI2A_L1N_T1A	AN5	AIO	CSI interface	AVDD12_CSI
CSI2A_L1P_T0C	AN6	AIO	CSI interface	AVDD12_CSI
CSI2A_L2N_T1C	AN4	AIO	CSI interface	AVDD12_CSI
CSI2A_L2P_T1B	AN3	AIO	CSI interface	AVDD12_CSI
CSI2B_L0N_T0B	AP5	AIO	CSI interface	AVDD12_CSI
CSI2B_L0P_T0A	AP6	AIO	CSI interface	AVDD12_CSI
CSI2B_L1N	AR5	AIO	CSI interface	AVDD12_CSI
CSI2B_L1P_T0C	AR6	AIO	CSI interface	AVDD12_CSI
CSI3A_L0N_T0B	AN2	AIO	CSI interface	AVDD12_CSI
CSI3A_L0P_T0A	AN1	AIO	CSI interface	AVDD12_CSI
CSI3A_L1N_T1A	AP3	AIO	CSI interface	AVDD12_CSI
CSI3A_L1P_T0C	AP2	AIO	CSI interface	AVDD12_CSI
CSI3A_L2N_T1C	AR3	AIO	CSI interface	AVDD12_CSI
CSI3A_L2P_T1B	AR2	AIO	CSI interface	AVDD12_CSI
CSI3B_L0N_T0B	AT2	AIO	CSI interface	AVDD12_CSI
CSI3B_L0P_T0A	AT1	AIO	CSI interface	AVDD12_CSI
CSI3B_L1N	AU2	AIO	CSI interface	AVDD12_CSI
CSI3B_L1P_T0C	AU1	AIO	CSI interface	AVDD12_CSI
CSI4A_L0N_T0B	AG6	AIO	CSI interface	AVDD12_CSI
CSI4A_L0P_T0A	AG5	AIO	CSI interface	AVDD12_CSI
CSI4A_L1N_T1A	AJ6	AIO	CSI interface	AVDD12_CSI
CSI4A_L1P_T0C	AH6	AIO	CSI interface	AVDD12_CSI
CSI4A_L2N_T1C	AJ4	AIO	CSI interface	AVDD12_CSI
CSI4A_L2P_T1B	AJ5	AIO	CSI interface	AVDD12_CSI
CSI4B_L0N_T0B	AK7	AIO	CSI interface	AVDD12_CSI
CSI4B_L0P_T0A	AJ7	AIO	CSI interface	AVDD12_CSI
CSI4B_L1N	AL6	AIO	CSI interface	AVDD12_CSI

Ball name	Ball location	Type	Description	Power domain
CSI4B_L1P_T0C	AL7	AIO	CSI interface	AVDD12_CSI
CSI5A_L0N_T0B	AH3	AIO	CSI interface	AVDD12_CSI
CSI5A_L0P_T0A	AH2	AIO	CSI interface	AVDD12_CSI
CSI5A_L1N_T1A	AK3	AIO	CSI interface	AVDD12_CSI
CSI5A_L1P_T0C	AJ3	AIO	CSI interface	AVDD12_CSI
CSI5A_L2N_T1C	AK1	AIO	CSI interface	AVDD12_CSI
CSI5A_L2P_T1B	AK2	AIO	CSI interface	AVDD12_CSI
CSI5B_L0N_T0B	AL3	AIO	CSI interface	AVDD12_CSI
CSI5B_L0P_T0A	AL2	AIO	CSI interface	AVDD12_CSI
CSI5B_L1N	AM1	AIO	CSI interface	AVDD12_CSI
CSI5B_L1P_T0C	AM2	AIO	CSI interface	AVDD12_CSI
SSUSB and DPTX				
SSUSB_RXN_P1	W36	AIO	SSUSB RXDATA	AVDD12_SSUSB
SSUSB_RXN_P2	Y34	AIO	SSUSB RXDATA	AVDD12_SSUSB
SSUSB_RXP_P1	W37	AIO	SSUSB TXDATA	AVDD12_SSUSB
SSUSB_RXP_P2	Y35	AIO	SSUSB TXDATA	AVDD12_SSUSB
SSUSB_TXN_P1	Y39	AIO	SSUSB RXDATA	AVDD12_SSUSB
SSUSB_TXN_P2	W33	AIO	SSUSB RXDATA	AVDD12_SSUSB
SSUSB_TXP_P1	Y38	AIO	SSUSB TXDATA	AVDD12_SSUSB
SSUSB_TXP_P2	W32	AIO	SSUSB TXDATA	AVDD12_SSUSB
DP_SBU1	AA37	AIO	DPTX interface	AVDD12_SSUSB
DP_SBU2	AA36	AIO	DPTX interface	AVDD12_SSUSB
WBG				
XIN_WBG	K8	AIO	GPS crystal clock input	AVDD15_WBG
GL1_I	H2	AIO	GPS L1 I channel	AVDD15_WBG
GL1_Q	H1	AIO	GPS L1 Q channel	AVDD15_WBG
GL5_I	J1	AIO	GPS L5 I channel	AVDD15_WBG
GL5_Q	J2	AIO	GPS L5 Q channel	AVDD15_WBG
BT_IN	H4	AIO	BT channel	AVDD15_WBG
BT_IP	G4	AIO	BT channel	AVDD15_WBG
BT_QN	J5	AIO	BT channel	AVDD15_WBG
BT_QP	J4	AIO	BT channel	AVDD15_WBG
WF0_IN	F3	AIO	WF0 channel	AVDD15_WBG
WF0_IP	E3	AIO	WF0 channel	AVDD15_WBG
WF0_QN	F2	AIO	WF0 channel	AVDD15_WBG
WF0_QP	F1	AIO	WF0 channel	AVDD15_WBG
WF1_IN	C2	AIO	WF1 channel	AVDD15_WBG
WF1_IP	D2	AIO	WF1 channel	AVDD15_WBG
WF1_QN	D1	AIO	WF1 channel	AVDD15_WBG
WF1_QP	C1	AIO	WF1 channel	AVDD15_WBG
USB				
USB_DP	AC36	AIO	USB D+ differential data line	AVDD33_USB
USB_DM	AC37	AIO	USB D- differential data line	AVDD33_USB
UFS				
UFS_PLL_CKREF	J37	AIO	Reference clock input for UFS PLL	AVDD15_UFS

Ball name	Ball location	Type	Description	Power domain
UFS_REFCK_OUT	H35	AIO	Reference clock output to UFS chip	AVDD15_UFS
UFS_TX0P	C39	AIO	UFS interface	AVDD15_UFS
UFS_TX0N	D39	AIO	UFS interface	AVDD15_UFS
UFS_TX1P	C37	AIO	UFS interface	AVDD15_UFS
UFS_TX1N	D37	AIO	UFS interface	AVDD15_UFS
UFS_RX0P	F36	AIO	UFS interface	AVDD15_UFS
UFS_RX0N	F37	AIO	UFS interface	AVDD15_UFS
UFS_RX1P	G39	AIO	UFS interface	AVDD15_UFS
UFS_RX1N	G38	AIO	UFS interface	AVDD15_UFS
UFS_RST_N	G36	AIO	UFS interface	AVDD15_UFS
PCIe				
CKM_XTAL_CK	W7	AIO	26 MHz clock input for PCIe	AVDD15_PCIE
PCIE_CKN	AA4	AIO	PCIe interface	AVDD15_PCIE
PCIE_CKP	AA3	AIO	PCIe interface	AVDD15_PCIE
PCIE_LN0_RXN	Y2	AIO	PCIe interface	AVDD15_PCIE
PCIE_LN0_RXP	Y1	AIO	PCIe interface	AVDD15_PCIE
PCIE_LN0_TXN	Y5	AIO	PCIe interface	AVDD15_PCIE
PCIE_LN0_TXP	Y6	AIO	PCIe interface	AVDD15_PCIE
General-purpose				
PERI_EN0	AU36	DIO	PERIPHERAL_EN0	DVDD18
PERI_EN1	AV37	DIO	PERIPHERAL_EN1	DVDD18
PERI_EN2	AV36	DIO	PERIPHERAL_EN2	DVDD18
PERI_EN3	AV35	DIO	PERIPHERAL_EN3	DVDD18
MISC				
TN_PLLGP	AC30	AIO	Reserved	
TP_PLLGP	AD20	AIO	Reserved	
Analog power				
AVDD08_DRV_DSI	U33	P	Analog power for DSI	AVDD08_DRV_DSI
AVDD12_APPLL	AF20	P	Analog power for AP PLL	AVDD12_APPLL
AVDD12_APUPLL	T20	P	Analog power for APUPLL	AVDD12_MDPLL
AVDD12_CKBUF_UFS	K31	P	Analog power for UFS	AVDD12_CKBUF_UFS
AVDD12_CKSQ	AD34	P	Analog power for clock square	AVDD12_CKSQ
AVDD12_CSI0	AN7	P	Analog power for CSI	AVDD12_CSI0
AVDD12_CSI1	AG7	P	Analog power for CSI	AVDD12_CSI1
AVDD12_DRF	AM39	P	Analog power for DRF	AVDD12_DRF
AVDD12_DSI	V39	P	Analog power for DSI	AVDD12_DSI
AVDD12_EMI0	K16	P	Analog power for DDRPHY	AVDD12_EMI0
AVDD12_EMI1	AR16	P	Analog power for DDRPHY	AVDD12_EMI1
AVDD12_EMI2	J24	P	Analog power for DDRPHY	AVDD12_EMI2
AVDD12_EMI3	AR24	P	Analog power for DDRPHY	AVDD12_EMI3
AVDD12_MDPLL	AJ24	P	Analog power for MDPLL	AVDD12_MDPLL
AVDD12_PCIE	AB1	P	Analog power for PCIe	AVDD12_PCIE
AVDD12_SSUSB_P1	AB39	P	Analog power for SSUSB	AVDD12_SSUSB_P1
AVDD12_SSUSB_P2	AC39	P	Analog power for SSUSB	AVDD12_SSUSB_P2

Ball name	Ball location	Type	Description	Power domain
AVDD12_TSFDC_BM	AN16	P	Analog power for TSFDC_BM	AVDD12_TSFDC_BM
AVDD12_TSFDC_RM	AJ15	P	Analog power for TSFDC_RM	AVDD12_TSFDC_RM
AVDD12_UFS_TX	L31	P	Analog power for UFS	AVDD12_UFS_TX
AVDD12_UFS_RX	M31	P	Analog power for UFS	AVDD12_UFS_RX
AVDD12_USB	AE39	P	Analog power for USB	AVDD12_USB
AVDD12_WBG	H7	P	Analog power for WBG	AVDD12_WBG
AVDD15_APPLL	AE20	P	Analog power 1.5V for AP PLL	AVDD15_APPLL
AVDD15_APUPLL	U19	P	Analog power 1.5V for APU PLL	AVDD15_APUPLL
AVDD15_CKSQ	AE33	P	Analog power 1.5V for clock square	AVDD15_CKSQ
AVDD15_CSI	AP7	P	Analog power 1.5V for CSI	AVDD15_CSI
AVDD15_DRF	AP33	P	Analog power 1.5V for digital RF	AVDD15_DRF
AVDD15_DSI	V34	P	Analog power 1.5V for DSI	AVDD15_DSI
AVDD15_EMI0	K15	P	Analog power 1.5V for DDRPHY	AVDD15_EMI0
AVDD15_EMI1	AP16	P	Analog power 1.5V for DDRPHY	AVDD15_EMI1
AVDD15_EMI2	K24	P	Analog power 1.5V for DDRPHY	AVDD15_EMI2
AVDD15_EMI3	AP24	P	Analog power 1.5V for DDRPHY	AVDD15_EMI3
AVDD15_MDPLL	AK24	P	Analog power 1.5V for MD PLL	AVDD15_MDPLL
AVDD15_PCIE	AA7	P	Analog power 1.5V for PCIe	AVDD15_PCIE
AVDD15_PROC	AG12	P	Analog power 1.5V for CPU	AVDD15_PROC
AVDD15_SSUSB_P1	AD38	P	Analog power 1.5V for SSUSB	AVDD15_SSUSB_P1
AVDD15_SSUSB_P2	AC38	P	Analog power 1.5V for SSUSB	AVDD15_SSUSB_P2
AVDD15_UFS	M30	P	Analog power 1.5V for UFS	AVDD15_UFS
AVDD15_USB	AE38	P	Analog power 1.5V for USB	AVDD15_USB
AVDD15_WBG	L1	P	Analog power 1.5V for WBG	AVDD15_WBG
AVDD33_USB	AD35	P	Analog power 3.3V for USB	AVDD33_USB
AVDD075_EMI0	K12	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI0	K14	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI1	AP11	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI1	AP14	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI2	K26	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI2	K29	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI3	AP26	P	DDRPHY power	AVDD075_EMI
AVDD075_EMI3	AP29	P	DDRPHY power	AVDD075_EMI
AVDD12_EMI0	K16	P	DDRPHY power	AVDD12_EMI
AVDD12_EMI1	AR16	P	DDRPHY power	AVDD12_EMI
AVDD12_EMI2	J24	P	DDRPHY power	AVDD12_EMI
AVDD12_EMI3	AR24	P	DDRPHY power	AVDD12_EMI
AVDDQ_EMI0	J14	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI0	J16	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI1	AR13	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI1	AR15	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI2	J25	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI2	J27	P	DDRPHY power	AVDDQ_EMI
AVDDQ_EMI3	AR25	P	DDRPHY power	AVDDQ_EMI

Ball name	Ball location	Type	Description	Power domain
AVDDQ_EMI3	AR27	P	DDRPHY power	AVDDQ_EMI
VDD2H_EMI0	J12	P	DRAM power	VDD2_EMI
VDD2H_EMI1	AR12	P	DRAM power	VDD2_EMI
VDD2H_EMI2	J29	P	DRAM power	VDD2_EMI
VDD2H_EMI3	AR29	P	DRAM power	VDD2_EMI
Digital power				
DVDD_VLPAO	AD25	P	Digital power input for voltage low power always-on design	-
DVDD_SCPAO	AC27	P	Digital power input for always-on feature processor	-
DVDD_APU	K17	P	Digital power input for APU	-
DVDD_CORE	K20	P	Digital power input for Vcore	-
DVDD_GPUSTACK	K2	P	Digital power input for GPU	-
DVDD_MM	AH10	P	Digital power input for multi-media	-
DVDD_MODEM	AH25	P	Digital power input for modem	-
DVDD_PROC_B	AD19	P	Digital power input for big core	-
DVDD_PROC_M	AE9	P	Digital power input for medium core	-
DVDD_PROC_L	AE14	P	Digital power input for little core	-
DVDD_SRAM_APU	M17	P	Digital power input for APU SRAM	-
DVDD_SRAM_CORE	R20	P	Digital power input for core SRAM	-
DVDD_SRAM_GPUSTACK	R21	P	Digital power input for GPU SRAM	-
DVDD_SRAM_MODEM	AF28	P	Digital power input for modem SRAM	-
DVDD_SRAM_MM	AK15	P	Digital power input for MM SRAM	-
DVDD_SRAM_PROC_B	AH16	P	Digital power input for big core SRAM	-
DVDD_SRAM_PROC_M	AA18	P	Digital power input for medium core SRAM	-
DVDD_SRAM_PROC_L	AB17	P	Digital power input for little core SRAM	-
DVDD075_I0BL	AT33	P	Digital power input for I/O	-
DVDD075_I0LB2	AH34	P	Digital power input for I/O	-
DVDD075_I0RM2	AC1	P	Digital power input for I/O	-
DVDD075_I0RT4	N6	P	Digital power input for I/O	-
DVDD075_MSDC1	N32	P	Digital power input for MSDC1	-
DVDD075_MSDC2	AF4	P	Digital power input for MSDC2	-
DVDD12_I0BL	AU38	P	Digital power input for I/O	-
DVDD12_I0LB2	AF33	P	Digital power input for I/O	-
DVDD12_I0RM2	AB7	P	Digital power input for I/O	-
DVDD12_I0RT4	N1	P	Digital power input for I/O	-
DVDD12_MSDC2	AF1	P	Digital power input for MSDC2	-
DVDD15_MSDC1	L34	P	Digital power input for MSDC1	-
DVDD18_I0BL	AU39	P	Digital power input for I/O	-
DVDD18_I0BM	AR23	P	Digital power input for I/O	-
DVDD18_I0BM2	AV19	P	Digital power input for I/O	-
DVDD18_I0BM3	AR20	P	Digital power input for I/O	-
DVDD18_I0BR	AV13	P	Digital power input for I/O	-
DVDD18_I0LB	AF39	P	Digital power input for I/O	-
DVDD18_I0LB2	AG39	P	Digital power input for I/O	-

Ball name	Ball location	Type	Description	Power domain
DVDD18_IORB	AN8	P	Digital power input for I/O	-
DVDD18_IORM	AG1	P	Digital power input for I/O	-
DVDD18_IORM2	AD1	P	Digital power input for I/O	-
DVDD18_IORT	V1	P	Digital power input for I/O	-
DVDD18_IORT2	U1	P	Digital power input for I/O	-
DVDD18_IORT3	T4	P	Digital power input for I/O	-
DVDD18_IORT4	N7	P	Digital power input for I/O	-
DVDD18_IOTL	F28	P	Digital power input for I/O	-
DVDD18_IOTM	F5	P	Digital power input for I/O	-
DVDD18_IOTM2	F14	P	Digital power input for I/O	-
DVDD18_IOTM3	F17	P	Digital power input for I/O	-
DVDD18_MSDC2	AE1	P	Digital power input for MSDC2	-
DVDD18_VQPS	J39	P	Digital power input for VQPS	-
DVDD28_MSDC1	M33	P	Digital power input for MSDC1	-
DVDD28_SIM1	K39	P	Digital power input for SIM1	-
DVDD28_SIM2	L39	P	Digital power input for SIM2	-
ESD				
CDM3P5A	AU17	G	Reserved	-
CDM5P5A	AU16	G	Reserved	-
Ground				
DVSS	A18	G	Ground	-

2.2 Electrical Characteristics

2.2.1 Absolute Maximum Ratings

Table 2-4. Absolute maximum ratings for power supply

Symbol or ball name	Description	Max.	Unit
AVDD08_DRV_DSI	Analog power	0.88	V
AVDD12_APPLL	Analog power	1.32	V
AVDD12_APUPLL	Analog power	1.32	V
AVDD12_CKBUF_UFS	Analog power	1.32	V
AVDD12_CKSQ	Analog power	1.32	V
AVDD12_CSI0	Analog power	1.32	V
AVDD12_CSI1	Analog power	1.32	V
AVDD12_DRF	Analog power	1.32	V
AVDD12_DSI	Analog power	1.32	V
AVDD12_MDPLL	Analog power	1.32	V
AVDD12_PCIE	Analog power	1.32	V
AVDD12_SSUSB_P1	Analog power	1.32	V
AVDD12_SSUSB_P2	Analog power	1.32	V
AVDD12_TSFDC_BM	Analog power	1.32	V
AVDD12_TSFDC_RM	Analog power	1.32	V
AVDD12_UFS_RX	Analog power	1.32	V
AVDD12_UFS_TX	Analog power	1.32	V
AVDD12_USB	Analog power	1.32	V
AVDD12_WBG	Analog power	1.32	V
AVDD15_APPLL	Analog power	1.65	V
AVDD15_APUPLL	Analog power	1.65	V
AVDD15_CKSQ	Analog power	1.65	V
AVDD15_CSI	Analog power	1.65	V
AVDD15_DRF	Analog power	1.65	V
AVDD15_DSI	Analog power	1.65	V
AVDD15_MDPLL	Analog power	1.65	V
AVDD15_PCIE	Analog power	1.65	V
AVDD15_PROC	Analog power	1.65	V
AVDD15_SSUSB_P1	Analog power	1.65	V
AVDD15_SSUSB_P2	Analog power	1.65	V
AVDD15_UFS	Analog power	1.65	V
AVDD15_USB	Analog power	1.65	V
AVDD15_WBG	Analog power	1.65	V
AVDD33_USB	Analog power	3.15	V
AVDDQ_EMI0/1/2/3	Analog DDRPHY power	0.63	V
AVDD075_EMI0/1/2/3	Analog DDRPHY power	0.9075	V
AVDD15_EMI0/1/2/3	Analog DDRPHY power	1.65	V
AVDD12_EMI0/1/2/3	Analog DDRPHY power	1.32	V
DVDD_VLPAO	Digital power	0.8	V
DVDD_SCPAO	Digital power	0.8	V

Symbol or ball name	Description	Max.	Unit
DVDD_APU	Digital power	0.85	V
DVDD_CORE	Digital power	0.8	V
DVDD_GPUSTACK	Digital power	0.9	V
DVDD_MM	Digital power	0.7	V
DVDD_MODEM	Digital power	0.8	V
DVDD_PROC_B	Digital power	1.1	V
DVDD_PROC_M	Digital power	1.1	V
DVDD_PROC_L	Digital power	1.05	V
DVDD_SRAM_APU	SRAM power	0.95	V
DVDD_SRAM_CORE	SRAM power	0.8	V
DVDD_SRAM_GPUSTACK	SRAM power	0.9	V
DVDD_SRAM_MODEM	SRAM power	0.8	V
DVDD_SRAM_MM	SRAM power	0.8	V
DVDD_SRAM_PROC_B	SRAM power	1.1	V
DVDD_SRAM_PROC_M	SRAM power	1.1	V
DVDD_SRAM_PROC_L	SRAM power	1.05	V
DVDD075_IOBL	Digital I/O power	0.825	V
DVDD075_IOLB2	Digital I/O power	0.825	V
DVDD075_IORM2	Digital I/O power	0.825	V
DVDD075_IORT4	Digital I/O power	0.825	V
DVDD075_MSDC1	Digital I/O power	0.825	V
DVDD075_MSDC2	Digital I/O power	0.825	V
DVDD12_IOBL	Digital I/O power	1.26	V
DVDD12_IOLB2	Digital I/O power	1.26	V
DVDD12_IORM2	Digital I/O power	1.26	V
DVDD12_IORT4	Digital I/O power	1.26	V
DVDD12_MSDC2	Digital I/O power	1.26	V
DVDD15_MSDC1	Digital I/O power	1.6	V
DVDD18_IOBL	Digital I/O power	1.98	V
DVDD18_IOBM	Digital I/O power	1.98	V
DVDD18_IOBM2	Digital I/O power	1.98	V
DVDD18_IOBM3	Digital I/O power	1.98	V
DVDD18_IOBR	Digital I/O power	1.98	V
DVDD18_IOLB	Digital I/O power	1.98	V
DVDD18_IOLB2	Digital I/O power	1.98	V
DVDD18_IORB	Digital I/O power	1.98	V
DVDD18_IORM	Digital I/O power	1.98	V
DVDD18_IORM2	Digital I/O power	1.98	V
DVDD18_IORT	Digital I/O power	1.98	V
DVDD18_IORT2	Digital I/O power	1.98	V
DVDD18_IORT3	Digital I/O power	1.98	V
DVDD18_IORT4	Digital I/O power	1.98	V
DVDD18_IOTL	Digital I/O power	1.98	V
DVDD18_IOTM	Digital I/O power	1.98	V
DVDD18_IOTM2	Digital I/O power	1.98	V

Symbol or ball name	Description	Max.	Unit
DVDD18_IOTM3	Digital I/O power	1.98	V
DVDD18_MSDC2	Digital I/O power	1.98	V
DVDD18_VQPS	Digital I/O power	1.98	V
DVDD28_MSDC1	Digital I/O power	3.3	V
DVDD28_SIM1	Digital I/O power	3.3	V
DVDD28_SIM2	Digital I/O power	3.3	V

2.2.2 Recommended Operating Conditions

Table 2-5. Recommended operating conditions for power supply

Symbol or ball name	Description	Min.	Typ.	Max.	Unit
AVDD08_DRV_DSI	Analog power	0.7125	0.75	0.84	V
AVDD12_APPLL	Analog power	1.14	1.2	1.26	V
AVDD12_APUPLL	Analog power	1.14	1.2	1.26	V
AVDD12_CKBUF_UFS	Analog power	1.14	1.2	1.26	V
AVDD12_CKSQ	Analog power	1.14	1.2	1.26	V
AVDD12_CSI0	Analog power	1.14	1.2	1.26	V
AVDD12_CSI1	Analog power	1.14	1.2	1.26	V
AVDD12_DRF	Analog power	1.14	1.2	1.26	V
AVDD12_DSI	Analog power	1.14	1.2	1.26	V
AVDD12_MDPLL	Analog power	1.14	1.2	1.26	V
AVDD12_PCIE	Analog power	1.14	1.2	1.26	V
AVDD12_SSUSB_P1	Analog power	1.14	1.2	1.26	V
AVDD12_SSUSB_P2	Analog power	1.14	1.2	1.26	V
AVDD12_TSFDC_BM	Analog power	1.14	1.2	1.26	V
AVDD12_TSFDC_RM	Analog power	1.14	1.2	1.26	V
AVDD12_UFS_RX	Analog power	1.14	1.2	1.26	V
AVDD12_UFS_TX	Analog power	1.14	1.2	1.26	V
AVDD12_USB	Analog power	1.14	1.2	1.26	V
AVDD12_WBG	Analog power	1.14	1.2	1.26	V
AVDD15_APPLL	Analog power	1.425	1.5	1.575	V
AVDD15_APUPLL	Analog power	1.425	1.5	1.575	V
AVDD15_CKSQ	Analog power	1.425	1.5	1.575	V
AVDD15_CSI	Analog power	1.425	1.5	1.575	V
AVDD15_DRF	Analog power	1.425	1.5	1.575	V
AVDD15_DSI	Analog power	1.425	1.5	1.575	V
AVDD15_MDPLL	Analog power	1.425	1.5	1.575	V
AVDD15_PCIE	Analog power	1.425	1.5	1.575	V
AVDD15_PROC	Analog power	1.425	1.5	1.575	V
AVDD15_SSUSB_P1	Analog power	1.425	1.5	1.575	V
AVDD15_SSUSB_P2	Analog power	1.425	1.5	1.575	V
AVDD15_UFS	Analog power	1.425	1.5	1.575	V
AVDD15_USB	Analog power	1.425	1.5	1.575	V
AVDD15_WBG	Analog power	1.425	1.5	1.575	V

Symbol or ball name	Description	Min.	Typ.	Max.	Unit
AVDD33_USB	Analog power	2.85	3.0	3.15	V
AVDDQ_EMI0/1/2/3	Analog power	0.27	0.3/0.5	0.57	V
AVDD075_EMI0/1/2/3	Analog power	0.5225	0.75	0.8663	V
AVDD15_EMI0/1/2/3	Analog power	1.425	1.5	1.575	V
AVDD12_EMI0/1/2/3	Analog power	1.14	1.2	1.26	V
DVDD18_IOBL	Digital power input for 1.8V I/O	1.62	1.8	1.98	V
DVDD18_IOBM					
DVDD18_IOBM2					
DVDD18_IOBM3					
DVDD18_IOBR					
DVDD18_IOLB					
DVDD18_IOLB2					
DVDD18_IORB					
DVDD18_IORM					
DVDD18_IORM2					
DVDD18_IORT					
DVDD18_IORT2					
DVDD18_IORT3					
DVDD18_IORT4					
DVDD18_IOTL					
DVDD18_IOTM					
DVDD18_IOTM2					
DVDD18_IOTM3					
DVDD18_MSDC2					
DVDD075_IOBL	Digital power input for 0.75V I/O	0.7	0.75	0.8	V
DVDD075_IOLB2					
DVDD075_IORM2					
DVDD075_IORT4					
DVDD075_MSDC1					
DVDD075_MSDC2					
DVDD12_IOBL	Digital power input for 1.2V I/O	1.14	1.2	1.26	V
DVDD12_IOLB2					
DVDD12_IORM2					
DVDD12_IORT4					
DVDD12_MSDC2					
DVDD15_MSDC1	Digital power input for 1.5V I/O	1.4	1.5	1.6	V
DVDD28_MSDC1	Digital power input for MSDC1	2.7	3	3.15	V
DVDD28_SIM1	Digital power input for SIM1	2.7	3	3.15	V
DVDD28_SIM2	Digital power input for SIM2	2.7	3	3.15	V
DVDD_CORE	Digital power input for core	0.72	0.8	0.88	V
		0.6525	0.725	0.7975	V
		0.585	0.65	0.715	V
		0.54	0.6	0.66	V
		0.5175	0.575	0.6325	V
		0.495	0.55	0.605	V

Symbol or ball name	Description	Min.	Typ.	Max.	Unit
DVDD_MM	Digital power input for MM	0.63	0.7	0.77	V
		0.585	0.65	0.715	V
		0.54	0.6	0.66	V
		0.5175	0.575	0.6325	V
DVDD_GPUSTACK	Digital power input for GPU	0.810	0.9	0.990	V
		0.72	0.8	0.88	V
		0.6075	0.675	0.7425	V
		0.5175	0.575	0.6325	V
DVDD_PROC_B	Digital power input for big core	0.45	0.5	0.55	V
		0.945	1.05	1.155	V
		0.810	0.9	0.990	V
		0.675	0.75	0.825	V
DVDD_PROC_M	Digital power input for medium core	0.495	0.55	0.605	V
		0.945	1.05	1.155	V
		0.810	0.9	0.990	V
		0.675	0.75	0.825	V
DVDD_PROC_L	Digital power input for little core	0.495	0.55	0.605	V
		0.945	1.05	1.155	V
		0.810	0.9	0.990	V
		0.675	0.75	0.825	V
DVDD_APU	Digital power input for APU	0.495	0.55	0.605	V
		0.765	0.85	0.935	V
		0.675	0.75	0.825	V
		0.585	0.65	0.715	V
DVDD_MODEM	Digital power input for modem	0.495	0.55	0.605	V
		0.450	0.5	0.550	V
		0.720	0.8	0.880	V
		0.675	0.75	0.825	V
DVDD_VLPAO	Digital power input for VLPAO	0.608	0.675	0.743	V
		0.563	0.625	0.688	V
		0.5175	0.575	0.6325	V
		0.72	0.8	0.88	V
DVDD_SCPAO	Digital power input for SCPAO	0.675	0.75	0.825	V
		0.495	0.55	0.605	V
		0.585	0.65	0.715	V
		0.5175	0.575	0.6325	V
DVDD_SRAM_CORE	Digital power input for Vcore SRAM	0.495	0.55	0.605	V
		0.72	0.8	0.88	V
		0.675	0.75	0.825	V
		0.495	0.55	0.605	V
DVDD_SRAM_APU	Digital power input for APU SRAM	0.765	0.85	0.935	V
		0.675	0.75	0.825	V
DVDD_SRAM_GPUSTACK	Digital power input for GPU SRAM	0.810	0.9	0.990	V

Symbol or ball name	Description	Min.	Typ.	Max.	Unit
		0.72	0.8	0.88	V
		0.675	0.75	0.825	V
DVDD_SRAM_MODEM	Digital power input for modem SRAM	0.72	0.8	0.88	V
		0.675	0.75	0.825	V
DVDD_SRAM_MM	Digital power input for MM SRAM	0.72	0.8	0.88	V
		0.675	0.75	0.825	V
		0.495	0.55	0.605	V
DVDD_SRAM_PROC_B	Digital power input for big core SRAM	0.945	1.05	1.155	V
		0.81	0.9	0.99	V
		0.675	0.75	0.825	V
DVDD_SRAM_PROC_M	Digital power input for medium core SRAM	0.945	1.05	1.155	V
		0.81	0.9	0.99	V
		0.675	0.75	0.825	V
DVDD_SRAM_PROC_L	Digital power input for little core SRAM	0.945	1.05	1.155	V
		0.81	0.9	0.99	V
		0.675	0.75	0.825	V

2.2.3 Storage Condition

- Shelf life in sealed bag: 24 months at < 40°C and < 90% relative humidity (RH).
- After the bag is opened, devices subjected to infrared reflow, vapor-phase reflow, or equivalent processing must be:
 - mounted within 168 hours in factory condition of 30°C/60% RH, or
 - stored at 20% RH.
- Devices should require baking before being mounted if they are placed
 - for 192 hours at 40°C +5°C/-0°C and < 5% RH in low temperature device containers, or
 - for 24 hours at 125°C +5°C/-0°C in high temperature device containers.

2.2.4 DC Electrical Characteristics

2.2.4.1 RTC DC Electrical Characteristics

Table 2-6. RTC DC electrical characteristics (DVDD18_IOTM = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	0.65*DVDD18_IOTM		DVDD18_IOTM + 0.3	V
VIL	Input logic low voltage	-0.3		0.35*DVDD18_IOTM	V
VOH	DC output logic high voltage	0.75*DVDD18_IOTM			V
VOL	DC output logic low voltage			0.25*DVDD18_IOTM	V
F _{RTC}	Input clock frequency		32		kHz
DC _{RTC}	Input signal duty cycle	45	50	55	%

2.2.4.2 SPI, I2S DC Electrical Characteristics

Table 2-7. SPI, I2S DC electrical characteristics (DVDD18_IOBM = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	$0.65 \cdot \text{DVDD18_IOBM}$		$\text{DVDD18_IOBM} + 0.3$	V
VIL	Input logic low voltage	-0.3		$0.35 \cdot \text{DVDD18_IOBM}$	V
VOH	DC output logic high voltage	$0.75 \cdot \text{DVDD18_IOBM}$			V
VOL	DC Output logic low voltage			$0.25 \cdot \text{DVDD18_IOBM}$	V

2.2.4.3 I2C/I3C DC Electrical Characteristics

Table 2-8. I2C/I3C DC electrical characteristics (DVDD18_IOBM = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	$0.7 \cdot \text{DVDD18_IOBM}$		$1.1 \cdot \text{DVDD18_IOBM}$	V
VIL	Input logic low voltage	$-0.1 \cdot \text{DVDD18_IOBM}$		$0.3 \cdot \text{DVDD18_IOBM}$	V
VOL	DC output logic low voltage			0.27	V

2.2.4.4 I2C3 DC Electrical Characteristics

Table 2-9. I2C3 DC electrical characteristics (DVDD18_IOBM = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	$0.7 \cdot \text{DVDD18_IOBM}$		$\text{DVDD18_IOBM} + 0.5$	V
VIL	Input logic low voltage	-0.5		$0.3 \cdot \text{DVDD18_IOBM}$	V
VOL	DC output logic low voltage			$0.2 \cdot \text{DVDD18_IOBM}$	V

2.2.4.5 MSDC DC Electrical Characteristics

Table 2-10. MSDC1 DC electrical characteristics (DVDD28_MSDC1 = 2.9V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	$0.625 \cdot \text{DVDD28_MSDC1}$		$\text{DVDD28_MSDC1} + 0.3$	V
VIL	Input logic low voltage	-0.3		$0.25 \cdot \text{DVDD28_MSDC1}$	V
VOH	DC output logic high voltage	$0.75 \cdot \text{DVDD28_MSDC1}$		$\text{DVDD28_MSDC1} + 0.3$	V
VOL	DC output logic low voltage	-0.3		$0.125 \cdot \text{DVDD28_MSDC1}$	V

Table 2-11. MSDC1 DC electrical characteristics (DVDD28_MSDC1 = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	$0.65 \cdot \text{DVDD28_MSDC1}$		$\text{DVDD28_MSDC1} + 0.3$	V
VIL	Input logic low voltage	-0.3		$0.35 \cdot \text{DVDD28_MSDC1}$	V
VOH	DC output logic high voltage	$\text{DVDD28_MSDC1} - 0.45$		$\text{DVDD28_MSDC1} + 0.3$	V
VOL	DC output logic low voltage	-0.3		0.45	V

Table 2-12. MSDC2 DC electrical characteristics (DVDD18_MSDC2 = 1.8V)

Parameter	Description	Min.	Typ.	Max.	Unit
VIH	Input logic high voltage	0.7*DVDD18_MSDC2		DVDD18_MSDC2 + 0.3	V
VIL	Input logic low voltage	-0.3		0.3*DVDD18_MSDC2	V
VOH	DC output logic high voltage	1.4			V
VOL	DC output logic low voltage			0.45	V

2.2.4.6 SIM DC Electrical Characteristics

Table 2-13. SIM DC electrical characteristics

Parameter	Condition	Symbol	Min.	Typ.	Max.	Unit
SIM1_SIO						
Input high voltage	DVDD28_SIM1 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.4	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.27	V
Input high voltage	DVDD28_SIM1 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V
Output high voltage		V _{oh}	2.6	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.4	V
SIM1_SCLK						
Input high voltage	DVDD28_SIM1 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.62	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.22	V
Input high voltage	DVDD28_SIM1 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V
Output high voltage		V _{oh}	2.7	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.4	V
SIM1_SRST						
Input high voltage	DVDD28_SIM1 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.62	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.36	V
Input high voltage	DVDD28_SIM1 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V
Output high voltage		V _{oh}	2.7	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.36	V
SIM2_SIO						
Input high voltage	DVDD28_SIM2 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.4	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.27	V
Input high voltage	DVDD28_SIM2 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V

Parameter	Condition	Symbol	Min.	Typ.	Max.	Unit
Output high voltage		V _{oh}	2.6	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.4	V
SIM2_SCLK						
Input high voltage	DVDD28_SIM2 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.62	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.22	V
Input high voltage	DVDD28_SIM2 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V
Output high voltage		V _{oh}	2.7	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.4	V
SIM2_SRST						
Input high voltage	DVDD28_SIM2 = 1.8V	V _{ih}	1.4	1.8	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.27	V
Output high voltage		V _{oh}	1.62	1.8	1.9	V
Output low voltage		V _{ol}	N/A	0.0	0.36	V
Input high voltage	DVDD28_SIM2 = 3.0V	V _{ih}	2.6	3.0	N/A	V
Input low voltage		V _{il}	N/A	0.0	0.4	V
Output high voltage		V _{oh}	2.7	3.0	3.1	V
Output low voltage		V _{ol}	N/A	0.0	0.36	V

2.2.5 AC Electrical Characteristics and Timing Diagram

2.2.5.1 External Memory Interface for LPDDR5X

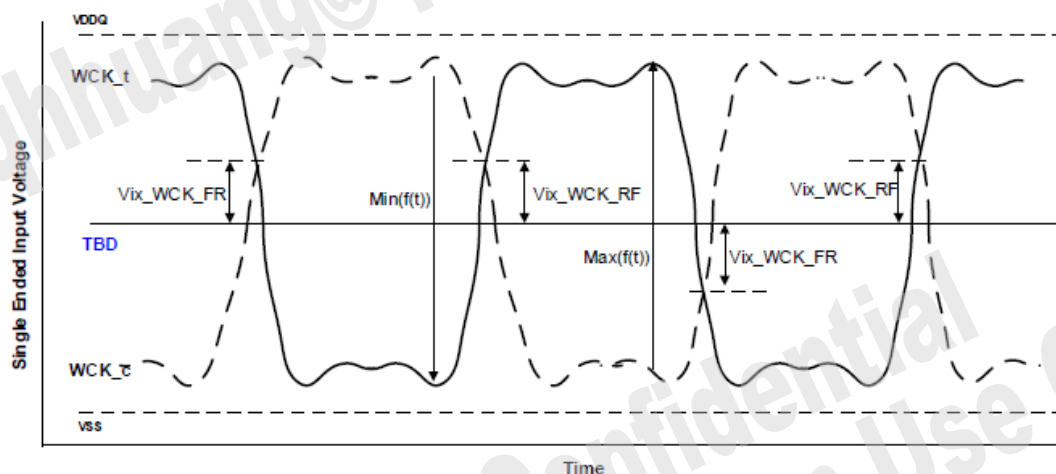


Figure 2-2. LPDDR5X WCK VIX definition

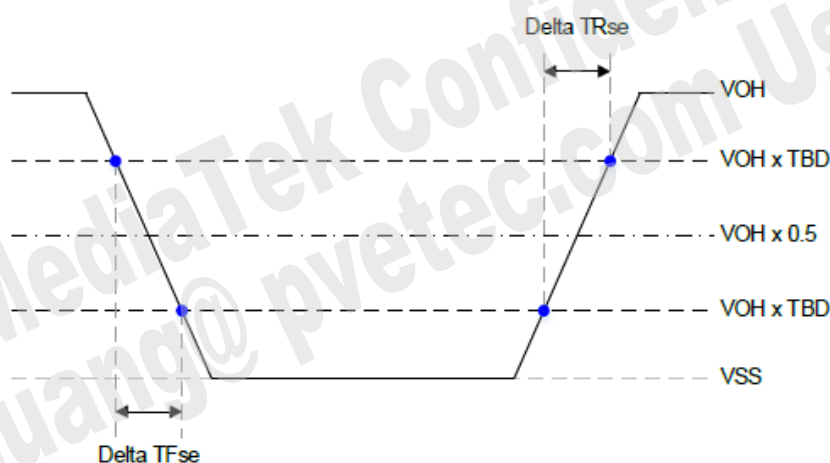


Figure 2-3. LPDDR5X single-ended output slew-rate definition

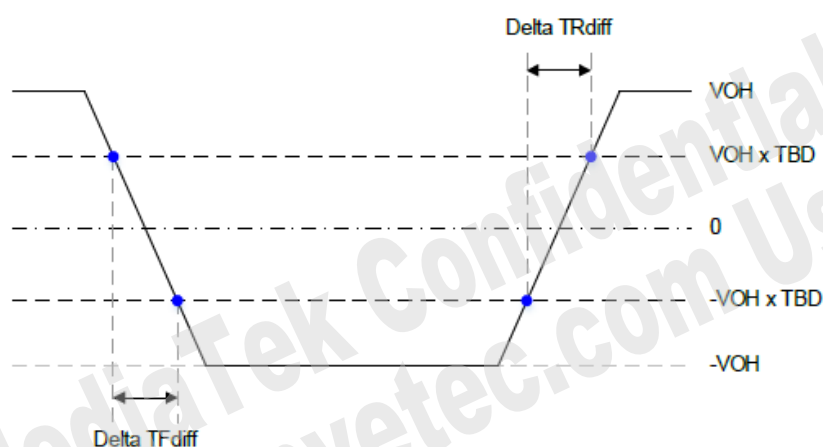


Figure 2-4. LPDDR5X differential output slew-rate definition

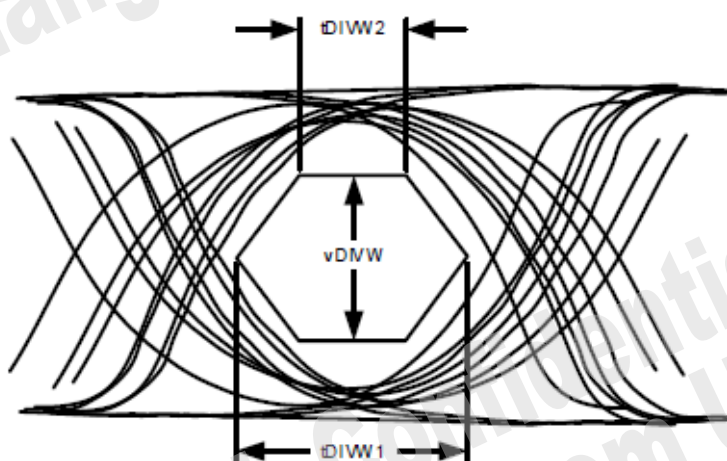


Figure 2-5. LPDDR5X Rx mask

Table 2-14. LPDDR5X AC timing parameter table of external memory interface

Symbol	Description	Min.	Typ.	Max.	Unit
Vix_WCK_ratio	WCK differential input crosspoint voltage ratio			20	%
Vix_CK_ratio	CK differential input crosspoint voltage ratio			25	%
vDIVW	DQ Rx mask height	80		100	mV
tDIVW1	DQ Rx mask width			0.35	UI
tDIVW2	DQ Rx mask width at vDIVW			0.18	UI

2.2.5.2 SPI AC Timing Characteristics

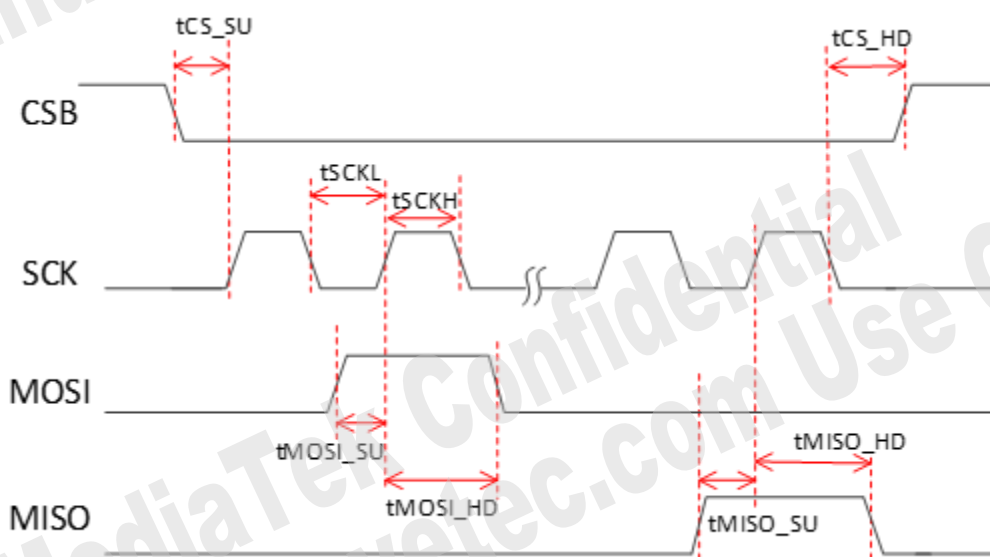


Figure 2-6. SPI timing diagram

Table 2-15. SPI AC timing parameters

Symbol	Description	Performance			Unit	Note
		Min.	Typ.	Max.		
fSCK	SPI master SCK clock frequency	-	-	52	MHz	
tMOSI_SU	MOSI to SCK rising setup time	6	-	-	ns	
tMOSI_HD	SCK rising to MOSI hold time	6	-	-	ns	
tSCKL	SCK low pulse	7.2	-	-	ns	
tSCKH	SCK high pulse	7.2	-	-	ns	
tCSB_SU ¹	CSB falling to SCK rising setup time	1.8	-	-	ns	
tCSB_HD ¹	SCK falling to CSB rising hold time	1.8	-	-	ns	
tMISO_SU ²	MISO to SCK rising setup time requirement	0	-	-	ns	
tMISO_HD ³	SCK rising to MISO hold time requirement	0	-	-	ns	

Notes:

1. In CS GPIO mode, SPI_CS is handled by software. Software should pull down SPI_CS pin before SPI starts transferring and pull up SPI_CS pin when SPI completes the transaction. Based on the sequence above, the minimum specification of tCSB_SU and tCSB_HD time cannot be satisfied.

2. To achieve the min. value of t_{MISO_SU} , the internal sample clock delay of SPI master should be adjusted.
3. MISO data valid time should be one cycle of f_{SCK} .
4. For dual mode or quad mode, all the output data pins can refer to the MOSI timing parameters, and all the input data pins can refer to the MISO timing parameters.

2.2.5.3 I2S AC Timing Characteristics

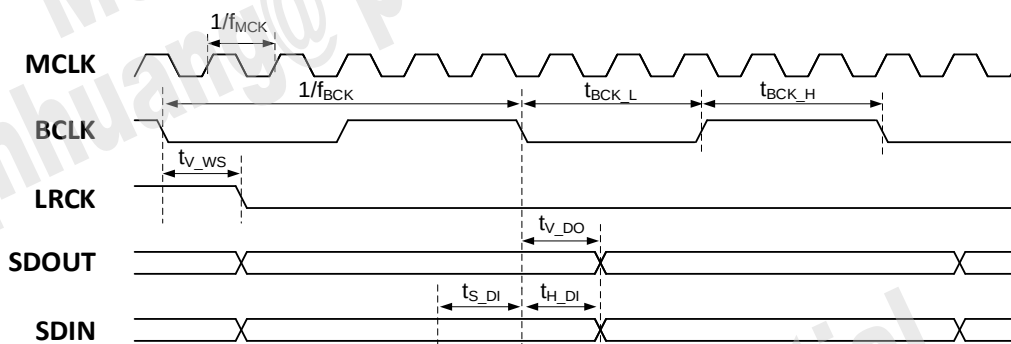


Figure 2-7. I2S master mode timing diagram

Table 2-16. I2S AC timing parameters

Parameter	Description	Min.	Typ.	Max.	Unit
f_s	Sampling frequency	8	-	192	kHz
f_{MCK}	Master clock frequency	0.768	-	49.152	MHz
f_{BCK}	Serial clock frequency	$32 * f_s$	-	$64 * f_s$	MHz
t_{BCK_H}	BCK high-level time	-	0.5	-	$1/f_{BCK}$
t_{BCK_L}	BCK low-level time	-	0.5	-	$1/f_{BCK}$
t_{v_ws}	Output delay time. BCLK low to LRCK valid	-	-	10	ns
t_{v_do}	Output delay time. BCLK low to SDOUT valid	-	-	10	ns
t_{S_DI}	DI setup time	18	-	-	ns
t_{H_DI}	DI hold time	18	-	-	ns

2.2.5.4 I2C AC Timing Characteristics

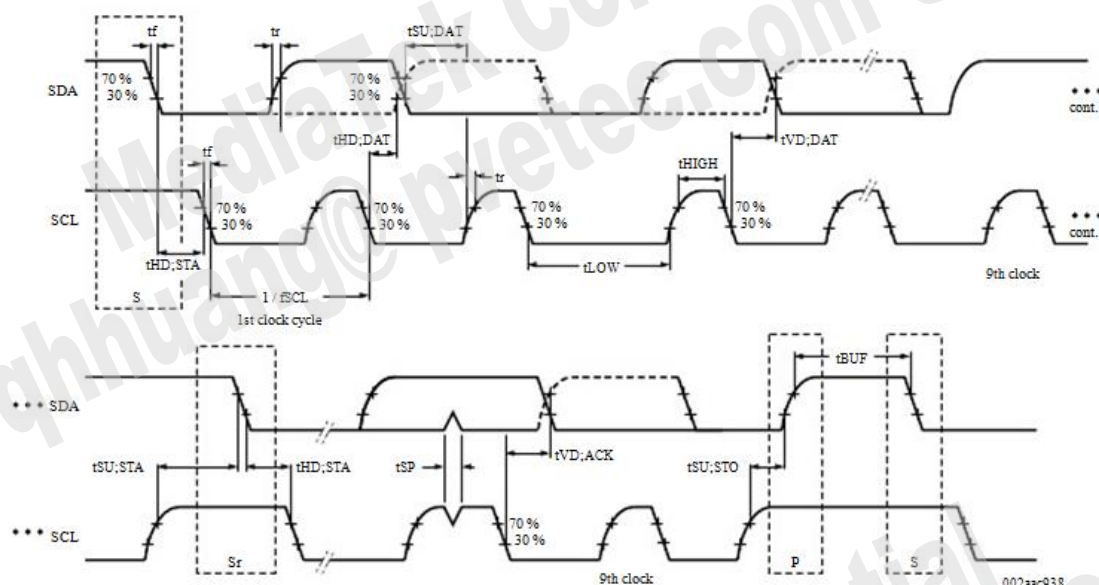


Figure 2-8. I2C timing diagram of standard mode (100 kHz) and fast mode (400 kHz)

Table 2-17. I2C AC timing parameters

Symbol	Parameter	Standard mode		Fast mode		Unit	Note
		Min.	Max.	Min.	Max.		
fSCL	SCL clock frequency	0	100	0	400	kHz	-
tHD;STA	Hold time (repeated) START condition	4.0	-	0.6	-	μs	-
tLOW	LOW period of the SCL clock	4.7	-	1.3	-	μs	-
tHIGH	HIGH period of the SCL clock	4.0	-	0.6	-	μs	-
tSU;STA	Set-up time for a repeated START condition	4.7	-	0.6	-	μs	-
tHD;DAT	Data hold time	5.0	-	0	-	μs	I2C-bus devices
tSU;DAT	Data set-up time	250	-	100	-	ns	-
tr	Rise time of both SDA and SCL signals	-	1000	20	300	ns	-
tf	Fall time of both SDA and SCL signals	-	300	20x (VDD/5.5V)	300	ns	VDD is I2C I/O voltage.
tSU;STO	Set-up time for STOP condition	4.0	-	0.6	-	ns	-
tVD;DAT	Data valid time	-	3.45	-	0.9	μs	-
tVD;ACK	Data valid acknowledgement time	-	3.45	-	0.9	μs	-

2.2.5.5 MSDC AC Timing Characteristics

2.2.5.5.1 Default Speed Timing

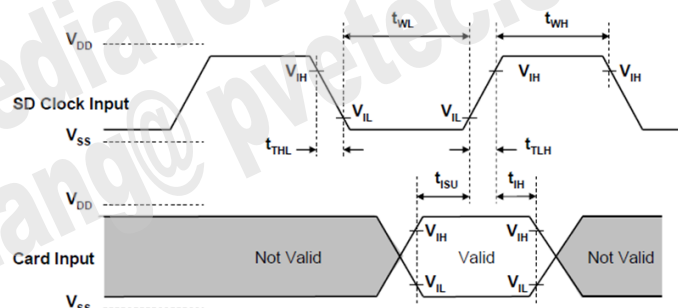


Figure 2-9. MSDC device input timing diagram of default speed

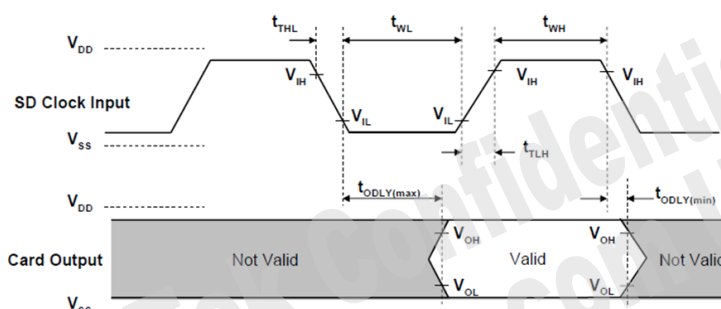


Figure 2-10. MSDC device output timing diagram of default speed

Table 2-18. MSDC device AC timing parameters of default speed

Parameter	Symbol	Min.	Max.	Unit	Remark
Clock (CLK) (All values are referred to min(V_{IH}) and max(V_{IL}).					
Clock frequency data transfer mode	f _{PP}	0	25	MHz	C _{CARD} ≤ 10 pF (1 card)
Clock frequency identification mode	f _{OD}	0/100	400	kHz	C _{CARD} ≤ 10 pF (1 card)
Clock low time	t _{WL}	10		ns	C _{CARD} ≤ 10 pF (1 card)
Clock high time	t _{WH}	10		ns	C _{CARD} ≤ 10 pF (1 card)
Clock rise time	t _{TLH}		10	ns	C _{CARD} ≤ 10 pF (1 card)
Clock fall time	t _{THL}		10	ns	C _{CARD} ≤ 10 pF (1 card)
Input CMD, DAT (referenced to CLK)					
Input setup time	T _{ISU}	5		ns	C _{CARD} ≤ 10 pF (1 card)
Input hold time	T _{IH}	5		ns	C _{CARD} ≤ 10 pF (1 card)
Output CMD, DAT (referenced to CLK)					
Output delay time during data transfer mode	T _{ODLY}	0	14	ns	C _L ≤ 40 pF (1 card)
Output delay time during identification mode	T _{ODLY}	0	50	ns	C _L ≤ 40 pF (1 card)

2.2.5.5.2 High-Speed Timing

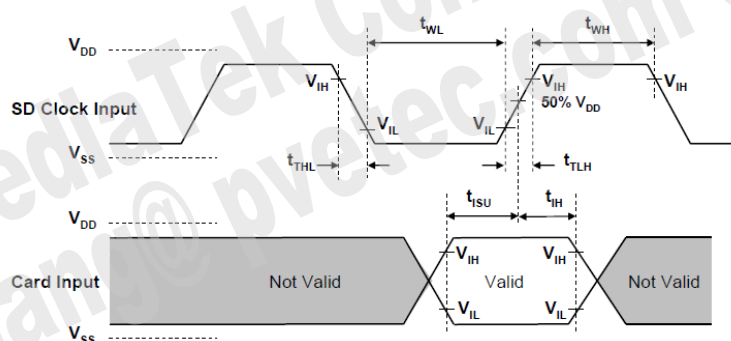


Figure 2-11. MSDC device input timing diagram of high-speed

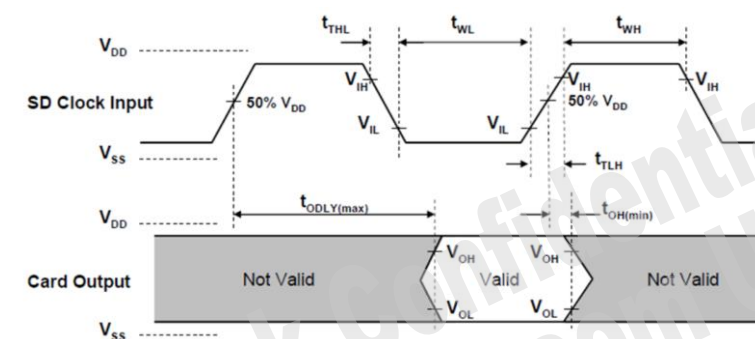


Figure 2-12. MSDC device output timing diagram of high-speed

Table 2-19. MSDC device AC timing parameters of high-speed

Parameter	Symbol	Min.	Max.	Unit	Remark
Clock (CLK) (All values are referred to min(V_{IH}) and max(V_{IL})).					
Clock frequency data transfer mode	f_{PP}	0	50	MHz	$C_{CARD} \leq 10$ pF (1 card)
Clock low time	t_{WL}	7		ns	$C_{CARD} \leq 10$ pF (1 card)
Clock high time	t_{WH}	7		ns	$C_{CARD} \leq 10$ pF (1 card)
Clock rise time	t_{TLH}		3	ns	$C_{CARD} \leq 10$ pF (1 card)
Clock fall time	t_{THL}		3	ns	$C_{CARD} \leq 10$ pF (1 card)
Input CMD, DAT (referenced to CLK)					
Input setup time	t_{ISU}	6		ns	$C_{CARD} \leq 10$ pF (1 card)
Input hold time	t_{IH}	2		ns	$C_{CARD} \leq 10$ pF (1 card)
Output CMD, DAT (referenced to CLK)					
Output delay time during data transfer mode	t_{ODLY}		14	ns	$C_L \leq 40$ pF (1 card)
Output hold time	t_{OH}	2.5		ns	$C_L \geq 15$ pF (1 card)
Total system capacitance for each line	C_L		40	pF	1 card

2.2.5.5.3 SDR12/SDR25/SDR50/SDR104 Mode Timing

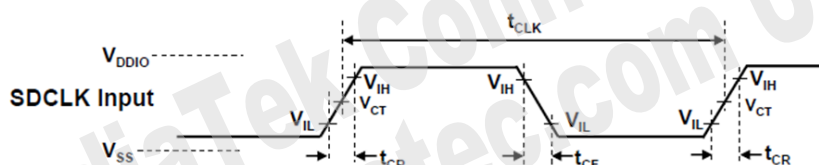


Figure 2-13. MSDC device clock timing diagram of SDR12/SDR25/SDR50/SDR104 modes

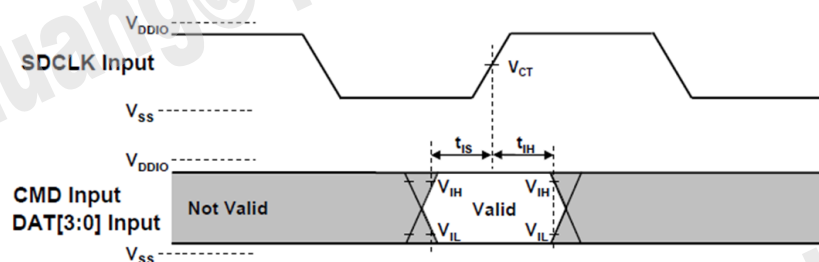


Figure 2-14. MSDC device input timing diagram of SDR50/SDR104 modes

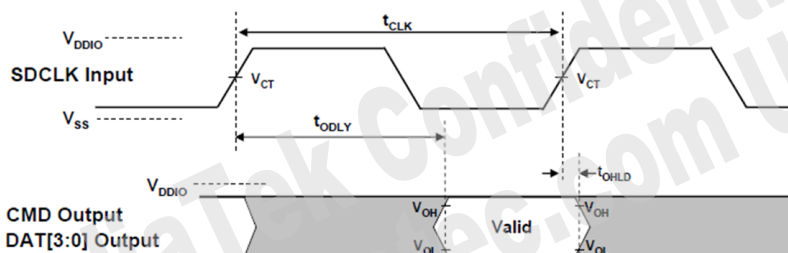


Figure 2-15. MSDC device output timing diagram of fixed data window (SDR12/SDR25/SDR50)

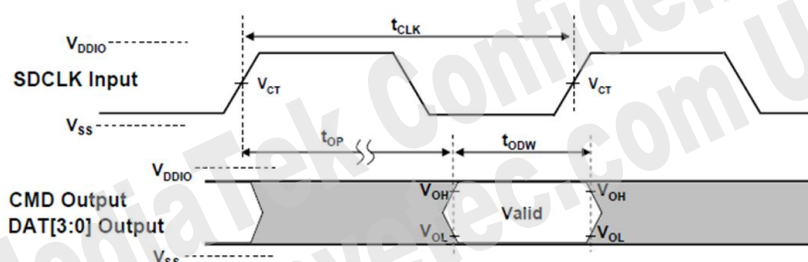


Figure 2-16. MSDC device output timing diagram of variable window (SDR104)

Table 2-20. MSDC device AC timing parameters of SDR12/SDR25/SDR50/SDR104 modes

Symbol	Min.	Max.	Unit	Remark
Clock (CLK)				
t_{CLK}	4.8	-	ns	208 MHz (max.), between rising edge, $V_{CT} = 0.975V$
t_{CR}, t_{CF}	-	$0.2 \cdot t_{CLK}$	ns	$t_{CR}, t_{CF} < 0.96$ ns (max.) at 208 MHz, $C_{CARD} = 10$ pF $t_{CR}, t_{CF} < 2.00$ ns (max.) at 100 MHz, $C_{CARD} = 10$ pF The absolute maximum value of t_{CR}, t_{CF} is 10 ns regardless of clock frequency.
Clock duty	30	70	%	
Input CMD, DAT (SDR104)				
t_{IS}	1.40	-	ns	$C_{CARD} = 10$ pF, $V_{CT} = 0.975V$
t_{IH}	0.80	-	ns	$C_{CARD} = 5$ pF, $V_{CT} = 0.975V$
Input CMD, DAT (SDR50)				
t_{IS}	3.00	-	ns	$C_{CARD} = 10$ pF, $V_{CT} = 0.975V$
t_{IH}	0.80	-	ns	$C_{CARD} = 5$ pF, $V_{CT} = 0.975V$
Output CMD, DAT (SDR12/SDR25/SDR50)				
t_{ODLY}	-	7.5	ns	$t_{CLK} \geq 10.0$ ns, $C_L = 30$ pF, using driver type B, for SDR50
t_{ODLY}	-	14	ns	$t_{CLK} \geq 20.0$ ns, $C_L = 40$ pF, using driver type B, for SDR25 and SDR12
T_{OH}	1.5	-	ns	Hold time at the t_{ODLY} (min.), $C_L = 15$ pF
Output CMD, DAT (SDR104)				
t_{OP}	0	2	UI	Card output phase
Δt_{OP}	-350	+1,550	ps	Delay variation due to temperature change (after tuning).
t_{ODW}	0.6	-	UI	$t_{ODW} = 2.88$ ns at 208 MHz

2.2.5.5.4 DDR50 Speed Mode Timing

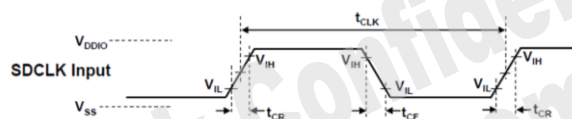


Figure 2-17. MSDC device clock timing diagram of DDR50 speed mode

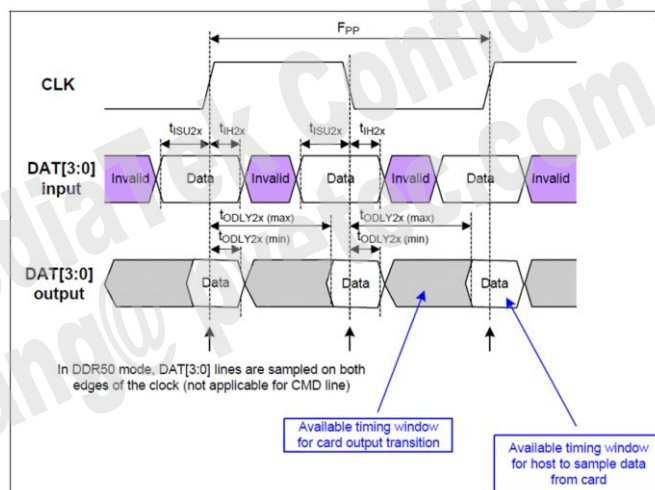


Figure 2-18. MSDC device input/output timing diagram of DDR50 speed mode

Table 2-21. MSDC device AC timing parameters of DDR50 speed mode

Parameter	Symbol	Min.	Max.	Unit	Remark
Input CMD (referenced to CLK rising edge)					
Input setup time	t_{ISU}	6	-	ns	$C_{CARD} \leq 10 \text{ pF}$ (1 card)
Input hold time	t_{IH}	0.8	-	ns	$C_{CARD} \leq 10 \text{ pF}$ (1 card)
Output CMD (referenced to CLK rising edge)					
Output delay time during data transfer mode	t_{ODLY}	-	13.7	ns	$C_L \leq 30 \text{ pF}$ (1 card)
Output hold time	t_{OH}	1.5	-	ns	$C_L \geq 15 \text{ pF}$ (1 card)
Input DAT (referenced to CLK rising and falling edge)					
Input setup time	t_{ISU}	3	-	ns	$C_{CARD} \leq 10 \text{ pF}$ (1 card)
Input hold time	t_{IH}	0.8	-	ns	$C_{CARD} \leq 10 \text{ pF}$ (1 card)
Output DAT (referenced to CLK rising and falling edge)					
Output delay time during data transfer mode	t_{ODLY}	-	7.0	ns	$C_L \leq 25 \text{ pF}$ (1 card)
Output hold time	t_{OH}	1.5	-	ns	$C_L \geq 15 \text{ pF}$ (1 card)

2.2.5.5.5 HS200 Speed Timing

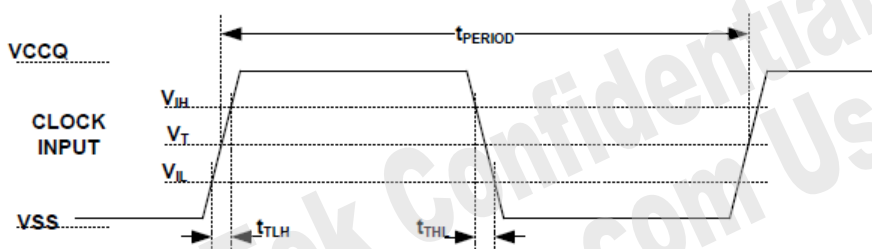
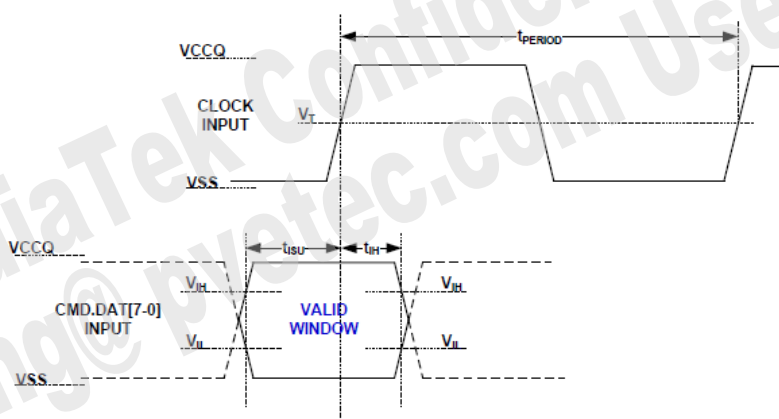


Figure 2-19. MSDC device clock timing diagram of HS200



NOTE 1 t_{ISU} and t_{IH} are measured at $V_{IL(max.)}$ and $V_{IH(min.)}$.

NOTE 2 V_{IH} denote $V_{IH(min.)}$ and V_{IL} denotes $V_{IL(max.)}$.

Figure 2-20. MSDC device input timing diagram of HS200

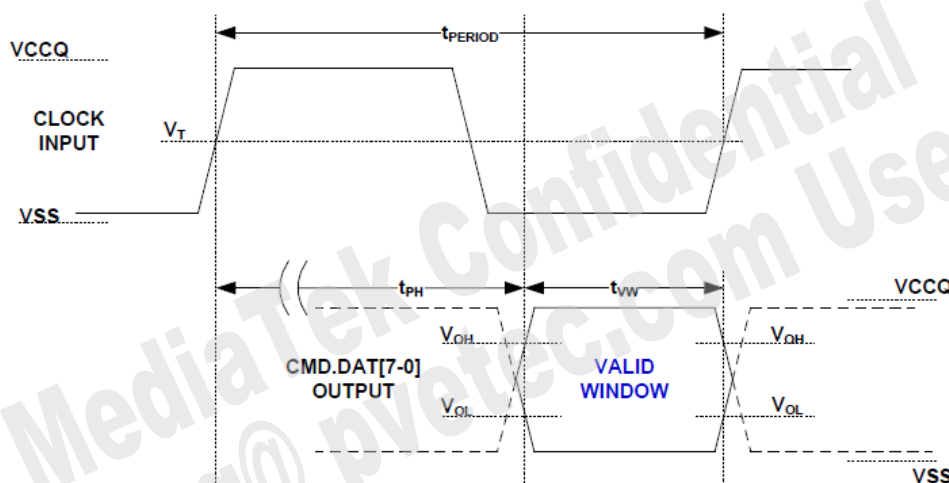


Figure 2-21. MSDC device output timing diagram of HS200

Table 2-22. MSDC device AC timing parameters of HS200

Symbol	Min.	Max.	Unit	Remark
Clock (CLK)				
t_{PERIOD}	5	-	ns	200 MHz (max.), between rising edge
t_{TLH}, t_{THL}	-	$0.2 * t_{PERIOD}$	ns	$t_{TLH}, t_{THL} < 1$ ns (max.) at 200 MHz, $C_{DEVICE} = 6$ pF The absolute maximum value of t_{TLH}, t_{THL} is 10 ns regardless of clock frequency.
Clock duty	30	70	%	
Input CMD, DAT				
t_{ISU}	1.40	-	ns	$C_{DEVICE} \leq 6$ pF
t_{IH}	0.80	-	ns	$C_{DEVICE} \leq 6$ pF
Output CMD, DAT				
t_{PH}	0	2	UI	Device output momentary phase from CLK input to CMD or DAT lines output. Does not include a long-term temperature drift.
ΔT_{PH}	-350	+1,550	ps	Delay variation due to temperature change (after tuning). Total allowable shift of output valid window

Symbol	Min.	Max.	Unit	Remark
	($\Delta T = -20^{\circ}\text{C}$)	($\Delta T = 90^{\circ}\text{C}$)		(T_{VW}) from last system tuning procedure ΔT_{PH} is 2,600 ps for ΔT from -25°C to 125°C during operation.
t_{VW}	0.575	-	UI	$t_{VW} = 2.88$ ns at 208 MHz
Note: Unit interval (UI) is one-bit nominal time. For example, UI = 5 ns at 200 MHz.				

2.2.5.6 SIM AC Timing Characteristics

Table 2-23. SIM AC timing parameters

Parameter	Condition	Symbol	Min.	Typ.	Max.	Unit
SIM1_SCLK						
Rise and fall time	DVDD28_SIM1 = 1.8V	T _{rise_fall}	N/A	50	50	ns
Clock duty		Duty	47	50	53	%
Rise and fall time	DVDD28_SIM1 = 3.0V	T _{rise_fall}	N/A	18	18	ns
Clock duty		Duty	47	50	53	%
SIM1_SIO						
Rise and fall time	DVDD28_SIM1 = 1.8V	T _{rise_fall}	N/A	50	1,000	ns
Rise and fall time	DVDD28_SIM1 = 3.0V	T _{rise_fall}	N/A	50	1,000	ns
SIM1_SRST						
Rise and fall time	DVDD28_SIM1 = 1.8V	T _{rise_fall}	N/A	18	1,000	ns
Rise and fall time	DVDD28_SIM1 = 3.0V	T _{rise_fall}	N/A	18	1,000	ns
SIM2_SCLK						
Rise and fall time	DVDD28_SIM2 = 1.8V	T _{rise_fall}	N/A	50	50	ns
Clock duty		Duty	47	50	53	%
Rise and fall time	DVDD28_SIM2 = 3.0V	T _{rise_fall}	N/A	18	18	ns
Clock duty		Duty	47	50	53	%
SIM2_SIO						
Rise and fall time	DVDD28_SIM2 = 1.8V	T _{rise_fall}	N/A	50	1,000	ns
Rise and fall time	DVDD28_SIM2 = 3.0V	T _{rise_fall}	N/A	50	1,000	ns
SIM2_SRST						
Rise and fall time	DVDD28_SIM2 = 1.8V	T _{rise_fall}	N/A	18	1,000	ns
Rise and fall time	DVDD28_SIM2 = 3.0V	T _{rise_fall}	N/A	18	1,000	ns

2.2.6 Clock Squarer

2.2.6.1 Block Description

The clock squarer is designed to receive clock signal from pin “X26M_IN” and distribute it to the chip internally.

2.2.6.2 Functional Specifications

See the following table for the functional specifications of clock squarer of pin “X26M_IN”.

Table 2-24. Clock squarer specifications

Symbol	Parameter	Min.	Typ.	Max.	Unit
Vin	Input signal amplitude	1,000	1,200	1,250	mVpp
DcyclN	Input signal duty cycle		50		%
DcyclOUT	Output signal duty cycle	DcyclN - 5		DcyclN + 5	%
	Maximum positive overshoot			1.3	V
	Minimum negative overshoot	-0.1			V

2.3 System Configuration

2.3.1 Mode Selection

Table 2-25. Mode selection

Ball name	Description
KPCOLO	0: Force USB download mode in bootrom 1: NA (default)
SCP_VREQ_VAO	0: Default 1: Enable MD_JTAG
[0] AUD_DAT_MOSI0 [1] AUD_DAT_MOSI1	00: Boot from UFS3.1-26M 01: Boot from UFS4.0-LS 10: Boot from UFS4.0-HS 11: Boot from UFS3.1-52M
[0] SCP_EINT0 [1] AUD_DAT_MOSI2	00: LP5X 01: Reserved for future use 10: Reserved for future use 11: Reserved for future use

2.3.2 Constant Tied Pins

Table 2-26. Constant tied pins

Ball name	Description
TESTMODE	Test mode (tied to GND)

2.4 Power-on Sequence

Refer to the PMIC datasheet for detailed timing sequence.

2.5 Package Information

2.5.1 Package Outlines

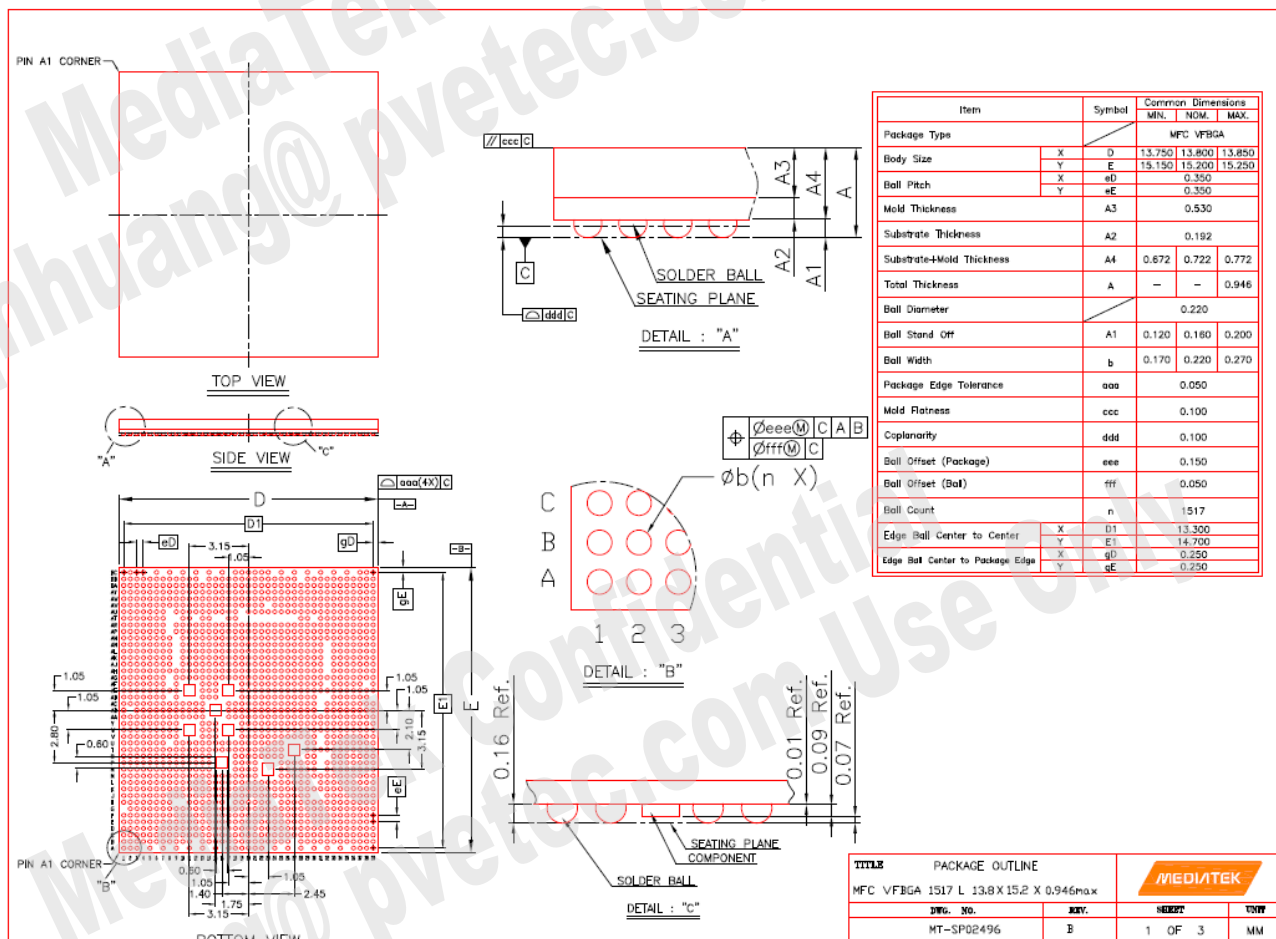


Figure 2-22. Outlines and dimensions of VFBGA 13.8 mm x 15.2 mm package

2.5.2 Ordering Information

Order#	Marking	Temp. range	Package
MT8676	See 2.5.3	-40 ~ 85°C	MFC VFBGA

2.5.3 Top Marking Definition

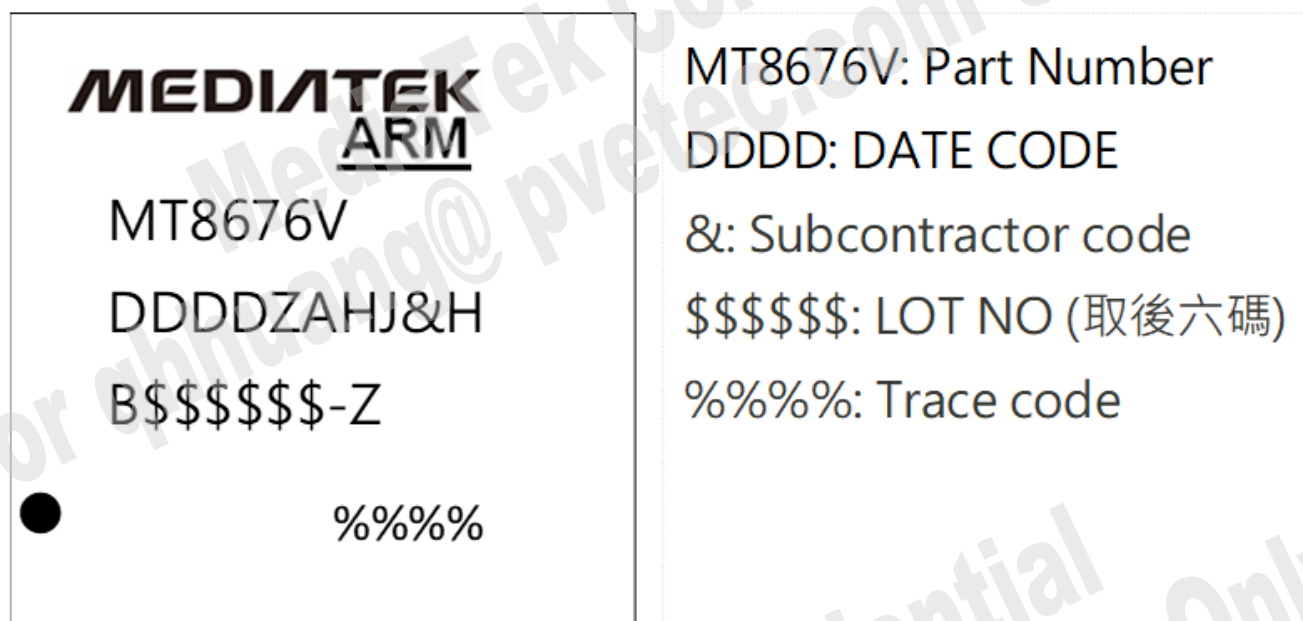


Figure 2-23. Top marking of MT8676

2.5.4 Thermal Operating Specifications

Table 2-27. Thermal operating specifications

Symbol	Description	Value	Unit	Note
	Max. operating junction temperature	125	°C	
	Package junction to case top thermal resistance	0.86	°C/Watt	

2.5.5 Lead-free Packaging

The chip is provided in a lead-free package and meets RoHS requirements.

2.5.6 MSL

The MSL of this chip is 3.

3 MCU and Bus Fabric

3.1 Microcontroller Unit System (MCUSYS)

3.1.1 Introduction

The microcontroller unit system (MCUSYS) is a subsystem responsible for running the operating system and application programs. It comprises two different types of CPU, including four Cortex-A510 cores and four Cortex-A710 cores, into a single cluster (DynamIQ). It also provides different levels of power efficiency and computing power to satisfy a wide range of system power and performance requirements. The power efficiency of four little cores (Cortex-A510) is specially optimized to minimize the power consumption in daily usage scenarios and lightweight applications. For performance driven applications, the combination of powerful three Cortex-A710 and one high-speed Cortex-A710 are capable of handling heavy tasks and providing the best user experiences.

The MCUSYS supports DVFS technology which allows the CPU to run at different frequencies and voltage configurations for different application requirements. Besides DVFS, the power of each CPU core can be turned off individually when not used. In the standby mode, the MCUSYS can be completely shut down to further save power consumption and optimize the battery usage on mobile devices.

3.1.2 Features

3.1.2.1 Little Cores, Cortex-A510 Specifications

- 4-core Arm® Cortex-A510 MPCore™
- Supports Armv9 architecture for both 32-bit and 64-bit execution states
- Armv9 extensions, Armv9 extensions, cryptography extensions, RAS extensions, Armv9 (LDAPR instructions only)
- Supports scalable vector extension (SVE) and SVE2 SIMD instruction set, offering advanced SIMD and floating-point architecture
- 32 KB L1 I-cache and 64 KB L1 D-cache
- 128 KB private L2 cache for each CPU
- DVFS technology with adaptive operating voltage

3.1.2.2 Big Cores, Cortex-A710 Specifications

- 4-core Arm® Makalu MPCore™
- Arm v9.0 architecture with 64-bit execution states
- Cryptographic and RAS extensions
- Supports SVE and SVE2
- Supports advanced SIMD and floating-point
- MPAM – Memory system partitioning and monitoring
- 64 KB L1 I-cache and 64 KB L1 D-cache
- 512 KB private L2 per core

- DVFS technology with adaptive operating voltage

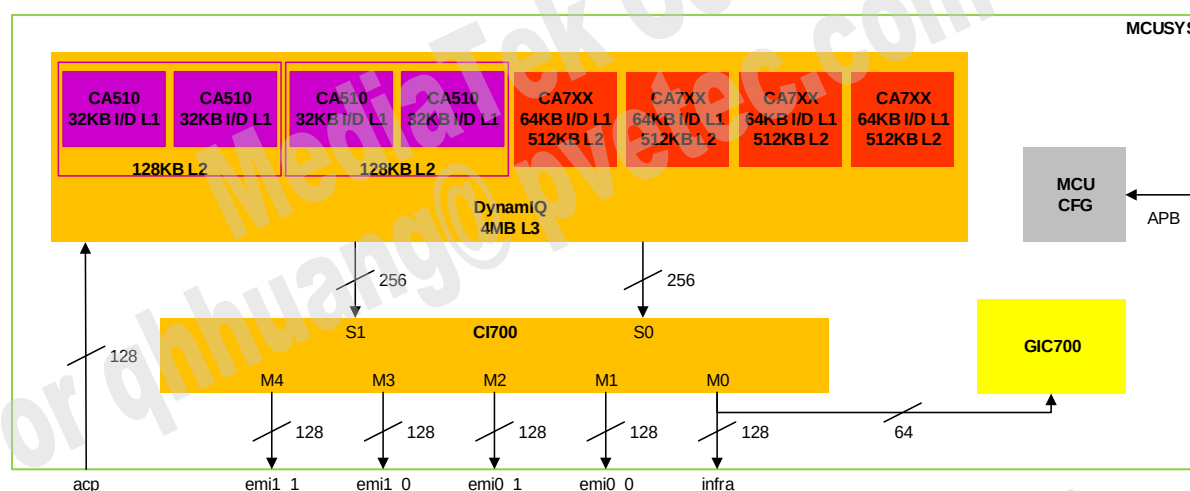
3.1.2.3 DynamIQ Specifications

- Arm® DynamIQ™ shared unit
- A combination of high-performance “big” CPUs and high-efficiency “little” CPUs in 1 cluster, with a shared coherent memory
- 8 MB shared memory for CPUs
- DVFS technology with adaptive operating voltage

3.1.2.4 Clock Modes between DynamIQ and AXI Bus Fabric

The DynamIQ and AXI bus fabric are synchronous. The AXI bus fabric also supports the dynamic clock management (DCM) mechanism to dynamically turn off the clock when no transaction is on the bus interface. The CPU, DynamIQ and AXI bus fabric all support DVFS technology to lower the power consumption.

3.1.3 Block Diagram



3.1.4 Interrupt Controller

MT8676 uses the Arm GIC700 interrupt controller for interrupt management. GIC700 is embedded inside the MCUSYS alongside BOOKER to minimize the interrupt handling latency. For the interrupt connected to GIC700, see the following table for details. The GIC interrupts are separated into two categories, the private peripheral interrupts (PPI) and shared peripheral interrupts (SPI). The PPI occupies the first 32 interrupt slots in GIC and are banked for each CPU core. The SPI begins from the 33rd interrupt and is shared by all CPU cores.

Table 3-1. Interrupt request list for GIC700

GIC ID	Interrupt source/name	Polarity	Trigger type
0	Software generated interrupt 0	H	Edge
1	Software generated interrupt 1	H	Edge
2	Software generated interrupt 2	H	Edge
3	Software generated interrupt 3	H	Edge
4	Software generated interrupt 4	H	Edge
5	Software generated interrupt 5	H	Edge
6	Software generated interrupt 6	H	Edge
7	Software generated interrupt 7	H	Edge
8	Software generated interrupt 8	H	Edge
9	Software generated interrupt 9	H	Edge
10	Software generated interrupt 10	H	Edge
11	Software generated interrupt 11	H	Edge
12	Software generated interrupt 12	H	Edge
13	Software generated interrupt 13	H	Edge
14	Software generated interrupt 14	H	Edge
15	Software generated interrupt 15	H	Edge
16	(Reserved)	-	-
17	(Reserved)	-	-
18	Hypervisor secure physical timer event	L	Level
19	Hypervisor secure virtual timer event	L	Level
20	Trace buffer extension interrupt request	L	Level
21	(Reserved)	-	-
22	Comms channel receive/transmit interrupt request	L	Level
23	PMU interrupt request	L	Level
24	CTI interrupt	H	Level
25	Virtual CPU interface maintenance interrupt	L	Level
26	Hypervisor physical timer event	L	Level
27	Virtual physical timer event	L	Level
28	Hypervisor virtual timer event	L	Level
29	Secure physical timer event	L	Level
30	Non-secure physical timer event	L	Level
31	(Reserved)	-	-
32	nCOREFAULTIRQ[0]	L	Level
33	nCOREFAULTIRQ[1]	L	Level
34	nCOREFAULTIRQ[2]	L	Level
35	nCOREFAULTIRQ[3]	L	Level
36	nCOREFAULTIRQ[4]	L	Level
37	nCOREFAULTIRQ[5]	L	Level
38	nCOREFAULTIRQ[6]	L	Level
39	nCOREFAULTIRQ[7]	L	Level
40	nCOMPLEXFAULTIRQ[0]	L	Level
41	nCOMPLEXFAULTIRQ[1]	L	Level
42	nCOMPLEXFAULTIRQ[2]	L	Level
43	nCOMPLEXFAULTIRQ[3]	L	Level

GIC ID	Interrupt source/name	Polarity	Trigger type
44	nCOMPLEXFAULTIRQ[4]	L	Level
45	nCOMPLEXFAULTIRQ[5]	L	Level
46	nCOMPLEXFAULTIRQ[6]	L	Level
47	nCOMPLEXFAULTIRQ[7]	L	Level
48	nCLUSTERFAULTIRQ	L	Level
49	nCOREERRIRQ[0]	L	Level
50	nCOREERRIRQ[1]	L	Level
51	nCOREERRIRQ[2]	L	Level
52	nCOREERRIRQ[3]	L	Level
53	nCOREERRIRQ[4]	L	Level
54	nCOREERRIRQ[5]	L	Level
55	nCOREERRIRQ[6]	L	Level
56	nCOREERRIRQ[7]	L	Level
57	nCOMPLEXERRIRQ[0]	L	Level
58	nCOMPLEXERRIRQ[1]	L	Level
59	nCOMPLEXERRIRQ[2]	L	Level
60	nCOMPLEXERRIRQ[3]	L	Level
61	nCOMPLEXERRIRQ[4]	L	Level
62	nCOMPLEXERRIRQ[5]	L	Level
63	nCOMPLEXERRIRQ[6]	L	Level
64	nCOMPLEXERRIRQ[7]	L	Level
65	nCLUSTERERRIRQ	L	Level
66	nCLUSTERCRITIRQ	L	Level
67	nCLUSTERPMUIRQ	L	Level
68	nMPAMSIRQ	L	Level
69	nMPAMNSIRQ	L	Level
70	CTIIRQ_CLUSTER	H	Level
71	INTREQPMU	H	Level
72	INTREQFAULTS	H	Level
73	INTREQFAULTNS	H	Level
74	INTREQERRS	H	Level
75	INTREQERRNS	H	Level
76	sec_vio_abort_n	L	Level
77	cpc_interrupt_n	L	Level
78	bus_parity_fail_irq	H	Level
79	cpueb_wdt_irq	H	Level
80	cpueb_irq[0]	H	Level
81	cpueb_irq[1]	H	Level
82	cpueb_irq[2]	H	Level
83	cpueb_irq[3]	H	Level
84	cpueb_irq[4]	H	Level
85	cpueb_irq[5]	H	Level
86	cpueb_irq[6]	H	Level
87	cpueb_irq[7]	H	Level
88	cpueb_irq[8]	H	Level

GIC ID	Interrupt source/name	Polarity	Trigger type
89	cpueb_irq[9]	H	Level
90	cpueb_irq[10]	H	Level
91	cpueb_irq[11]	H	Level
92	cpueb_irq[12]	H	Level
93	cpueb_irq[13]	H	Level
94	cpueb_irq[14]	H	Level
95	cpueb_irq[15]	H	Level
96	ses_irq[0]	H	Level
97	ses_irq[1]	H	Level
98	ses_irq[2]	H	Level
99	ses_irq[3]	H	Level
100	ses_irq[4]	H	Level
101	ses_irq[5]	H	Level
102	ses_irq[6]	H	Level
103	ses_irq[7]	H	Level
104	mpsys_drfc_irq[0]	H	Level
105	mpsys_drfc_irq[1]	H	Level
106	mpsys_drfc_irq[2]	H	Level
107	core0_drfc_irq[0]	H	Level
108	core0_drfc_irq[1]	H	Level
109	core0_drfc_irq[2]	H	Level
110	core1_drfc_irq[0]	H	Level
111	core1_drfc_irq[1]	H	Level
112	core1_drfc_irq[2]	H	Level
113	core2_drfc_irq[0]	H	Level
114	core2_drfc_irq[1]	H	Level
115	core2_drfc_irq[2]	H	Level
116	core3_drfc_irq[0]	H	Level
117	core3_drfc_irq[1]	H	Level
118	core3_drfc_irq[2]	H	Level
119	core4_drfc_irq[0]	H	Level
120	core4_drfc_irq[1]	H	Level
121	core4_drfc_irq[2]	H	Level
122	core5_drfc_irq[0]	H	Level
123	core5_drfc_irq[1]	H	Level
124	core5_drfc_irq[2]	H	Level
125	core6_drfc_irq[0]	H	Level
126	core6_drfc_irq[1]	H	Level
127	core6_drfc_irq[2]	H	Level
128	core7_drfc_irq[0]	H	Level
129	core7_drfc_irq[1]	H	Level
130	core7_drfc_irq[2]	H	Level
131	cmo_setway_mon_hint_all	H	Level
132	apb_dbg_int_out	H	Level
133	ubm_irq	H	Level

3.1.5 Register Definition

See chapter 1.1 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.2 On-Chip Memory Controller

3.2.1 Introduction

The on-chip memory controller provides the boot ROM (read only memory) and SRAM (static random-access memory) resources.

3.2.2 Features

The memory controller has the following features:

- 256 KB on-chip ROM, with memory access protection and detection
- 256 KB on-chip SRAM, with memory access protection and detection
- Chip ID

The following table is the memory map of on-chip ROM and SRAM.

Table 3-2. Memory map of on-chip memory controller

Bank	Start address	End address	Size	Device
0	0x0000_0000	0x0003_FFFF	256 KB	ROM
	0x0010_0000	0x0013_FFFF	256 KB	SRAM

3.2.3 Block Diagram

The on-chip memory controller consists of an SRAM controller, a ROM controller, an AXI-FPC bus bridge, three bus interface units, a setting register and a chip ID unit (see [Figure 3-1](#)). Detailed functionality is described in the following sections.

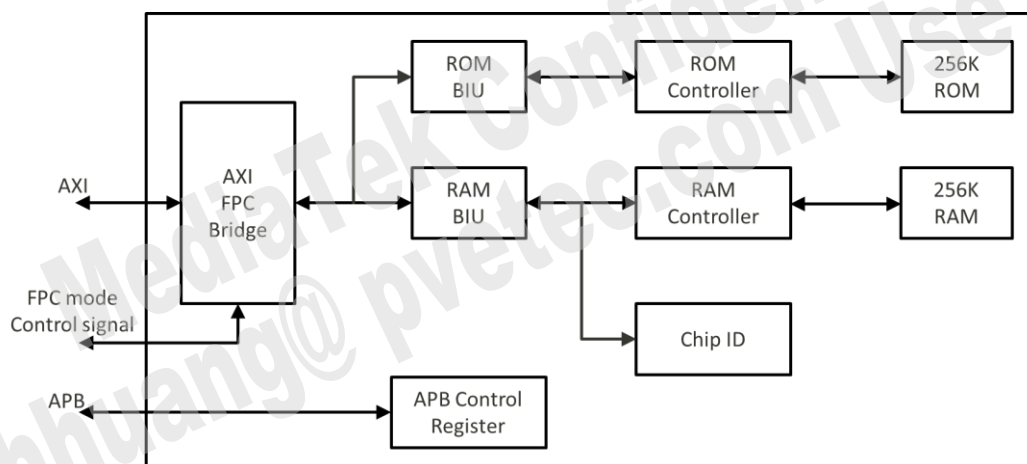


Figure 3-1. Block diagram of on-chip memory controller

3.2.3.1 Boot ROM Power Down Mode

The “boot ROM power down” mode (also referred as “power down mode”) is used in the following scenarios.

1. After the system boot, the boot ROM is powered down and prevented from any probe of ROM content.
2. In MCDI (multi-core-deep-idle), it is the bootstrap for the suspend/resume CPU.

The power down mode can be entered by setting `SRAMROM_SEC_CTRL.sramrom_sw_rom_pd_reg = 1`. The bus interface unit returns a far jump instruction when receiving the read transactions. The jump address offset is always zero, so the program enters an infinite loop until being reset.

The power down mode can also be entered by discounting specified timing cycles. The amount of timing cycles can be specified by setting `SRAMROM_BOOT_ADDR` register. Once the counter reaches zero, the SRAMROM automatically enters the power down mode.

3.2.3.2 Boot ROM FPC Mode

The “boot ROM FPC (function pattern code)” mode is mainly used in the function pattern mode. When the chip is trapped into the FPC mode, the AXI-FPC bridge blocks all the transactions to the ROM address by returning a far jump instruction with jump address specified in `SRAMROM_FPC_BOOT_ADDR` of the default value of 0. The AXI-FPC bridge then automatically unblocks the transaction when the FPC program is downloaded to the SRAM memory address space.

3.2.3.3 On-Chip SRAM Security Protection

The on-chip SRAM can be partitioned into four regions with different security protection configurations. The bus interface unit performs the permission check based on the settings, records the first violated address in the `SEC_VIO_ADDR` registers, and issues interrupts to the host processor. The violation interrupt can be cleared by a write to `SEC_VIO_ACK`.

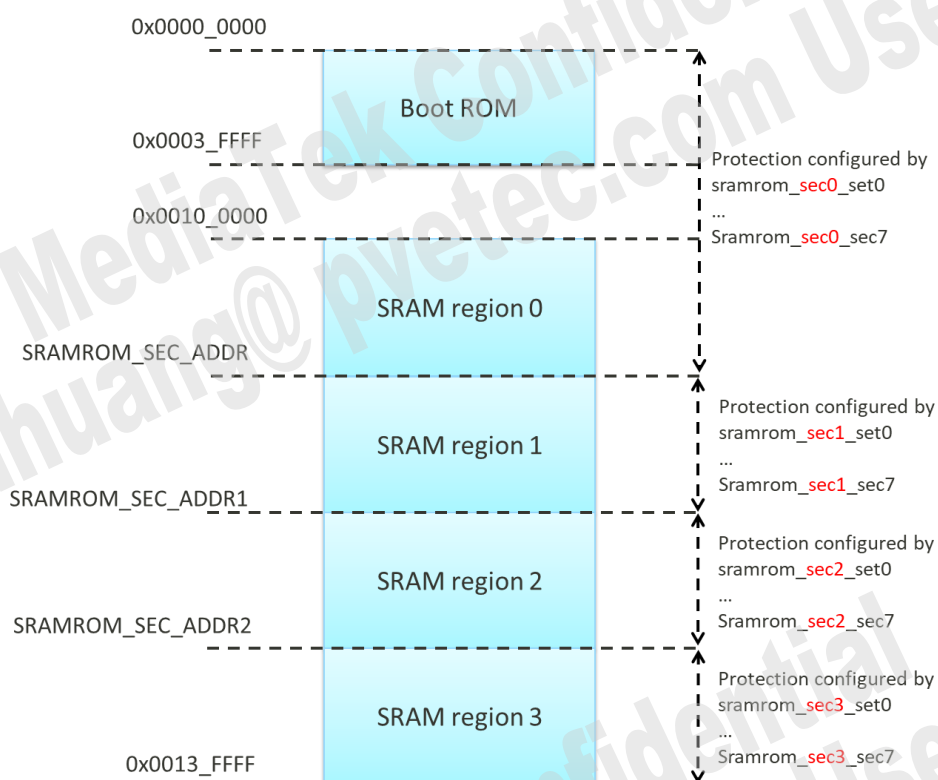


Figure 3-2. Security memory protection scheme

3.2.4 Register Definition

See chapter 1.2 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.3 External Interrupt Controller (EINTC)

3.3.1 Introduction

The external interrupt controller (EINTC) processes all off-chip interrupt sources and forwards interrupt request signals to the AP MCU.

3.3.2 Features

The EINTC supports up to 123 external interrupt signals and performs the following processes to the interrupt signals coming from external sources:

- Polarity inversion
- Edge/Level trigger selection
- De-bounce with a configurable 32 kHz clock (optional)

According to the register configuration, the external interrupt source is forwarded to the AP MCU built-in interrupt controller with different IRQ signals, `eint_irq` or `eint_direct_irq`. The EINTC generates wake-up events to the AP MCU.

3.3.3 Block Diagrams

[Figure 3-3](#) is the example block diagram of an EINTC. Every functional block is controlled by the corresponding control registers defined in the register table (see [3.3.4](#)).

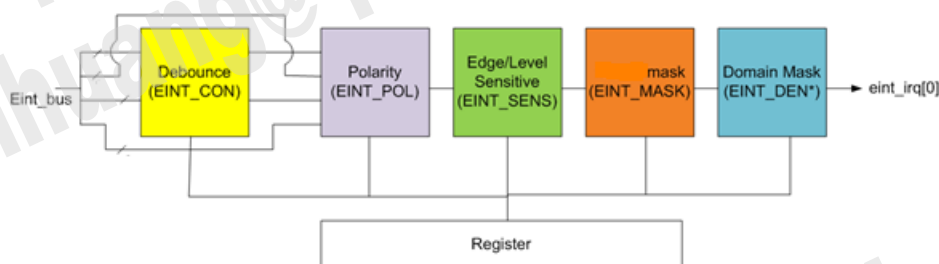


Figure 3-3. Block diagram of EINTC

[Figure 3-4](#) shows the secure view of the EINT module which has the following properties.

- Secure write and global read
- Uses `secure_en` to control security
- Supports security for the EINT listed in [Table 3-5](#)
- If one is in a secure world, the interrupt cannot appear in a non-secure world.

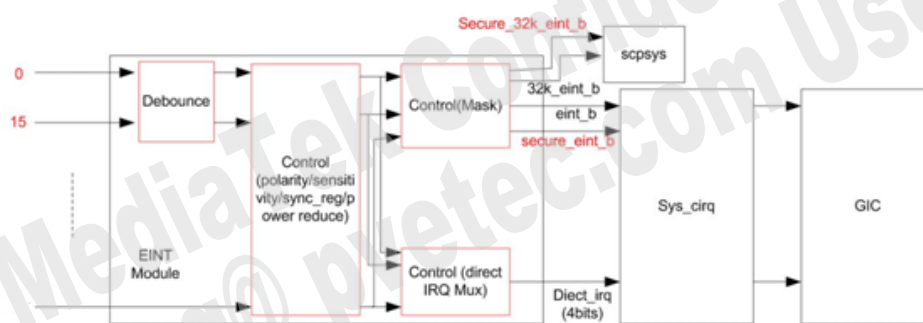


Figure 3-4. Secure view of external interrupt controller

Normally, the external interrupt source goes through the de-bounce unit which is driven by a 32 kHz clock and triggers the corresponding CPU with eint_irq. Therefore, the minimum latency from eint_bus to eint_irq is 30.52 μ s.

To program de-bounce settings (to enable, reset and de-bounce time window), mask the IRQ and wait for at least 62.5 μ s (32k @ 2T) + de-bounce time window to unmask to avoid possible glitches.

The following tables list the signal connections to the interrupt controller of the AP MCU.

Table 3-3. External interrupt request signal connection

IRQ name	GIC Interrupt ID
eint_irq	341
eint_irq_secure	342
eint_event_b	343
eint_event_secure_b	344
eint_direct_irq[0]	345
eint_direct_irq[1]	346
eint_direct_irq[2]	347
eint_direct_irq[3]	348

Table 3-4. Definitions of domains

Domain number	Target CPU
0	Application CPU

Table 3-5. EINT table

EINT name	EINT signals	PAD name	Support hardware de-bounce	Support security
EINT_T_0	EINT0	SCP_EINT0	V	V
EINT_T_1	EINT1	SCP_EINT1	V	V
EINT_T_2	EINT2	SCP_EINT2	V	V
EINT_T_3	EINT3	SCP_EINT3	V	V
EINT_T_4	EINT7	EINT_CHG_IRQB	V	V
EINT_T_5	EINT9	EINT_CTP0	V	V
EINT_T_6	EINT10	EINT_CTP1	V	V

EINT name	EINT signals	PAD name	Support hardware de-bounce	Support security
EINT_T_7	EINT11	SPI2_CK	V	V
EINT_T_8	EINT12	SPI2_CSB	V	V
EINT_T_9	EINT13	SPI2_MI	V	V
EINT_T_10	EINT14	SPI2_MO	V	V
EINT_T_11	EINT40	LCM_RST_aux		
EINT_T_12	EINT41	LCM_RST		
EINT_T_13	EINT42	DSI_TE_aux		
EINT_T_14	EINT43	DSI_TE		
EINT_T_15	EINT54	SCP_SPI_CK		
EINT_T_16	EINT55	SCP_SPI_CSB		
EINT_T_17	EINT56	SCP_SPI_MI		
EINT_T_18	EINT57	SCP_SPI_MO		
EINT_T_19	EINT58	INT_SIM1		
EINT_T_20	EINT59	INT_SIM2		
EINT_T_21	EINT60	GPIO_CTP0_RST		
EINT_T_22	EINT61	GPIO_CTP1_RST		
EINT_W_0	EINT8	EINT_FPS	V	V
EINT_W_1	EINT31	KPCOL1	V	V
EINT_W_2	EINT32	KPCOLO	V	V
EINT_W_3	EINT33	KPROW0	V	V
EINT_W_4	EINT34	KPROW1	V	V
EINT_W_5	EINT35	AP_GOOD		
EINT_W_6	EINT50	SPI1_CK		
EINT_W_7	EINT51	SPI1_CSB		
EINT_W_8	EINT52	SPI1_MI		
EINT_W_9	EINT53	SPI1_MO		
EINT_W_10	EINT66	SIM1_SCLK		
EINT_W_11	EINT67	SIM1_SRST		
EINT_W_12	EINT68	SIM1_SIO		
EINT_W_13	EINT69	SIM2_SCLK		
EINT_W_14	EINT70	SIM2_SRST		
EINT_W_15	EINT71	SIM2_SIO		
EINT_W_16	EINT72	MSDC1_CLK		
EINT_W_17	EINT73	MSDC1_CMD		
EINT_W_18	EINT74	MSDC1_DAT0		
EINT_W_19	EINT75	MSDC1_DAT1		
EINT_W_20	EINT76	MSDC1_DAT2		
EINT_W_21	EINT77	MSDC1_DAT3		
EINT_S_0	EINT15	I2S0_BCK	V	V
EINT_S_1	EINT16	I2S0_LRCK	V	V
EINT_S_2	EINT17	I2S0_DI	V	V
EINT_S_3	EINT18	I2S0_DO	V	V

EINT name	EINT signals	PAD name	Support hardware de-bounce	Support security
EINT_S_4	EINT19	I2S1_BCK	V	V
EINT_S_5	EINT20	I2S1_LRCK	V	V
EINT_S_6	EINT21	I2S1_DI	V	V
EINT_S_7	EINT22	I2S1_DO	V	V
EINT_S_8	EINT23	I2S2_BCK	V	V
EINT_S_9	EINT24	I2S2_LRCK	V	V
EINT_S_10	EINT25	I2S2_DI	V	V
EINT_S_11	EINT26	I2S2_DO	V	V
EINT_S_12	EINT27	SPI3_CK	V	V
EINT_S_13	EINT28	SPI3_CSB	V	V
EINT_S_14	EINT29	SPI3_MI	V	V
EINT_S_15	EINT30	SPI3_MO	V	V
EINT_S_16	EINT36	SPI4_CK		
EINT_S_17	EINT37	SPI4_CSB		
EINT_S_18	EINT38	SPI4_MI		
EINT_S_19	EINT39	SPI4_MO		
EINT_S_20	EINT84	NFC_CLK_REQ		
EINT_S_21	EINT85	BPI_D_BUS0		
EINT_S_22	EINT86	BPI_D_BUS1		
EINT_S_23	EINT87	BPI_D_BUS2		
EINT_S_24	EINT88	BPI_D_BUS3		
EINT_S_25	EINT89	BPI_D_BUS4		
EINT_S_26	EINT90	BPI_D_BUS5		
EINT_S_27	EINT91	BPI_D_BUS6		
EINT_S_28	EINT92	BPI_D_BUS7		
EINT_S_29	EINT93	BPI_D_BUS8		
EINT_S_30	EINT94	BPI_D_BUS9		
EINT_S_31	EINT95	BPI_D_BUS10		
EINT_S_32	EINT96	BPI_D_BUS11		
EINT_S_33	EINT97	BPI_D_BUS12		
EINT_S_34	EINT98	BPI_D_BUS13		
EINT_S_35	EINT99	BPI_D_BUS14		
EINT_S_36	EINT100	BPI_D_BUS15		
EINT_S_37	EINT101	BPI_D_BUS16		
EINT_S_38	EINT102	BPI_D_BUS17		
EINT_S_39	EINT103	BPI_D_BUS18		
EINT_S_40	EINT104	BPI_D_BUS19		
EINT_S_41	EINT105	MIPI0_D_SCLK		
EINT_S_42	EINT106	MIPI0_D_SDATA		
EINT_S_43	EINT107	MIPI1_D_SCLK		
EINT_S_44	EINT108	MIPI1_D_SDATA		
EINT_S_45	EINT109	MIPI2_D_SCLK		

EINT name	EINT signals	PAD name	Support hardware de-bounce	Support security
EINT_S_46	EINT110	MIPI2_D_SDATA		
EINT_S_47	EINT111	MIPI3_D_SCLK		
EINT_S_48	EINT112	MIPI3_D_SDATA		
EINT_S_49	EINT113	MIPI4_D_SCLK		
EINT_S_50	EINT114	MIPI4_D_SDATA		
EINT_S_51	EINT118	CAM_RST3		
EINT_S_52	EINT122	CAM_RST7		
EINT_R_0	EINT4	EINT4	V	V
EINT_R_1	EINT5	EINT5	V	V
EINT_R_2	EINT6	EINT6	V	V
EINT_R_3	EINT44	SPIO_HOLD		
EINT_R_4	EINT45	SPIO_WP		
EINT_R_5	EINT46	SPIO_CK		
EINT_R_6	EINT47	SPIO_CSB		
EINT_R_7	EINT48	SPIO_MI		
EINT_R_8	EINT49	SPIO_MO		
EINT_R_9	EINT62	PERI_EN3		
EINT_R_10	EINT63	PERI_EN2		
EINT_R_11	EINT64	PERI_EN1		
EINT_R_12	EINT65	PERI_EN0		
EINT_R_13	EINT78	MSDC2_CLK		
EINT_R_14	EINT79	MSDC2_CMD		
EINT_R_15	EINT80	MSDC2_DAT0		
EINT_R_16	EINT81	MSDC2_DAT1		
EINT_R_17	EINT82	MSDC2_DAT2		
EINT_R_18	EINT83	MSDC2_DAT3		
EINT_R_19	EINT115	CAM_RST0		
EINT_R_20	EINT116	CAM_RST1		
EINT_R_21	EINT117	CAM_RST2		
EINT_R_22	EINT119	CAM_RST4		
EINT_R_23	EINT120	CAM_RST5		
EINT_R_24	EINT121	CAM_RST6		
EINT_C_0	SSUSB_EINT_OUT[0]	X		
EINT_C_1	SSUSB_EINT_OUT[1]	X		
EINT_C_2	SSUSB_EINT_OUT[2]	X		
EINT_C_3	SSUSB_EINT_OUT[3]	X		
EINT_C_4	SSUSB_EINT_OUT[4]	X		
EINT_C_5	SSUSB_EINT_OUT[5]	X		

EINT name	EINT signals	PAD name	Support hardware de-bounce	Support security
EINT_C_6	SSUSB_EINT_OUT[6]	X		
EINT_C_7	SSUSB_EINT_OUT[7]	X		
EINT_C_8	Reserved	X		
EINT_C_9	Reserved	X		
EINT_C_10	Reserved	X		
EINT_C_11	Reserved	X		
EINT_C_12	pmifspi_pmic_eint[0]	X		
EINT_C_13	pmifspi_pmic_eint[1]	X		
EINT_C_14	pmifspmi_pmic_eint[0]	X		
EINT_C_15	pmifspmi_pmic_eint[1]	X		
EINT_C_16	spmi_slv_eint_o	X		
EINT_C_17	spmi2_slv_eint_o	X		
EINT_C_18	Reserved	X		
EINT_C_19	Reserved	X		

3.3.4 Register Definition

See chapter 1.3 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.3.5 Programming Guide

3.3.5.1 Register Bit Set/Clear

The following registers have specific bit set/clear registers for efficient operations.

- EINT_MASK
- EINT_SOFT
- EINT_DBNC
- EINT_POL
- EINT_SENS

Write 1 to a specific bit of the set/clear register to set up or clear the corresponding bit in the status register.

3.3.5.2 EINT De-bounce Control Sequence

1. Set up EINT_CON (PRESCALER, POL, CNT) and enable de-bounce (EN).
2. Wait for 3*32K (~100 us).

3. Write RSTDBC (self-clear).
4. Wait for 3*32K (~100 us).
5. Write EINT_INTACK to ack/clear all statuses.
6. Unmask EINT_MASK.

3.3.5.3 ACK for EINT Control Sequence

3.3.5.3.1 ACK for EINT_N/EINT_S/EINT_E/EINT_W

1. Set up EINT MASK.
2. Write EINT_INTACK to ack/clear all statuses.
3. Unmask EINT_MASK.

3.4.1 Introduction

The system interrupt controller (SYS_CIRQ) is a low power (also a second level) interrupt controller designed to work outside the MCUSYS that the MCUSYS can be completely turned off to improve the system power consumption without losing interrupts.

The SYS_CIRQ supports up to 784 interrupts which can configure the following attributes individually.

- The 784 interrupts feed through the SYS_CIRQ and connect to the GIC in the MCUSYS. When the SYS_CIRQ is enabled, it records the edge-sensitive interrupts and generates a pulse signal to the CPU GIC when the flush command is executed.

Figure 3-5 is the system level block diagram of the SYS CIRQ.

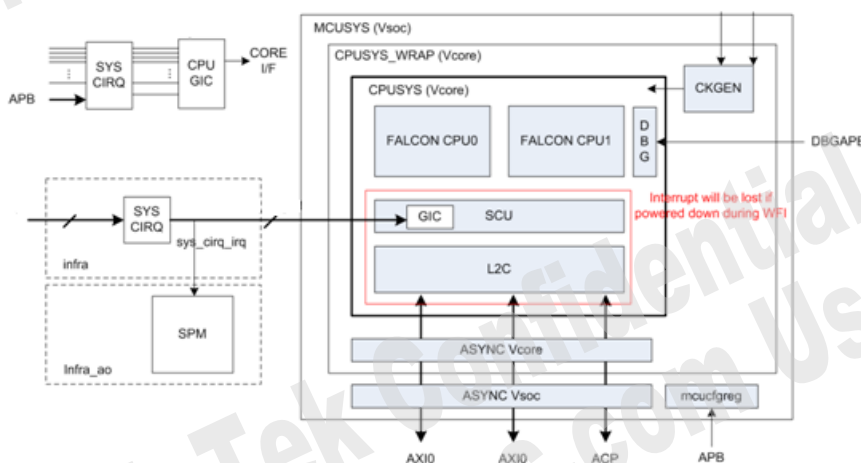


Figure 3-5. System level block diagram of SYS CIRQ

In the normal mode (where the MCUSYS GIC is active), the SYS_CIRQ is disabled and interrupts are directly issued to the MCUSYS. When the MCUSYS enters the sleep mode where the GIC is power down, the SYS_CIRQ is enabled to monitor all edge-trigger interrupts (which shall be lost in this scenario) to be recorded in the SYS_CIRQ_STA register, and can be restored to the GIC by the software or the SYS_CIRQ flush function.

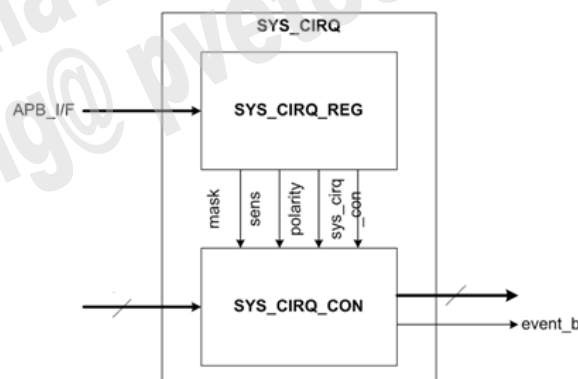


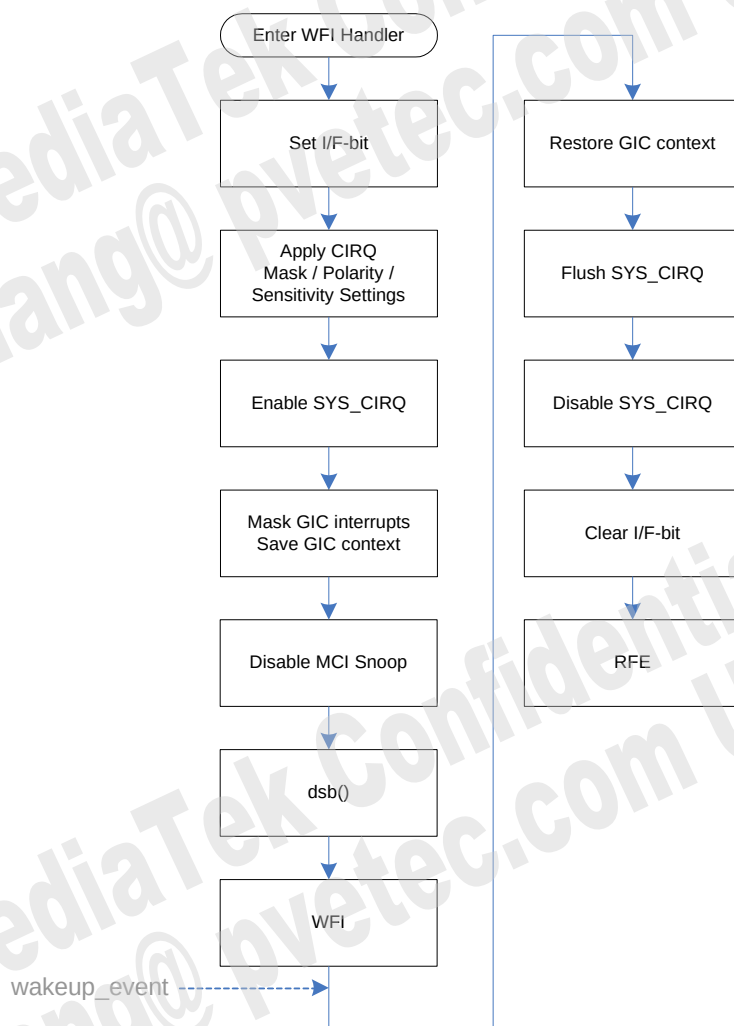
Figure 3-6. Block diagram of SYS_CIRQ

[Figure 3-6](#) is the architecture of the SYS_CIRQ. Its register (SYS_CIRQ_REG) stores the mask/sensitivity/polarity attributes of each interrupt signal, and SYS_CIRQ_CON is used to mask and detect edge-triggered interrupts.

3.4.4 Register Definition

See chapter 1.4 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.4.4.1 Software Flow



3.5 Infrastructure System Configuration Module (INFRACFG)

3.5.1 Introduction

The infrastructure system configuration module (INFRACFG) provides reset, clock and miscellaneous control signals in the infrastructure system.

3.5.2 Features

The INFRACFG provides the following control signals to the functional blocks inside the infrastructure system:

- Software reset signals
- Clock gating control signals
- Dynamic clock management control signals
- AXI bus fabric control signals
- Sleep protect control signals

3.5.2.1 DCM in Details

The dynamic clock management (DCM) function slows down the clock frequency for power saving when the system is in the idle state automatically.

The clock frequency in the DCM mode is set to quarter of the original clock. The ratio of clock frequency slow-down is controlled by the INFRA_DCMFSEL register.

After the bus idle signal is low, it takes several cycles of latency to make the slow-down clock return to the normal frequency. The cycle number varies with the runtime status of the clock gating logic and causes minor performance impact. To minimize the impact when the system is in heavy load status, the INFRA_DCMDBC register controls the cycle count once the bus idle signal is asserted. Set the debounce cycle to be longer and enable the function to reduce the probability of the system entering the DCM mode.

3.5.2.2 AXI Fabric Control

The AXI fabric control registers prevent the system bus from hanging up caused by improper accesses while some parts of the system are in the power-down state. The sleep protector can get the functions above. Refer to the corresponding control registers in [3.5.3](#).

3.5.3 Register Definition

See chapter 1.5 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.6 External Memory Interface (EMI)

3.6.1 Introduction

The external memory interface (EMI) controller schedules requests from the masters and issues commands to the DRAM controller (DRAMC) in an efficient way. The block conducts flow control for the DRAMC and masters to avoid DRAM stall or data overflow/underflow. It also minimizes the latency of processor path to enhance the performance and tries to increase the DRAM efficiency. The block also informs clock control to gate the clock for the time when it does not find any transaction. This block is designed to supply an EMI clock frequency of 728 MHz that can be scaled down to 660 MHz, 625 MHz, 466 MHz, 333 MHz, and 200 MHz.

3.6.2 Features

The EMI controller receives AXI master commands and issues them to the the DRAMC. It supports all AXI transaction type commands, except for the fixed and cache commands. There are plenty of schedule options to schedule the command, which are:

- Starvation control
- Bandwidth regulator
- Latency regulator
- High priority
- Page hit control
- Read/Write turnaround prevent control
- Memory-protect unit (DRAM and APB)

3.6.3 Block Diagram

In MT8676, the EMI controller is used as a bridge of several systems and the DRAMC. For cortex APMCU system, or multimedia masters (e.g., display, video encoder, video decoder, and camera), or GPU, APU and peripherals, two 128-bit AXI ports are provided for the connection. Besides, there are two 128-bit AXI ports for connecting to the modem MCU and modem 2G/3G/4G/5G hardware.

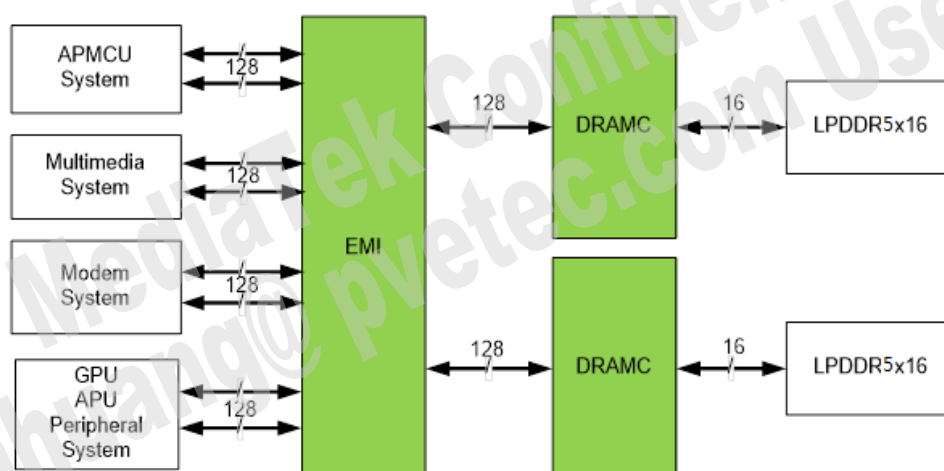


Figure 3-7. EMI/DRAM controller top connection

3.6.4 Register Definition

See chapter 1.6 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.7 DRAM Controller (DRAMC)

3.7.1 Overview

DRAM controller (DRAMC) supports the DRAM bus configuration of LPDDR5X 16-bit @3,750 MHz (7,500 Mbps/bit channel).

See the following table for the DRAM bus signals.

Table 3-6. DRAM bus signal list (refer to the DRAMC side) (LPDDR5/5X)

Signal name	Type	Description
CK_t	Input	DRAM clock signal
CK_c	Input	DRAM clock invert signal
CA[6:0]	Input	Address for all memories/CA bus for LPDDR5/5X
CS0/CS1	Input	RANK1~RANK0 selection signal
DQ[15:0]	I/O	Data bus for LPDDR5/5X
DMI[1:0]	Input	Data mask inversion
RDQS_t[1:0]	I/O	Read data strobe
RDQS_c[1:0]	Output	Differential read data strobe in LPDDR5/5X
WCK_t[1:0]	Input	Data clocks
WCK_c[1:0]	Input	Data clock invert signal
ZQ	I/O	Output driving calibration
RESET_n	Input	Reset

See below for the DRAM bus command truth table.

Table 3-7. DRAM bus command truth table (LPDDR5/5X)

CAS	Any	H	L	L	H	H	WS_WR	WS_R D	WS_FS		R1	1, 2, 7, 13
		X	DC0	DC1	DC2	DC3	WRX	WXSA	WXSBI B3		F1	
MULTI PURPOSE COMMAND (MPC)	Any	H	L	L	L	L	H	H	OP7		R1	1, 2
		X	OP0	OP1	OP2	OP3	OP4	OP5	OP6		F1	
SELF REFRESH ENTRY (SRE)	Any	H	L	L	L	H	L	H	H		R1	1, 2, 9
		X	V	V	V	V	V	DSM	PD		F1	
SELF REFRESH EXIT (SRX)	Any	H	L	L	L	H	L	H	L		R1	1, 2
		X	V	V	V	V	V	V	V		F1	
MODE REGISTER WRITE-1 (MRW-1)	Any	H	L	L	L	H	H	L	H		R1	1, 2
		X	MA0	MA1	MA2	MA3	MA4	MA5	MA6		F1	
MODE REGISTER WRITE-2 (MRW-2)	Any	H	L	L	L	H	L	L	OP7		R1	1, 2
		X	OP0	OP1	OP2	OP3	OP4	OP5	OP6		F1	
MODE REGISTER READ (MRR)	Any	H	L	L	L	H	H	L	L		R1	1, 2
		X	MA0	MA1	MA2	MA3	MA4	MA5	MA6		F1	
WRITE FIFO (WFF)	Any	H	L	L	L	L	L	H	H		R1	1, 2
		X	L	L	L	L	L	L	L		F1	
READ FIFO (RFF)	Any	H	L	L	L	L	L	H	L		R1	1, 2
		X	L	L	L	L	L	L	L		F1	
READ DQ CALIBRATION (RDC)	Any	H	L	L	L	L	H	L	H		R1	1, 2
		X	L	L	L	L	L	L	L		F1	

SDRAM COMMAND	BK ORG (BG, 16B, 8B)	SDR CMD PIN	DDR COMMAND PINS								CK t edge	Notes	
		CS	CA0	CA1	CA2	CA3	CA4	CA5	CA6				
DESELECT (DES)	Any	L	X	X	X	X	X	X	X	X		R1	1, 2
		X	X	X	X	X	X	X	X	X		F1	
NO OPERATION (NOP)	Any	H	L	L	L	L	L	L	L	L		R1	1, 2
		X	X	X	X	X	X	X	X	X		F1	
POWER DOWN ENTRY (PDE)	Any	H	L	L	L	L	L	L	L	H		R1	1, 2, 12
		L	X	X	X	X	X	X	X	X		R2	
ACTIVATE-1 (ACT-1)	Any	H	H	H	H	R14	R15	R16	R17		R1	1, 2, 3, 4	
	BG	X	BA0	BA1	BG0	BG1							
	16B	X	BA0	BA1	BA2	BA3	R11	R12	R13		F1		
ACTIVATE-2 (ACT-2)	Any	H	H	H	L	R7	R8	R9	R10		R1	1, 2, 4	
		X	R0	R1	R2	R3	R4	R5	R6		F1		
		8B	X	BA0	BA1	BA2	BA3	V	V	AB			F1
PRECHARGE (PRE) (Per Bank, All Banks)	Any	H	L	L	L	H	H	H	H		R1	1, 2, 3, 5	
	BG	X	BA0	BA1	BG0	BG1							
	16B	X	BA0	BA1	BA2	BA3	V	V	AB		F1		
REFRESH (REF) (Per Bank, All Banks)	Any	H	L	L	L	H	H	H	L		R1	1, 2, 3, 5, 14	
	BG	X	BA0	BA1	BG0	BG1	SB0	V					
	16B	X	BA0	BA1	BA2	BA3	SB0	V	AB		F1		
MASK WRITE (MWR)	Any	H	L	H	L	C0	C3	C4	C5		R1	1, 2, 3, 6, 7	
	BG	X	BA0	BA1	BG0	BG1	C1	C2	AP		F1		
	16B	X	BA0	BA1	BA2	BA3	C1	C2	AP		F1		
WRITE (WR16 or WR)	Any	H	L	H	H	C0	C3	C4	C5		R1	1, 2, 3, 6, 7, 8	
	BG	X	BA0	BA1	BG0	BG1	C1	C2	AP		F1		
	16B	X	BA0	BA1	BA2	BA3	C1	C2	AP		F1		
WRITE32 (WR32)	BG/16B	H	L	L	H	L	C3	C4	C5		R1	1, 2, 3, 6, 7, 8	
	BG	X	BA0	BA1	BG0	BG1	C1	C2	AP		F1		
	16B	X	BA0	BA1	BA2	BA3	C1	C2	AP		F1		
READ (RD16 or RD)	Any	H	H	L	L	C0	C3	C4	C5		R1	1, 2, 3, 6, 7, 10, 11	
	BG	X	BA0	BA1	BG0	BG1	C1	C2	AP		F1		
	16B	X	BA0	BA1	BA2	BA3	C1	C2	AP		F1		
READ32 (RD32)	BG/16B	H	H	L	H	C0	C3	C4	C5		R1	1, 2, 3, 6, 7, 10, 11	
	BG	X	BA0	BA1	BG0	BG1	C1	C2	AP		F1		
	16B	X	BA0	BA1	BA2	BA3	C1	C2	AP		F1		

3.7.2 Features

- Supports high-priority side band signal for reducing request latency
- A request cannot cross the page boundary.
- Supports 4X (LPDDR5) frequency mode (frequency ratio of DRAMC:DRAM = 1:4) for timing optimization
- Supports the dual-scheduler function for 1T command rate under 4X frequency mode for performance optimization
- Supports power-down and self-refresh for power saving
- Supports clock stop for power saving
- Supports input DQS/DQ timing calibration for PVT variation
- Supports read/write command out of order control
- Supports LPDDR5/5X byte mode
- Supports 10 shuffle frequencies
- Supports per bank refresh

3.7.2.1 Reference

LPDDR5 spec: <https://www.jedec.org/standards-documents/docs/jesd209-5b>

3.7.3 Block Diagram

The major blocks of the DRAM controller are command decoder, command pool, bus scheduler, timing controller DDR PHY and response generator.

The requests from the arbiter are pushed to the command pool to wait for execution in order. The bus scheduler inspects the precharge/active pool and command FIFO and decides which DRAM bus command, e.g., PRECHARGE, ACTIVE, READ or WRITE, is issued to the DRAM bus. The goals of bus scheduler are to raise the bus utilization rate and lower the response latency. The timing control unit is responsible for the integrity of the DRAM bus timing such as pre-charge to active delay (tRP), active to command delay (tRCD) and bus turnaround time. The bus scheduler refers to the information and chooses the next DRAM bus command. The DRAM interface unit is responsible for generating DRAM bus commands, transmitting data and DQS to DDR DRAM and receiving data and DQS from the DDR DRAM. The response generator produces the response signals for all DRAM agents such as DLE and RDAT.

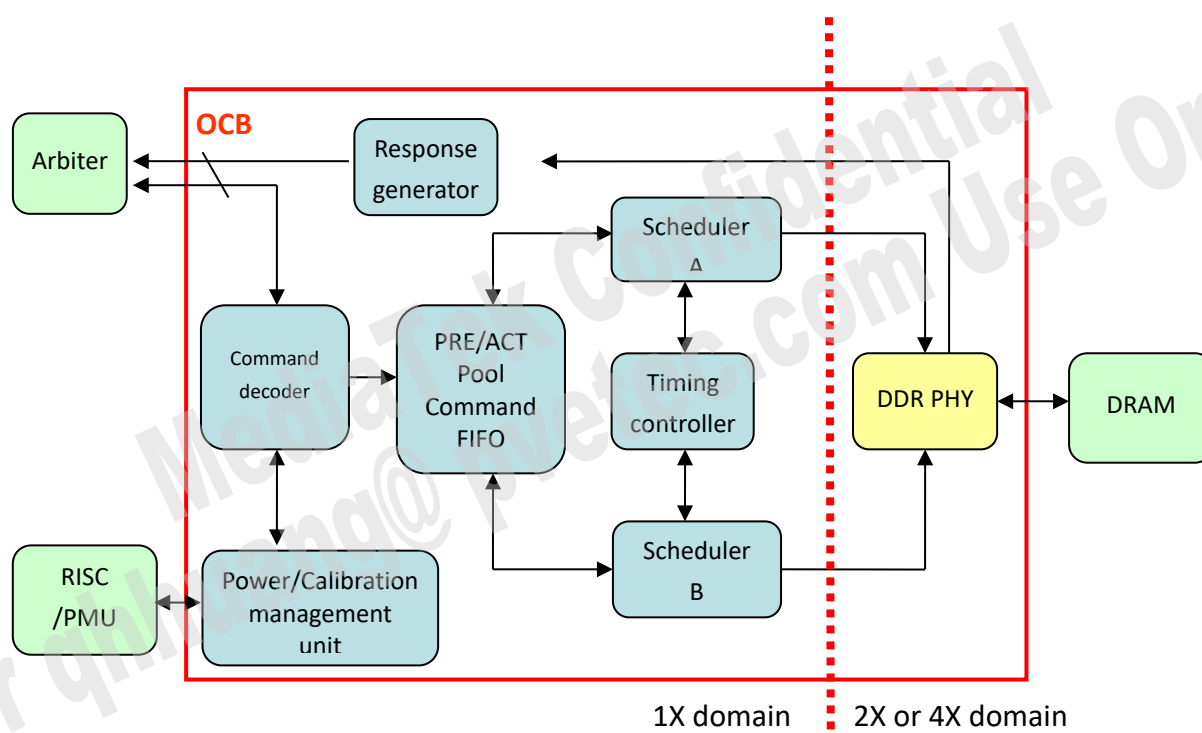


Figure 3-8. Block diagram of DRAM controller

3.7.4 Register Definition

See chapter 1.7 of "MT8676 5G Connected Cockpit Application Processor Software Register Table".

3.7.5 Programming Guide

Steps of DRAM initialization

1. Set up the DRAM AC timing parameter.

2. Follow the DRAM specification to complete the DRAM initialization including mode-register programming.
3. Enable calibration for the DQ/DQS window.
4. Set up the refresh-rate counter.
5. Normal operation.

3.8 Application Processor DMA (AP_DMA)

3.8.1 Introduction

The MT8676 application processor DMA (AP_DMA) serves data transfers between the memory and peripherals.

3.8.2 Features

- Supports up to 29 channels of simultaneous data transfers
- Compliant with system bus (AXI)
- Data FIFO of 128 bits embedded in the DMA channel
- Peripherals and channels:
 - UART x 4 (Tx/Rx channels are separated, 8 channels in total)
 - BTIF x 1 (Tx/Rx channels are separated, 2 channels in total)
 - I2C x 19 (I2C x 7, I3C x 12)
 - Triggers DMA transmission through the handshaking signal from a peripheral
- Source/destination configuration
 - Only one side (either the source or the destination) is programmable; the other side can be selected as the specified peripheral.
- Burst size/burst length of 8 bytes/1 beat
- TrustZone
 - The corresponding channel is set as the secure channel, if SEC_EN (AP_DMA_I2Cx_SEC_EN, AP_DMA_UARTx_TX_SEC_EN or AP_DMA_UARTx_RX_SEC_EN)[0] = 1
 - If the channel is set to a secure one, it can issue secure requests, and its configuration registers can only be accessed via secure masters.
- Interrupt notification when the FIFO data is over/under a certain threshold.
- Scheduling scheme
 - Round-Robin (RR): If many channels are triggered simultaneously, the triggering priority will depend on the channel number. The channel with a smaller number has a higher priority, e.g., channel 0 > channel 1 > channel 2.
- Cache coherency not supported

3.8.3 Block Diagram

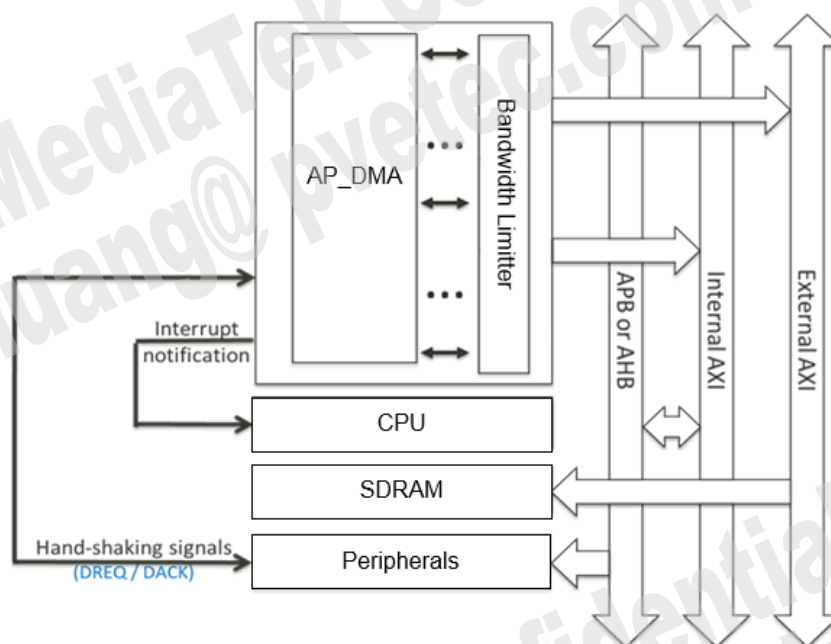


Figure 3-9. Block diagram of AP_DMA

Figure 3-9 is the basic block diagram of the AP_DMA with a total of 29 channels in the DMA.

There are 3 interfaces of the AP_DMA: External AXI, internal AXI and APB.

Interface	Connected to	Purpose	Data size (per request)
External AXI	Main AXI bus fabric	Provides external memory access	8 bytes
Internal AXI	Peripheral AXI bus fabric	Connectivity for peripherals (e.g., I ² C, BTIF and UART)	1 byte
APB		Accesses to configuration space of DMA	

All control registers are divided into two groups: Global registers and local registers.

Register	Description
Global	Contain common status and configurations
Local	Based on channel-dependent configuration and exist for every DMA channel.

Table 3-8. APDMA matrix

AP_DMA		Source			
		SDRAM	SYSRAM	I2C	UART
Destination	SDRAM	X	X	V	V
	SYSRAM	X	X	V	V
	I2C	V	V	X	X
	UART	V	V	X	X

3.8.3.1 UART Virtual FIFO (VFF) DMA

The VFF functions like the ring buffer which uses two address pointers (VFF_WPT/VFF_RPT) to control its condition. Based on the address pointers, two parameters, “VFF_VALID_SIZE” and “VFF_LEFT_SIZE” are defined to represent valid data and available space in the VFF. Refer to [Figure 3-10](#) for the relationship between the UART VFF read and write pointers.

- When TX_VFF_WPT is wrapped to a ring head again, invert TX_VFF_WPT_WRAP (AP_DMA_UART_x_TX_VFF_WPT)[16] for UART Tx VFF DMA.
- When RX_VFF_RPT is wrapped to a ring head again, invert RX_VFF_RPT_WRAP (AP_DMA_UART_x_RX_VFF_RPT)[16] for UART Rx VFF DMA.

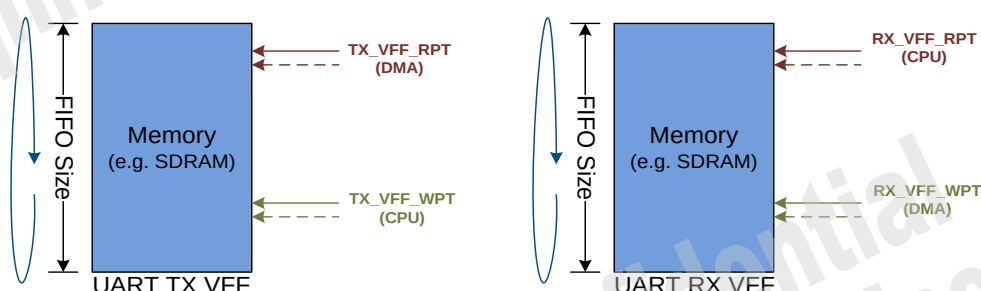
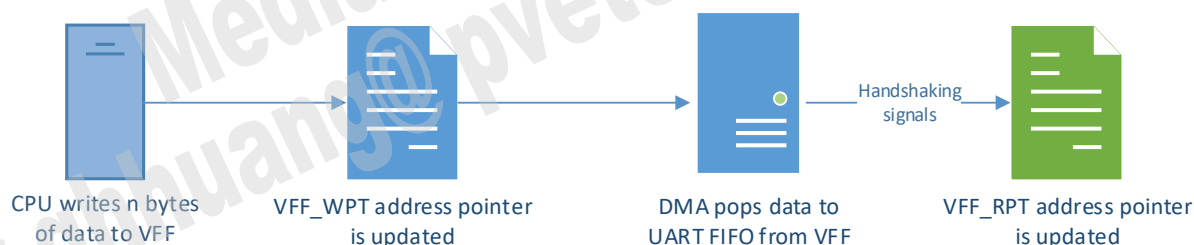


Figure 3-10. UART VFF pointers relationship

3.8.3.1.1 UART Tx VFF DMA



When the CPU writes n bytes of data to the VFF, the VFF_WPT address pointer is updated. The DMA then pops data to UART FIFO from VFF with handshaking signals and updates the VFF_RPT address pointer.

Condition	Result	VFF
UART FIFO available and VFF_VALID SIZE $\neq$ 0	VFF data not yet sent to UART FIFO	Data byte stored = VFF_VALID_SIZE (VFF_WPT - VFF_RPT)
		Available space = VFF_LEFT_SIZE (VFF_LEN - VFF_VALID_SIZE)

There is a threshold (TX_VFF_THRE) to trigger the interrupt. If VFF_LEFT_SIZE $\geq$ TX_VFF_THRE, the CPU can push data after receiving IRQ. In other words, if there is a lot of available space (VFF_LEFT_SIZE) in the VFF, the DMA will trigger IRQ which informs the CPU to push data.

For better bandwidth efficiency, every read request from the Tx DMA to the Tx VFF is 8 bytes. If $VFF_VALID_SIZE < 8$ bytes, the DMA does not issue any read request to the Tx VFF, even if the DMA FIFO is empty. It could be problematic if the total data size pushed to the VFF by the CPU is not 8-byte aligned, the last remaining data (< 8 bytes) can be kept in the VFF permanently. To resolve this issue, the software should set up the “Tx FLUSH” register to enable the DMA to issue a read request to get the last remaining data in the VFF.

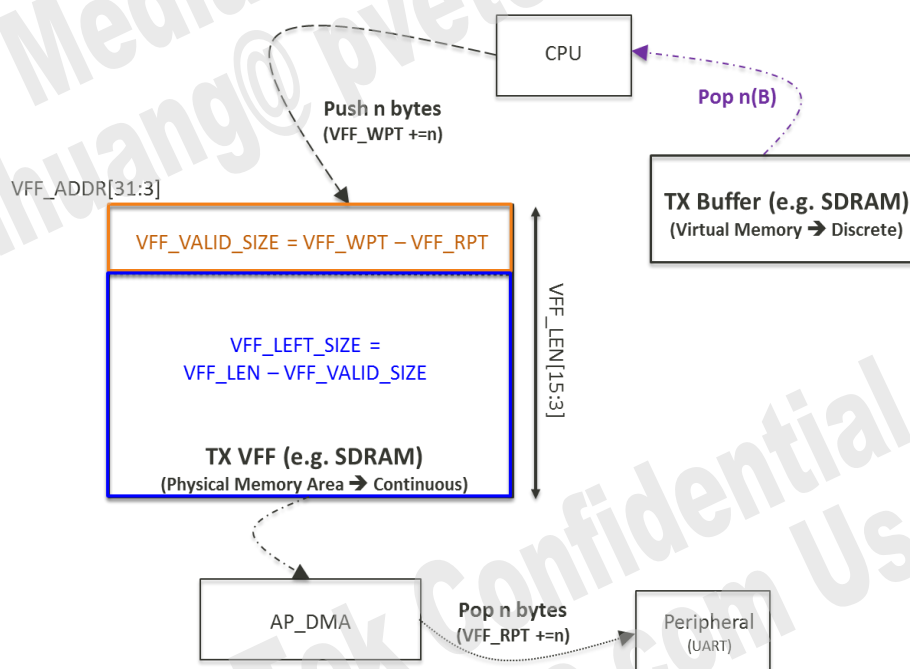


Figure 3-11. UART Tx VFF data flow diagram

3.8.3.1.2 UART Rx VFF DMA

In the Rx VFF DMA mode, the data transfer direction is opposite to that in the Tx VFF DMA mode. After the UART pushes data to the VFF with handshaking signals, VFF_WPT is then updated by the hardware. On the other hand, VFF_RPT is updated by the CPU after CPU pops data from the VFF if $VFF_VALID_SIZE > 0$.

There is also a threshold (RX_VFF_THRE) to trigger interrupts. If $VFF_VALID_SIZE \geq RX_VFF_THRE$, the CPU can pop data after receiving IRQ. In other words, if there is a lot of valid data stored in the VFF, the DMA will trigger IRQ which informs the CPU to pop data. If the CPU cannot pop data from the VFF immediately, the data from the UART might be lost. To avoid this, the DMA can inform the UART through side-band signals, and the CPU will receive IRQ if $VFF_LEFT_SIZE \leq RX_FLOW_CTRL_THRE$. The software can assert RTS if IRQ is received.

Every write request from the Rx DMA to the VFF is 8 bytes. If the total data from the UART is not 8-byte aligned, the last remaining data (< 8 bytes) will be kept in the DMA FIFO permanently. There are two methods to solve this issue:

1. Set up the “Rx FLUSH” register manually, and DMA will issue a write request to pop the remaining data from the DMA FIFO to the VFF.

- Issue a flush request to the DMA through the UART with side-band signals when the UART FIFO is empty, and timeout happens (i.e., data is not pushed towards the UART FIFO in the 4-byte baud-rate period). After the flush request is issued, the interrupt flag (FLAG1::AP_DMA_UART_x_RX_INT_FLAG[1]) will be raised.

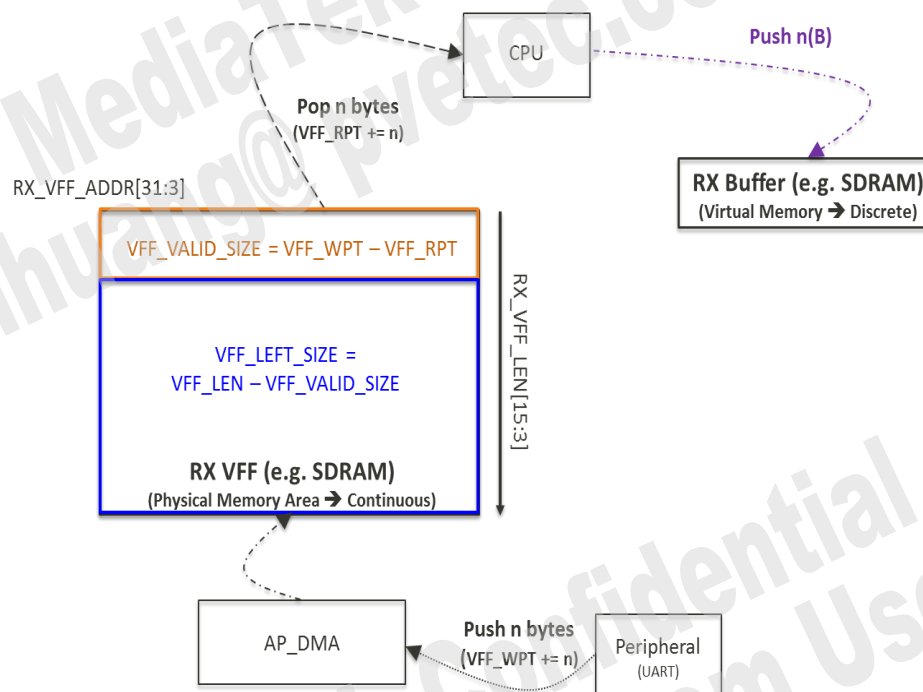


Figure 3-12. UART Rx VFF data flow diagram

3.8.3.2 Peripheral (I²C) DMA

The behavior of the I²C DMA is similar to that of the command queue DMA (CQ_DMA).

I ² C DMA	CQ_DMA
Source/Destination: I ² C FIFO	Source/Destination: SDRAM
Handshaking signals (DREQ/DACK)	
Fixed channels	

The DMA direction (DIR::AP_DMA_I2C_x_CON[0]) is controlled by the I²C register (SLAVE_ADDR[0]) and has the following features:

- Every DMA channel supports both Tx (I²C: write bit) and Rx (I²C: read bit) directions.
- The transfer length and memory address register settings are also divided into Tx and Rx parts.
- DMA supports all kinds of transfer formats, except for when the direction is changed (write and then read); however, it can be resolved when the side-band signal (TX2RX) is sent to the DMA when DMA_EN::CONTROL (I2Cx_Base + 0x10)[2] = 1 and DIR_CHANGE::CONTROL(I2Cx_Base + 0x10)[4] = 1.

Figure 3-13 is an example of the direction change transfer and the corresponding settings of DMA and I²C are listed in Table 3-9 which should be set before the I²C transaction (START::START(I2Cx_Base + 0x24)[0]).

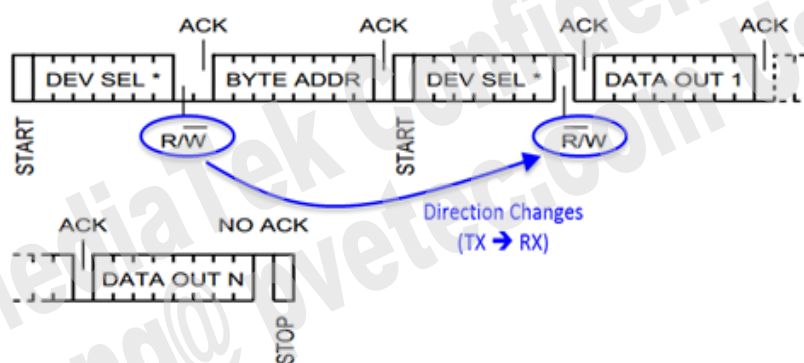


Figure 3-13. I2C sequential random read protocol

Table 3-9. AP_DMA and I2C sequential random read settings

AP DMA		I ² C	
Tx transfer length (AP_DMA_I2C_x_TX_LEN[0:15])	1	1	Tx transfer length (TRANSFER_LEN[0:15])
Rx transfer length (AP_DMA_I2C_x_RX_LEN[0:15])	N	N	Rx transfer length (TRANSFER_LEN_AUX[0:15])
Direction (DIR::AP_DMA_I2C_x_CON[0])	Tx (Controlled by I2C)	Write (0)	Direction (SLAVE_ADDR[0])
Enable DMA (EN::AP_DMA_I2C_x_EN[0])	1	1	Enable DMA (DMA_EN::CONTROL[2])
Direction change	(Note)	1	Direction change (DMA_EN::CONTROL[4])

Note: When the I²C changes the DMA direction (Tx -> Rx), it will send the side-band signal to the DMA, and the DMA shall be enabled again until the "Rx transfer length" is received.

3.8.3.3 Warm/Hard Reset

The warm/hard reset is in the DMA global control and every DMA engine.

Table 3-10. Warm/Hard reset features

	Warm reset	Hard reset
Engine	Reset after current transaction finished	Reset immediately
Side effect	No bus hang	Bus can go down due to unfinished transaction
Mechanism	When the software restarts/re-clears all engines in the DMA, it can set WARM_RST to 1 and wait for (poll) all global running statuses to be 0. Hardware will clear WARM_RST to 0 automatically after reset is completed.	When the software restarts/re-clears all engines in the DMA without waiting, it can set HARD_RST to 1 and then back to 0 to finish the global hard reset. Note that this action can break the bus protocol to cause system hang.

3.8.3.4 Pause and Resume

The “pause and resume” function is only available for the peripheral DMA (I²C channel). See [Table 3-11](#) for the mechanism.

Table 3-11. Example of pause and resume functions

Step	Address	Register name	Local address	R/W	Value	Description
1	AP_DMA+0x08	AP_DMA_I2C_x_EN	AP_DMA_I2C_x_EN[0]	W	1'b1	Start DMA program necessary settings then set EN (AP_DMA+0x08)[0] to 1.
2	AP_DMA+0x10	AP_DMA_I2C_x_STOP	AP_DMA_I2C_x_STOP[1]	W	1'b1	Pauses DMA Set PAUSE (AP_DMA+0x10)[1] to 1.
3	AP_DMA+0x10	AP_DMA_I2C_x_STOP	AP_DMA_I2C_x_STOP[1]	W	1'b0	Resumes DMA Set PAUSE (AP_DMA+0x10)[1] to 0.
4	AP_DMA+0x08	AP_DMA_I2C_x_EN	AP_DMA_I2C_x_EN[0]	R	-	Waits for DMA to finish EN (AP_DMA+0x08)[0] will become 0, and flag will be set to 1.

Note: The software can repeat steps 2 and 3 many times when the DMA is running. Users can monitor the idle bit to see if the DMA pauses. The DMA will not pause immediately and will wait for the last transaction to finish.

3.8.4 Register Definition

See chapter 1.8 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.8.5 Programming Sequence for Different Types of Channels

3.8.5.1 Peripheral DMA

I²C is one example of a peripheral DMA. See [Table 3-12](#) for the reference settings.

Table 3-12. Reference settings for peripheral DMA (I2C)

Step	Address	Register name	Local address	R/W	Value	Description
1	AP_DMA+0x18	AP_DMA_I2C_x_CON	AP_DMA_I2C_x_CON[0]	W	1'b0 1'b1	Half duplex peripheral DMA direction 0: Tx 1: Rx
2	AP_DMA+0x24 (TX) AP_DMA+0x28 (RX)	AP_DMA_I2C_x_TX_LEN AP_DMA_I2C_x_RX_LEN	AP_DMA_I2C_x_TX_LEN [15:0] AP_DMA_I2C_x_RX_LEN [15:0]	W	Length	Peripheral DMA transfer length The register can be any byte alignment.
3	AP_DMA+0x1C (TX) AP_DMA+0x54 (TX) AP_DMA+0x20 (RX) AP_DMA+0x58 (RX)	AP_DMA_I2C_x_TX_MEM_ADDR AP_DMA_I2C_x_TX_MEM_ADDR2 AP_DMA_I2C_x_RX_MEM_ADDR AP_DMA_I2C_x_RX_MEM_ADDR2	AP_DMA_I2C_x_TX_MEM_ADDR[31:0] AP_DMA_I2C_x_TX_MEM_ADDR2[3:0] AP_DMA_I2C_x_RX_MEM_ADDR[31:0] AP_DMA_I2C_x_RX_MEM_ADDR2[3:0]	W	Address	Peripheral DMA memory address The register can be any byte alignment.
4	AP_DMA+0x04	AP_DMA_I2C_x_INT_EN	AP_DMA_I2C_x_INT_EN [0] AP_DMA_I2C_x_INT_EN [1] AP_DMA_I2C_x_INT_EN [2]	W	1'b1	Enables interrupt [0]: Enable interrupt for TX_FLAG [1]: Enable interrupt for RX_FLAG [2]: Enable interrupt for Tx-to-Rx
5	AP_DMA+0x08	AP_DMA_I2C_x_EN	P_DMA_I2C_x_EN[0]	W	1'b1	Enables peripheral DMA 0: Disable 1: Enable
6	-	-	-	-	-	Waits for interrupt
7	AP_DMA+0x00	AP_DMA_I2C_x_INT_FLGA	AP_DMA_I2C_x_INT_FLGA[0] AP_DMA_I2C_x_INT_FLGA[1] AP_DMA_I2C_x_INT_FLGA[2]	W	1'b1	Clears interrupt flag [0]: Clear interrupt flag for TX_FLAG [1]: Clear interrupt flag for RX_FLAG [2]: Clear interrupt flag for Tx-to-Rx

3.8.5.2 Virtual FIFO DMA Tx

UART Tx is one example of a virtual FIFO DMA Tx. See [Table 3-13](#) for the reference settings.

Table 3-13. Reference settings for virtual FIFO DMA Tx (UART_TX)

Step	Address	Register name	Local address	R/W	Value	Description
1	AP_DMA+0x0C	AP_DMA_UART_x_TX_RST	AP_DMA_UART_x_TX_RST[0]	W	1'b1	Sets up channel warm reset
2	AP_DMA+0x0C	AP_DMA_UART_x_TX_RST	AP_DMA_UART_x_TX_RST[0]	R	-	Waits for AP_DMA_UART_x_TX_RST[0] = 0
3	AP_DMA+0x24	AP_DMA_UART_x_TX_VFF_LEN	AP_DMA_UART_x_TX_VFF_LEN[15:3]	W	Length	Sets up virtual size
4	AP_DMA+0x1C AP_DMA+0x54	AP_DMA_UART_x_TX_VFF_ADDR AP_DMA_UART_x_TX_VFF_ADDR2	AP_DMA_UART_x_TX_VFF_ADDR[31:3] AP_DMA_UART_x_TX_VFF_ADDR2[3:0]	W	Address	Sets up virtual FIFO address
5	AP_DMA+0x28	AP_DMA_UART_x_TX_VFF_THRE	AP_DMA_UART_x_TX_VFF_THRE[15:0]	W	Threshold	VFF threshold in byte alignment
6	AP_DMA+0x04	AP_DMA_UART_x_TX_INT_EN	AP_DMA_UART_x_TX_INT_EN[0]	W	1'b1	Controls interrupt enabling or not 0: Disable 1: Enable
7	AP_DMA+0x08	AP_DMA_UART_x_TX_EN	AP_DMA_UART_x_TX_EN[0]	W	1'b1	Enables UART Tx virtual FIFO DMA 0: Disable 1: Enable
8	Waiting for interrupt					
9	AP_DMA+0x00	AP_DMA_UART_x_TX_INT_FLAG	AP_DMA_UART_x_TX_INT_FLAG[0]	W	1'b0	Clears interrupt flag
10	Push data to virtual FIFO					
11	AP_DMA+0x2C	AP_DMA_UART_x_TX_VFF_WPT	AP_DMA_UART_x_TX_VFF_WPT[15:0]	W	-	Write pointer of byte alignment FIFO
12	Repeat steps 8 to 11					
13	AP_DMA+0x10	AP_DMA_UART_x_TX_STOP	AP_DMA_UART_x_TX_STOP[0]	W	1'b0	Stops UART Tx virtual FIFO

3.8.5.3 Virtual FIFO DMA Rx

UART Rx is one example of a virtual FIFO DMA Rx. See [Table 3-14](#) for the reference settings.

Table 3-14. Reference settings for virtual FIFO DMA Rx (UART_RX)

Step	Address	Register name	Local address	R/W	Value	Description
1	AP_DMA+0x0C	AP_DMA_UART_x_RX_RST	AP_DMA_UART_x_RX_RST[0]	W	1'b1	Sets up channel warm reset
2	AP_DMA+0x0C	AP_DMA_UART_x_RX_RST	AP_DMA_UART_x_RX_RST[0]	R	-	Waits for AP_DMA_UART_x_RX_RST[0] = 0
3	AP_DMA+0x24	AP_DMA_UART_x_RX_VFF_LEN	AP_DMA_UART_x_RX_VFF_LEN[15:3]	W	Length	Sets up virtual size

Step	Address	Register name	Local address	R/W	Value	Description
4	AP_DMA+0x1C AP_DMA+0x54	AP_DMA_UART_x_RX_VFF_ADDR AP_DMA_UART_x_RX_VFF_ADDR2	AP_DMA_UART_x_RX_VFF_ADDR[31:3] AP_DMA_UART_x_RX_VFF_ADDR2[3:0]	W	Address	Sets up virtual FIFO address
5	AP_DMA+0x28	AP_DMA_UART_x_RX_VFF_THRE	AP_DMA_UART_x_RX_VFF_THRE[15:0]	W	Threshold	VFF threshold in byte alignment
6	AP_DMA+0x04	AP_DMA_UART_x_RX_INT_EN	AP_DMA_UART_x_RX_INT_EN[0]	W	1'b1	Controls interrupt enabling or not 0: Disable 1: Enable
7	AP_DMA+0x08	AP_DMA_UART_x_RX_EN	AP_DMA_UART_x_RX_EN[0]	W	1'b1	Enables UART Rx virtual FIFO DMA 0: Disable 1: Enable
8	Waiting for interrupt					
9	AP_DMA+0x00	AP_DMA_UART_x_RX_INT_FLAG	AP_DMA_UART_x_RX_INT_FLAG[0]	W	1'b0	Clears interrupt flag
10	Pop data to virtual FIFO					
11	AP_DMA+0x30	AP_DMA_UART_x_RX_VFF_RPT	AP_DMA_UART_x_RX_VFF_RPT[15:0]	W	-	Read pointer of byte alignment FIFO
12	AP_DMA+0x04	AP_DMA_UART_x_RX_INT_EN	AP_DMA_UART_x_RX_INT_EN[0]	W	1'b1	Controls interrupt enabling 0: Disable 1: Enable
13	Repeat steps 8 to 12					
14	AP_DMA+0x10	AP_DMA_UART_x_RX_STOP	AP_DMA_UART_x_RX_STOP[0]	W	1'b0	Stops UART Rx virtual FIFO

3.9 Command Queue DMA (CQ_DMA)

3.9.1 Introduction

The command queue DMA (CQDMA, or CQ_DMA) enables data transfers between different slaves, which there are several slaves in a platform, and the major one is external memory (e.g., SDRAM).

3.9.2 Features

- Supports 4 GDMA channels
- Compliant with the system bus (AXI)
 - Supports unaligned data transfer (byte alignment)
- A data FIFO of 16 x 8 bytes embedded in each DMA channel
- Source/Destination configuration
 - Both source and destination are programmable.
- Burst size of 8 bytes
- Burst length of 1 to 7 beats
- DMA transfer length (maximum and minimum) (unit: byte):
 - Without address-wrapping -> (256M - 1, 1)
 - With address-wrapping -> (512M - 2, 1)
- Source or destination address fix mode
- Source or destination address wrapping mode
- Source and destination address increment mode
- Fixed pattern mode
- Bandwidth limiter
- TrustZone:
 - The corresponding channel is treated as a secure channel if SEC_EN(CQ_DMA_G_DMA_SEC_EN)[0] = 1.
 - If the channel is set to a secure one, it can issue secure requests, and its configuration registers can only be accessed via secure masters (interrupt notification).
 - Completion of transfers for a specified amount count
 - Supports the lock mechanism to SEC_EN and DOMAIN in every channel
 - SEC_EN and DOMAIN in every channel in a secure world by default
 - LOCK and DLOCK in every channel in a secure world by default
- Supports descriptor mode

3.9.3 Block Diagram

The CQDMA serves for memory copies and registers read/write in batches, and it supports the address wrapping function, burst mode transfer, increment and fixed mode on the source/destination address which both addresses are programmed as memory space or control register space.

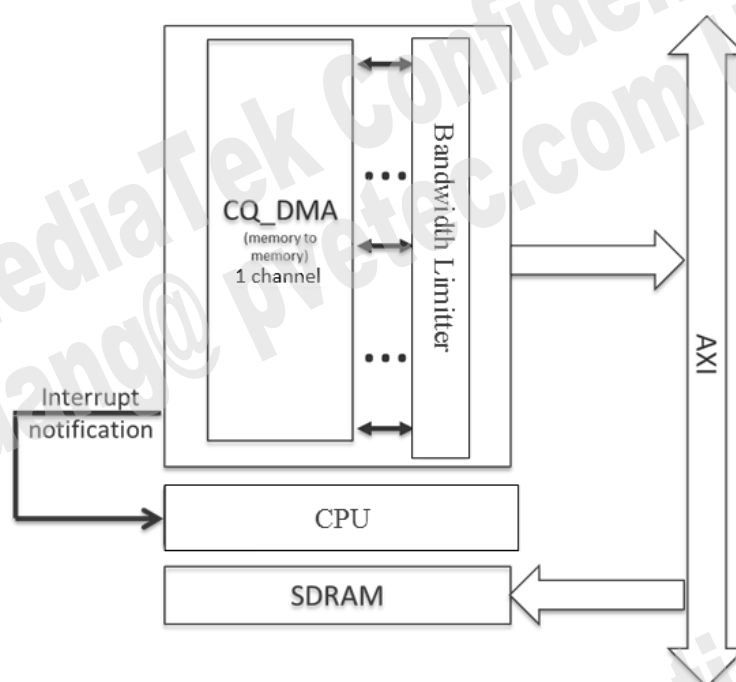


Figure 3-14. Block diagram of CQDMA

3.9.4 Special Modes

3.9.4.1 Fixed Pattern Mode

In the fix-pattern mode, the read AXI path in the CQDMA is blocked that it can only issue write requests of the fixed pattern to the destination address. The descriptions of the parameters in [Table 3-15](#) are as follows:

- CQ_DMA_G_DMA_LEN1 (CQ_DMA+0x0024): The transaction size
- CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020): The destination address
- CQ_DMA_G_DMA_SRC_ADDR (CQ_DMA+0x001C): The pattern

Table 3-15. Reference settings for fixed pattern mode

Step	Address	Register name	Local address	R/W	Value	Description
1	CQ_DMA+0x24	CQ_DMA_G_DMA_x_LEN1	CQ_DMA_G_DMA_x_LEN1[19:0]	W	Length	General DMA transfer length
2	CQ_DMA+0x20	CQ_DMA_G_DMA_x_DST_ADDR CQ_DMA_G_DMA_x_DST_ADDR2	CQ_DMA_G_DMA_x_DST_ADDR[31:0] CQ_DMA_G_DMA_x_DST_ADDR2[3:0]	W	Address	General DMA destination address
3	CQ_DMA+0x1c	CQ_DMA_G_DMA_x_SRC_ADDR CQ_DMA_G_DMA_x_SRC_ADDR2	CQ_DMA_G_DMA_x_SRC_ADDR[31:0] CQ_DMA_G_DMA_x_SRC_ADDR2[3:0]	W	Pattern	General DMA fixed pattern
4	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[0]	W	1'b1	0: Not use fixed pattern 1: Use fixed pattern
5	CQ_DMA+0x8	CQ_DMA_G_DMA_x_EN	CQ_DMA_G_DMA_x_EN[0]	W	1'b1	0: Disable 1: Enable

3.9.4.2 Address Wrapping Mode

This section describes the wrapping mechanism, where WRAP_EN (CQ_DMA+0x0018) [15] is the enable bit of the address wrapping function. There are two address wrapping modes that are determined by WRAP_SEL (CQ_DMA+0x0018) [20].

Read Address Wrapping Mode

1. See the left part of [Figure 3-15](#) for the dataflow of the read address wrapping mode (WRAP_EN (CQ_DMA+0x0018) [15] = 1 and WRAP_SEL (CQ_DMA+0x0018) [20] = 0).
2. Set CQ_DMA_G_DMA_SRC_ADDR (CQ_DMA+0x001C) and CQ_DMA_G_DMA_JUMP_ADDR (CQ_DMA+0x002C) as the start address and jump address respectively.
3. CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020) is the destination address.
4. Use CQ_DMA_G_DMA_LEN1 (CQ_DMA+0x0024) and CQ_DMA_G_DMA_LEN2 (CQ_DMA+0x0028) as the transfer length. There are two sub-transitions.
5. Copy CQ_DMA_G_DMA_LEN1 (CQ_DMA+0x0024) size data from CQ_DMA_G_DMA_SRC_ADDR (CQ_DMA+0x001C) to CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020).
6. Copy CQ_DMA_G_DMA_LEN2 (CQ_DMA+0x0028) size data from CQ_DMA_G_DMA_JUMP_ADDR (CQ_DMA+0x002C) to (CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020) + CQ_DMA_G_DMA_LEN1 (CQ_DMA+0x0024)) continuously.

Table 3-16. Reference settings for read address wrapping mode

Step	Address	Register name	Local address	R/W	Value	Description
1	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[15]	W	1'b1	Enables general DMA wrap or double buffer
2	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[20]	W	1'b0	0: Read pointer (source) 1: Write pointer (destination)
3	CQ_DMA+0x1c	CQ_DMA_G_DMA_x_SRC_ADDR CQ_DMA_G_DMA_x_SRC_ADDR2	CQ_DMA_G_DMA_x_SRC_ADDR[31:0] CQ_DMA_G_DMA_x_SRC_ADDR2[3:0]	W	Address	General DMA SRC address
4	CQ_DMA+0x2c	CQ_DMA_G_DMA_x_JUMP_ADDR CQ_DMA_G_DMA_x_JUMP_ADDR2	CQ_DMA_G_DMA_x_JUMP_ADDR[31:0] CQ_DMA_G_DMA_x_JUMP_ADDR2[3:0]	W	Address	General DMA jump address
5	CQ_DMA+0x24	CQ_DMA_G_DMA_x_LEN1	CQ_DMA_G_DMA_x_LEN1[19:0]	W	Length	General DMA transfer length
6	CQ_DMA+0x28	CQ_DMA_G_DMA_x_LEN2	CQ_DMA_G_DMA_x_LEN2[19:0]	W	Length	General DMA transfer length
7	CQ_DMA+0x20	CQ_DMA_G_DMA_x_DST_ADDR CQ_DMA_G_DMA_x_DST_ADDR2	CQ_DMA_G_DMA_x_DST_ADDR[31:0] CQ_DMA_G_DMA_x_DST_ADDR2[3:0]	W	Address	General DMA destination address
8	CQ_DMA+0x8	CQ_DMA_G_DMA_x_EN	CQ_DMA_G_DMA_x_EN[0]	W	1'b1	0: Disable 1: Enable

Write Address Wrapping Mode

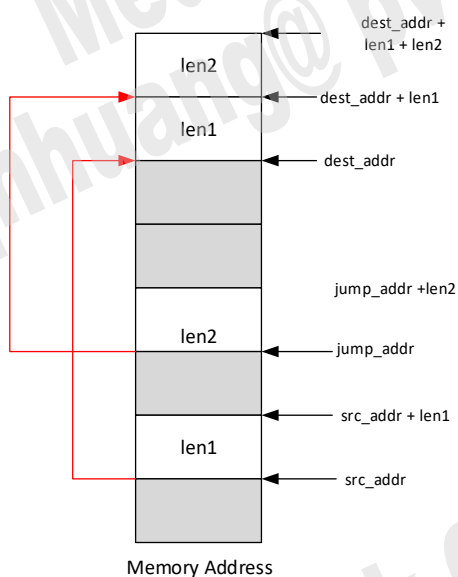
See the right part of [Figure 3-15](#) for the write address wrapping mode (WRAP_EN (CQ_DMA+0x0018) [15] = 1 and "WRAP_SEL (CQ_DMA+0x0018) [20] = 1). The configuration and workflow follow the same logic as the read address wrapping mode, but the direction is inverse.

Table 3-17. Reference settings for write address wrapping mode

Step	Address	Register name	Local address	R/W	Value	Description
1	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[15]	W	1'b1	Enables general DMA wrap or double buffer
2	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[20]	W	1'b1	0: Read pointer (source) 1: Write pointer (destination)
3	CQ_DMA_0x1c	CQ_DMA_G_DMA_x_SRC_ADDR CQ_DMA_G_DMA_x_SRC_ADDR2	CQ_DMA_G_DMA_x_SRC_ADDR[31:0] CQ_DMA_G_DMA_x_SRC_ADDR2[3:0]	W	Address	General DMA SRC address
4	CQ_DMA+0x24	CQ_DMA_G_DMA_x_LEN1	CQ_DMA_G_DMA_x_LEN1[19:0]	W	Length	General DMA transfer length
5	CQ_DMA+0x28	CQ_DMA_G_DMA_x_LEN2	CQ_DMA_G_DMA_x_LEN2[19:0]	W	Length	General DMA transfer length
6	CQ_DMA+0x20	CQ_DMA_G_DMA_x_DST_ADDR CQ_DMA_G_DMA_x_DST_ADDR2	CQ_DMA_G_DMA_x_DST_ADDR[31:0] CQ_DMA_G_DMA_x_DST_ADDR2[3:0]	W	Address	General DMA destination address
7	CQ_DMA+0x2c	CQ_DMA_G_DMA_x_JUMP_ADDR CQ_DMA_G_DMA_x_JUMP_ADDR2	CQ_DMA_G_DMA_x_JUMP_ADDR[31:0] CQ_DMA_G_DMA_x_JUMP_ADDR2[3:0]	W	Address	General DMA jump address
8	CQ_DMA+0x8	CQ_DMA_G_DMA_x_EN	CQ_DMA_G_DMA_x_EN[0]	W	1'b1	0: Disable 1: Enable

Read Address Wrapping

(wrap_en = 1 / wrap_sel = 0)



Write Address Wrapping

(wrap_en = 1 / wrap_sel = 1)

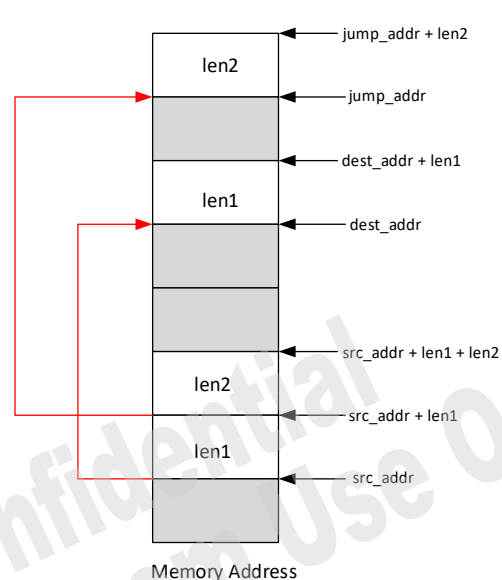


Figure 3-15. CQ_DMA address wrapping

3.9.4.3 Source/Destination Address Fixed Mode

For cases of a fixed write address (WADDR_FIX_EN (CQ_DMA+0x0018) [3]), the AXI write burst type is FIXED. Note that the restriction of the destination address (CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020)) must be in the SRAM, and the source address (CQ_DMA_G_DMA_SRC_ADDR (CQ_DMA+0x001C)) can be in the SRAM or the SDRAM. The behavior of a fixed read address (RADDR_FIX_EN (CQ_DMA+0x0018) [4]) is the same as the fixed write address above, but the direction and the restriction are both inverse.

The source address (CQ_DMA_G_DMA_SRC_ADDR (CQ_DMA+0x001C)) must be in the SRAM, and the destination address (CQ_DMA_G_DMA_DST_ADDR (CQ_DMA+0x0020)) can be in the SRAM or the SDRAM.

Table 3-18. Reference settings for source address fix mode

Step	Address	Register name	Local address	R/W	Value	Description
1	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[4]	W	1'b1	General DMA fixed read address
2	CQ_DMA+0x1c	CQ_DMA_G_DMA_x_SRC_ADDR CQ_DMA_G_DMA_x_SRC_ADDR2	CQ_DMA_G_DMA_x_SRC_ADDR[31:0] CQ_DMA_G_DMA_x_SRC_ADDR2[3:0]	W	Address	General DMA SRC address
3	CQ_DMA+0x24	CQ_DMA_G_DMA_x_LEN1	CQ_DMA_G_DMA_x_LEN1[19:0]	W	Length	General DMA transfer length
4	CQ_DMA+0x20	CQ_DMA_G_DMA_x_DST_ADDR CQ_DMA_G_DMA_x_DST_ADDR2	CQ_DMA_G_DMA_x_DST_ADDR[31:0] CQ_DMA_G_DMA_x_DST_ADDR2[3:0]	W	Address	General DMA destination address
5	CQ_DMA+0x8	CQ_DMA_G_DMA_x_EN	CQ_DMA_G_DMA_x_EN[0]	W	1'b1	0: Disable 1: Enable

Table 3-19. Reference settings for destination address fix mode

Step	Address	Register name	Local address	R/W	Value	Description
1	CQ_DMA+0x18	CQ_DMA_G_DMA_x_CON	CQ_DMA_G_DMA_x_CON[3]	W	1'b1	General DMA fixed write address
2	CQ_DMA+0x1c	CQ_DMA_G_DMA_x_SRC_ADDR CQ_DMA_G_DMA_x_SRC_ADDR2	CQ_DMA_G_DMA_x_SRC_ADDR[31:0] CQ_DMA_G_DMA_x_SRC_ADDR2[3:0]	W	Address	General DMA SRC address
3	CQ_DMA+0x24	CQ_DMA_G_DMA_x_LEN1	CQ_DMA_G_DMA_x_LEN1[19:0]	W	Length	General DMA transfer length
4	CQ_DMA+0x20	CQ_DMA_G_DMA_x_DST_ADDR CQ_DMA_G_DMA_x_DST_ADDR2	CQ_DMA_G_DMA_x_DST_ADDR[31:0] CQ_DMA_G_DMA_x_DST_ADDR2[3:0]	W	Address	General DMA destination address
5	CQ_DMA+0x8	CQ_DMA_G_DMA_x_EN	CQ_DMA_G_DMA_x_EN[0]	W	1'b1	0: Disable 1: Enable

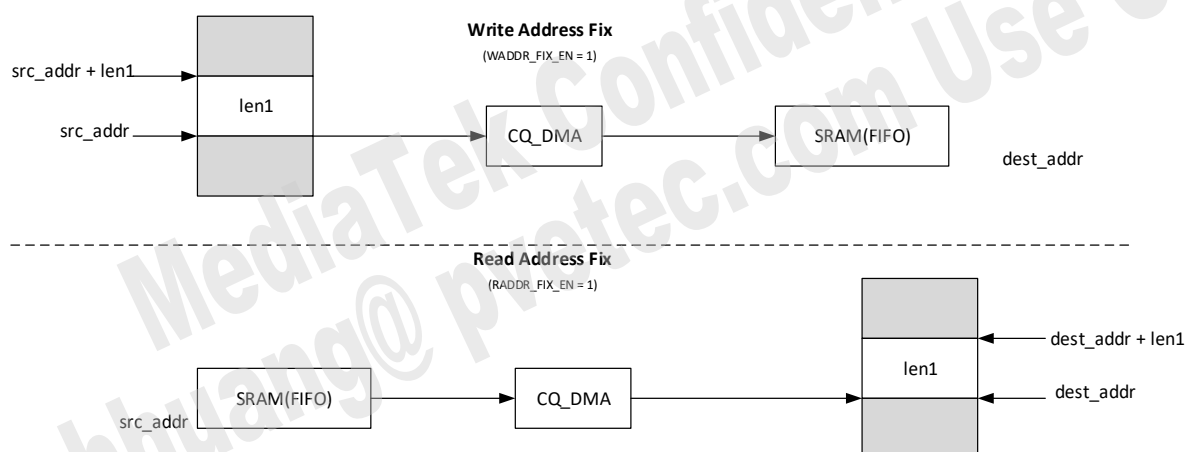


Figure 3-16. CQ_DMA address fixed mode

3.9.4.4 Descriptor Mode

See [Figure 3-17](#) for the descriptor mode mechanism. Descriptor-based DMA transfers require a set of parameters stored in the memory to initiate a DMA sequence. The descriptor contains all the same parameters programmed into the DMA control register set. The descriptor base address (CQ_DMA+0x006c) should be set first to enable the descriptor DMA mode (CQ_DMA+0x007c)[1].

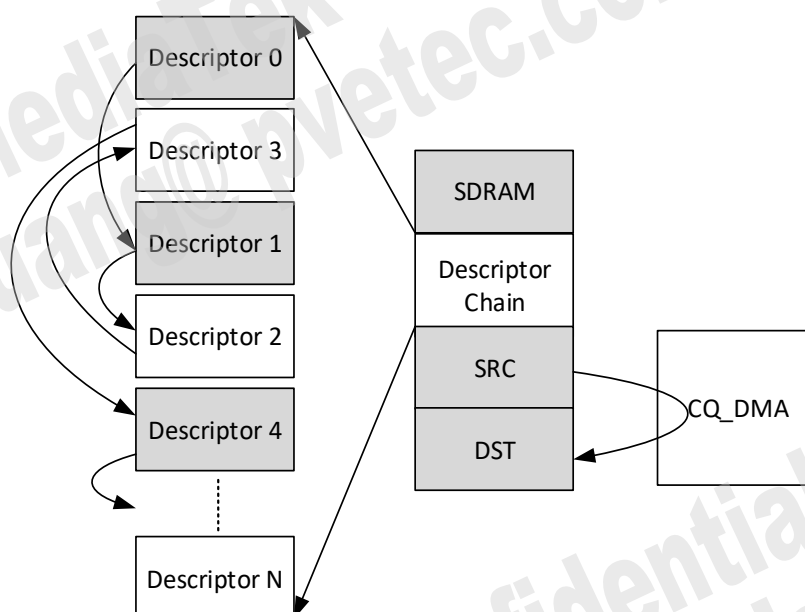


Figure 3-17. CQ_DMA descriptor mode diagram

3.9.4.5 Bandwidth Limiter

In case that the bus traffic is already heavy but DMA still needs to issue requests, there is an option to hold the DMA operation from issuing a read transaction. This transaction is issued with delay for a pre-configured clock cycle period from

0 to 1,023. The slow-down counter only slows down the read transaction without affecting the write transaction. However, the overall throughput is reduced.

SLOW_EN (CQ_DMA+0x0018) [2] is the bit to enable the bandwidth limiter function; and SLOW_CNT (CQ_DMA+0x0018) [14:5] is the delay cycle period. These fields should be configured before enabling DMA transfer.

3.9.4.6 Interrupt Logic

The completion of the transfer results in assertion of interrupt. Each channel has its own interrupt number.

3.9.5 Register Definition

See chapter 1.9 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

3.10 System Companion Processor (SCP)

3.10.1 Introduction

The system companion processor (SCP) is a tiny processor subsystem with MDSP RV55NN processor and peripherals. The power domain of the sensor hub subsystem is always-on and the clock source is very low power. The special design in the power domain and the clock source makes the subsystem suitable to run applications, such as voice wake-up, sensor hub and future tasks when the entire SoC is in the suspend mode. The subsystem is connected to the infra-bus and therefore can access DRAM, audio system SRAM and the other hardware resources through the infra-bus. The subsystem also has another slave interface for the APMCU to configure the subsystem.

3.10.2 Features

The MDSP RV55NN processor is a 32-bit microprocessor core with the following features:

- 32 KB I-cache and 32 KB D-cache
- AXI interface & prefetch
- 8-stage pipeline

The SCP features:

- 1 MDSP RV55NN core
- 2 MB L2TCM
- An AXI master interface for the SoC site memory access (registers/DRAM)
- An AXI slave interface for APMCU configuration
- Inter-processor interrupts to APMCU
- 6 32-bit down-count timers for RV55NN core, clock source is selectable
- An interrupt controller for RV55NN core
- Supports 32-input external interrupt with debounce function
- 7 I2Cs
- 4 SPIs
- 2 UARTs
- 32-bit GPIO
- Direct path to the PMIC wrapper
- Watchdog timer

3.10.3 Block Diagram

The following diagram shows the SCP subsystem and the related connections with AP SoC.

Figure 3-18. Block diagram of SCP

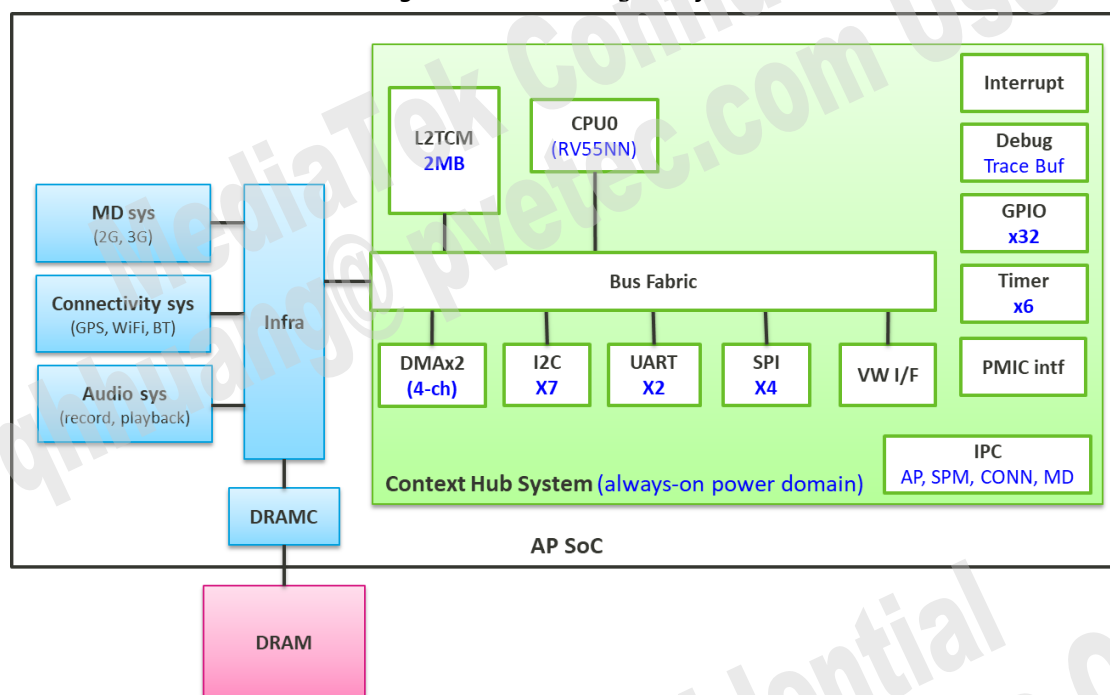


Table 3-20. SCP frequency

SCP frequency (MHz)	SCP clock source
800	ULPOSC

3.10.4 Register Definition

See chapter 1.10 of "MT8676 5G Connected Cockpit Application Processor Software Register Table".

3.11 System Level Cache/Buffer (SLC/SLB)

3.11.1 Introduction

The system level cache (SLC) controller schedules requests from the external memory interface (EMI) and issues commands to the DRAM in an efficient way. It also minimizes the latency of processor path to enhance the performance and reduce DRAM accessing power.

The system level buffer (SLB) controller schedules requests from the masters and issues commands to the SRAM in an efficient way.

The SLC/SLB also informs clock control to gate the clock when it does not find any transaction. The SLC/SLB is designed to supply the same clock with the EMI.

3.11.2 Features

- SLC features
 - 4 MB with 16-way cache
 - 64-byte cache line (block) size
 - Write-through policy
 - Bypass mode with coherency
 - Tag access on demand; no data operation
 - Reserved way allocation
 - Constrained read/write allocation
 - Supports 64-byte-aligned 64B/128B payload (all write strobe bits must be asserted)
 - Single outstanding per-ARID
 - MPLRU replacement policy
 - Speculative fetch
 - EMI early abort for bandwidth reduction
- SLB features
 - SLC and SLB can be runtime partitioned on the basis of way.
 - SLB can be accessed via CEN_EMI path and dedicated SLB bus.
 - Individual SLB port
 - Memory allocated address (0x0f00_0000~0x0f3f+fff)
 - Max. 256-bit interface (for 728 MHz, total bandwidth = 23 GB/s x DRAM channel)
 - Pipeline latency: 7T
 - Via EMI
 - Configurable memory allocated address
 - AXI addressing: Access range must not cross 128 bytes boundary
 - Pipeline latency: 15T

3.11.3 Block Diagram

The SLB controller connects three systems via two AXI ports and supports connection to SRAM devices. This is for the multimedia/AI/audio system.

The SLB controller connects two systems via the EMI and supports connection with SRAM/DRAM devices. This is for the APMCU and GPU.

In the SLC/SLB controller, the APB interface is for programming registers then initializing the DRAM or other parameter settings.

SLC/SLB Interface

- ① Cen2chn interface
- ② AXI interface
- ③ AXI interface

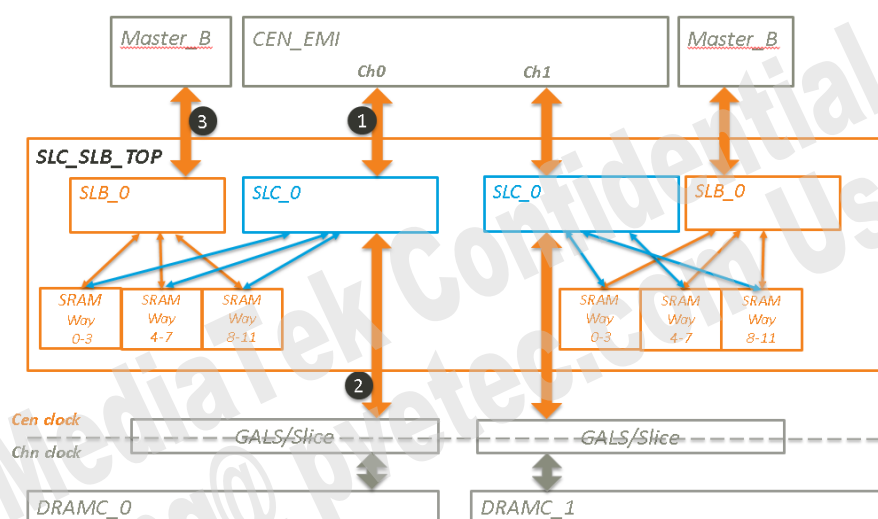


Figure 3-19. EMI/SLC controller top connection

3.11.4 Register Definition

See chapter 1.11 of "MT8676 5G Connected Cockpit Application Processor Software Register Table".

4 Clock and Power Control

4.1 Top Clock Generator (TOPCKGEN)

4.1.1 Introduction

This chapter introduces the top clock generator (TOPCKGEN) and the clock architecture.

4.1.2 Features

The TOPCKGEN is responsible for generating the following clock signals:

- Free clock generation for the whole chip
- Infrastructure and peripheral system clock, including the top level AXI fabric clock
- Multimedia system clock
- Pad macro clocks to be synchronized with one of the above systems

The TOPCKGEN module provides the clock source selection that each clock has several clock source selections and can be turned off. When switching certain clock from frequency A to frequency B, make sure they are available.

This module comprises the glitch-free clock MUX and a digital clock divider to generate various clock frequencies.

4.1.3 Block Diagram

4.1.3.1 Clock Architecture

There are clock generators not only in the top-level hierarchy but also in every partition/system. [Figure 4-1](#) shows the location of the top-level clock generator.

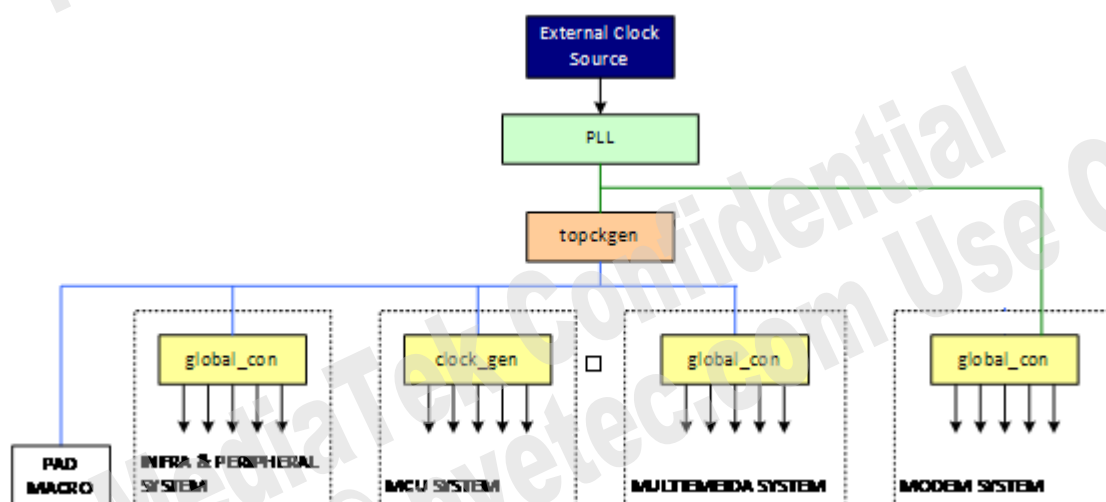


Figure 4-1. Block diagram of clock architecture

4.1.3.2 Clock Multiplier

The clock selection and generation have similar structures (see [Figure 4-2](#)). Several clock sources are provided, and one can be selected by specified register settings. The turn-off bit is provided to stop the clock output.

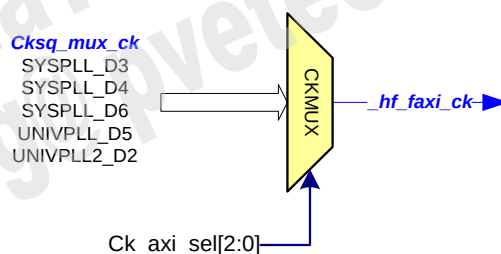


Figure 4-2. Example of clock multiplier

4.1.4 Clock PLL

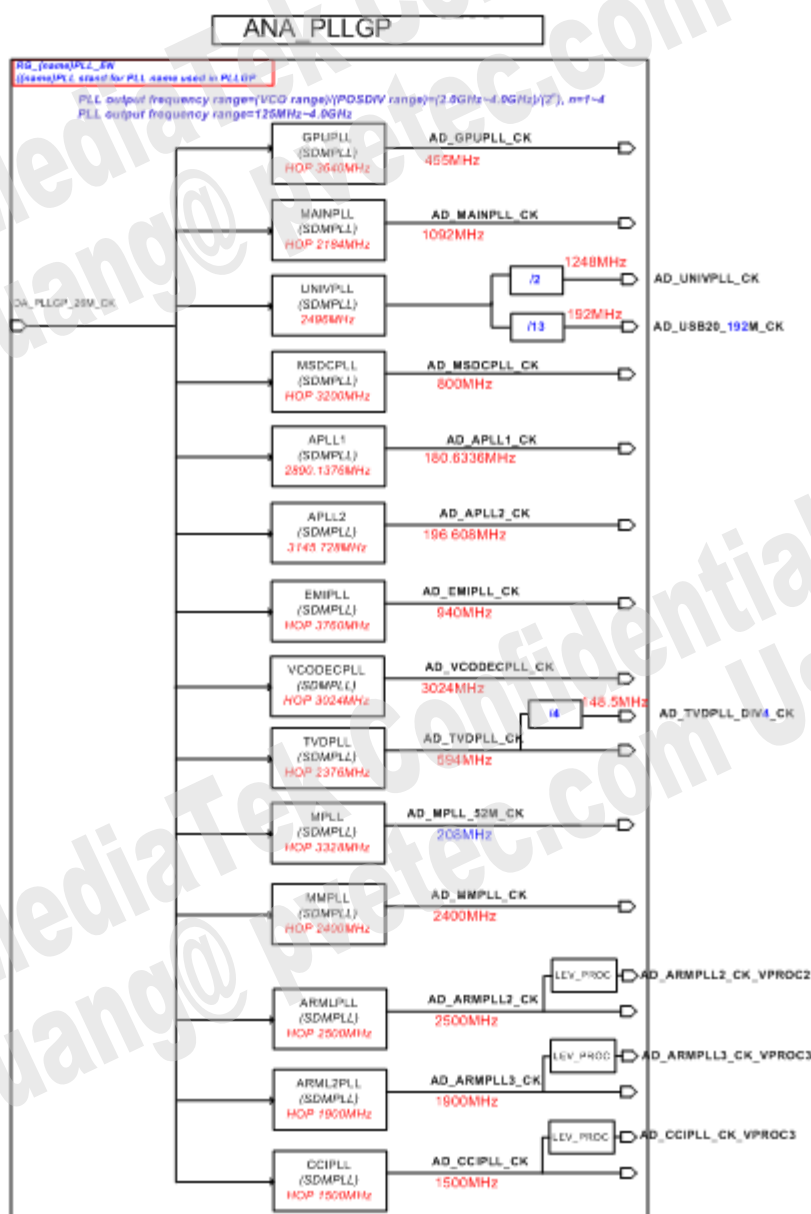


Figure 4-3. PLL block diagram (for reference only)

4.1.5 PLL Related Control

The following table lists all PLLs inside the application system.

The enabling of PLL can be switched between the software control and the hardware control. The hardware control is from SCPSYS.

The hopping and SSC features can also be switched between the software control and the hardware control. The hardware control is from FHCTL.

Table 4-1. PLL related control

PLL	Capability	Control by FHCTL	Control by SPM
ARMPLL_L	Hopping, SSC	Y	N
ARMLPLL_BL (0, 1, 2, 3)	Hopping, SSC	Y	N
CCIPLL	Hopping, SSC	Y	N
MAINPLL	Hopping, SSC	Y	Y
UNIVPLL	Fix	N	Y
MSDCPLL	Hopping, SSC	Y	N
MMPLL	Hopping, SSC	Y	N
TVDPLL	Hopping, SSC	Y	N
ADSPPLL	Hopping, SSC	Y	N
APLL1	Fix	N	N
APLL2	Fix	N	N
MIPI	SSC	N	N
MPLL	Hopping, SSC	Y	N
APUPLL	Hopping, SSC	Y	N
IMGPLL	Hopping, SSC	Y	N

4.1.6 Clock Gating

The clock gating for the TOPCKGEN module is listed in the following table where the DCM and turn-off settings are provided.

Table 4-2. Clock gating settings

Register name	Bit	Default	Function name	Description
CLK_MODE	8	1'b0	pdn_md_32k	Turns off 32K clock source to MD
CLK_SCP_CFG_0	[0]	1'b0	sc_26ck_off_en	Turns on SCPSYS control path to gate 26 MHz
	[2]	1'b0	sc_axick_off_en	Turns on SCPSYS control path to gate hf_faxi_ck
	[5]	1'b0	sc_md_32k_off_en	Turns on SCPSYS control path to gate MD 32 kHz
	[6]	1'b0	sc_md_26m_off_en	Turns on SCPSYS control path to gate MD 32 kHz
	[9]	1'b0	sc_mac_26m_off_en	Turns on SCPSYS control path to gate MIPI 26 MHz
CLK_SCP_CFG_1	[0]	1'b0	sc_axi_26m_sel_en	Turns on SCPSYS control path to switch hf_faxi_ck to 26 MHz
	[2]	1'b0	sc_scpcck_26m_sel_en	Turns on SCPSYS control path to switch hf_fscp_ck to 26 MHz
	[3]	1'b0	sc_pmicpick_26m_sel_en	Turns on SCPSYS control path to switch hf_fpmicspi_ck to 26 MHz

4.1.7 Frequency Meter

There is one frequency meter inside the TOPCKGEN. There are two input clock sources, one is for PLLs and TEST clock; the other is for the clocks generated from the TOPCKGEN.

There is PAD output that can observe frequency directly instead of reading results from the frequency meter through DEBUG_MON[0].

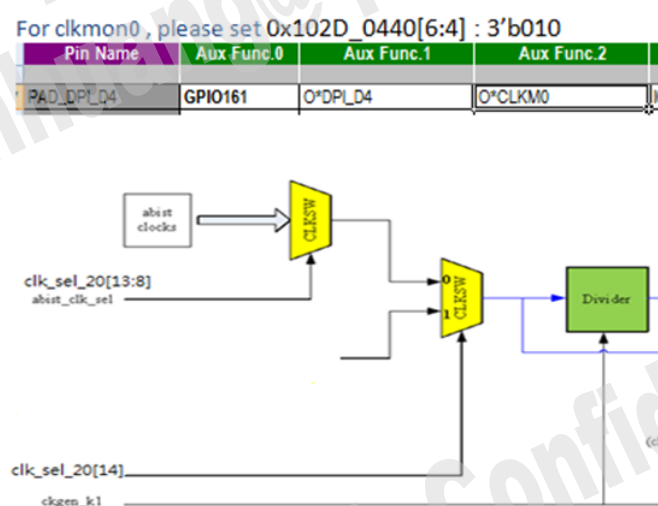
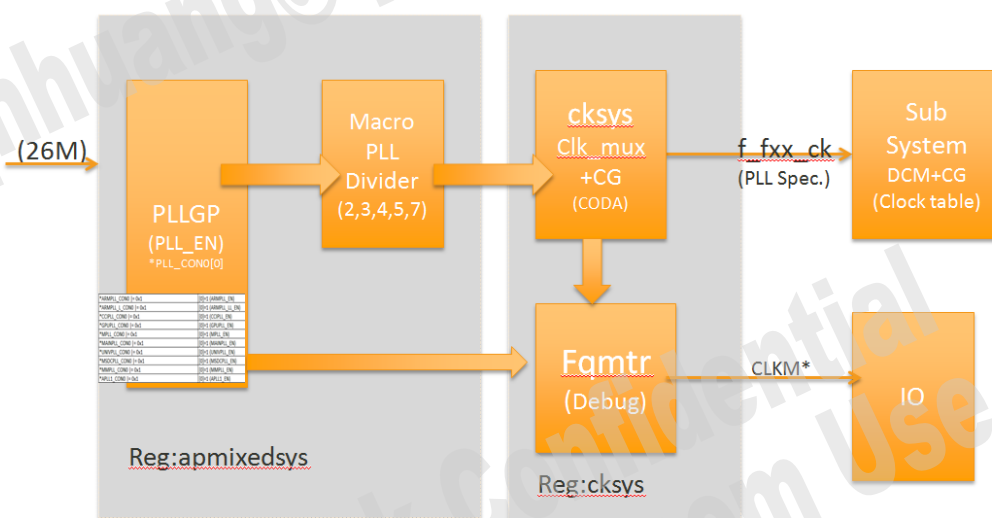


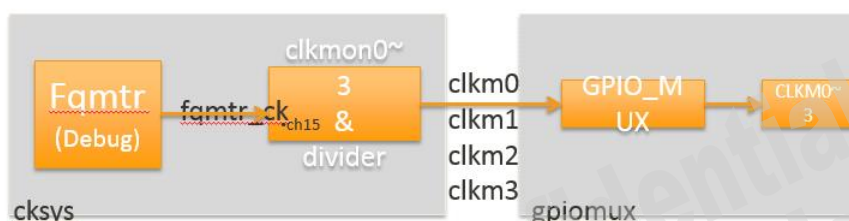
Figure 4-4. TOPCKGEN frequency meter structure

Clock architecture



Fqmtr clock to IO path

pad_name	skip	cond	GPIO idx	MODE
PAD_SPI4_MI	0	13 ENT13	B GPIO13 O ANT_SE0_SPI4_MI	N SOP_SF11MO_UO_O_QUM0
PAD_SPI4_CSB	0	14 ENT14	B GPIO14 O ANT_SE0_SPI4_CSB	O SOP_SF11MO_UO_O_QUM1
PAD_SPI4_MO	0	15 ENT15	B GPIO15 O ANT_SE0_SPI4_MO	O SOP_SF11MO_UO_O_QUM2
PAD_SPI4_CLK	0	16 ENT16	B GPIO16 O ANT_SE0_SPI4_CLK	O SOP_SF11MO_UO_O_QUM3



CKMUX Clock off

The clock can be turned on/off through changing the value of `pdn_*`. However, this control bit cannot be switched along with `clk_*_sel` and `clk_*_inv`.

It is recommended to change `pdn_*` with SET and CLEAR

10000004		CLK_CFG_UPDATE														Clock Configuration Update Register														00000000	
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16															
Name		msdc30_1_ck_update	msdc50_0_ck_update	msdc50_0_hclk_ck_update	spl_ck_update	uart_ck_update	camtg4_ck_update	camtg3_ck_update	camtg2_ck_update	camtg2_ck_update	mfg_ck_update	ipu_if_ck_update	dsp7_c_k_update	dsp6_c_k_update	dsp5_c_k_update	dsp4_c_k_update															
Type		WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO															
Reset		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0															
Name	dsp3_c_k_update	dsp2_c_k_update	dsp1_c_k_update	dsp0_ck_update	ccu_ck_update	cam_ck_update	dpe_ck_update	ipe_ck_update	img2_c_k_update	img1_c_k_update	mde_c_k_update	disp_ck_update	bus_axi_mem_c_k_update	scp_ck_update	spm_ck_update	axi_ck_update															
Type	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO															
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															

10000010					CLK_CFG_0												Function Clock Selection Register 0												00000000			
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16																
Name	pdn_bus_axi_mem			clk_bus_axi_mem_inv			clk_bus_axi_mem_sel			pdn_scp_r			clk_scp_inv		clk_scp_sel																	
Type	RW			RW			RW			RW			RW		RW																	
Reset	0			0			0	0	0	0		0		0	0	0																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																
Name	pdn_spm			clk_spm_inv			clk_spm_sel			pdn_axi_i			clk_axi_inv		clk_axi_sel																	
Type	RW			RW			RW			RW			RW		RW																	
Reset	0			0			0	0	0	0		0		0	0	0																

Frequency meter

- FW setting
- For ABIST
 - clk_dbg_cfg[1: 0](0x1000010C): fqmtr_bank_sel: , select 0
 - clk_dbg_cfg[21:16](0x1000010C): abist_clk_sel[5:0] : , select abist_clk0 ~ 63
- For CKGEN
 - clk_dbg_cfg[1:0](0x1000010C): fqmtr_bank_sel: , select 1
 - clk_dbg_cfg[13:8](0x1000010C): ckgen_clk_sel[5:0] : , select ckgen_clk0 ~ 63
- clk_misc_cfg_0[31:24](0x10000110) : abist_k1[7:0] --> set to 0 for non-divider mode
- freq meter enable CLK26CALI_0[12] (0x100000220): set to 1, enable meter
- freq meter enable CLK26CALI_1[25:16] (0x10000224): set load_cnt to 0x1ff, measure cycle
- abist_tri_cal CLK26CALI_0[4] (0x10000220) : set to 1, start measure
- Wait abist_tri_cal CLK26CALI_0[4] (0x10000220) : became 1'b0 , calibration is completed.
- Read abist_cal_cnt CLK26CALI_1(0x10000224[15:0])
- For example:

Write	CLK26CALI_1	0x10000224	0x01FF0000	[25:16]:load_cnt
Write	CLK_MISC_CFG_0	0x10000110	0x00000000	[31:24]:setup meter div
Write	CLK_DBG_CFG	0x1000010C	0x00250000	[14]=1:abist_clk [12:8]:select ULPOSC1_CK
Write	CLK26CALI_0	0x10000220	0x00000081	[0]:Start

- Measure Freq = Read value (CLK26CALI_1[15:0]) * 26 / (CLK26CALI_1[25:16]+ 1);

4.1.8 Register Definition

See chapter 2.1 of "MT8676 5G Connected Cockpit Application Processor Software Register Table".

4.1.9 Programming Guide

4.1.9.1 Clock Off

The clock can be turned on/off through changing the value of pdn_*. However, this control cannot be switched along with clk_*_sel and clk_*_inv. It is better to change pdn_* with the set and clear functions (CLK_CFG_*_SET and CLK_CFG_*_CLR) as there may be a clock with multi-bit pdn_* which are planned to avoid "read/modify/write" from different subsystems (APSYS and MDSYS) at the same time.

4.1.9.2 Clock Switching

- Make sure clock A and clock B are available before changing the setting of clk_*_sel. When switch to a non-exist clock, the clock switch is stuck until the non-exist clock is turned on to free the clock switch.
- Supports multi-clock switching at the same time (without changing pdn_*)

Switching from clock A to clock B

1. Make sure clock B is ready.

2. Change clk_*_sel.
3. Write *_ck_update
4. Wait until chg_sta = 1'b0 (optional).
5. Turn off clock A (optional).

4.1.9.3 Switch AXI to 26 MHz by SCPSYS

The reflection time is about 13T 26 MHz if all clocks are counted as 26 MHz. Refer to the following formula:

$$2T \text{ Bus Clock}_{(*1)} + 3T \text{ Current Clock}_{(*2)} + 4T \text{ Reference Clock}_{(*3)} + 1T \text{ Bus Clock}_{(*4)} + 3T \text{ Target Clock}_{(*5)}$$

Comment	Description
Bus clock	26 MHz
Ref. clock	26 MHz, balance with bus clock
*1	2T sync
*2	3T sync
*3	4T sync
*4	1T control. For async CLKSW such as hf_faxi_ck used, it is 2T sync.
*5	2T sync

4.2 Top Reset Generation Unit (TOPRGU)

4.2.1 Introduction

The top reset generation unit (TOPRGU) contains a watchdog timer (WDT) and generates reset signals and distributes them to each system.

4.2.2 Features

- Hardware reset signals for the whole chip
- Software controllable reset for each system
- Watchdog timer
- Reset output signals for companion chips
- Dedicated reset input from external MCU for debugging
- In the IOC mode, AP only issues an interrupt to IOC when an exception occurs.

4.2.3 Block Diagram

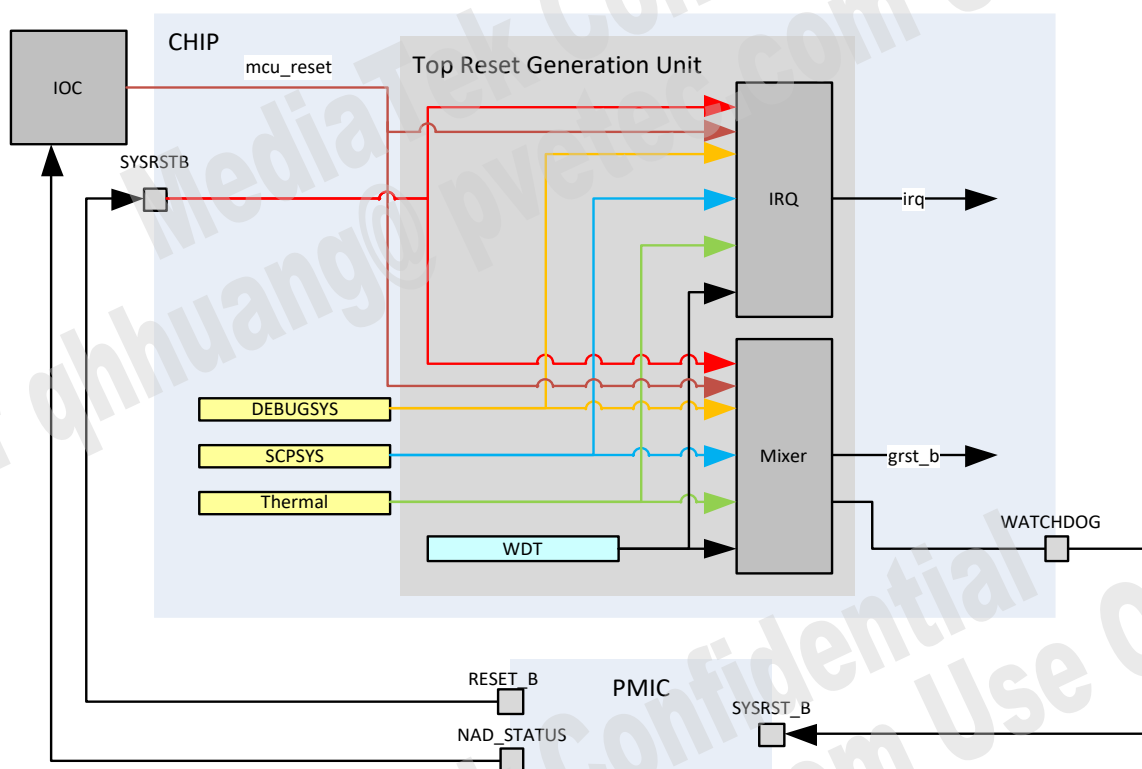


Figure 4-5. Block diagram of TOPRGU

4.2.4 Register Definition

See chapter 2.2 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

4.2.5 Programming Guide

4.2.5.1 TOPRGU Initialization

Enable the dual-mode reset when the TOPRGU is firstly initialized. Since WDT_MODE cannot be reset and the WDT (including WDT_SWRST) IRQ mode is also disabled if system reset is triggered through WDT_SWRST, the registers can only be reset by SYSRSTB.

The following registers cannot be reset by the TOPRGU:

- WDT_MODE
- WDT_STA
- WDT_NONRST_REG
- WDT_NONRST_REG2
- WDT_REQ_MODE
- WDT_REQ_IRQ_EN

4.2.5.2 Watchdog Timer

- Trigger WDT_RESTART right after WDT_LENGTH is updated.
- WDT_SWRST can be triggered without wdt_en set to 1'b1.
- Trigger WDT_RESTART before setting wdt_en to 1'b1.

4.2.5.3 IRQ Mode

Since the dual mode reset is on by default, all reset requests are in the IRQ mode also by default. This means that the interrupt is triggered (instead of the system reset) immediately. To trigger the system reset but not the interrupt, change the corresponding configuration of each reset request which can be configured as reset or IRQ separately.

4.2.5.4 Dual Mode Reset

The dual mode reset is the system watchdog reset after the TOPRGU triggers the interrupt. The watchdog timer should be enabled to fulfill this function.

In this mode, the watchdog timer is *AUTO-RESTART* after the interrupt is triggered. The AP should clear WDT_STA after receiving the interrupt from the TOPRGU. The watchdog timer is restarted to {WDT_LENGTH, {9{1'b1}}} once the interrupt is issued, and the watchdog reset is triggered after the timer expires.

- Set wdt_en = 1'b1.
- Set dual_mode = 1'b1.
- Set wdt_irq, thermal_irq, scpsys_irq or debug_irq to 1'b1.

4.2.5.5 History

WDT_LENGTH should be reset, or the system should enter the RESET loop if WDT_LENGTH is set very short before the reset is triggered.

4.3 Frequency Hopping Controller (FHCTL)

4.3.1 Introduction

The frequency hopping controller (FHCTL) helps the AP resolve desense issues. The RF victims are 2G, 3G, BT, FM, Wi-Fi, GPS, etc.; and the aggressor in the AP is the clock generated from the PLL. The harmony of all clock frequencies may cause band desense of the RF system.

4.3.2 Features

The FHCTL receives the command from the CPU to trigger two mechanisms: **spread spectrum clocking (SSC)** and **frequency hopping**.

Name	Capability	Range
MPLL	Hopping, SSC	{-8%,0}
MEMPLL	Hopping, SSC	{0,0}
MAINPLL	Hopping, SSC	{-2%,0}
MSDCPLL	Hopping, SSC	{-8%,0}
MMPLL	Hopping, SSC	{-2%,0}
ADSPPLL	Hopping, SSC	{-2%,0}
TVDPLL	Hopping, SSC	{-8%,0}
IMGPLL	Hopping, SSC	{-2%,0}

4.3.3 Block Diagram

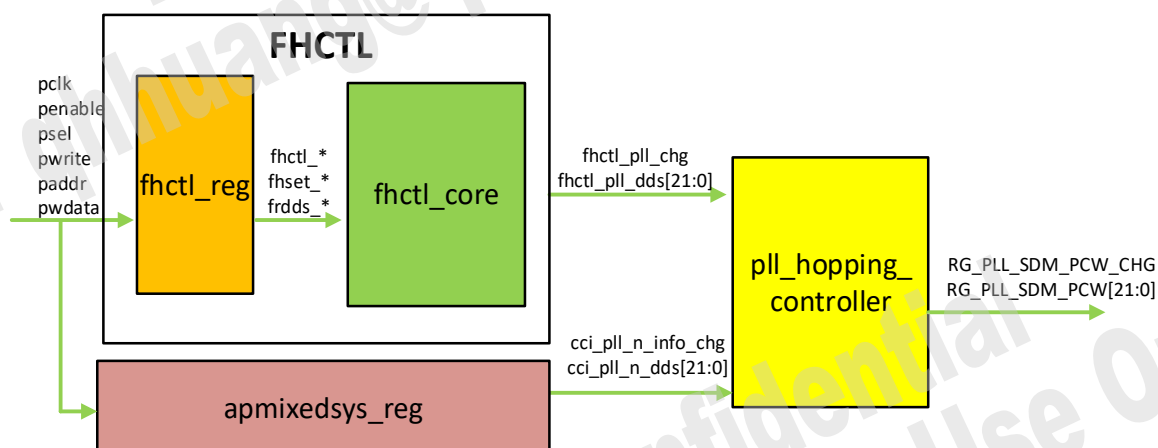


Figure 4-6. Block diagram of FHCTL

4.3.4 Register Definition

See chapter 2.3 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

4.3.5 Theory of Operation

4.3.5.1 Dynamic Voltage and Frequency Scaling (DVFS)

- Enable
 - SFSTRx_EN
- Set soft-start slope in S0.
 - For all PLLs except MEMPLL and MPLL, the slope is controlled by FHCTL_SLOPE0.
 - For MEMPLL and MPLL, the slope is controlled by FHCTL_SLOPE1.
 - Mappings of S1~S5 slopes
 - $S1 = \frac{1}{2} S0$, $S2 = \frac{1}{2} S1$, $S3 = \frac{1}{2} S2$, $S4 = \frac{1}{2} S3$, $S5 = \frac{1}{2} S4$ (auto-mapping)
- Trigger DVFS by writing FHCTLx_DVFS[21:0].
 - FHCTL_CORE will auto-calculate the delta frequency then reach to the target frequency with unit-slope or multi-slope mechanism.
- The overshoot will be OK if the rule is followed.
 - $S5 < 2.5\text{MHz/us}$, $S_{n-1} = 2S_n$ ($n = 0 \sim 4$)
 - Period of each stage $> 4\text{ us}$ (excluding initial stage)
 - $\Delta f_0 = 310\text{ MHz}$, $\Delta f_1 = 150\text{ MHz}$, $\Delta f_2 = 70\text{ MHz}$, $\Delta f_3 = 30\text{ MHz}$, $\Delta f_4 = 10\text{ MHz}$
- Hardware will auto-detect the Δf and use FSM to control the hopping slope.
 - $S0 = (\text{FHCTL_SFDY}[21:0] * 2^{-14} * 26\text{ MHz}) / [(\text{FHCTL_SFDT} + 2) * 38.4\text{ ns}]$
For example, set FHCTL_SFDY[21:0] to max. 15,511, $S_0 = (0.94671630859375 * 26\text{ MHz}) / (8 * 38.4) = 80.01\text{ MHz}$
 - If $\Delta f > 310\text{ MHz}$, it will be auto-separated into 6 hopping stages.
 - f_{tar0} : ($f_{tar} - 310\text{MHz}$), S0
 - f_{tar1} : ($f_{tar} - 150\text{MHz}$), S1
 - f_{tar2} : ($f_{tar} - 70\text{MHz}$), S2
 - f_{tar3} : ($f_{tar} - 30\text{MHz}$), S3
 - f_{tar4} : ($f_{tar} - 10\text{MHz}$), S4
 - f_{tar} , S5
- For all PLLs except MEMPLL and MPLL
 - FHCTL_SLOPE0
 - Default slope: 20 MHz
 - FHCTL_SFDT0 = 8'd6
 - FHCTL_SFDY0 = 22'b00000000_11110100101100
- For MEMPLL and MPLL
 - FHCTL_SLOPE1
 - Default slope: 20 MHz
 - FHCTL_SFDT1 = 8'd6
 - FHCTL_SFDY1 = 22'b00000000_11110100101100

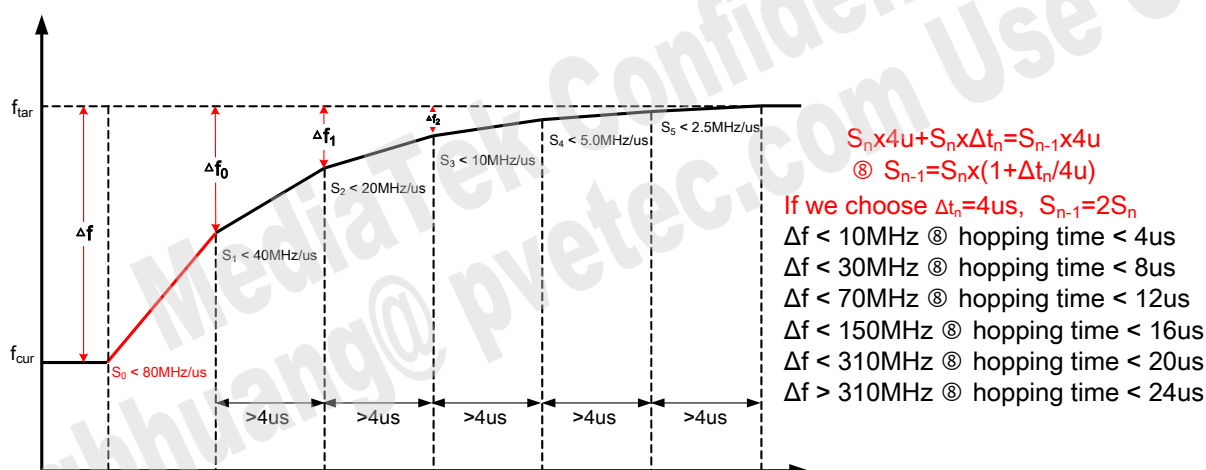


Figure 4-7. Hopping stage diagram

4.3.5.2 SSC

- Set SSC swing range in FHCTLx_UPDNLMT.
 - Upper bound: FRDDsx_UPLMT[15:0]
 - Lower bound: FRDDsx_DNLMT[15:0]
 - SSC upper bound:
 - FHCTLx_PLL_ORG[21:0] + {FRDDsx_UPLMT[15:0], 5'b0}
 - SSC lower bound:
 - FHCTLx_PLL_ORG[21:0] - {FRDDsx_DNLMT[15:0], 5'b0}
- SSC slope
 - FRDDsx_DYS
 - FRDDsx_DTS
- Enable (Set this up after FHCTLx_UPDNLMT is initialized.)
 - FRDDsx_EN

4.3.6 Programming Guide

4.3.6.1 Initialize FHCTL for Each PLL

- Initialize PLL by apmixedsys
- Initialize FHCTL basic setting

- Turn on the clock of each FHCTL_CORE
 - FHCTL_CLK_CON
- Release software-reset of each FHCTL_CORE
 - FHCTL_RST_CON
- Align frequency (SDM_PCW[21:0]) with apmixedsys/DDRPHY
 - Setting

- Setting up FHCTLx_DDS will update the SDM_PCW value into FHCTL_CORE FSM and trigger one TGL signal for PLL to latch this value.
- Mapping of FHCTL core
- Switch to FHCTL_CORE controller
 - FHCTL_HP_EN
- Turn on FHCTL_CORE to make DVFS/SSC work
 - FHCTLx_EN
 - If this bit is turned off, all the features (DVFS/SSC) will not work and the frequency will jump to the original frequency in FHCTLx_DDS[21:0].

4.3.6.2 Enable/Disable Hopping

- Enable hopping
 - Turn on the PLL based on apmixedsys/DDRPHY PLL related registers
 - Controlled by top clock owner
 - Configure FHCTL to align the PLL frequency (SDM_PCW[21:0])
 - Switch control by pll_en from 1'b0 to 1'b1
 - FHCTL_HP_EN
- Disable hopping
 - Disable all FHCTL features to keep the SDM_PCW stable
 - SSC/DVFS
 - Configure apmixedsys/DDRPHY register to align SDM_PCW value
 - Switch control by pll_en from 1'b1 to 1'b0
 - FHCTL_HP_EN

5 Peripherals

5.1 Peripheral System Configuration Module (PERICFG)

5.1.1 Introduction

The peripheral system configuration module (PERICFG) provides reset, clock and miscellaneous control signals in the peripheral system and IPs. Each module inside the peripheral system has its own software reset and clock gated control (power-down control). The hardware dynamic clock management (DCM) of the peripheral system is also contained.

5.1.2 Features

The PERICFG provides the following control signals to the functional blocks inside the peripheral system:

- Software reset signals
- Clock gating control signals
- DCM control signals
- AXI bus fabric control signals
- Sleep protection control signals

5.1.2.1 Dynamic Clock Management (DCM)

The DCM function slows down the clock frequency for power saving when the system is in the idle state automatically.

The clock frequency in the DCM mode is set to quarter of the original clock. The ratio of the clock frequency slow-down is controlled by the PERI_DCMFSEL register.

After the bus idle signal is low, it takes several cycles of latency to make the slow-down clock return to the normal frequency. The cycle number varies with the runtime status of the clock gating logic and somehow causes some minor performance impacts. To minimize the impacts, when the system is in a heavy load status, the PERI_DCMDBC register controls the cycle count once the bus idle signal is asserted. Setting the debounce cycle to be longer and enabling the function can reduce the probability of the system entering DCM mode.

5.1.2.2 AXI Fabric Control

The AXI fabric control registers prevent the system bus from hanging up caused by improper accesses while some parts of the system are in the power-down state. The sleep protector can get the functions above. Refer to the corresponding control registers in [3.5.3](#).

5.1.3 Register Definition

See chapter 5.1 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.2 General-Purpose Input/Output (GPIO) Control

5.2.1 General Descriptions

The I/O pins can be programmed for multiple purposes, including general-purpose input/output (GPIO), NAND, SPI, etc. By setting up the GPIO_MODE register, a specific I/O is selected for a specific function.

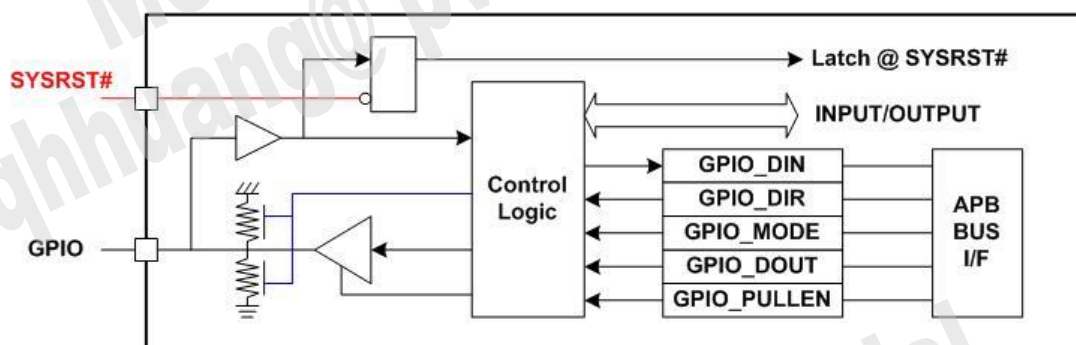


Figure 5-1. GPIO block diagram

All functions should comply with the priority rule. When there is more than one I/O set as the same **output** function, all of the selected I/Os are able to output specific signals. When there is more than one I/O set as the same **input** (or bi-directional) function, only the I/O with the largest GPIO index works functionally.

Table 5-1. Summary of configuration registers of GPIOs

Address base name	Address
IOCFG_BL_BASE	0x11D3_0000
IOCFG_BM_BASE	0x11D4_0000
IOCFG_BR_BASE	0x11D5_0000
IOCFG_LB_BASE	0x11EA_0000
IOCFG_LT_BASE	0x11EB_0000
IOCFG_RM_BASE	0x11C0_0000
IOCFG_RT_BASE	0x11B2_0000
IOCFG_TL_BASE	0x11F3_0000
IOCFG_TM_BASE	0x11F2_0000
GPIO_BASE	0x1000_5000

Name	IES	SMT	PU	PD	PUPD
PAD_ANT_SELO	IOCFG_RT_BASE+0x0090[0]	IOCFG_RT_BASE+0x0170[0]	IOCFG_RT_BASE+0x00e0[0]	IOCFG_RT_BASE+0x00b0[0]	-
PAD_ANT_SEL1	IOCFG_RT_BASE+0x0090[1]	IOCFG_RT_BASE+0x0170[1]	IOCFG_RT_BASE+0x00e0[1]	IOCFG_RT_BASE+0x00b0[1]	-
PAD_ANT_SEL10	IOCFG_RT_BASE+0x0090[2]	IOCFG_RT_BASE+0x0170[2]	IOCFG_RT_BASE+0x00e0[2]	IOCFG_RT_BASE+0x00b0[2]	-
PAD_ANT_SEL11	IOCFG_RT_BASE+0x0090[3]	IOCFG_RT_BASE+0x0170[3]	IOCFG_RT_BASE+0x00e0[3]	IOCFG_RT_BASE+0x00b0[3]	-
PAD_ANT_SEL12	IOCFG_RT_BASE+0x0090[4]	IOCFG_RT_BASE+0x0170[4]	IOCFG_RT_BASE+0x00e0[4]	IOCFG_RT_BASE+0x00b0[4]	-

Name	IES	SMT	PU	PD	PUPD
PAD_ANT_SEL13	IOCFG_RT_BASE+0x0090[5]	IOCFG_RT_BASE+0x0170[5]	IOCFG_RT_BASE+0x00e0[5]	IOCFG_RT_BASE+0x00b0[5]	-
PAD_ANT_SEL14	IOCFG_RT_BASE+0x0090[6]	IOCFG_RT_BASE+0x0170[6]	IOCFG_RT_BASE+0x00e0[6]	IOCFG_RT_BASE+0x00b0[6]	-
PAD_ANT_SEL2	IOCFG_RT_BASE+0x0090[7]	IOCFG_RT_BASE+0x0170[7]	IOCFG_RT_BASE+0x00e0[7]	IOCFG_RT_BASE+0x00b0[7]	-
PAD_ANT_SEL3	IOCFG_RT_BASE+0x0090[8]	IOCFG_RT_BASE+0x0170[8]	IOCFG_RT_BASE+0x00e0[8]	IOCFG_RT_BASE+0x00b0[8]	-
PAD_ANT_SEL4	IOCFG_RT_BASE+0x0090[9]	IOCFG_RT_BASE+0x0170[9]	IOCFG_RT_BASE+0x00e0[9]	IOCFG_RT_BASE+0x00b0[9]	-
PAD_ANT_SEL5	IOCFG_RT_BASE+0x0090[10]	IOCFG_RT_BASE+0x0170[10]	IOCFG_RT_BASE+0x00e0[10]	IOCFG_RT_BASE+0x00b0[10]	-
PAD_ANT_SEL6	IOCFG_RT_BASE+0x0090[11]	IOCFG_RT_BASE+0x0170[11]	IOCFG_RT_BASE+0x00e0[11]	IOCFG_RT_BASE+0x00b0[11]	-
PAD_ANT_SEL7	IOCFG_RT_BASE+0x0090[12]	IOCFG_RT_BASE+0x0170[12]	IOCFG_RT_BASE+0x00e0[12]	IOCFG_RT_BASE+0x00b0[12]	-
PAD_ANT_SEL8	IOCFG_RT_BASE+0x0090[13]	IOCFG_RT_BASE+0x0170[13]	IOCFG_RT_BASE+0x00e0[13]	IOCFG_RT_BASE+0x00b0[13]	-
PAD_ANT_SEL9	IOCFG_RT_BASE+0x0090[14]	IOCFG_RT_BASE+0x0170[14]	IOCFG_RT_BASE+0x00e0[14]	IOCFG_RT_BASE+0x00b0[14]	-
PAD_AP_GOOD	IOCFG_BM_BASE+0x0080[0]	IOCFG_BM_BASE+0x0140[24]	IOCFG_BM_BASE+0x00d0[0]	IOCFG_BM_BASE+0x00a0[0]	-
PAD_AUD_CLK_MOSI	IOCFG_TM_BASE+0x0080[0]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[0]	IOCFG_TM_BASE+0x00a0[0]	-
PAD_AUD_DAT_MISO0	IOCFG_TM_BASE+0x0080[1]	IOCFG_TM_BASE+0x0110[0]	IOCFG_TM_BASE+0x00c0[1]	IOCFG_TM_BASE+0x00a0[1]	-
PAD_AUD_DAT_MISO1	IOCFG_TM_BASE+0x0080[2]	IOCFG_TM_BASE+0x0110[1]	IOCFG_TM_BASE+0x00c0[2]	IOCFG_TM_BASE+0x00a0[2]	-
PAD_AUD_DAT_MISO2	IOCFG_TM_BASE+0x0080[3]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[3]	IOCFG_TM_BASE+0x00a0[3]	-
PAD_AUD_DAT_MOSI0	IOCFG_TM_BASE+0x0080[4]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[4]	IOCFG_TM_BASE+0x00a0[4]	-
PAD_AUD_DAT_MOSI1	IOCFG_TM_BASE+0x0080[5]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[5]	IOCFG_TM_BASE+0x00a0[5]	-
PAD_AUD_DAT_MOSI2	IOCFG_TM_BASE+0x0080[6]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[6]	IOCFG_TM_BASE+0x00a0[6]	-
PAD_AUD_I2C_SCL	IOCFG_TM_BASE+0x0080[7]	IOCFG_TM_BASE+0x0110[15]	IOCFG_TM_BASE+0x00c0[7]	IOCFG_TM_BASE+0x00a0[7]	-
PAD_AUD_I2C_SDA	IOCFG_TM_BASE+0x0080[8]	IOCFG_TM_BASE+0x0110[15]	IOCFG_TM_BASE+0x00c0[8]	IOCFG_TM_BASE+0x00a0[8]	-
PAD_AUD_NLE_MOSI0	IOCFG_TM_BASE+0x0080[9]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[9]	IOCFG_TM_BASE+0x00a0[9]	-
PAD_AUD_NLE_MOSI1	IOCFG_TM_BASE+0x0080[10]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[10]	IOCFG_TM_BASE+0x00a0[10]	-
PAD_AUD_SYNC_MOSI	IOCFG_TM_BASE+0x0080[11]	IOCFG_TM_BASE+0x0110[14]	IOCFG_TM_BASE+0x00c0[11]	IOCFG_TM_BASE+0x00a0[11]	-
PAD_BPI_D_BUS0	IOCFG_BM_BASE+0x0080[1]	IOCFG_BM_BASE+0x0140[27]	IOCFG_BM_BASE+0x00d0[1]	IOCFG_BM_BASE+0x00a0[1]	-
PAD_BPI_D_BUS1	IOCFG_BM_BASE+0x0080[2]	IOCFG_BM_BASE+0x0140[27]	IOCFG_BM_BASE+0x00d0[2]	IOCFG_BM_BASE+0x00a0[2]	-

Name	IES	SMT	PU	PD	PUPD
PAD_BPI_D_BUS10	IOCFG_BM_BASE+0x0080[3]	IOCFG_BM_BASE+0x0140[0]	IOCFG_BM_BASE+0x00d0[3]	IOCFG_BM_BASE+0x00a0[3]	-
PAD_BPI_D_BUS11	IOCFG_BM_BASE+0x0080[4]	IOCFG_BM_BASE+0x0140[1]	IOCFG_BM_BASE+0x00d0[4]	IOCFG_BM_BASE+0x00a0[4]	-
PAD_BPI_D_BUS12	IOCFG_BM_BASE+0x0080[5]	IOCFG_BM_BASE+0x0140[2]	IOCFG_BM_BASE+0x00d0[5]	IOCFG_BM_BASE+0x00a0[5]	-
PAD_BPI_D_BUS13	IOCFG_BM_BASE+0x0080[6]	IOCFG_BM_BASE+0x0140[3]	IOCFG_BM_BASE+0x00d0[6]	IOCFG_BM_BASE+0x00a0[6]	-
PAD_BPI_D_BUS14	IOCFG_BM_BASE+0x0080[7]	IOCFG_BM_BASE+0x0140[4]	IOCFG_BM_BASE+0x00d0[7]	IOCFG_BM_BASE+0x00a0[7]	-
PAD_BPI_D_BUS15	IOCFG_BM_BASE+0x0080[8]	IOCFG_BM_BASE+0x0140[5]	IOCFG_BM_BASE+0x00d0[8]	IOCFG_BM_BASE+0x00a0[8]	-
PAD_BPI_D_BUS16	IOCFG_BM_BASE+0x0080[9]	IOCFG_BM_BASE+0x0140[6]	IOCFG_BM_BASE+0x00d0[9]	IOCFG_BM_BASE+0x00a0[9]	-
PAD_BPI_D_BUS17	IOCFG_BM_BASE+0x0080[10]	IOCFG_BM_BASE+0x0140[7]	IOCFG_BM_BASE+0x00d0[10]	IOCFG_BM_BASE+0x00a0[10]	-
PAD_BPI_D_BUS18	IOCFG_BM_BASE+0x0080[11]	IOCFG_BM_BASE+0x0140[8]	IOCFG_BM_BASE+0x00d0[11]	IOCFG_BM_BASE+0x00a0[11]	-
PAD_BPI_D_BUS19	IOCFG_BM_BASE+0x0080[12]	IOCFG_BM_BASE+0x0140[9]	IOCFG_BM_BASE+0x00d0[12]	IOCFG_BM_BASE+0x00a0[12]	-
PAD_BPI_D_BUS2	IOCFG_BM_BASE+0x0080[13]	IOCFG_BM_BASE+0x0140[27]	IOCFG_BM_BASE+0x00d0[13]	IOCFG_BM_BASE+0x00a0[13]	-
PAD_BPI_D_BUS3	IOCFG_BM_BASE+0x0080[14]	IOCFG_BM_BASE+0x0140[27]	IOCFG_BM_BASE+0x00d0[14]	IOCFG_BM_BASE+0x00a0[14]	-
PAD_BPI_D_BUS4	IOCFG_BM_BASE+0x0080[15]	IOCFG_BM_BASE+0x0140[28]	IOCFG_BM_BASE+0x00d0[15]	IOCFG_BM_BASE+0x00a0[15]	-
PAD_BPI_D_BUS5	IOCFG_BM_BASE+0x0080[16]	IOCFG_BM_BASE+0x0140[28]	IOCFG_BM_BASE+0x00d0[16]	IOCFG_BM_BASE+0x00a0[16]	-
PAD_BPI_D_BUS6	IOCFG_BM_BASE+0x0080[17]	IOCFG_BM_BASE+0x0140[28]	IOCFG_BM_BASE+0x00d0[17]	IOCFG_BM_BASE+0x00a0[17]	-
PAD_BPI_D_BUS7	IOCFG_BM_BASE+0x0080[18]	IOCFG_BM_BASE+0x0140[28]	IOCFG_BM_BASE+0x00d0[18]	IOCFG_BM_BASE+0x00a0[18]	-
PAD_BPI_D_BUS8	IOCFG_BM_BASE+0x0080[19]	IOCFG_BM_BASE+0x0140[29]	IOCFG_BM_BASE+0x00d0[19]	IOCFG_BM_BASE+0x00a0[19]	-
PAD_BPI_D_BUS9	IOCFG_BM_BASE+0x0080[20]	IOCFG_BM_BASE+0x0140[10]	IOCFG_BM_BASE+0x00d0[20]	IOCFG_BM_BASE+0x00a0[20]	-
PAD_BPI_WAKEUP_PMIC	IOCFG_RT_BASE+0x0090[15]	IOCFG_RT_BASE+0x0170[31]	IOCFG_RT_BASE+0x00e0[15]	IOCFG_RT_BASE+0x00b0[15]	-
PAD_CAM_CLK0	IOCFG_RT_BASE+0x0090[16]	IOCFG_RT_BASE+0x0170[15]	IOCFG_RT_BASE+0x00e0[16]	IOCFG_RT_BASE+0x00b0[16]	-
PAD_CAM_CLK1	IOCFG_RT_BASE+0x0090[17]	IOCFG_RT_BASE+0x0170[16]	IOCFG_RT_BASE+0x00e0[17]	IOCFG_RT_BASE+0x00b0[17]	-
PAD_CAM_CLK2	IOCFG_BR_BASE+0x0060[0]	IOCFG_BR_BASE+0x00c0[0]	IOCFG_BR_BASE+0x0080[0]	IOCFG_BR_BASE+0x0070[0]	-
PAD_CAM_CLK3	IOCFG_BR_BASE+0x0060[1]	IOCFG_BR_BASE+0x00c0[1]	IOCFG_BR_BASE+0x0080[1]	IOCFG_BR_BASE+0x0070[1]	-
PAD_CAM_CLK4	IOCFG_RT_BASE+0x0090[18]	IOCFG_RT_BASE+0x0170[17]	IOCFG_RT_BASE+0x00e0[18]	IOCFG_RT_BASE+0x00b0[18]	-
PAD_CAM_CLK5	IOCFG_RT_BASE+0x0090[19]	IOCFG_RT_BASE+0x0170[18]	IOCFG_RT_BASE+0x00e0[19]	IOCFG_RT_BASE+0x00b0[19]	-

Name	IES	SMT	PU	PD	PUPD
PAD_CAM_CLK6	IOCFG_BR_BASE+0x0060[2]	IOCFG_BR_BASE+0x00c0[2]	IOCFG_BR_BASE+0x0080[2]	IOCFG_BR_BASE+0x0070[2]	-
PAD_CAM_CLK7	IOCFG_BR_BASE+0x0060[3]	IOCFG_BR_BASE+0x00c0[3]	IOCFG_BR_BASE+0x0080[3]	IOCFG_BR_BASE+0x0070[3]	-
PAD_CAM_I3C_SCL2	IOCFG_RT_BASE+0x0090[20]	IOCFG_RT_BASE+0x0170[19]	IOCFG_RT_BASE+0x00e0[20]	IOCFG_RT_BASE+0x00b0[20]	-
PAD_CAM_I3C_SCL4	IOCFG_RT_BASE+0x0090[21]	IOCFG_RT_BASE+0x0170[20]	IOCFG_RT_BASE+0x00e0[21]	IOCFG_RT_BASE+0x00b0[21]	-
PAD_CAM_I3C_SCL8	IOCFG_BR_BASE+0x0060[4]	IOCFG_BR_BASE+0x00c0[4]	IOCFG_BR_BASE+0x0080[4]	IOCFG_BR_BASE+0x0070[4]	-
PAD_CAM_I3C_SCL9	IOCFG_BR_BASE+0x0060[5]	IOCFG_BR_BASE+0x00c0[5]	IOCFG_BR_BASE+0x0080[5]	IOCFG_BR_BASE+0x0070[5]	-
PAD_CAM_I3C_SDA2	IOCFG_RT_BASE+0x0090[22]	IOCFG_RT_BASE+0x0170[21]	IOCFG_RT_BASE+0x00e0[22]	IOCFG_RT_BASE+0x00b0[22]	-
PAD_CAM_I3C_SDA4	IOCFG_RT_BASE+0x0090[23]	IOCFG_RT_BASE+0x0170[22]	IOCFG_RT_BASE+0x00e0[23]	IOCFG_RT_BASE+0x00b0[23]	-
PAD_CAM_I3C_SDA8	IOCFG_BR_BASE+0x0060[6]	IOCFG_BR_BASE+0x00c0[6]	IOCFG_BR_BASE+0x0080[6]	IOCFG_BR_BASE+0x0070[6]	-
PAD_CAM_I3C_SDA9	IOCFG_BR_BASE+0x0060[7]	IOCFG_BR_BASE+0x00c0[7]	IOCFG_BR_BASE+0x0080[7]	IOCFG_BR_BASE+0x0070[7]	-
PAD_CAM_RST0	IOCFG_RT_BASE+0x0090[24]	IOCFG_RT_BASE+0x0170[30]	IOCFG_RT_BASE+0x00e0[24]	IOCFG_RT_BASE+0x00b0[24]	-
PAD_CAM_RST1	IOCFG_RT_BASE+0x0090[25]	IOCFG_RT_BASE+0x0170[23]	IOCFG_RT_BASE+0x00e0[25]	IOCFG_RT_BASE+0x00b0[25]	-
PAD_CAM_RST2	IOCFG_BR_BASE+0x0060[8]	IOCFG_BR_BASE+0x00c0[13]	IOCFG_BR_BASE+0x0080[8]	IOCFG_BR_BASE+0x0070[8]	-
PAD_CAM_RST3	IOCFG_BR_BASE+0x0060[9]	IOCFG_BR_BASE+0x00c0[15]	IOCFG_BR_BASE+0x0080[9]	IOCFG_BR_BASE+0x0070[9]	-
PAD_CAM_RST4	IOCFG_RT_BASE+0x0090[26]	IOCFG_RT_BASE+0x0170[24]	IOCFG_RT_BASE+0x00e0[26]	IOCFG_RT_BASE+0x00b0[26]	-
PAD_CAM_RST5	IOCFG_RT_BASE+0x0090[27]	IOCFG_RT_BASE+0x0170[25]	IOCFG_RT_BASE+0x00e0[27]	IOCFG_RT_BASE+0x00b0[27]	-
PAD_CAM_RST6	IOCFG_BR_BASE+0x0060[10]	IOCFG_BR_BASE+0x00c0[14]	IOCFG_BR_BASE+0x0080[10]	IOCFG_BR_BASE+0x0070[10]	-
PAD_CAM_RST7	IOCFG_BR_BASE+0x0060[11]	IOCFG_BR_BASE+0x00c0[8]	IOCFG_BR_BASE+0x0080[11]	IOCFG_BR_BASE+0x0070[11]	-
PAD_CHG_I2C_SCL5	IOCFG_TM_BASE+0x0080[12]	IOCFG_TM_BASE+0x0110[2]	IOCFG_TM_BASE+0x00c0[12]	IOCFG_TM_BASE+0x00a0[12]	-
PAD_CHG_I2C_SDA5	IOCFG_TM_BASE+0x0080[13]	IOCFG_TM_BASE+0x0110[3]	IOCFG_TM_BASE+0x00c0[13]	IOCFG_TM_BASE+0x00a0[13]	-
PAD_CONN_BT_CLK	IOCFG_RT_BASE+0x0090[28]	IOCFG_RT_BASE+0x0180[2]	-	-	IOCFG_RT_BASE+0x00d0[0]
PAD_CONN_BT_DATA	IOCFG_RT_BASE+0x0090[29]	IOCFG_RT_BASE+0x0180[2]	-	-	IOCFG_RT_BASE+0x00d0[1]
PAD_CONN_HRS_T_B	IOCFG_RT_BASE+0x0090[30]	IOCFG_RT_BASE+0x0180[0]	IOCFG_RT_BASE+0x00e0[28]	IOCFG_RT_BASE+0x00b0[28]	-
PAD_CONN_PMI_C_EN	IOCFG_RT_BASE+0x0090[31]	IOCFG_RT_BASE+0x0180[1]	IOCFG_RT_BASE+0x00e0[29]	IOCFG_RT_BASE+0x00b0[29]	-
PAD_CONN_TOP_CLK	IOCFG_RT_BASE+0x00a0[0]	IOCFG_RT_BASE+0x0180[2]	-	-	IOCFG_RT_BASE+0x00d0[2]

Name	IES	SMT	PU	PD	PUPD
PAD_CONN_TOP_DATA	IOCFG_RT_BASE+0x00a0[1]	IOCFG_RT_BASE+0x0180[2]	-	-	IOCFG_RT_BASE+0x00d0[3]
PAD_CONN_WB_PTA	IOCFG_RT_BASE+0x00a0[2]	IOCFG_RT_BASE+0x0180[2]	-	-	IOCFG_RT_BASE+0x00d0[4]
PAD_CONN_WF_CTRL0	IOCFG_RT_BASE+0x00a0[3]	IOCFG_RT_BASE+0x0180[3]	-	-	IOCFG_RT_BASE+0x00d0[5]
PAD_CONN_WF_CTRL1	IOCFG_RT_BASE+0x00a0[4]	IOCFG_RT_BASE+0x0180[3]	-	-	IOCFG_RT_BASE+0x00d0[6]
PAD_CONN_WF_CTRL2	IOCFG_RT_BASE+0x00a0[5]	IOCFG_RT_BASE+0x0180[3]	-	-	IOCFG_RT_BASE+0x00d0[7]
PAD_CONN_WF_CTRL3	IOCFG_RT_BASE+0x00a0[6]	IOCFG_RT_BASE+0x0180[3]	-	-	IOCFG_RT_BASE+0x00d0[8]
PAD_CONN_WF_CTRL4	IOCFG_RT_BASE+0x00a0[7]	IOCFG_RT_BASE+0x0180[3]	-	-	IOCFG_RT_BASE+0x00d0[9]
PAD_DIGRF_IRQ	IOCFG_LB_BASE+0x0060[0]	IOCFG_LB_BASE+0x00b0[15]	IOCFG_LB_BASE+0x0080[0]	IOCFG_LB_BASE+0x0070[0]	-
PAD_DISP_PWM	IOCFG_BM_BASE+0x0080[21]	IOCFG_BM_BASE+0x0140[21]	IOCFG_BM_BASE+0x00d0[21]	IOCFG_BM_BASE+0x00a0[21]	-
PAD_DISP_PWM_1	IOCFG_BM_BASE+0x0080[22]	IOCFG_BM_BASE+0x0140[21]	IOCFG_BM_BASE+0x00d0[22]	IOCFG_BM_BASE+0x00a0[22]	-
PAD_DMIC_CLK	IOCFG_TM_BASE+0x0080[14]	IOCFG_TM_BASE+0x0110[16]	IOCFG_TM_BASE+0x00c0[14]	IOCFG_TM_BASE+0x00a0[14]	-
PAD_DMIC_DAT	IOCFG_TM_BASE+0x0080[15]	IOCFG_TM_BASE+0x0110[16]	IOCFG_TM_BASE+0x00c0[15]	IOCFG_TM_BASE+0x00a0[15]	-
PAD_DSI_TE	IOCFG_TM_BASE+0x0080[16]	IOCFG_TM_BASE+0x0110[12]	IOCFG_TM_BASE+0x00c0[16]	IOCFG_TM_BASE+0x00a0[16]	-
PAD_DSI_TE_AUX	IOCFG_TM_BASE+0x0080[17]	IOCFG_TM_BASE+0x0110[10]	IOCFG_TM_BASE+0x00c0[17]	IOCFG_TM_BASE+0x00a0[17]	-
PAD_EINT4	IOCFG_BM_BASE+0x0080[23]	IOCFG_BM_BASE+0x0140[20]	IOCFG_BM_BASE+0x00d0[23]	IOCFG_BM_BASE+0x00a0[23]	-
PAD_EINT5	IOCFG_BM_BASE+0x0080[24]	IOCFG_BM_BASE+0x0140[20]	IOCFG_BM_BASE+0x00d0[24]	IOCFG_BM_BASE+0x00a0[24]	-
PAD_EINT6	IOCFG_BM_BASE+0x0080[25]	IOCFG_BM_BASE+0x0140[20]	IOCFG_BM_BASE+0x00d0[25]	IOCFG_BM_BASE+0x00a0[25]	-
PAD_EINT_CHG_IRQB	IOCFG_TM_BASE+0x0080[18]	IOCFG_TM_BASE+0x0110[18]	IOCFG_TM_BASE+0x00c0[18]	IOCFG_TM_BASE+0x00a0[18]	-
PAD_EINT_CTP0	IOCFG_TM_BASE+0x0080[19]	IOCFG_TM_BASE+0x0110[12]	IOCFG_TM_BASE+0x00c0[19]	IOCFG_TM_BASE+0x00a0[19]	-
PAD_EINT_CTP1	IOCFG_TM_BASE+0x0080[20]	IOCFG_TM_BASE+0x0110[4]	IOCFG_TM_BASE+0x00c0[20]	IOCFG_TM_BASE+0x00a0[20]	-
PAD_EINT_FPS	IOCFG_RM_BASE+0x0060[0]	IOCFG_RM_BASE+0x00e0[6]	IOCFG_RM_BASE+0x0090[0]	IOCFG_RM_BASE+0x0070[0]	-
PAD_GPIO_CTP0_RST	IOCFG_TM_BASE+0x0080[21]	IOCFG_TM_BASE+0x0110[12]	IOCFG_TM_BASE+0x00c0[21]	IOCFG_TM_BASE+0x00a0[21]	-
PAD_GPIO_CTP1_RST	IOCFG_TM_BASE+0x0080[22]	IOCFG_TM_BASE+0x0110[11]	IOCFG_TM_BASE+0x00c0[22]	IOCFG_TM_BASE+0x00a0[22]	-
PAD_GPIO_FPS_RST	IOCFG_RM_BASE+0x0060[1]	IOCFG_RM_BASE+0x00e0[7]	IOCFG_RM_BASE+0x0090[1]	IOCFG_RM_BASE+0x0070[1]	-
PAD_GPS_L1_ELNA_EN	IOCFG_RT_BASE+0x00a0[8]	IOCFG_RT_BASE+0x0180[1]	IOCFG_RT_BASE+0x00e0[30]	IOCFG_RT_BASE+0x00b0[30]	-

Name	IES	SMT	PU	PD	PUPD
PAD_GPS_L5_ELNA_EN	IOCFG_RT_BASE+0x00a0[9]	IOCFG_RT_BASE+0x0180[1]	IOCFG_RT_BASE+0x00e0[31]	IOCFG_RT_BASE+0x00b0[31]	-
PAD_I2C_SCL10	IOCFG_RT_BASE+0x00a0[10]	IOCFG_RT_BASE+0x0170[26]	IOCFG_RT_BASE+0x00f0[0]	IOCFG_RT_BASE+0x00c0[0]	-
PAD_I2C_SCL11	IOCFG_RT_BASE+0x00a0[11]	IOCFG_RT_BASE+0x0170[27]	IOCFG_RT_BASE+0x00f0[1]	IOCFG_RT_BASE+0x00c0[1]	-
PAD_I2C_SCL12	IOCFG_BR_BASE+0x0060[12]	IOCFG_BR_BASE+0x00c0[9]	IOCFG_BR_BASE+0x0080[12]	IOCFG_BR_BASE+0x0070[12]	-
PAD_I2C_SCL13	IOCFG_BR_BASE+0x0060[13]	IOCFG_BR_BASE+0x00c0[10]	IOCFG_BR_BASE+0x0080[13]	IOCFG_BR_BASE+0x0070[13]	-
PAD_I2C_SCL3	IOCFG_TL_BASE+0x0040[0]	IOCFG_TL_BASE+0x0090[0]	IOCFG_TL_BASE+0x0060[0]	IOCFG_TL_BASE+0x0050[0]	-
PAD_I2C_SCL6	IOCFG_LB_BASE+0x0060[1]	IOCFG_LB_BASE+0x00b0[0]	IOCFG_LB_BASE+0x0080[1]	IOCFG_LB_BASE+0x0070[1]	-
PAD_I2C_SDA10	IOCFG_RT_BASE+0x00a0[12]	IOCFG_RT_BASE+0x0170[28]	IOCFG_RT_BASE+0x00f0[2]	IOCFG_RT_BASE+0x00c0[2]	-
PAD_I2C_SDA11	IOCFG_RT_BASE+0x00a0[13]	IOCFG_RT_BASE+0x0170[29]	IOCFG_RT_BASE+0x00f0[3]	IOCFG_RT_BASE+0x00c0[3]	-
PAD_I2C_SDA12	IOCFG_BR_BASE+0x0060[14]	IOCFG_BR_BASE+0x00c0[11]	IOCFG_BR_BASE+0x0080[14]	IOCFG_BR_BASE+0x0070[14]	-
PAD_I2C_SDA13	IOCFG_BR_BASE+0x0060[15]	IOCFG_BR_BASE+0x00c0[12]	IOCFG_BR_BASE+0x0080[15]	IOCFG_BR_BASE+0x0070[15]	-
PAD_I2C_SDA3	IOCFG_TL_BASE+0x0040[1]	IOCFG_TL_BASE+0x0090[1]	IOCFG_TL_BASE+0x0060[1]	IOCFG_TL_BASE+0x0050[1]	-
PAD_I2C_SDA6	IOCFG_LB_BASE+0x0060[2]	IOCFG_LB_BASE+0x00b0[1]	IOCFG_LB_BASE+0x0080[2]	IOCFG_LB_BASE+0x0070[2]	-
PAD_I2S0_BCK	IOCFG_LB_BASE+0x0060[3]	IOCFG_LB_BASE+0x00b0[2]	IOCFG_LB_BASE+0x0080[3]	IOCFG_LB_BASE+0x0070[3]	-
PAD_I2S0_DI	IOCFG_LB_BASE+0x0060[4]	IOCFG_LB_BASE+0x00b0[3]	IOCFG_LB_BASE+0x0080[4]	IOCFG_LB_BASE+0x0070[4]	-
PAD_I2S0_DO	IOCFG_LB_BASE+0x0060[5]	IOCFG_LB_BASE+0x00b0[4]	IOCFG_LB_BASE+0x0080[5]	IOCFG_LB_BASE+0x0070[5]	-
PAD_I2S0_LRCK	IOCFG_LB_BASE+0x0060[6]	IOCFG_LB_BASE+0x00b0[5]	IOCFG_LB_BASE+0x0080[6]	IOCFG_LB_BASE+0x0070[6]	-
PAD_I2S1_BCK	IOCFG_LB_BASE+0x0060[7]	IOCFG_LB_BASE+0x00b0[6]	IOCFG_LB_BASE+0x0080[7]	IOCFG_LB_BASE+0x0070[7]	-
PAD_I2S1_DI	IOCFG_LB_BASE+0x0060[8]	IOCFG_LB_BASE+0x00b0[7]	IOCFG_LB_BASE+0x0080[8]	IOCFG_LB_BASE+0x0070[8]	-
PAD_I2S1_DO	IOCFG_LB_BASE+0x0060[9]	IOCFG_LB_BASE+0x00b0[8]	IOCFG_LB_BASE+0x0080[9]	IOCFG_LB_BASE+0x0070[9]	-
PAD_I2S1_LRCK	IOCFG_LB_BASE+0x0060[10]	IOCFG_LB_BASE+0x00b0[9]	IOCFG_LB_BASE+0x0080[10]	IOCFG_LB_BASE+0x0070[10]	-
PAD_I2S2_BCK	IOCFG_LB_BASE+0x0060[11]	IOCFG_LB_BASE+0x00b0[10]	IOCFG_LB_BASE+0x0080[11]	IOCFG_LB_BASE+0x0070[11]	-
PAD_I2S2_DI	IOCFG_LB_BASE+0x0060[12]	IOCFG_LB_BASE+0x00b0[11]	IOCFG_LB_BASE+0x0080[12]	IOCFG_LB_BASE+0x0070[12]	-
PAD_I2S2_DO	IOCFG_LB_BASE+0x0060[13]	IOCFG_LB_BASE+0x00b0[12]	IOCFG_LB_BASE+0x0080[13]	IOCFG_LB_BASE+0x0070[13]	-
PAD_I2S2_LRCK	IOCFG_LB_BASE+0x0060[14]	IOCFG_LB_BASE+0x00b0[13]	IOCFG_LB_BASE+0x0080[14]	IOCFG_LB_BASE+0x0070[14]	-

Name	IES	SMT	PU	PD	PUPD
PAD_I3C_SCL0	IOCFG_BM_BASE+0x0080[26]	IOCFG_BM_BASE+0x0140[11]	IOCFG_BM_BASE+0x00d0[26]	IOCFG_BM_BASE+0x00a0[26]	-
PAD_I3C_SCL1	IOCFG_BM_BASE+0x0080[27]	IOCFG_BM_BASE+0x0140[12]	IOCFG_BM_BASE+0x00d0[27]	IOCFG_BM_BASE+0x00a0[27]	-
PAD_I3C_SCL7	IOCFG_BM_BASE+0x0080[28]	IOCFG_BM_BASE+0x0140[13]	IOCFG_BM_BASE+0x00d0[28]	IOCFG_BM_BASE+0x00a0[28]	-
PAD_I3C_SDA0	IOCFG_BM_BASE+0x0080[29]	IOCFG_BM_BASE+0x0140[14]	IOCFG_BM_BASE+0x00d0[29]	IOCFG_BM_BASE+0x00a0[29]	-
PAD_I3C_SDA1	IOCFG_BM_BASE+0x0080[30]	IOCFG_BM_BASE+0x0140[15]	IOCFG_BM_BASE+0x00d0[30]	IOCFG_BM_BASE+0x00a0[30]	-
PAD_I3C_SDA7	IOCFG_BM_BASE+0x0080[31]	IOCFG_BM_BASE+0x0140[16]	IOCFG_BM_BASE+0x00d0[31]	IOCFG_BM_BASE+0x00a0[31]	-
PAD_INT_SIM1	IOCFG_TM_BASE+0x0080[23]	IOCFG_TM_BASE+0x0110[9]	IOCFG_TM_BASE+0x00c0[23]	IOCFG_TM_BASE+0x00a0[23]	-
PAD_INT_SIM2	IOCFG_TM_BASE+0x0080[24]	IOCFG_TM_BASE+0x0110[9]	IOCFG_TM_BASE+0x00c0[24]	IOCFG_TM_BASE+0x00a0[24]	-
PAD_JTCK	IOCFG_BM_BASE+0x0090[0]	IOCFG_BM_BASE+0x0140[22]	IOCFG_BM_BASE+0x00e0[0]	IOCFG_BM_BASE+0x00b0[0]	-
PAD_JTDI	IOCFG_BM_BASE+0x0090[1]	IOCFG_BM_BASE+0x0140[22]	IOCFG_BM_BASE+0x00e0[1]	IOCFG_BM_BASE+0x00b0[1]	-
PAD_JTDO	IOCFG_BM_BASE+0x0090[2]	IOCFG_BM_BASE+0x0140[22]	IOCFG_BM_BASE+0x00e0[2]	IOCFG_BM_BASE+0x00b0[2]	-
PAD_JTMS	IOCFG_BM_BASE+0x0090[3]	IOCFG_BM_BASE+0x0140[22]	IOCFG_BM_BASE+0x00e0[3]	IOCFG_BM_BASE+0x00b0[3]	-
PAD_JTRSTN	IOCFG_BM_BASE+0x0090[4]	IOCFG_BM_BASE+0x0140[21]	IOCFG_BM_BASE+0x00e0[4]	IOCFG_BM_BASE+0x00b0[4]	-
PAD_KPCOLO	IOCFG_BM_BASE+0x0090[5]	IOCFG_BM_BASE+0x0140[26]	-	-	IOCFG_BM_BASE+0x00c0[0]
PAD_KPCOL1	IOCFG_BM_BASE+0x0090[6]	IOCFG_BM_BASE+0x0140[26]	-	-	IOCFG_BM_BASE+0x00c0[1]
PAD_KPROW0	IOCFG_BM_BASE+0x0090[7]	IOCFG_BM_BASE+0x0140[17]	-	-	IOCFG_BM_BASE+0x00c0[2]
PAD_KPROW1	IOCFG_BM_BASE+0x0090[8]	IOCFG_BM_BASE+0x0140[25]	-	-	IOCFG_BM_BASE+0x00c0[3]
PAD_LCM_RST	IOCFG_TM_BASE+0x0080[25]	IOCFG_TM_BASE+0x0110[12]	IOCFG_TM_BASE+0x00c0[25]	IOCFG_TM_BASE+0x00a0[25]	-
PAD_LCM_RST_AUX	IOCFG_TM_BASE+0x0080[26]	IOCFG_TM_BASE+0x0110[5]	IOCFG_TM_BASE+0x00c0[26]	IOCFG_TM_BASE+0x00a0[26]	-
PAD_MD_UCNT	IOCFG_LB_BASE+0x0060[15]	IOCFG_LB_BASE+0x00b0[15]	IOCFG_LB_BASE+0x0080[15]	IOCFG_LB_BASE+0x0070[15]	-
PAD_MIPI0_D_S_CLK	IOCFG_BL_BASE+0x0050[0]	IOCFG_BL_BASE+0x0090[8]	IOCFG_BL_BASE+0x0070[0]	IOCFG_BL_BASE+0x0060[0]	-
PAD_MIPI0_D_S_DATA	IOCFG_BL_BASE+0x0050[1]	IOCFG_BL_BASE+0x0090[8]	IOCFG_BL_BASE+0x0070[1]	IOCFG_BL_BASE+0x0060[1]	-
PAD_MIPI1_D_S_CLK	IOCFG_BL_BASE+0x0050[2]	IOCFG_BL_BASE+0x0090[0]	IOCFG_BL_BASE+0x0070[2]	IOCFG_BL_BASE+0x0060[2]	-
PAD_MIPI1_D_S_DATA	IOCFG_BL_BASE+0x0050[3]	IOCFG_BL_BASE+0x0090[1]	IOCFG_BL_BASE+0x0070[3]	IOCFG_BL_BASE+0x0060[3]	-
PAD_MIPI2_D_S_CLK	IOCFG_BL_BASE+0x0050[4]	IOCFG_BL_BASE+0x0090[2]	IOCFG_BL_BASE+0x0070[4]	IOCFG_BL_BASE+0x0060[4]	-

Name	IES	SMT	PU	PD	PUPD
PAD_MIPI2_D_S DATA	IOCFG_BL_BASE+0x 0050[5]	IOCFG_BL_BASE+0x 0090[3]	IOCFG_BL_BASE+0x 0070[5]	IOCFG_BL_BASE+0x 0060[5]	-
PAD_MIPI3_D_S CLK	IOCFG_BL_BASE+0x 0050[6]	IOCFG_BL_BASE+0x 0090[4]	IOCFG_BL_BASE+0x 0070[6]	IOCFG_BL_BASE+0x 0060[6]	-
PAD_MIPI3_D_S DATA	IOCFG_BL_BASE+0x 0050[7]	IOCFG_BL_BASE+0x 0090[5]	IOCFG_BL_BASE+0x 0070[7]	IOCFG_BL_BASE+0x 0060[7]	-
PAD_MIPI4_D_S CLK	IOCFG_BL_BASE+0x 0050[8]	IOCFG_BL_BASE+0x 0090[6]	IOCFG_BL_BASE+0x 0070[8]	IOCFG_BL_BASE+0x 0060[8]	-
PAD_MIPI4_D_S DATA	IOCFG_BL_BASE+0x 0050[9]	IOCFG_BL_BASE+0x 0090[7]	IOCFG_BL_BASE+0x 0070[9]	IOCFG_BL_BASE+0x 0060[9]	-
PAD_MIPI_M_SC LK	IOCFG_LB_BASE+0x 0060[16]	IOCFG_LB_BASE+0x 00b0[14]	IOCFG_LB_BASE+0x 0080[16]	IOCFG_LB_BASE+0x 0070[16]	-
PAD_MIPI_M_S DATA	IOCFG_LB_BASE+0x 0060[17]	IOCFG_LB_BASE+0x 00b0[14]	IOCFG_LB_BASE+0x 0080[17]	IOCFG_LB_BASE+0x 0070[17]	-
PAD_MSDC1_CL K	IOCFG_LT_BASE+0x 0020[0]	IOCFG_LT_BASE+0x 0090[0]	-	-	IOCFG_LT_BASE+0x 0030[0]
PAD_MSDC1_C MD	IOCFG_LT_BASE+0x 0020[1]	IOCFG_LT_BASE+0x 0090[1]	-	-	IOCFG_LT_BASE+0x 0030[1]
PAD_MSDC1_DA T0	IOCFG_LT_BASE+0x 0020[2]	IOCFG_LT_BASE+0x 0090[2]	-	-	IOCFG_LT_BASE+0x 0030[2]
PAD_MSDC1_DA T1	IOCFG_LT_BASE+0x 0020[3]	IOCFG_LT_BASE+0x 0090[3]	-	-	IOCFG_LT_BASE+0x 0030[3]
PAD_MSDC1_DA T2	IOCFG_LT_BASE+0x 0020[4]	IOCFG_LT_BASE+0x 0090[4]	-	-	IOCFG_LT_BASE+0x 0030[4]
PAD_MSDC1_DA T3	IOCFG_LT_BASE+0x 0020[5]	IOCFG_LT_BASE+0x 0090[5]	-	-	IOCFG_LT_BASE+0x 0030[5]
PAD_MSDC2_CL K	IOCFG_RM_BASE+0 x0060[2]	IOCFG_RM_BASE+0 x00e0[0]	-	-	IOCFG_RM_BASE+0 x0080[0]
PAD_MSDC2_C MD	IOCFG_RM_BASE+0 x0060[3]	IOCFG_RM_BASE+0 x00e0[1]	-	-	IOCFG_RM_BASE+0 x0080[1]
PAD_MSDC2_DA T0	IOCFG_RM_BASE+0 x0060[4]	IOCFG_RM_BASE+0 x00e0[2]	-	-	IOCFG_RM_BASE+0 x0080[2]
PAD_MSDC2_DA T1	IOCFG_RM_BASE+0 x0060[5]	IOCFG_RM_BASE+0 x00e0[3]	-	-	IOCFG_RM_BASE+0 x0080[3]
PAD_MSDC2_DA T2	IOCFG_RM_BASE+0 x0060[6]	IOCFG_RM_BASE+0 x00e0[4]	-	-	IOCFG_RM_BASE+0 x0080[4]
PAD_MSDC2_DA T3	IOCFG_RM_BASE+0 x0060[7]	IOCFG_RM_BASE+0 x00e0[5]	-	-	IOCFG_RM_BASE+0 x0080[5]
PAD_NFC_CLK_R EQ	IOCFG_TL_BASE+0x 0040[2]	IOCFG_TL_BASE+0x 0090[3]	IOCFG_TL_BASE+0x 0060[2]	IOCFG_TL_BASE+0x 0050[2]	-
PAD_PERIEN_CH G_ENB	IOCFG_TM_BASE+0 x0080[27]	IOCFG_TM_BASE+0 x0110[17]	IOCFG_TM_BASE+0 x00c0[27]	IOCFG_TM_BASE+0 x00a0[27]	-
PAD_PERI_EN0	IOCFG_BM_BASE+0 x0090[9]	IOCFG_BM_BASE+0 x0140[20]	IOCFG_BM_BASE+0 x00e0[5]	IOCFG_BM_BASE+0 x00b0[5]	-
PAD_PERI_EN1	IOCFG_BM_BASE+0 x0090[10]	IOCFG_BM_BASE+0 x0140[21]	IOCFG_BM_BASE+0 x00e0[6]	IOCFG_BM_BASE+0 x00b0[6]	-
PAD_PERI_EN2	IOCFG_BM_BASE+0 x0090[11]	IOCFG_BM_BASE+0 x0140[18]	IOCFG_BM_BASE+0 x00e0[7]	IOCFG_BM_BASE+0 x00b0[7]	-
PAD_PERI_EN3	IOCFG_BM_BASE+0 x0090[12]	IOCFG_BM_BASE+0 x0140[19]	IOCFG_BM_BASE+0 x00e0[8]	IOCFG_BM_BASE+0 x00b0[8]	-

Name	IES	SMT	PU	PD	PUPD
PAD_RTC32K_CK	IOCFG_TM_BASE+0x0080[28]	IOCFG_TM_BASE+0x0110[17]	IOCFG_TM_BASE+0x00c0[28]	IOCFG_TM_BASE+0x00a0[28]	-
PAD_SCP_EINT0	IOCFG_TM_BASE+0x0080[29]	IOCFG_TM_BASE+0x0110[6]	IOCFG_TM_BASE+0x00c0[29]	IOCFG_TM_BASE+0x00a0[29]	-
PAD_SCP_EINT1	IOCFG_TM_BASE+0x0080[30]	IOCFG_TM_BASE+0x0110[6]	IOCFG_TM_BASE+0x00c0[30]	IOCFG_TM_BASE+0x00a0[30]	-
PAD_SCP_EINT2	IOCFG_TM_BASE+0x0080[31]	IOCFG_TM_BASE+0x0110[6]	IOCFG_TM_BASE+0x00c0[31]	IOCFG_TM_BASE+0x00a0[31]	-
PAD_SCP_EINT3	IOCFG_TM_BASE+0x0090[0]	IOCFG_TM_BASE+0x0110[6]	IOCFG_TM_BASE+0x00d0[0]	IOCFG_TM_BASE+0x00b0[0]	-
PAD_SCP_I3C_SCL0	IOCFG_TM_BASE+0x0090[1]	IOCFG_TM_BASE+0x0110[8]	IOCFG_TM_BASE+0x00d0[1]	IOCFG_TM_BASE+0x00b0[1]	-
PAD_SCP_I3C_SCL2	IOCFG_TM_BASE+0x0090[2]	IOCFG_TM_BASE+0x0110[7]	IOCFG_TM_BASE+0x00d0[2]	IOCFG_TM_BASE+0x00b0[2]	-
PAD_SCP_I3C_SCL3	IOCFG_TM_BASE+0x0090[3]	IOCFG_TM_BASE+0x0110[9]	IOCFG_TM_BASE+0x00d0[3]	IOCFG_TM_BASE+0x00b0[3]	-
PAD_SCP_I3C_SDA0	IOCFG_TM_BASE+0x0090[4]	IOCFG_TM_BASE+0x0110[8]	IOCFG_TM_BASE+0x00d0[4]	IOCFG_TM_BASE+0x00b0[4]	-
PAD_SCP_I3C_SDA2	IOCFG_TM_BASE+0x0090[5]	IOCFG_TM_BASE+0x0110[7]	IOCFG_TM_BASE+0x00d0[5]	IOCFG_TM_BASE+0x00b0[5]	-
PAD_SCP_I3C_SDA3	IOCFG_TM_BASE+0x0090[6]	IOCFG_TM_BASE+0x0110[9]	IOCFG_TM_BASE+0x00d0[6]	IOCFG_TM_BASE+0x00b0[6]	-
PAD_SCP_SPI_CK	IOCFG_TM_BASE+0x0090[7]	IOCFG_TM_BASE+0x0110[7]	IOCFG_TM_BASE+0x00d0[7]	IOCFG_TM_BASE+0x00b0[7]	-
PAD_SCP_SPI_CS_B	IOCFG_TM_BASE+0x0090[8]	IOCFG_TM_BASE+0x0110[7]	IOCFG_TM_BASE+0x00d0[8]	IOCFG_TM_BASE+0x00b0[8]	-
PAD_SCP_SPI_MI	IOCFG_TM_BASE+0x0090[9]	IOCFG_TM_BASE+0x0110[8]	IOCFG_TM_BASE+0x00d0[9]	IOCFG_TM_BASE+0x00b0[9]	-
PAD_SCP_SPI_MO	IOCFG_TM_BASE+0x0090[10]	IOCFG_TM_BASE+0x0110[8]	IOCFG_TM_BASE+0x00d0[10]	IOCFG_TM_BASE+0x00b0[10]	-
PAD_SCP_VREQ_VAO	IOCFG_TM_BASE+0x0090[11]	IOCFG_TM_BASE+0x0110[17]	IOCFG_TM_BASE+0x00d0[11]	IOCFG_TM_BASE+0x00b0[11]	-
PAD_SIM1_SCLK	IOCFG_LT_BASE+0x0020[6]	IOCFG_LT_BASE+0x0090[6]	-	-	IOCFG_LT_BASE+0x0030[6]
PAD_SIM1_SIO	IOCFG_LT_BASE+0x0020[7]	IOCFG_LT_BASE+0x0090[6]	-	-	IOCFG_LT_BASE+0x0030[7]
PAD_SIM1_SRST	IOCFG_LT_BASE+0x0020[8]	IOCFG_LT_BASE+0x0090[6]	-	-	IOCFG_LT_BASE+0x0030[8]
PAD_SIM2_SCLK	IOCFG_LT_BASE+0x0020[9]	IOCFG_LT_BASE+0x0090[7]	-	-	IOCFG_LT_BASE+0x0030[9]
PAD_SIM2_SIO	IOCFG_LT_BASE+0x0020[10]	IOCFG_LT_BASE+0x0090[7]	-	-	IOCFG_LT_BASE+0x0030[10]
PAD_SIM2_SRST	IOCFG_LT_BASE+0x0020[11]	IOCFG_LT_BASE+0x0090[7]	-	-	IOCFG_LT_BASE+0x0030[11]
PAD_SPI0_CK	IOCFG_RM_BASE+0x0060[8]	IOCFG_RM_BASE+0x00e0[9]	-	-	IOCFG_RM_BASE+0x0080[6]
PAD_SPI0_CSB	IOCFG_RM_BASE+0x0060[9]	IOCFG_RM_BASE+0x00e0[9]	-	-	IOCFG_RM_BASE+0x0080[7]
PAD_SPI0_HOLD	IOCFG_RM_BASE+0x0060[10]	IOCFG_RM_BASE+0x00e0[10]	-	-	IOCFG_RM_BASE+0x0080[8]

Name	IES	SMT	PU	PD	PUPD
PAD_SPI0_MI	IOCFG_RM_BASE+0x0060[11]	IOCFG_RM_BASE+0x00e0[9]	-	-	IOCFG_RM_BASE+0x0080[9]
PAD_SPI0_MO	IOCFG_RM_BASE+0x0060[12]	IOCFG_RM_BASE+0x00e0[9]	-	-	IOCFG_RM_BASE+0x0080[10]
PAD_SPI0_WP	IOCFG_RM_BASE+0x0060[13]	IOCFG_RM_BASE+0x00e0[10]	-	-	IOCFG_RM_BASE+0x0080[11]
PAD_SPI1_CK	IOCFG_RM_BASE+0x0060[14]	IOCFG_RM_BASE+0x00e0[7]	IOCFG_RM_BASE+0x0090[2]	IOCFG_RM_BASE+0x0070[2]	-
PAD_SPI1_CSB	IOCFG_RM_BASE+0x0060[15]	IOCFG_RM_BASE+0x00e0[7]	IOCFG_RM_BASE+0x0090[3]	IOCFG_RM_BASE+0x0070[3]	-
PAD_SPI1_MI	IOCFG_RM_BASE+0x0060[16]	IOCFG_RM_BASE+0x00e0[7]	IOCFG_RM_BASE+0x0090[4]	IOCFG_RM_BASE+0x0070[4]	-
PAD_SPI1_MO	IOCFG_RM_BASE+0x0060[17]	IOCFG_RM_BASE+0x00e0[8]	IOCFG_RM_BASE+0x0090[5]	IOCFG_RM_BASE+0x0070[5]	-
PAD_SPI2_CK	IOCFG_TM_BASE+0x0090[12]	IOCFG_TM_BASE+0x0110[19]	IOCFG_TM_BASE+0x00d0[12]	IOCFG_TM_BASE+0x00b0[12]	-
PAD_SPI2_CSB	IOCFG_TM_BASE+0x0090[13]	IOCFG_TM_BASE+0x0110[20]	IOCFG_TM_BASE+0x00d0[13]	IOCFG_TM_BASE+0x00b0[13]	-
PAD_SPI2_MI	IOCFG_TM_BASE+0x0090[14]	IOCFG_TM_BASE+0x0110[21]	IOCFG_TM_BASE+0x00d0[14]	IOCFG_TM_BASE+0x00b0[14]	-
PAD_SPI2_MO	IOCFG_TM_BASE+0x0090[15]	IOCFG_TM_BASE+0x0110[22]	IOCFG_TM_BASE+0x00d0[15]	IOCFG_TM_BASE+0x00b0[15]	-
PAD_SPI3_CK	IOCFG_TL_BASE+0x0040[3]	IOCFG_TL_BASE+0x0090[2]	IOCFG_TL_BASE+0x0060[3]	IOCFG_TL_BASE+0x0050[3]	-
PAD_SPI3_CSB	IOCFG_TL_BASE+0x0040[4]	IOCFG_TL_BASE+0x0090[2]	IOCFG_TL_BASE+0x0060[4]	IOCFG_TL_BASE+0x0050[4]	-
PAD_SPI3_MI	IOCFG_TL_BASE+0x0040[5]	IOCFG_TL_BASE+0x0090[2]	IOCFG_TL_BASE+0x0060[5]	IOCFG_TL_BASE+0x0050[5]	-
PAD_SPI3_MO	IOCFG_TL_BASE+0x0040[6]	IOCFG_TL_BASE+0x0090[2]	IOCFG_TL_BASE+0x0060[6]	IOCFG_TL_BASE+0x0050[6]	-
PAD_SPI4_CK	IOCFG_TL_BASE+0x0040[7]	IOCFG_TL_BASE+0x0090[3]	IOCFG_TL_BASE+0x0060[7]	IOCFG_TL_BASE+0x0050[7]	-
PAD_SPI4_CSB	IOCFG_TL_BASE+0x0040[8]	IOCFG_TL_BASE+0x0090[3]	IOCFG_TL_BASE+0x0060[8]	IOCFG_TL_BASE+0x0050[8]	-
PAD_SPI4_MI	IOCFG_TL_BASE+0x0040[9]	IOCFG_TL_BASE+0x0090[3]	IOCFG_TL_BASE+0x0060[9]	IOCFG_TL_BASE+0x0050[9]	-
PAD_SPI4_MO	IOCFG_TL_BASE+0x0040[10]	IOCFG_TL_BASE+0x0090[4]	IOCFG_TL_BASE+0x0060[10]	IOCFG_TL_BASE+0x0050[10]	-
PAD_SPMI_M_SCL	IOCFG_RM_BASE+0x0060[18]	IOCFG_RM_BASE+0x00e0[11]	IOCFG_RM_BASE+0x0090[6]	IOCFG_RM_BASE+0x0070[6]	-
PAD_SPMI_M_SDA	IOCFG_RM_BASE+0x0060[19]	IOCFG_RM_BASE+0x00e0[12]	IOCFG_RM_BASE+0x0090[7]	IOCFG_RM_BASE+0x0070[7]	-
PAD_SPMI_P_SCL	IOCFG_RM_BASE+0x0060[20]	IOCFG_RM_BASE+0x00e0[13]	IOCFG_RM_BASE+0x0090[8]	IOCFG_RM_BASE+0x0070[8]	-
PAD_SPMI_P_SDA	IOCFG_RM_BASE+0x0060[21]	IOCFG_RM_BASE+0x00e0[14]	IOCFG_RM_BASE+0x0090[9]	IOCFG_RM_BASE+0x0070[9]	-
PAD_SRCLKENAO	IOCFG_TM_BASE+0x0090[16]	IOCFG_TM_BASE+0x0110[17]	IOCFG_TM_BASE+0x00d0[16]	IOCFG_TM_BASE+0x00b0[16]	-
PAD_URXD0	IOCFG_BM_BASE+0x0090[13]	IOCFG_BM_BASE+0x0140[23]	IOCFG_BM_BASE+0x00e0[9]	IOCFG_BM_BASE+0x00b0[9]	-

Name	IES	SMT	PU	PD	PUPD
PAD_UTXD0	IOCFG_BM_BASE+0x0090[14]	IOCFG_BM_BASE+0x0140[23]	IOCFG_BM_BASE+0x00e0[10]	IOCFG_BM_BASE+0x00b0[10]	-
PAD_WATCHDOG	IOCFG_TM_BASE+0x0090[17]	IOCFG_TM_BASE+0x0110[13]	IOCFG_TM_BASE+0x00d0[17]	IOCFG_TM_BASE+0x00b0[17]	-

5.2.2 Register Definition

See chapter 3.2 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.3 Keypad Scanner

5.3.1 Introduction

The keypad (KP) supports 3*3 key matrix (9 keys).

5.3.2 Features

The KP interface includes 3 columns and 3 rows (see the block diagram in the later chapter). The key detection block provides the key pressed, key released and debounce mechanisms.

5.3.3 Block Diagram

(3x3) key matrix

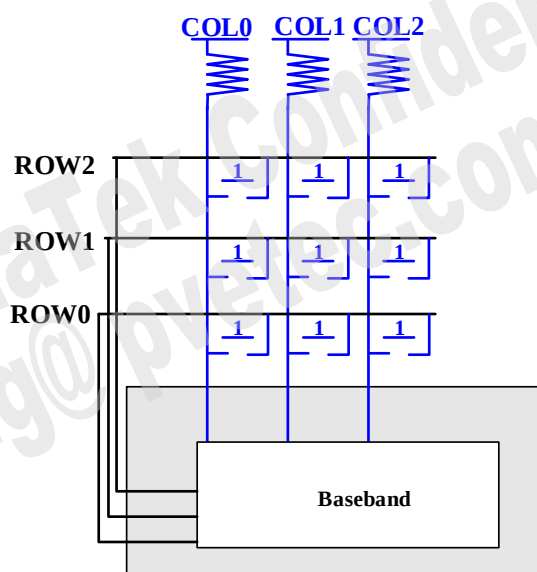


Figure 5-2. 3*3 keypad matrix (9 keys)

The key detection block senses the change and recognizes whether a key has been pressed or released each time. Whenever the key status changes and becomes stable, a KEYPAD IRQ will be issued. Then the MCU can read the key(s) pressed directly in the KP_MEM1, KP_MEM2, KP_MEM3, KP_MEM4 and KP_MEM5 registers. The status register can only be changed by the key-pressed detection FSM.

5.3.4 Theory of Operations

KP detects one key, or two keys pressed simultaneously in any combination. [Figure 5-3](#) and [Figure 5-4](#) show the one-key pressed condition. [Figure 5-5](#) illustrates the case where two keys are pressed. Since the key-pressed detection depends on the HIGH or LOW level of the external keypad interface, if the keys are pressed at the same time and one key exists in the

same column and the same row as the other key(s), the pressed key cannot be correctly decoded. For example, if there are three keys pressed: key1 = (x1, y1), key2 = (x2, y2), and key3 = (x1, y2), both key3 and key4 = (x2, y1) will be detected, and therefore they cannot be distinguished correctly. Hence, KP detects only one key, or two keys pressed simultaneously in any combination. More than two keys pressed simultaneously in a specific pattern will retrieve wrong information.

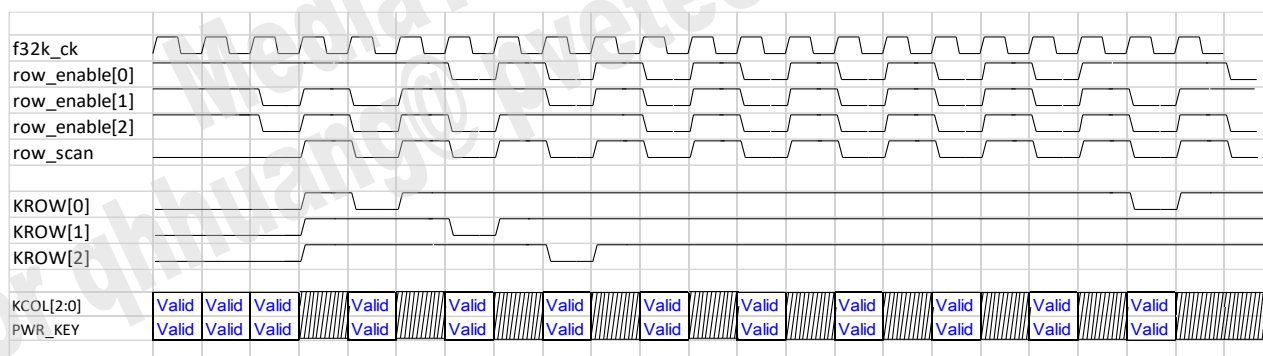


Figure 5-3. Signal key scan waveform

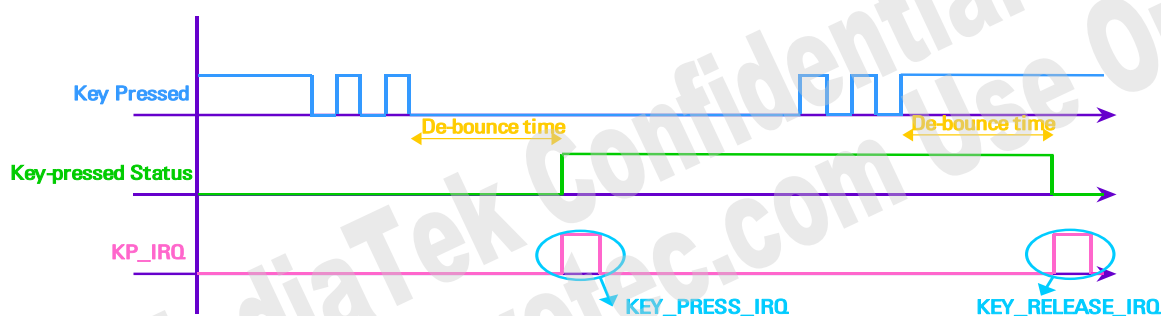


Figure 5-4. One-key-pressed with debounce mechanism denoted

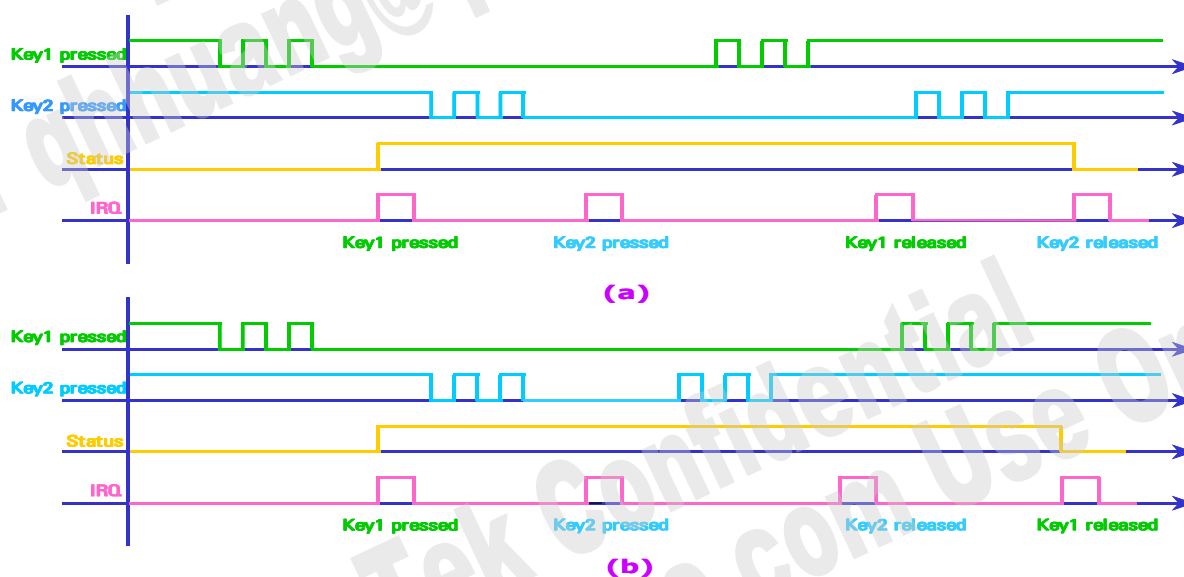


Figure 5-5. Two-keys-pressed sequence

5.3.5 Programming Guide

This section describes the operating sequence, as shown in the table below.

Table 5-2. Keypad setting flow

Step	Address	Register name	Local address	R/W	Value	Description
Set up keypad						
1	Keypad base address + 0x018	KP_DEBOUNCE	DEBOUNCE	RW	14'h400	Set up debounce time. Debounce time = KP_DEBOUNCE/32 ms.
Wait for the keypad to issue interrupt; and read keypad MEM register.						

5.3.6 Register Definition

See chapter 3.3 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.4 Universal Asynchronous Receiver/Transmitter

5.4.1 Introduction

The universal asynchronous receiver/transmitter (UART) provides full duplex serial communication channels between the chip and external devices. The UART has M16C450 and M16550A operation modes, which are compatible with a range of standard software drivers. The extensions are designed to be broadly software compatible with 16550A variants, but certain areas offer no consensus.

In common with M16550A, the UART supports word lengths from five to eight bits, an optional parity bit and one or two stop bits, and this word length is fully programmable by a CPU interface. A 16-bit programmable baud rate generator and an 8-bit scratch register are included, along with separate transmission and received FIFOs. Eight modem control lines and a diagnostic loop-back mode are provided. The UART also includes two direct memory access (DMA) handshake lines which are used to indicate when the FIFOs are ready to transfer data to CPU. Interrupts can be generated from any of the several sources.

After hardware reset, the UART is in M16C450 mode. Its FIFOs can be enabled and the UART can enter M16550A mode. The UART adds further functionality beyond M16550A mode. Each of the extended functions can be selected individually under software control.

5.4.2 Features

The features of UART are as follows:

- Provides 3 channels of UARTs
- UART1 is a 4-pin (Tx, Rx, CTS, RTS) UART channel.
- UART0 and UART2 are 2-pin (Tx, Rx) UART channels.
- Supports both M16C450 and M16550A operation modes
- Compatible with standard software drivers
- Transfer system: Asynchronous
- Data length: 5 to 8 bits
- Hardware flow control: CTS/RTS-based automatic transmission and reception of control
- Software flow control: Uses special character Xon/Xoff to do software flow control
- Baud rate is programmable from 300 bps to 3 Mbps.
- Baud rate error: Less than 0.25 %
- Interrupt request: Receive interrupts/transmit interrupts
- Data transfer: DMA (transmit/receive) transfer is supported.

5.4.3 Block Diagram

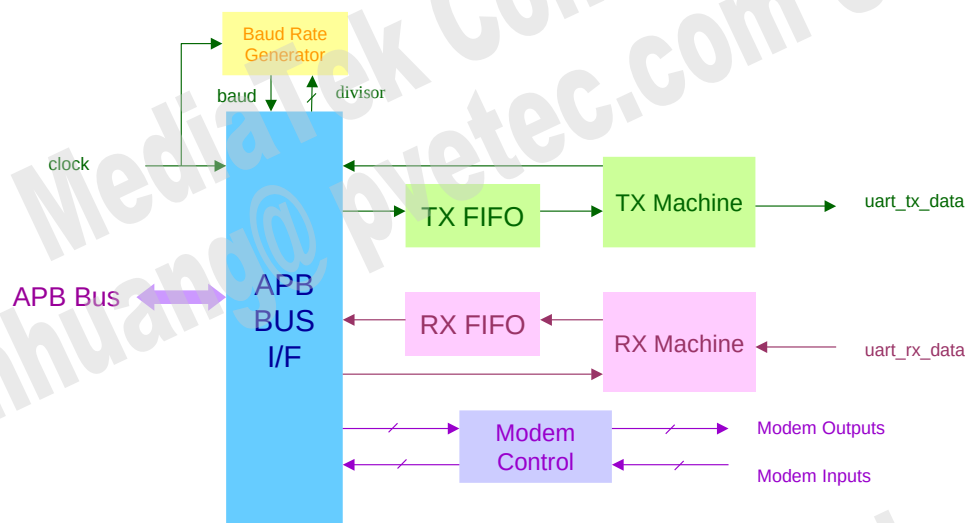


Figure 5-6. Block diagram of UART

Figure 5-6 shows the block diagram of UART. It consists of the first in first out (FIFO), the finite state machine (FSM), APB BUS I/F and modem control. Because of full-duplex serial communication, UART has Tx and Rx channels, and each channel contains a FSM (Tx FSM & Rx FSM) and a 32-byte FIFO (Tx FIFO & Rx FIFO). The FSM is used to indicate the current transfer stage of Tx and Rx channel. The Tx FIFO and Rx FIFO store the data to be sent or the data received from outside. Through APB BUS I/F, the system can access the configuration and status registers of UART, read the received data or write the data to be sent. UART can support the function of hardware flow control through modem control. The UART signal descriptions are included in Table 5-3.

Table 5-3. Signal descriptions

Signal name	Signal type	Description
uart_tx_data	Output	Serial data transmit
uart_rx_data	Input	Serial data receive
RTS	Output	Request to send handshaking signal
CTS	Input	Clear to send handshaking signal

5.4.4 Communication Protocol

UART communication protocol is as follows:

- 1-bit start bit must be low.
- The data bit length is 5~8 bits.
- The parity check can be odd parity or even parity.
- The stop bit length is 1~2 bit(s). It must be high.

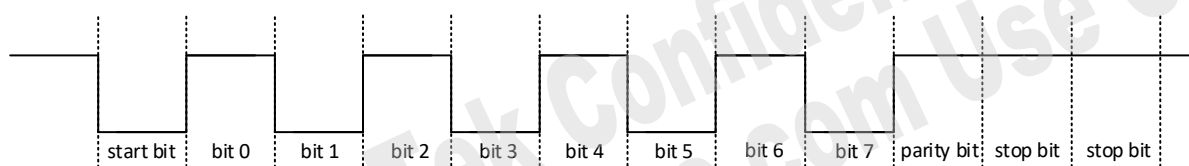


Figure 5-7. UART communication protocol

5.4.5 Theory of Operations

UART Enhancement Features

The UART provides more powerful enhancements than the industry-standard 16550:

Hardware flow control: Use two dedicated signals, clear to send (CTS) and request to send (RTS) signals, to indicate UART is ready to get data or send data. When CTS is low, UART can start to transmit data. As long as CTS is active, UART is not allowed to send data. RTS going low means UART FIFO in received circuit is sufficient to receive data. UART is not allowed to receive data when RTS is high. This feature is very useful when the interrupt service routine (ISR) latency is hard to predict and control in the embedded applications. The MCU is relieved of having to fetch the received data within a fixed amount of time.

Software flow control: Use special character Xon/Xoff to do software flow control. Special character Xon/Xoff is software programmable. When Xoff is received, UART transmission is halted. The transmission will not be resumed until Xon is received.

Notes:

1. In order to enable any of the enhancements, the enhanced mode bit, EFR[4], must be set. If EFR[4] is not set, IER[7:5], FCR[5:4], and MCR[7:6] cannot be written. The enhanced mode bit ensures that the UART is backward compatible with software that has been written for 16C450 and 16550A devices.
2. When the oversampling ratio between UART clock and baud rate is less than 8, it is necessary to enable guard time function in your UART Tx device to make MediaTek UART Rx work properly. Otherwise, frame error could happen and the received data could get corrupted.

UART Interrupt

UART generates several interrupts. The interrupt types are shown in [Table 5-5](#).

Table 5-4. UART interrupt control bits and interrupt factors

UARTn+0004h Interrupt Enable Register									UARTn_IER							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name									CTSI	RTSI	XOFFI		EDSSI	ELSI	ETBEI	ERBFI
Type									R/W							
Reset									0							

IER[3:0] are modified when LCR[7] = 0.

IER[7:4] are modified when LCR[7] = 0 & EFR[4] = 1.

UARTn+0008h

Interrupt Identification Register

UARTn_IIR

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name									FIFOE		ID4	ID3	ID2	ID1	ID0	NINT
Type									RO							
Reset									0	0	0	0	0	0	0	1

ID4 and ID3 are presented only when EFR[4] = 1.

Table 5-5. UART interrupt types

Interrupt type	Interrupt request bit (UARTn_IER)	Interrupt identification (UARTn_IIR)	Interrupt factor	Notes
Received	ERBFI	IIR[5:0] = 000100b	Rx buffer contains data.	-
	-	IIR[5:0] = 001100b	Timeout on character in Rx FIFO	-
Transmit	ETBEI	IIR[5:0] = 000010b	Tx holding register is empty or the contents of the Tx FIFO have been reduced to its trigger level.	-
Communication error	ELSI	IIR[5:0] = 000110b	When frame error/parity error/break interrupt/FIFO overrun happens, the interrupt will be generated.	For the detailed interrupt status, refer to LSR[4:1].
Modem	EDSSI	IIR[5:0] = 000000b	Modem status change	For the detailed interrupt status, refer to MSR[4:1].
Enhancement feature	CTSI	IIR[5:0] = 100000b	When a rising edge is detected on the CTS, the interrupt will be generated.	Available when enhanced feature is enabled. (EFR[4] = 1)
	RTSI	IIR[5:0] = 100000b	When a rising edge is detected on the RTS, the interrupt will be generated.	
	XOFF1	IIR[5:0] = 010000b	When an XOFF character is received, the interrupt will be generated.	

Data Transmission

If the Tx holding register (THR) is empty (FIFOs are disabled) or Tx FIFO is reduced to its trigger level (FIFOs are enabled), Tx holding register empty (THRE) bit of the LSR is "1" and the transmitted data can be written in the THR.

When THR is not empty (FIFOs are disabled) or Tx FIFO is increased to its trigger level (FIFOs are enabled), Tx starts transmission automatically.

Software can write transmitted data into THR by asserting transmit interrupt (IIR[5:0] = 000010b) or polling the THRE bit status as "1" directly.

If FIFOs are enabled, the transmitted data can be written into the THR. The data will be transferred to Tx FIFO directly.

Data Reception

If the Rx buffer is becoming full or a byte is being transferred into Rx FIFO, the DR bit of the LSR is "1" and the received data can be read by Rx buffer register (RBR).

Software can read the received data when receive interrupt (IIR[5:0] = 000100b) is asserted or UART is polling the DR bit status directly.

If FIFOs are enabled, the received data in the Rx FIFO can be read by reading RBR.

Register Description

UART_BASE:

UART0 register base address is 0x1100_2000.

UART1 register base address is 0x1000_E000.

UART2 register base address is 0x1000_F000.

Table 5-6. UART register map

Address	Name	Description
UART_BASE+0x0C	LCR	Line control register (LCR)
UART_BASE+0x24	HIGHSPEED	High speed UART
UART_BASE+0x28	SAMPLE_COUNT	SAMPLE_COUNT
UART_BASE+0x2C	SAMPLE_POINT	SAMPLE_POINT
UART_BASE+0x34	RATEFIX_AD	Rate fix address
UART_BASE+0x3C	GUARD	Guard time added register
UART_BASE+0x40	ESCAPE_DAT	Escape character register
UART_BASE+0x44	ESCAPE_EN	Escape enable register
UART_BASE+0x48	SLEEP_EN	Sleep enable register
UART_BASE+0x4C	VFIFO_EN	Virtual FIFO enable register
UART_BASE+0x50	RXTRI_AD	Rx trigger address
UART_BASE+0x54	FRACDIV_L	Fractional divider LSB address
UART_BASE+0x58	FRACDIV_M	Fractional divider MSB address
UART_BASE+0x5C	FCR_RD	FIFO control register
UART_BASE+0x60	TX_ACTIVE_EN	Tx active enable address

Condition: LCR[7] == 0			Condition: LCR[7] == 1	
Address	Name	Description	Name	Description
UART_BASE+0x00	THR RBR	Tx holding register/ Rx buffer register	DLL	Divisor latch (LS)
UART_BASE+0x04	IER	Interrupt enable register	DLM	Divisor latch (MS)
Condition: LCR != 0xBF			Condition: LCR == 0xBF	
Address	Name	Description	Name	Description
UART_BASE+0x08	FCR IIR	FIFO control register/ Interrupt identification register	EFR	Enhanced feature register
UART_BASE+0x10	MCR	Modem control register	XON1	XON1
UART_BASE+0x14	LSR	Line status register	XON2	XON2
UART_BASE+0x18	MSR	Modem status register	XOFF1	XOFF1
UART_BASE+0x1C	SCR	Scratch register	XOFF2	XOFF2

UARTn+0000h

Divisor Latch (LS)

UARTn_DLL

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name									DLL[7:0]							
Type									R/W							
Reset									1							

UARTn+0004h									UARTn_DLM							
Divisor (MS)																
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name									DLM[7:0]							
Type									R/W							
Reset									0							

Note:

DLL & DLM can only be updated if LCR[7] is set ("1"). Note that division by 1 generates a BAUD signal that is constantly high. The following table shows the divisor that needs to generate a given baud rate from CLK inputs of 26 MHz.

e.g., clock source: 26 MHz, baud rate: 4,800 bps

HIGHSPEED (0x24) = 0 case: $26 \text{ MHz} / 4,800 / 16 \approx 339 = 0x153 \rightarrow \text{DLL: } 0x01, \text{DLM: } 0x53$

HIGHSPEED (0x24) = 1 case: $26 \text{ MHz} / 4,800 / 8 \approx 677 = 0x2A3 \rightarrow \text{DLL: } 0x02, \text{DLM: } 0xA3$

HIGHSPEED (0x24) = 2 case: $26 \text{ MHz} / 4,800 / 4 \approx 1354 = 0x54A \rightarrow \text{DLL: } 0x05, \text{DLM: } 0x4A$

HIGHSPEED (0x24) = 3 case: $26 \text{ MHz} / 4,800 / 256 + 1 \approx 22 = 0x16 \rightarrow \text{DLL: } 0x00, \text{DLM: } 0x16$

UARTn+0024h								HIGH SPEED UART								UARTn_HIGHSPEED							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name															SPEED [1:0]								
Type															R/W								
Reset															0								

SPEED UART sample counter base

0: Based on $16 * \text{baud_pulse}$, $\text{baud_rate} = \text{system clock frequency} / 16 / \{\text{DLM}, \text{DLL}\}$

1: Based on $8 * \text{baud_pulse}$, $\text{baud_rate} = \text{system clock frequency} / 8 / \{\text{DLM}, \text{DLL}\}$

2: Based on $4 * \text{baud_pulse}$, $\text{baud_rate} = \text{system clock frequency} / 4 / \{\text{DLM}, \text{DLL}\}$

3: Based on $\text{sample_count} * \text{baud_pulse}$, $\text{baud_rate} = \text{system clock frequency} / (\text{sample_count} + 1) / \{\text{DLM}, \text{DLL}\}$

UARTn+0028h									SAMPLE_COUNT									UARTn_SAMPLE_COUNT							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name									SAMPLECOUNT [7:0]																
Type									R/W																
Reset									0																

When HIGHSPEED = 3, the sample_count is the threshold value for UART sample counter (sample_num).

Sample Count = $\text{clock source} / \text{baud rate} / \{\text{DLL}, \text{DLM}\} - 1$

e.g., clock source: 26 MHz, baud rate: 4,800 bps; DLL: 0x00, DLM: 0x16

High speed (0x24) = 0&1&2 case: No need to set SAMPLE_COUNT

High speed (0x24) = 3 case: $26 \text{ MHz} / 4,800 / 0x16 - 1 \approx 245 \rightarrow \text{SAMPLE_COUNT} = 245$

UARTn+002Ch								SAMPLE_POINT								UARTn_SAMPLE_POINT							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
Name									SAMPLEPOINT [7:0]														
Type									R/W														
Reset									Ffh														

When HIGHSPEED = 3, UART will get the input data when sample_count = sample_num.

The SAMPLE_POINT is usually (SAMPLE_COUNT/2).

e.g., clock source: 26 MHz, baud rate: 4,800 bps; DLL: 0x00, DLM: 0x16&SAMPLE_COUNT = 245

Sample point = $245/2 = 123$ (sample the central point to decrease the inaccuracy)

UARTn+0054h									Fractional Divider LSB Address									UARTn_FRACDIV_L							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
Name									FRACDIV_L																
Type									R/W																
Reset									0	0	0	0	0	0	0	0									

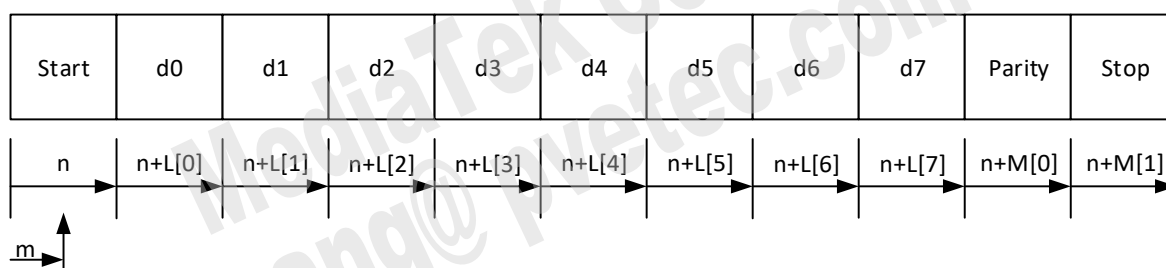
FRACDIV_L: Add sampling count (+1) from state data7 to state data0 in order to improve fractional divisor.

UARTn+0058h								Fractional Divider MSB Address								UARTn_FRACDIV_M			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name															FRACDIV_M				
Type															R/W				
Reset															0	0			

FRACDIV_M: Add sampling count in state stop and state parity in order to improve fractional divisor.

FRACDIV_L/FRACDIV_M: Add one sampling period to each symbol in order to increase the baud rate accuracy.

bit_extend register = FRACDIV_L[7:0]
FRACDIV_M[1:0]



Bit extend number = $\text{ROUND}((\text{clock source}/\text{baud rate}/\{\text{DLL}, \text{DLM}\} - (\text{SAMPLE_COUNT} + 1)) * 10)$

e.g., clock source: 26 MHz, baud rate: 4,800 bps; DLL: 0x00, DLM: 0x16&SAMPLE_COUNT = 245

Bit extend number = $\text{ROUND}((26 \text{ MHz}/4,800/0x16 - (245+1)) * 10) = 2 \rightarrow$ Need to compensate for 2 bits of one frame (e.g., FRACDIV_L = 0x44, FRACDIV_M = 0x00)

Refer to [Table 5-7](#) for the details.

Table 5-7. Bit extend number reference

Bit extend number	FRACDIV_M	FRACDIV_L
0	0x00	0x00
1	0x00	0x10
2	0x00	0x44
3	0x00	0x92
4	0x01	0x29
5	0x01	0xaa
6	0x01	0xb6
7	0x01	0xdb

Bit extend number	FRACDIV_M	FRACDIV_L
8	0x01	0xef
9	0x01	0xff
10	0x03	0xff

5.4.6 Register Definition

See chapter 3.4 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.4.7 Programming Guide

5.4.7.1 Baud rate setting and UART initialization

Table 5-8 shows the suggestion for UART baud rate setting from clock inputs of 30 MHz.

Table 5-8. Suggestion for UART baud rate setting

Baud rate	HIGHSPEED	{DLM, DLL}	SAMPLE_COUNT	SAMPLE_POINT	FRACDIV_M	FRACDIV_L
3,000,000	3	0x00, 0x01	0x09	0x03	0x0	0x0
2,000,000	3	0x00, 0x01	0x0E	0x06	0x0	0x0
1,000,000	3	0x00, 0x01	0x1D	0x0D	0x0	0x0
500,000	3	0x00, 0x01	0x3B	0x1C	0x0	0x0
250,000	3	0x00, 0x01	0x77	0x3A	0x0	0x0
153,600	3	0x00, 0x01	0xC2	0x60	0x0	0x92
115,200	3	0x00, 0x02	0x81	0x3F	0x0	0x44
76,800	3	0x00, 0x02	0xC2	0x60	0x0	0x92
57,600	3	0x00, 0x03	0xAC	0x55	0x1	0xB6
38,400	3	0x00, 0x04	0xC2	0x60	0x0	0x92
28,800	3	0x00, 0x05	0xCF	0x66	0x0	0x92
19,200	3	0x00, 0x07	0xDE	0x6E	0x0	0x44
9,600	3	0x00, 0x0D	0xEF	0x76	0x1	0x92
7,200	3	0x00, 0x11	0xF4	0x79	0x1	0x01
4,800	3	0x00, 0x19	0xF9	0x7B	0x0	0x0

Use register DLL, DLM, HIGHSPEED, SAMPLE_COUNT, SAMPLE_POINT, FRACDIV_M and FRACDIV_L to set up baud rate. After setting up the baud rate, UART can start transmission by filling the Tx FIFO and receiving data from Rx FIFO.

Table 5-9 is an example for generating baud rate 115,200 bps by clock inputs of 30 MHz:

Table 5-9. UART baud rate setting example

Step	Description	Related register setting
1	Select UART sample counter base to SPEED 3.	HIGHSPEED = 0x3

Step	Description	Related register setting
2	Set up the sample counter.	SAMPLE_COUNT = 0x81 SAMPLE_POINT = 0x3F FRACDIV_L = 0x44 FRACDIV_M = 0x1
3	Switch register to divisor mode (register map condition 2) to do divisor latch setting. UARTn_LCR[7] = 1	LCR = 0x80
4	Set up the divisor latch.	DLL = 0x2 DLM = 0x0
5	Set up the guard time.	GUARD
6	Switch register to normal mode (register map condition 1). UARTn_LCR[7] = 0	LCR = 0x00

Table 5-10. UART hardware initialization

Step	Description	Related register setting
1	Baud rate setting: Refer to Table 5-9 .	-
2	Enable enhanced feature. (Register is accessible only when LCR = BF'h.)	LCR = 0xBF EFR = 0x10 LCR = 0x00
3	Enable FIFO control.	FCR
4	Word length (LCR[1:0]), parity (LCR[5:4]), STOP (LCR[2]) bit settings	LCR
5	Enable interrupt.	IER

The suggested ED software programming sequence:

1. DRV_WriteReg32(UART_BASE+0x24, 0x00000003); //high-speed UART
2. DRV_WriteReg32(UART_BASE+0x28, 0x00000081); //sample_count
3. DRV_WriteReg32(UART_BASE+0x2C, 0x0000003F); //sample_point
4. DRV_WriteReg32(UART_BASE+0x4C, 0x00000001); //Enable Rx DMA
5. DRV_WriteReg32(UART_BASE+0x54, 0x00000044); //FRACDIV_L
6. DRV_WriteReg32(UART_BASE+0x58, 0x00000001); //FRACDIV_M
7. DRV_WriteReg32(UART_BASE+0x0C, 0x000000BF); //LCR=0xBF, change to Condition 2
8. DRV_WriteReg32(UART_BASE+0x00, 0x00000002); //DLL, LS
9. DRV_WriteReg32(UART_BASE+0x04, 0x00000000); //DLM, MS
10. DRV_WriteReg32(UART_BASE+0x08, 0x00000010); // Enable enhancement features
11. DRV_WriteReg32(UART_BASE+0x0C, 0x00000000); //LCR !=0xBF, change to Condition 1
12. DRV_WriteReg32(UART_BASE+0x08, 0x00000031); //FIFO trigger threshold and enable FIFO
13. DRV_WriteReg32(UART_BASE+0x0C, 0x00000003); //8-bit word length
14. DRV_WriteReg32(UART_BASE+0x04, 0x00000001); //Enable Rx interrupt

5.4.7.2 Transmission

Data Transmission

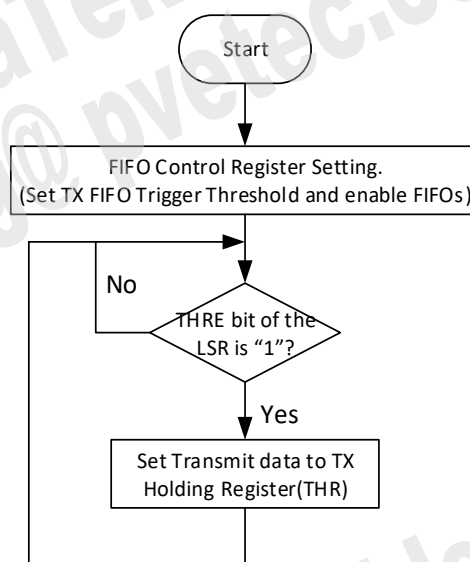


Figure 5-8. UART data transmission with polling the THRE bit status

FIFO register setting:

- Enable FIFO: Set x'08[0] to 1'b1.
- Tx FIFO trigger threshold x'08[5:4]
- 2'b00: 1 byte (down to trigger)
- 2'b01: 4 bytes (down to trigger)
- 2'b10: 8 bytes (down to trigger)
- 2'b11: 14 bytes (down to trigger)

Data Reception

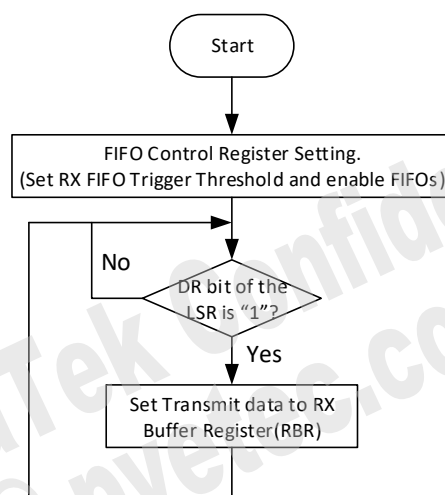


Figure 5-9. UART data reception with polling the DR bit status

FIFO register setting:

- Enable FIFO: Set x'08[0] to 1'b1.
- Rx FIFO trigger threshold x'08[7:6]
- 2'b00: 1 byte (up to trigger)
- 2'b01: 6 bytes (up to trigger)
- 2'b10: 12 bytes (up to trigger)
- 2'b11: Register x'50[4:0] (up to trigger)

5.5 Super Speed Universal Serial Bus (SSUSB)

5.5.1 Introduction

The SuperSpeed universal serial bus (SSUSB) provides a USB3.1 port with the USB3.1 Gen1 peripheral role, and the port is with the USB2.0 dual role capability. It contains a pair of U3 Gen1 PHY and U3 Gen1 MAC for SuperSpeed connection, and a pair of U2 PHY and U2 MAC for high-, full- and low-speed connections.

The USB2.0 dual role capability allows the port to support On-the-Go (OTG) host and the peripheral. For a USB2.0 host role, this port is controlled by the eXtensible host controller (xHC) which is used to manage all devices connected through its root hub ports. For a peripheral role, the port is controlled by the device (DEV) controller.

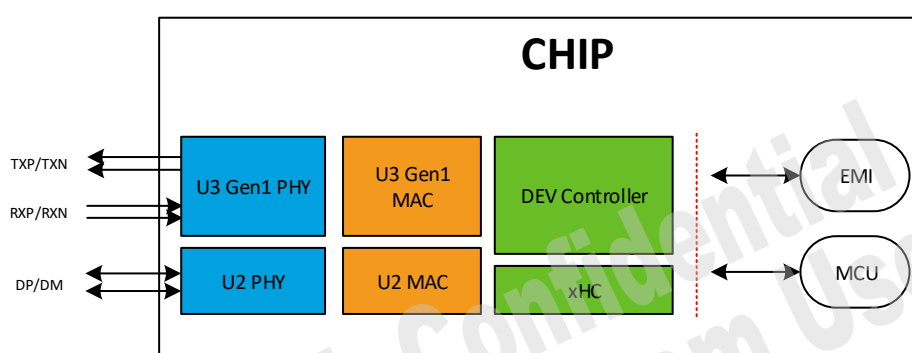


Figure 5-10. Block diagram of SSUSB

5.5.2 Features

- Configurable to be the USB3.1 Gen1 peripheral role
- Configurable to be the USB2.0 host role
- Shared hardware on the dual function
- Proprietary application layer device controller with linked list queue and scatter/gather DMA
- eXtensible host controller interface (xHCI) based host controller
- Embedded USB3 Gen1 PHY with the 32-bit/125 MHz PIPE interface
- Embedded USB2 PHY with the 16-bit/30 MHz UTMI interface
- High-/Full-speed OTG host and peripheral compliant with OTG supplement version 2.0

5.5.3 Host Function Architecture

5.5.3.1 Architecture Overview

The architecture of the USB2.0 host is illustrated as the following figure, which includes one U2PHY and one MAC to handle the protocol packets. All resources of the endpoint and the device are handled by an xHCI controller. Firmware can dynamically allocate resource for different endpoints. The DMA channel of U2 port can be multiplexed in the system.

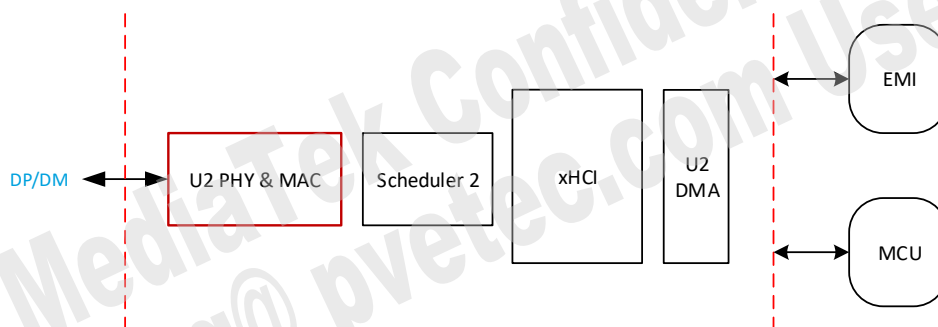


Figure 5-11. USB2.0 host architecture

5.5.3.2 Host Features

- Supports linking with high-, full- and low-speed devices
- Lower power management (LPM) on the U2 port
- Hardware configurable up to 2 USB2 ports with dedicated 480-Mbs bandwidth and shared DMA
- Control/bulk/interrupt/isochronous PIPE type (currently no stream supported)
- Compatible with connecting to the U2 hub
- Smart scheduling algorithm
- Up to 15 devices
- Up to 32 endpoints configuration

5.5.4 Device Function Architecture

5.5.4.1 Device Features

- Supports linking with super-, high- and full-speed hosts
- Embedded queue management function with scatter/gather DMA capability
- LPM on the U2 port
- U0/U1/U2/U3 state on the U3 port
- 16 KB of on-chip data RAM
- Hardware configurable up to 8 OUT endpoints and 8 IN endpoints
- Hardware configurable up to 4 packet slots for each endpoint separately
- Firmware configurable FIFO size allocation for each endpoint separately
- Firmware configurable transfer type to bulk/interrupt/isochronous for each endpoint

5.5.4.2 Architecture Overview

The architecture of the device is illustrated in [Figure 5-12](#). The linked-list queue of the device is basically inherited from the MediaTek unified USB IP with the similar descriptor definition.

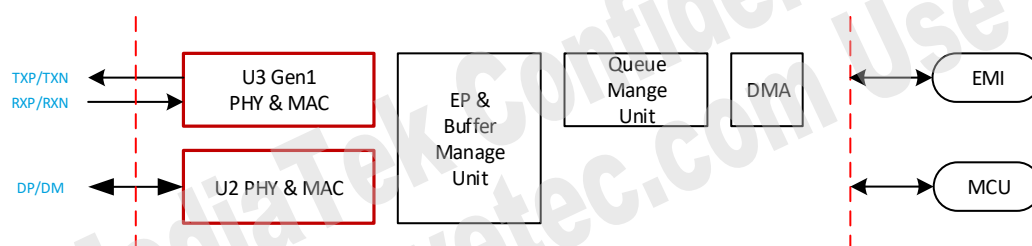


Figure 5-12. SSUSB device architecture

5.5.5 Register Definition

See chapter 3.5 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.6 Serial Peripheral Interface (SPI) Master

5.6.1 Introduction

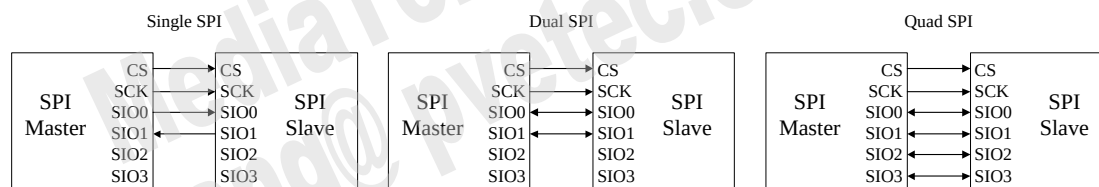


Figure 5-13. Pin connection between SPI master and SPI slave

The serial peripheral interface (SPI) is a bit-serial transmission protocol that supports single mode (4-pin), dual mode (4-pin) and quad mode (6-pin) for increased data throughput. The maximum serial clock (SCK) frequency is 52 MHz. Note that single/dual mode can support full/half duplex mode, but quad mode only supports half duplex mode.

[Figure 5-13](#) is an example of depicting the connection between the SPI master and the SPI slave. The SPI interface is the master responsible for the data transmission with the slave. There are eight SPI masters in MT8676. One is QSPI master, the other is single mode SPI master.

Table 5-11. SPI master controller interface

Signal name	Type	Description
CS	O	Low active chip selection signal
SCK	O	(Bit) serial clock (Max. SCK rate is 52 MHz.)
SIO0/MOSI	I/O	Data signal 0
SIO1/MISO	I/O	Data signal 1
SIO2/WP	I/O	Data signal 2
SIO3/HLOD	I/O	Data signal 3

Table 5-12. SPI master abbreviations and full forms

Abbreviation	Definition
MOSI	Master out slave in
MISO	Master in slave out
CPOL	Clock polarity
CPHA	Clock phase

5.6.2 Features

The features of the SPI controller (master) are listed below:

- The QSPI supported SCK is up to 52 MHz.
- The single mode SPI supported SCK is up to 52 MHz.

- The SPI master controller supports single mode (4-pin), dual mode (4-pin) and quad mode (6-pin). The controller can automatically set the port direction for data input/output, if the registers SPI_PIN_MODE, SPI_HALF_DUPLEX_EN and SPI_HALF_DUPLEX_DIR are already set.
- Configurable CS_N setup time, hold time and idle time are shown in [Figure 5-14](#).

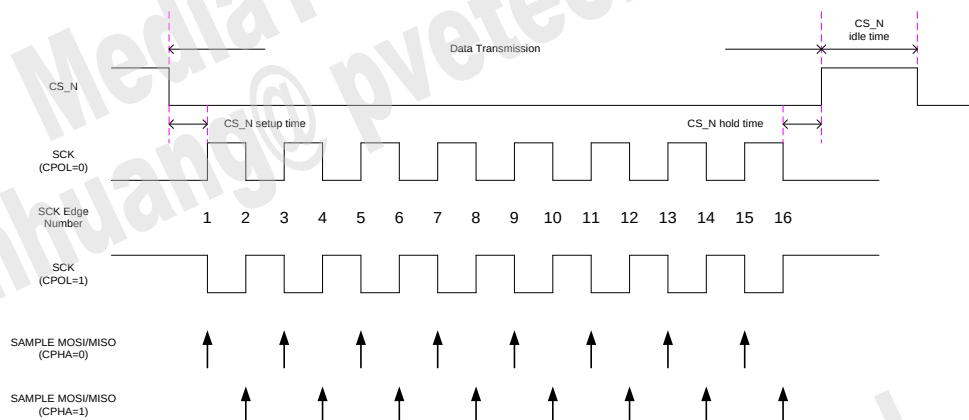


Figure 5-14. SPI transmission formats

- CS_N setup time, hold time and idle time can be adjusted by setting up the corresponding registers.
- Programmable SCK high time and low time can be set separately. Thus, for a given baud rate, SCK with a wide range of duty cycle can be generated.
- Configurable transmitting and receiving bit order: Two options of the bit order - MSB or LSB first
- Two configurable modes for the source of the data to be transmitted:
 - In Tx DMA mode, the SPI controller automatically fetches the data to be put on the MOSI line from memory.
 - In Tx PIO mode, the data to be transmitted on the MOSI line are written to FIFO by software before the start of the transaction.

(Notes: 1. The value of SPI_TX_SRC (SPIn Base address+0x0008) [31:0] must be 4-byte aligned.

2. Tx data must be always prepared before transaction. In DMA mode, TX_DMA_EN (SPIn Base address+0x0018) [11] must be set to 1'b1. In PIO mode, software must put data into Tx FIFO through SPI_TX_DATA (SPIn Base address+0x0010) [31:0] register.)

- The depth of the Tx FIFO is 32 bytes:
 - In Tx DMA mode, the data to be put on the MOSI line are prepared in advance in the memory that the SPI controller automatically reads the data from it.
 - In TXPIO mode, writing to SPI_TX_DATA (SPIn Base address+0x0010) [31:0] register means to write 4 bytes to the Tx FIFO. The Tx FIFO pointer automatically moves towards the next 4 bytes.
- Two configurable modes for the destination of the data to be received:
 - In Rx DMA mode, the SPI controller automatically stores the received data (from the MISO line) to the memory.
 - In Rx PIO mode, the received data are stored in the Rx FIFO of the SPI controller. The processor must read back the data by itself.

(Note: The value of SPI_RX_DST (SPIn Base address+0x000C) [31:0] must be 4-byte aligned.)

- The depth of the Rx FIFO is 32 bytes:
 - In Rx DMA mode, the received data from the MISO line are moved to the memory automatically by the SPI controller.
 - In Rx PIO mode, reading from SPI_RX_DATA (SPIn Base address+0x0014) [31:0] register means to read 4 bytes from Rx FIFO which its pointer automatically moves towards the next 4 bytes.
- The programmable byte length for transmission:

- PACKET_LENGTH (SPIn Base address+0x0004) [31:16] defines the number of bytes in one packet
- PACKET_LOOP_CNT (SPIn Base address+0x0004) [15:8] defines the number of packets within one transaction.
- The number of bytes in one packet = PACKET_LENGTH+1
- The number of packets in one transaction = PACKET_LOOP_CNT+1
- The total bytes of one transaction = (PACKET_LENGTH+1) * (PACKET_LOOP_CNT+1)
- The unlimited length for transmission is achieved by the operation in PAUSE mode.
 - In PAUSE mode, the CS_N signal keeps being active (normally low) after the transmission. At this time, the SPI controller is in the PAUSE_IDLE state and ready to receive the resume command. The state transition is shown in [Figure 5-15](#).

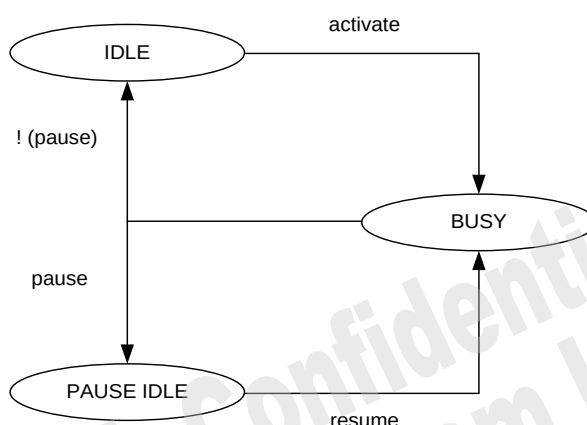


Figure 5-15. Operation flow with/without PAUSE mode

- Configurable option to control the CS_N de-assert between byte transfers. The controller supports a special transmission format called “CS_N de-assert mode” that its waveform is illustrated in [Figure 5-16](#).

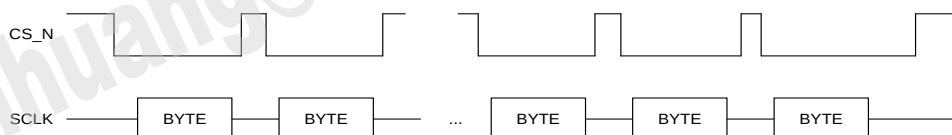


Figure 5-16. CS_N de-assert mode

- When the SPI master controller operates in dual/quad mode with half duplex, the transmission package includes three parts: command phase, address phase, and data phase.
 - The command phase always operates in single mode.
 - The address phase cannot transmit/receive data.
 - Data phase operation depends on SPI_PIN_MODE and SPI_HALF_DUPLEX_DIR settings. The command and address phases are useful for special applications, such as reading/writing serial flash data.
- Four communication modes are available (mode 0, 1, 2, and 3) that they define the SCLK edge on which the MOSI line toggles and the master samples the MISO line. They also define the SCLK steady level - the clock level, high or low, when the clock is not active. Each mode is formally defined with a pair of parameters namely “clock polarity” (CPOL) and “clock phase” (CPHA).

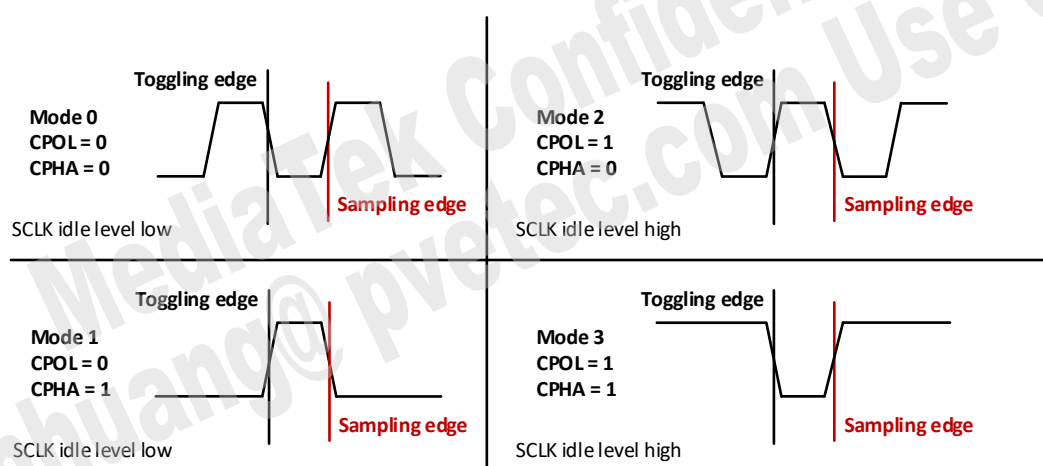


Figure 5-17. Waveforms of the four communication modes

- CS GPIO mode (SPIn Base address+0x0018) [18]: The SPI_CS pin can be replaced by any GPIO pin. The software controls a GPIO pin as an SPI_CS pin. By this means, a single SPI master can be selected and communicates with multi-SPI slave. [Figure 5-18](#) illustrates the block diagram of the multi-slave topology for single mode. The CS pins for device 2, 3, 4, ..., are replaced by the GPIO pins.

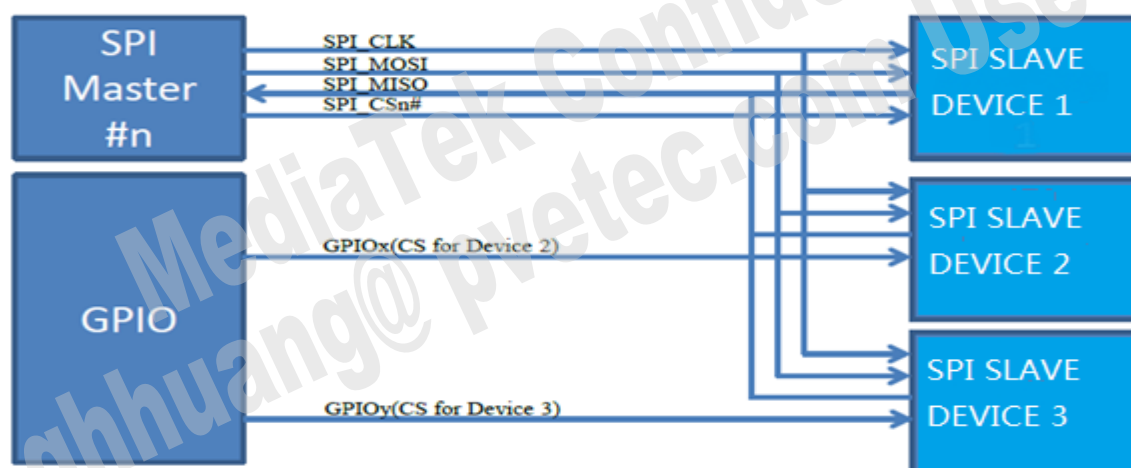


Figure 5-18. Pin connection between the master and the multi-slave for single mode

5.6.3 Block Diagram

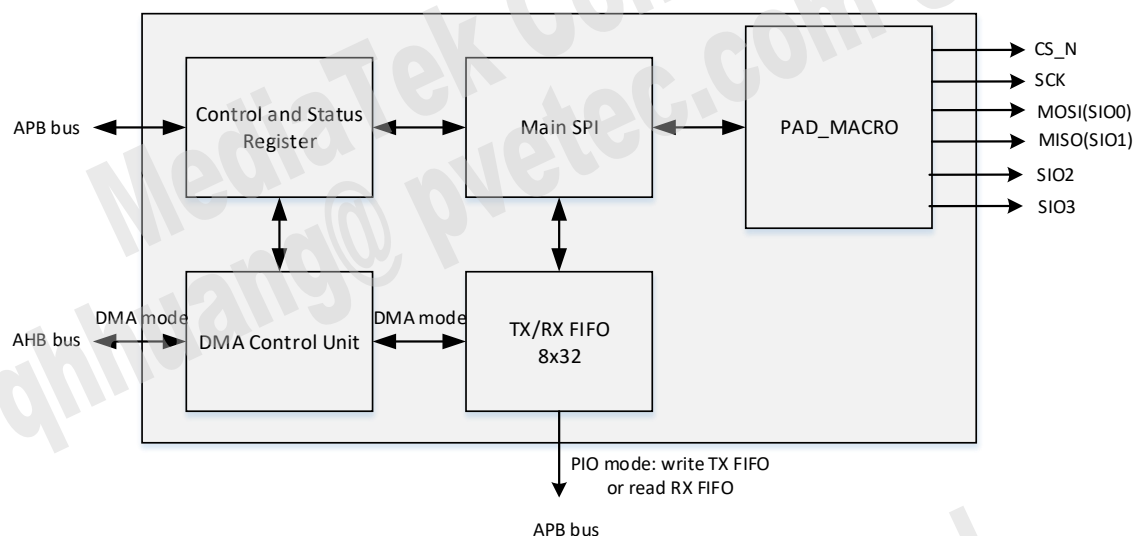


Figure 5-19. Block diagram of the SPI master

Figure 5-19 is the block diagram of the SPI master which consists of the control and status register, DMA control unit, main SPI, FIFO and PAD_MACRO.

Block	Description
Control and status register	Receives commands from the system
DMA control unit	Communicates with SYSRAM when SPI is set to DMA mode
Main SPI	Functional unit
Tx/Rx FIFO 8x32	Both Tx and Rx have an 8x32-bit FIFO for restoring data.
PAD_MACRO	Controls the SPI data capture and transmission to/from SPI I/F

In PIO mode, the software can write data into Tx FIFO via SPI_TX_DATA (SPIn Base address+0x0010) [31:0] or read data from Rx FIFO via SPI_RX_DATA (SPIn Base address+0x0014) [31:0].

In DMA mode, the SPI can automatically get data from or send data to SYSRAM via the AHB after the software configures DMA parameters.

The descriptions of the SPI master pins are listed in Table 5-11 and the IO operates in 1.8V.

5.6.4 Theory of Operations

5.6.4.1 SPI Transaction Format

As mentioned previously in section 5.6.2, the SPI master supports single/dual/quad mode, and single/dual modes can support full/half duplex mode while quad mode can only support half duplex mode. A general summary table is as shown below. More details are to be explained in the following sections.

Table 5-13. SPI transaction format summary

Mode	Register	Value
Single mode and full duplex mode	SPI_CFG3[1:0]	2'b00
	SPI_CFG3[3]	1'b0
Single mode and half duplex mode	SPI_CFG3[1:0]	2'b00
	SPI_CFG3[3]	1'b1
Dual mode and half duplex mode	SPI_CFG3[1:0]	2'b01
	SPI_CFG3[3]	1'b1
Dual mode and full duplex mode	SPI_CFG3[1:0]	2'b01
	SPI_CFG3[3]	1'b0
Quad mode and half duplex mode	SPI_CFG3[1:0]	2'b10
	SPI_CFG3[3]	1'b1

5.6.4.1.1 SPI Single Mode and Full Duplex Mode Transaction

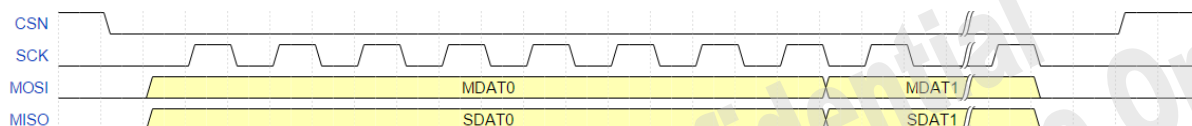


Figure 5-20. SPI single mode and full duplex mode transaction format

In DMA mode, data to be transferred should be prepared in the memory in advance.

In PIO mode, the system should push the data that are to be transferred into SPI Tx FIFO first. After receiving the START (SPI Base address+0x0018) [0] command, the SPI sends data to the slave continuously and receive the data from the slave at the same time.

If register SPI_CFG3[1:0] is set to 2'b00 and SPI_CFG3[3] is set to 1'b0, the SPI works in single mode and full duplex mode.

The waveform on the SPI pins is shown in Figure 5-20. MDATA is the data transferred from the SPI master to the slave; and SDATA is the data transferred from the SPI slave to the master.

5.6.4.1.2 SPI Single Mode and Half Duplex Mode Transaction

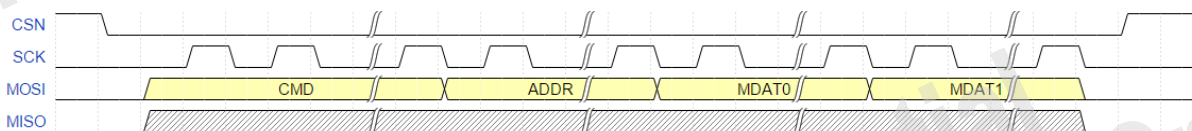


Figure 5-21. SPI single mode and half duplex mode Tx transaction format

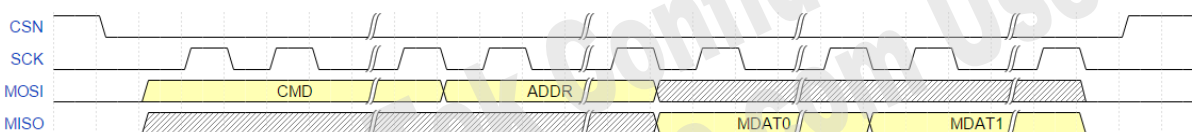


Figure 5-22. SPI single mode and half duplex mode Rx transaction format

If register SPI_CFG3[1:0] is set to 2'b00 and register SPI_CFG3[3] is set to 1'b1, the SPI works in single mode and half duplex mode.

If register SPI_CFG3[2] is set to 1'b0, the SPI master transfers data to the slave while no data is transferred from the slave to the master. The waveform of the SPI pins is shown in [Figure 5-21](#).

If register SPI_CFG3[2] is set to 1'b1, the SPI master receives data from the slave while no data is transferred from the master to the slave. The waveform of the SPI pins is shown in [Figure 5-22](#).

The CMD and ADDR lengths can be configured in register SPI_CFG3 and are not counted in packet_length in register SPI_CFG1. If the CMD length is 0, the ADDR length is forced to be 0, too. In this case, only data are transferred.

5.6.4.1.3 SPI Dual Mode and Half Duplex Mode Transaction

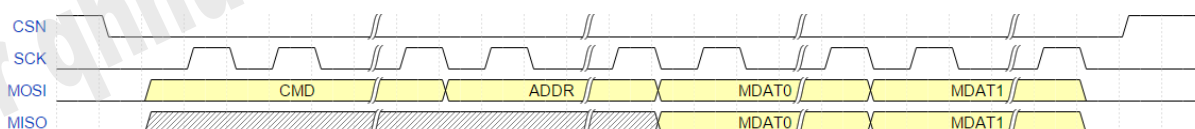


Figure 5-23. SPI dual mode and half duplex mode Tx transaction format

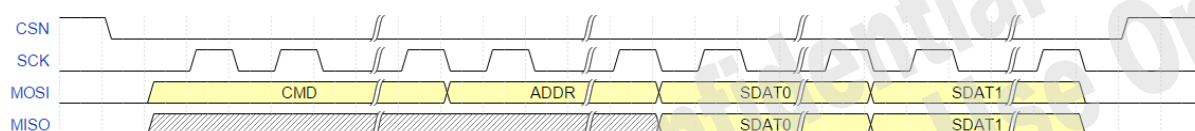


Figure 5-24. SPI dual mode and half duplex mode Rx transaction format

If register SPI_CFG3[1:0] is set to 2'b01 and register SPI_CFG3[3] is set to 1'b1, SPI works in dual mode and half duplex mode.

If register SPI_CFG3[2] is set to 1'b0, the SPI master transfers data to the slave while no data is transferred from the slave to the master. The waveform of the SPI pins is shown in [Figure 5-23](#).

If register SPI_CFG3[2] is set to 1'b1, the SPI master receives data from the slave while no data is transferred from the master to the slave. The waveform of the SPI pins is shown in [Figure 5-24](#).

5.6.4.1.4 SPI Dual Mode and Full Duplex Mode Transaction

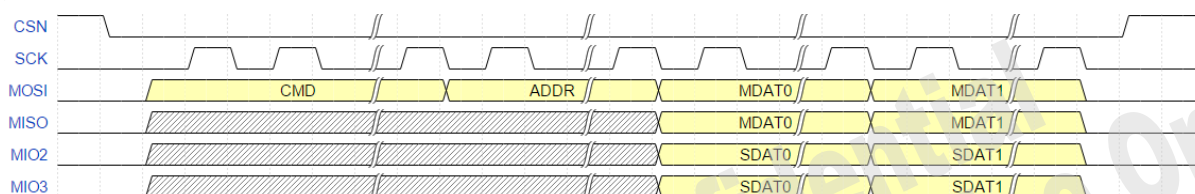


Figure 5-25. SPI dual mode and full duplex mode transaction format

If register SPI_CFG3[1:0] is set to 2'b01 and register SPI_CFG3[3] is set to 1'b0, the SPI works in dual mode and full duplex mode. The waveform of the SPI pins is shown in [Figure 5-25](#). SPI need two more pins if it works in dual mode and full duplex mode.

5.6.4.1.5 SPI Quad Mode and Half Duplex Mode Transaction

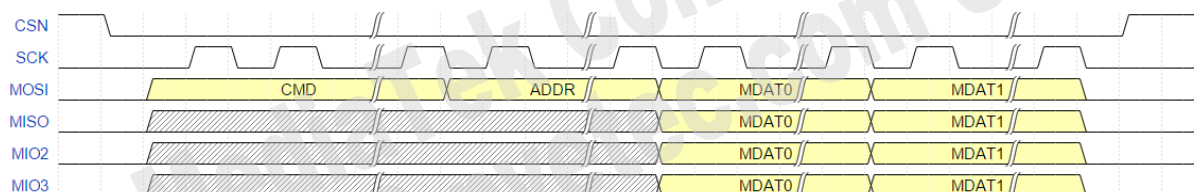


Figure 5-26. SPI quad mode and half duplex mode Tx transaction format

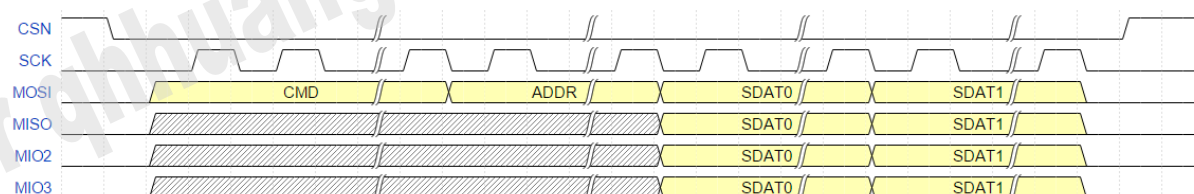


Figure 5-27. SPI quad mode and half duplex mode Rx transaction format

If register SPI_CFG3[1:0] is set to 2'b10 and register SPI_CFG3[3] is set to 1'b1, the SPI works in quad mode and half duplex mode.

If register SPI_CFG3[2] is set to 1'b0, the SPI master transfers data to the slave while no data is transferred from the slave to the master. The waveform if the SPI pins is shown in [Figure 5-26](#).

If register SPI_CFG3[2] is set to 1'b1, the SPI master receives data from the slave while no data is transferred from the master to the slave. The waveform of the SPI pins is shown in [Figure 5-27](#).

5.6.4.1.6 3-wire SPI Transaction Format

The 3-wire SPI master can support half duplex output/input mode.

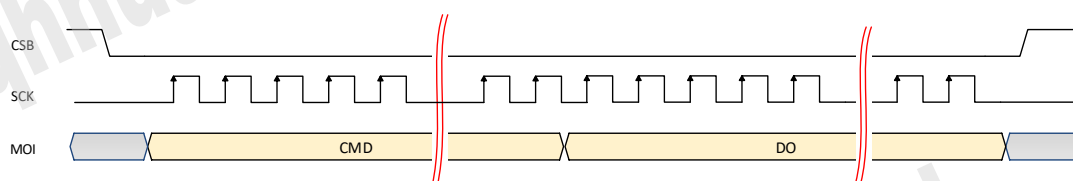


Figure 5-28. SPI half duplex output mode transaction format

In half duplex output mode, the waveform on SPI pins is shown in [Figure 5-28](#). The CMD is the command transferred from SPI master to SPI slave. The length of the CMD is 1-byte. DO is the data transferred from SPI master to SPI slave. The length of the DO can be both 1-byte and multi-bits.

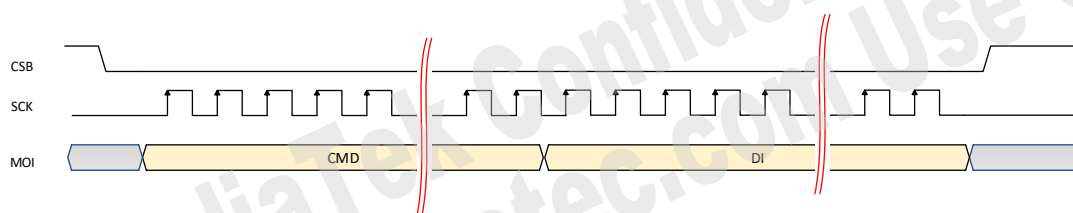


Figure 5-29. SPI half duplex input mode transaction format

In half duplex input mode, the waveform on SPI pins is shown in [Figure 5-29](#). The CMD is the command transferred from SPI master to SPI slave. The length of the CMD can be 1-byte. DI is the data transferred from SPI slave to SPI master. The length of the DI can be both 1-byte and multi-bits.

5.6.4.2 SPI Master Sampling Time Adjustment

When the SPI is operating at a high rate, the circuit delay and the EVB delay need to be taken into consideration. [Figure 5-30](#) gives a general view of how circuit delay and EVB delay influence SPI sampling result.

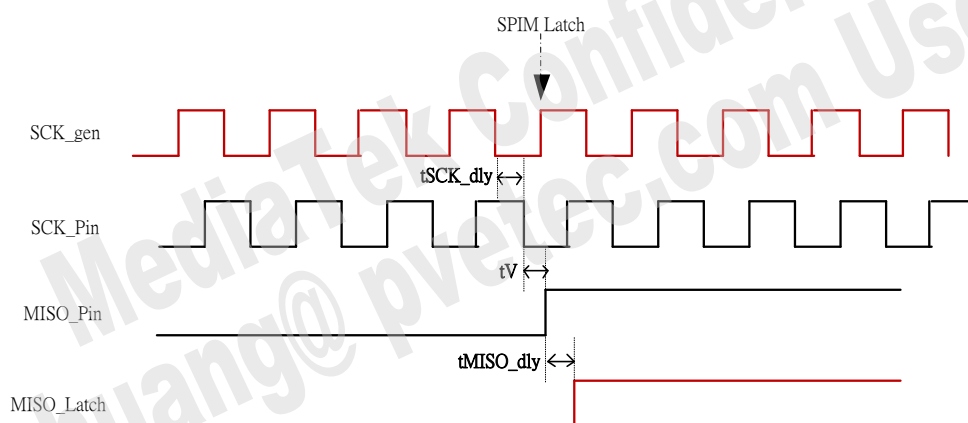


Figure 5-30. SPI master sampling time

The SPI master generates a SCK_gen inside the SoC. Due to circuit delay, it takes SCK_gen waveform t_{SCK_dly} time to arrive at SCK pin. Then, the SPI slave receives the SCK_pin waveform, and puts data on the MISO_pin after t_V time. At last, MISO data costs t_{MISO_dly} time to reach the SPI master for sampling. If the total delay – $t_{SCK_dly} + t_V + t_{MISO_dly} + t_{EVB_dly}$ exceeds half the SCK period, the SPI master would sample the wrong MISO data. In this case, GET_TICK_DLY (SPIn Base address+0x0018) [25:22] should be set to postpone the SPI sampling time.

The following formula shows how the appropriate GET_TICK_DLY is calculated:

$$GET_TICK_DLY * T_{clk} - \frac{1}{2} T_{SCK} < T_{total_dly} < GET_TICK_DLY * T_{clk} + \frac{1}{2} T_{SCK}$$

Note: T_{clk} is the module clock period. T_{SCK} is the SCK period. $T_{total_dly} = t_{SCK_dly} + t_V + t_{MISO_dly} + t_{EVB_dly}$.

5.6.4.3 SPI Master Bus Busy Status and Transaction Complete Interrupt

The SPI master provides the busy status of the bus: BUSY (SPIn Base address+0x0020) [0].

When BUSY (SPIn Base address+0x0020) [0] = 0, an SPI transaction is presently under way. The software must NOT try to start a new SPI transaction.

When BUSY (SPIn Base address+0x0020) [0] = 1, no SPI transaction is ongoing. The software may start a new SPI transaction and it could confirm the SPI bus status by polling the bit of spi_busy beforehand.

When a transaction is completed and interrupt FINISH_IE (SPIn Base address+0x0018) [16] is enabled, the SPI master controller sets FINISH (SPIn Base address+0x001C) [0] and asserts an SPI interrupt to notify the software that the SPIM transaction is completed.

If both PAUSE mode PAUSE_EN (SPIn Base address+0x0018) [4] and PAUSE interrupt PAUSE_IE (SPIn Base address+0x0018) [17] are enabled, the SPI controller sets PAUSE (SPIn Base address+0x001C) [1] and asserts an SPI interrupt to notify the software that the SPIM transaction is completed. At this time, the SPI controller is in PAUSE_IDLE state, and is ready to receive the resume command by RESUME (SPIn Base address+0x0018) [1].

5.6.5 Register Definition

See chapter 3.6 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.6.6 Top Programming Outline

5.6.6.1 SPI DMA Mode Guide

Table 5-14. SPI master DMA mode guide

Step	Sequence	REG_Name	REG_Value	Address
1	Basic configuration	SPI_CFG0 SPI_CFG1 SPI_CFG3	USER_DEFINE	SPIn Base Addr+0x00[31:0] SPIn Base Addr+0x04[31:0] SPIn Base Addr+0x40[31:0]
2	Configure DMA address (which supports max. 36-bit address).	SPI_TX_SRC SPI_TX_EXT_ADDR SPI_RX_DST SPI_RX_EXT_ADDR	USER_DEFINE	SPIn Base Addr+0x08[31:0] SPIn Base Addr+0x2C[3:0] SPIn Base Addr+0x0C[31:0] SPIn Base Addr+0x30[3:0]
3	Enable DMA mode.	SPI_CMD	2'b11	SPIn Base Addr+0x18[11:10]
4	Set interrupt enabled.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[17:16]
5	Configure SPI transaction format.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[9:3] SPIn Base Addr+0x18[15:12] SPIn Base Addr+0x18[24:18]
6	Trigger DMA.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[1:0]
7	CPU to wait for interrupt.	-	-	-
8	Clear interrupt flag.	Read SPI_IRQ[0] or when use pause mode, read SPI_IRQ[1] to clear interrupt.	USER_DEFINE	SPIn Base Addr+0x1C[1:0]

Step	Sequence	REG_Name	REG_Value	Address
9	Get the data received from the buffer.	Read data starting from "destination address".	-	-

5.6.6.2 SPI FIFO Mode Guide

Table 5-15. SPI master FIFO mode guide

Step	Sequence	REG_Name	REG_Value	Address
1	Basic configuration	SPI_CFG0 SPI_CFG1 SPI_CFG3	USER_DEFINE	SPIn Base Addr+0x00[31:0] SPIn Base Addr+0x04[31:0] SPIn Base Addr+0x40[31:0]
2	Write data into Tx FIFO.	Write to SPI_TX_DATA	USER_DEFINE	SPIn Base Addr+0x10[31:0]
3	Set interrupt enabled.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[17:16]
3	Configure SPI transaction format.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[9:3] SPIn Base Addr+0x18[15:12] SPIn Base Addr+0x18[24:18]
4	Trigger FIFO.	SPI_CMD	USER_DEFINE	SPIn Base Addr+0x18[1:0]
5	CPU to wait for interrupt	-	-	-
6	Clear interrupt flag.	Read SPI_IRQ[0] or when use pause mode, read SPI_IRQ[1] to clear interrupt.	USER_DEFINE	SPIn Base Addr+0x1C[1:0]
7	Get the data received from the Rx FIFO.	Read from SPI_RX_DATA.	-	SPIn Base Addr+0x14[31:0]

5.6.7 AC Timing

5.6.7.1 SPI Master AC Timing

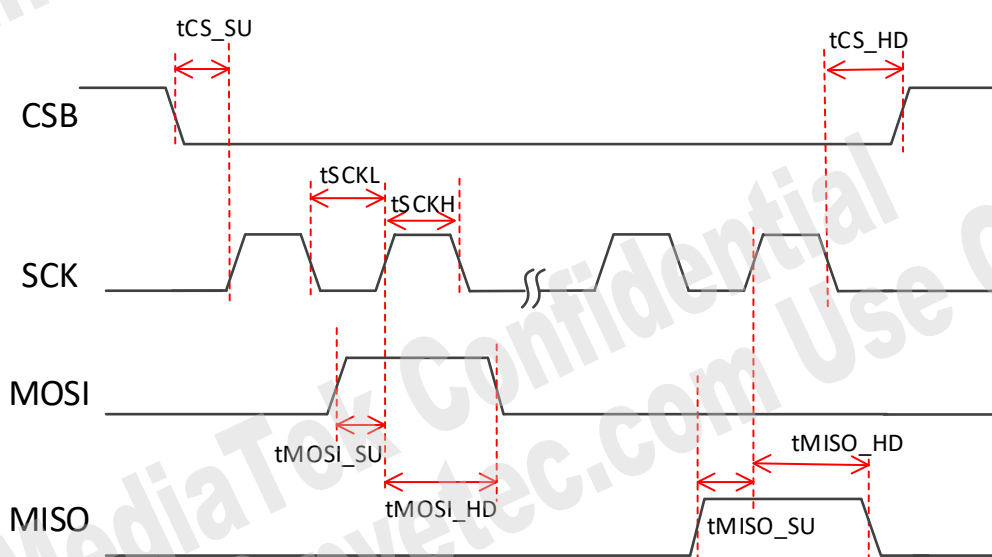


Figure 5-31. SPI master timing diagrams

Table 5-16. SPI master electrical specifications

Symbol	Description	Performance			Unit	Note
		Min.	Typ.	Max.		
fSCK	SPI master SCK clock frequency	-	-	52	MHz	
tMOSI_SU	MOSI to SCK rising setup time	6.6	-	-	ns	Tsck/2-Tskew-Tmargin
tMOSI_HD	SCK rising to MOSI hold time	6.6	-	-	ns	Tsck/2-Tskew-Tmargin
tSCKL	SCK low pulse	7.2	-	-	ns	Tsck/2*0.75
tSCKH	SCK high pulse	7.2	-	-	ns	Tsck/2*0.75
tCSB_SU ¹	CSB falling to SCK rising setup time	1.8	-	-	ns	Tbclk-Tskew-Tmargin
tCSB_HD ¹	SCK falling to CSB rising hold time	1.8	-	-	ns	Tbclk-Tskew-Tmargin
tMISO_SU ²	MISO to SCK rising setup time requirement	0	-	-	ns	
tMISO_HD ³	SCK rising to MISO hold time requirement	0	-	-	ns	

Notes:

1. In CS GPIO mode, the SPI_CS is handled by software. The software should pull down SPI_CS pin before the SPI starts transferring and pull up SPI_CS pin when the SPI completes the transaction. Based on the sequence above, the minimum specification of tCSB_SU and tCSB_HD time can be satisfied.
2. To achieve the min. value of tMISO_SU, the internal sample clock delay of the SPI master should be adjusted.
3. The MISO data valid time should be one cycle of fSCK.
4. For dual mode or quad mode, all the output data pins can refer to the MOSI timing parameters, and all the input data pins can refer to the MISO timing parameters.

5.7 MMC and SD Controller (MSDC)

5.7.1 Introduction

The MMC and SD controller (MSDC) fully supports the functions listed below. Register accessing is through AHB.

- SD memory card specification version 3.0
- SDIO card specification version 3.0
- eMMC 4.5

5.7.2 Features

There are two ports of MSDC, MSDC1 and MSDC2. Detailed features of each MSDC are as follows:

MSDC1

- 32-bit access on AHB for control registers
- Built-in CRC circuit
- Supports basic DMA mode and descriptor mode
- Interrupt capabilities
- Supports bus speed mode:
 - Default speed mode
 - High-speed mode
 - SDR12 mode
 - SDR25 mode
 - SDR50 mode
 - SDR104 mode
 - DDR50 mode
- Supports 1-bit/4-bit SD bus width

MSDC2

- 32-bit access on AHB for control registers
- Built-in CRC circuit
- Supports basic DMA mode and descriptor mode
- Interrupt capabilities
- Supports bus speed mode:
 - Default speed mode
 - High-speed mode
 - SDR12 mode
 - SDR25 mode
 - SDR50 mode
 - SDR104 mode
 - DDR50 mode
- Supports 1-bit/4-bit SD bus width

5.7.3 Block Diagram

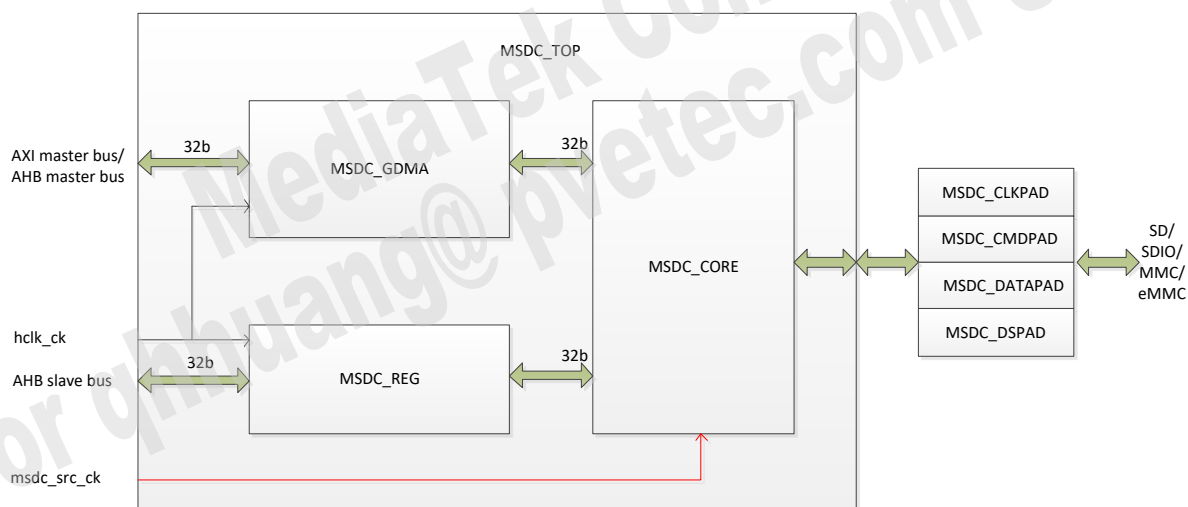


Figure 5-32. Block diagram of MSDC

Table 5-17. MSDC functions and address

MSDC list	Base address	Feature
MSDC1	0x11240000	SD3.0/SDIO3.0/eMMC4.5
MSDC2	0x11242000	SDIO3.0

MSDC is mainly composed of three parts: MSDC_GDMA, MSDC_CORE and MSDC_REG.

- MSDC_REG: The register to configure MSDC
- MSDC_CORE: The main controller of MSDC, implementing the transfer between the host and the device
- MSDC_GDMA: The DMA engine, transferring data between MSDC and memory

5.7.4 Register Definition

See chapter 3.7 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.7.5 Programming Guide

To start MSDC function, program the settings in the following manner.

5.7.5.1 Recommended MSDC Command Sequence

For SD command without response

- Check whether SDC_STA.SDCBUSY is 0 before starting to issue this command.

- When this command is done, check the MSDC_INT.SD_CMDRDY, MSDC_INT.SD_CMDTO, MSDC_INT.SD_RESP_CRCERR bits for the status.

For SD command with response

- Check whether SDC_STA.SDCBUSY is 0 before starting to issue this command
- When this command is done, check the MSDC_INT.SD_CMDRDY, MSDC_INT.SD_CMDTO, MSDC_INT.SD_RESP_CRCERR bits for the status.
- The response can be found in SDC_RESP0 ~ SDC_RESP3.

For SD command with data read/write transfer

- Check whether SDC_STA.SDCBUSY is 0 before starting to issue this command.
- When this command is done, check the MSDC_INT.SD_CMDRDY/SD_CMDTO/SD_RESP_CRCERR bits for the command phase status.
- The response can be found in SDC_RESP0 ~ SDC_RESP3.
- Enable DMA if needed (DMA_CTRL register should be programmed).
- The DMA_CTRL register should be programmed after the command register is programmed.
- The PIO mode can be used to move data (MSDC_FIFOCs, MSDC_RXDAT, MSDC_TXDAT registers).
- The PIO mode and DMA mode must not be switched in the same transfer; otherwise, the result will be unexpected.
- Check MSDC_INT.SD_XFER_COMPLETE/DMA_DONE/SD_DATTO/SD_DATA_CRCERR for the data phase status. Detailed timing is shown in the timing diagrams in the following sections.

For SD which software can always check SDC_STA.SDCBUSY before issuing a new command

- SDC_STA_CMDDBUSY is left to be referenced.

5.7.5.2 Single Block Read/Write in PIO Mode Flow Chart

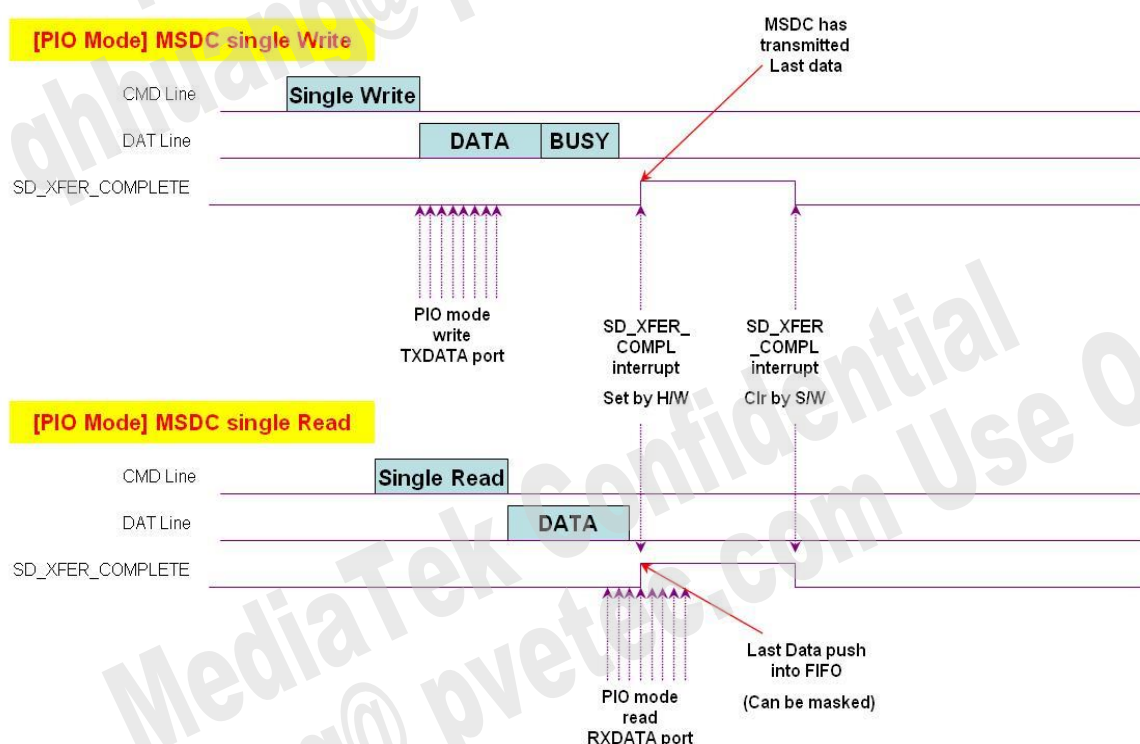


Figure 5-33. PIO single block timing

5.7.5.3 Multiple Block Read/Write in PIO Mode Flow Chart

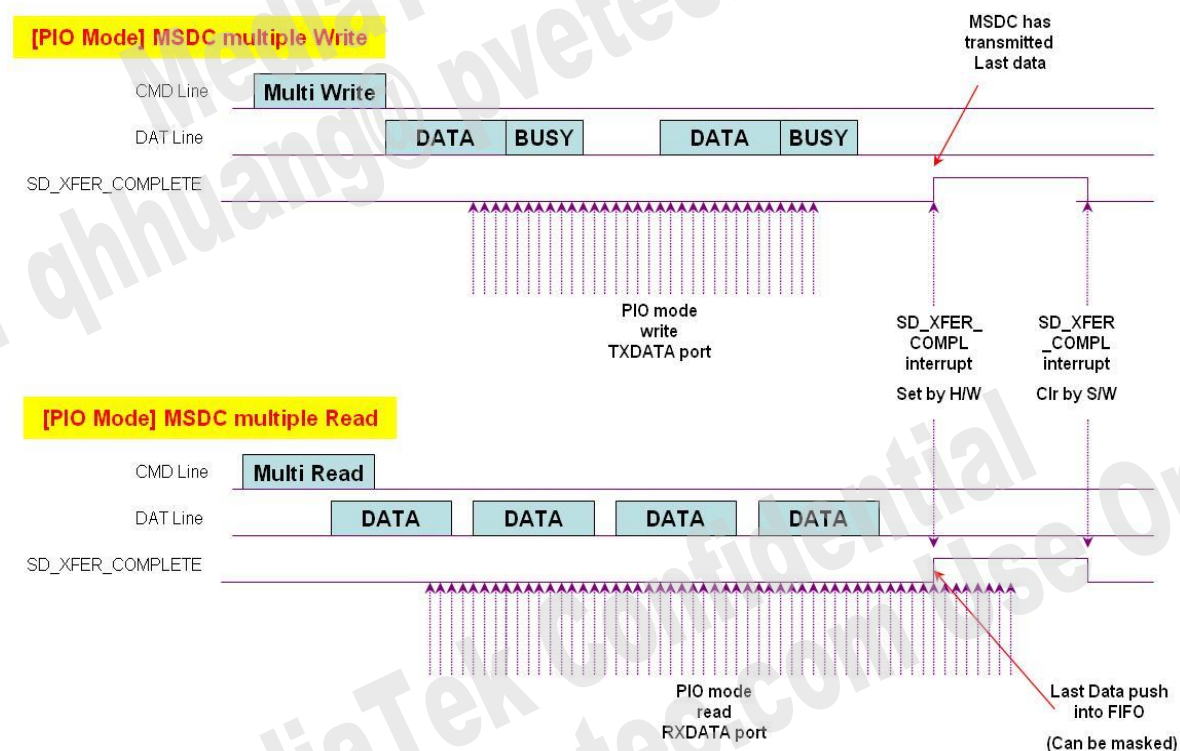


Figure 5-34. PIO multiple block timing

5.7.5.4 MSDC_INT.SD_XFER_COMPLETE/DMA_DONE Timing Diagram

DMA read

When MSDC_CFG.PIO_MODE (MSDC0 base_address+0x0000) [3] is not set, MSDC works in the DMA mode. MSDC receives the data from the device and writes it to the target SDRAM address through the MSDC_GDMA control. Software needs to configure the DMA_SA (MSDC0 base_address+0x0090) [31:0] register as the start address in the SDRAM. After the transfer is finished, two interrupts will be generated, and software needs to clear the interrupt bit after receiving it.

DMA write

When MSDC_CFG.PIO_MODE (MSDC0 base_address+0x0000) [3] is not set, MSDC works in the DMA mode. MSDC gets the data from the SDRAM through the MSDC GDMA control and writes it to device. Software needs to configure the DMA_SA register as the start address in SDRAM. After the transfer is finished, two interrupts will be generated, and software needs to clear the interrupt bit after receiving it.

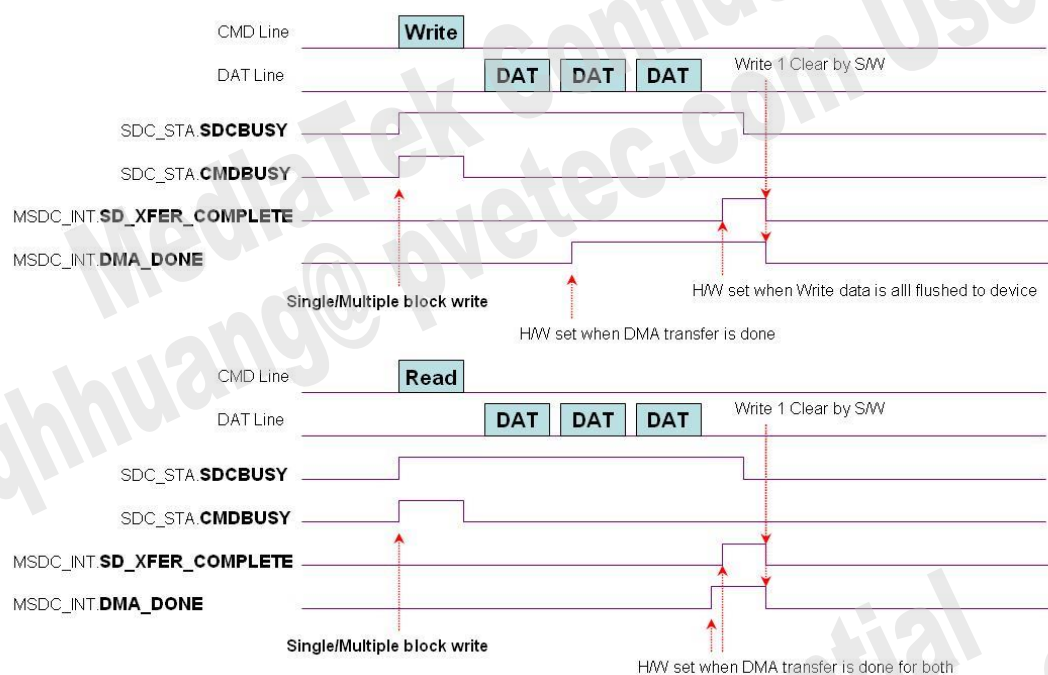


Figure 5-35. DMA single/multiple block timing

5.7.5.5 MSDC Tuning Support

The host controller provides a tuning algorithm to guarantee that the host can sample the data, command response and CRC status stably from the device. Both command and data have pad_delay. The tuning flow is as follows.

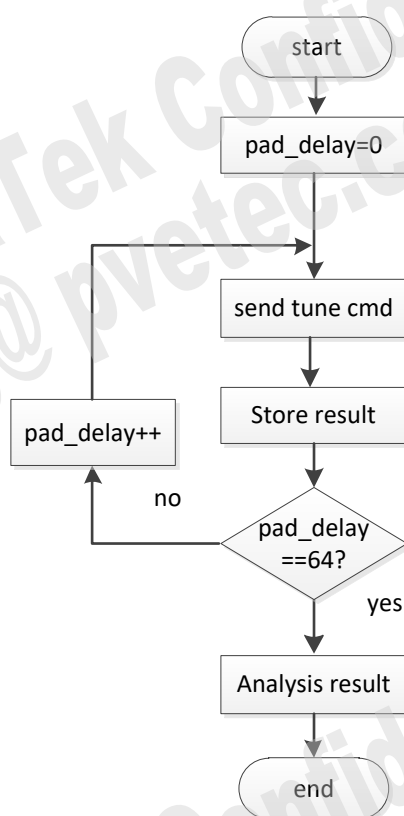


Figure 5-36. Tuning flow

5.8 Inter-IC (I2C)/Improved Inter-IC (I3C) Controller

5.8.1 Introduction

The inter-IC (I2C)/improved inter-IC (I3C) controller is a two-wire serial interface. The two signals are SCL and SDA. Both SCL and SDA are bi-directional data signals that can be driven by either the master or the slave. This generic controller supports the master role and conforms to the I2C/I3C specification.

5.8.2 Features

The features of I2C/I3C controller are as follows:

- I2C compliant master mode operation
- Adjustable clock speed for LS/FS mode operation
- Supports 7-bit/10-bit addressing
- Supports high-speed mode
- Supports slave clock extension
- Manual transfer mode
- Multi-write per transfer
- Multi-read per transfer
- Multi-transfer per transaction
- Combined format transfer with length change capability
- Active drive/wired-and I/O configuration
- Start multiple transfer repeatedly
- Single data rate transfer
- Dynamic address assignment
- Common command code
- Secure slave address

5.8.2.1 Manual Transfer Mode

The controller supports the manual transfer mode. When the manual mode is selected, in addition to the slave address register, the controller has a built-in 8-byte deep FIFO for I2C and a 16-byte deep FIFO for I3C, which allows the MCU to prepare bytes of data for a write transfer, or to read bytes of data for a read transfer.

5.8.2.2 Transfer Format Support

This controller is designed to be as generic as possible in order to support a wide range of devices that might utilize different combinations of transfer formats. [Figure 5-37](#) to [Figure 5-44](#) illustrate the transfer format types supported through different software configurations.

Terminology

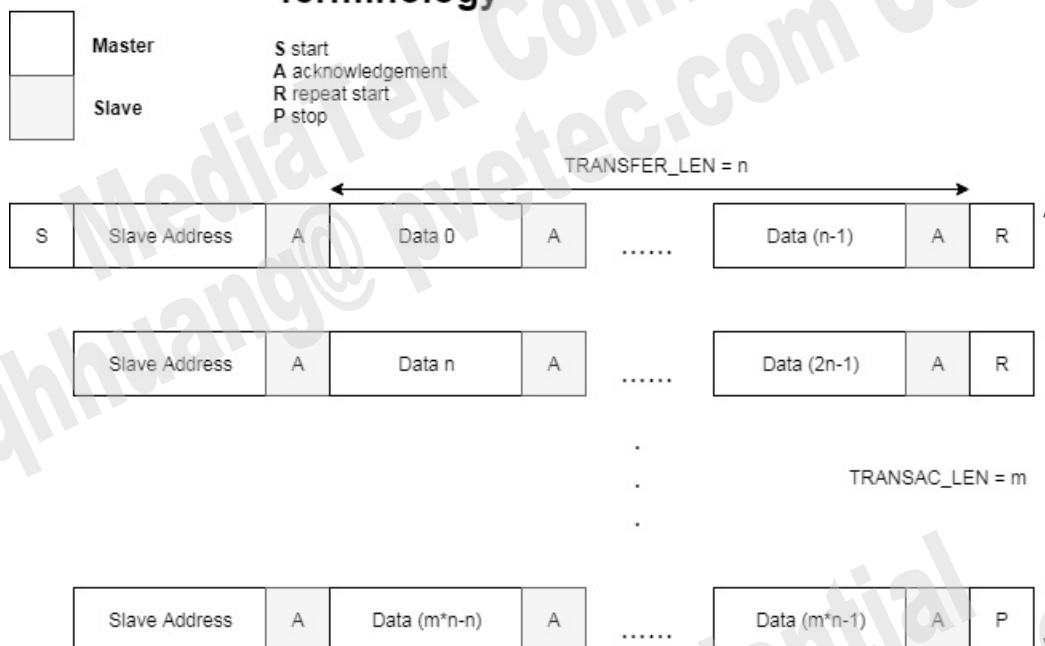


Figure 5-37. I2C transaction (write)

Terminology

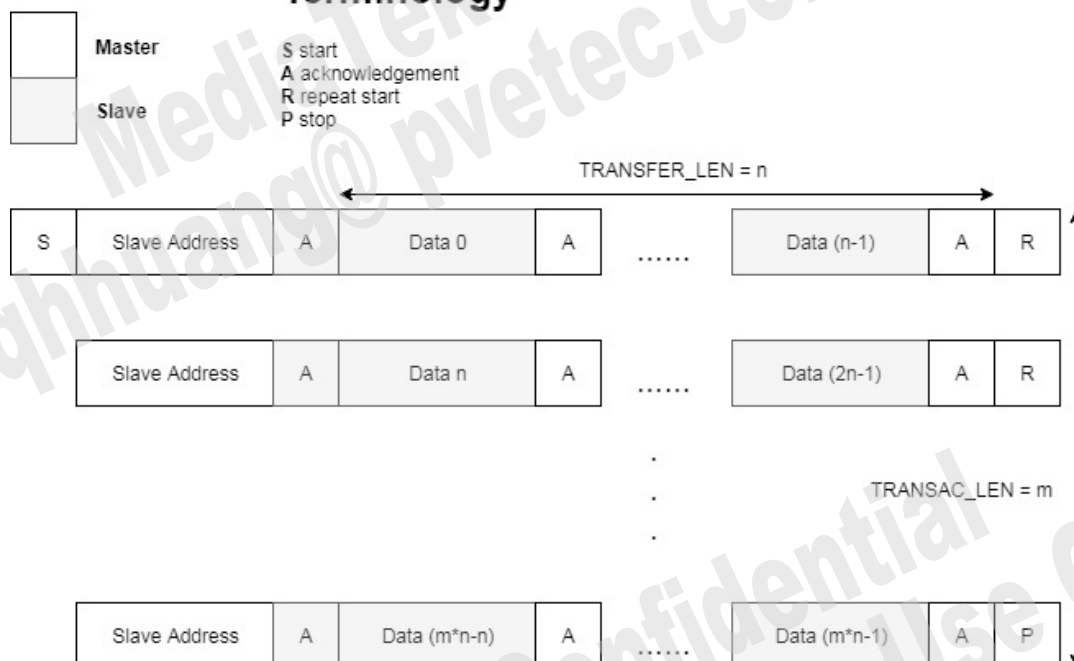


Figure 5-38. I2C transaction (read)

Terminology

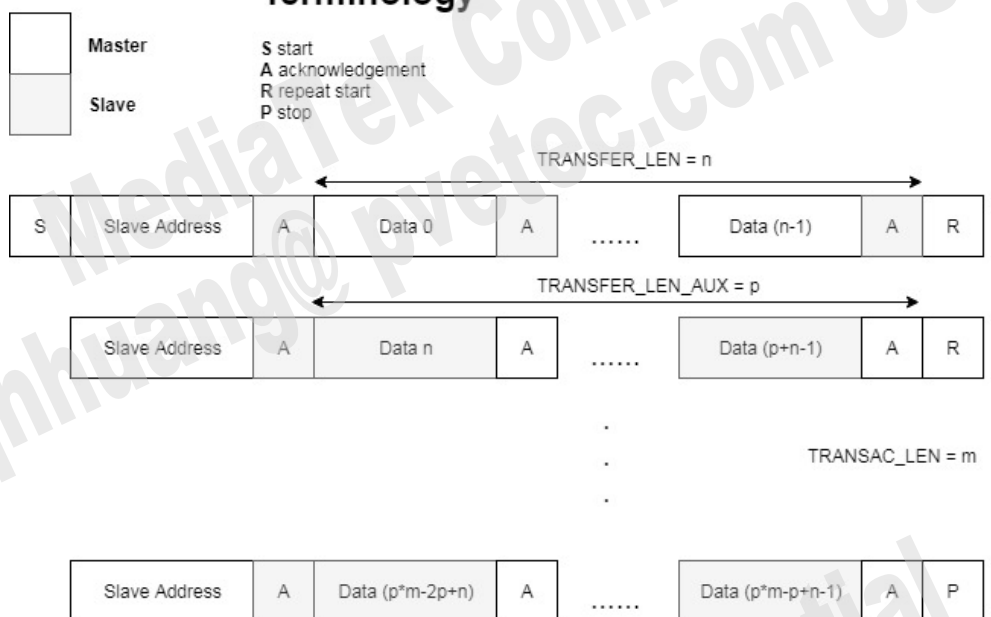


Figure 5-39. I2C transaction (read after write)

Terminology

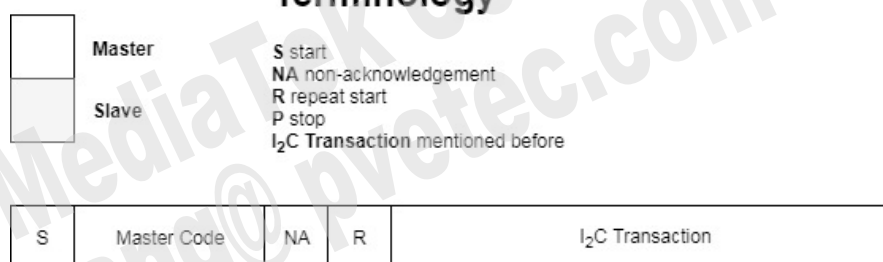


Figure 5-40. I2C high-speed mode

Terminology



S start
A acknowledgement
R repeat start
P stop
T transfer bit

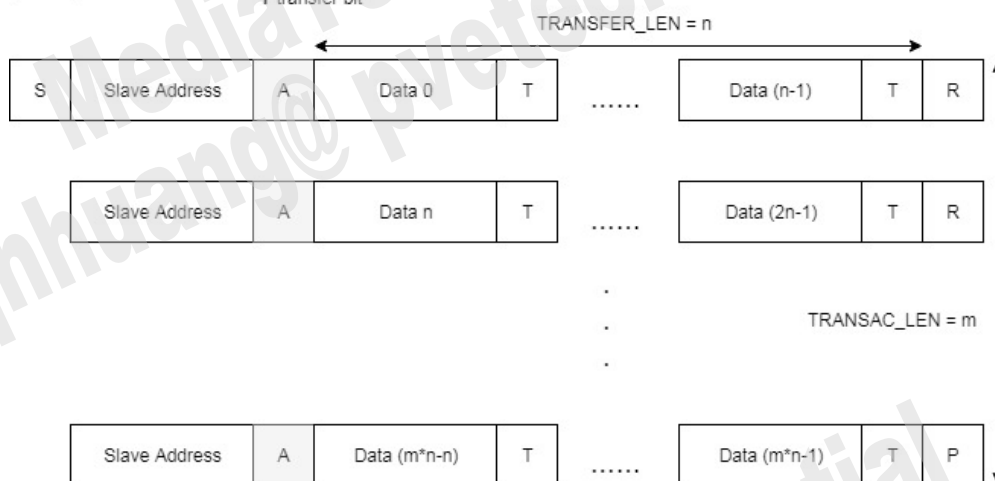


Figure 5-41. I3C transaction (write)

Terminology



S start
A acknowledgement
R repeat start
P stop
T transfer bit

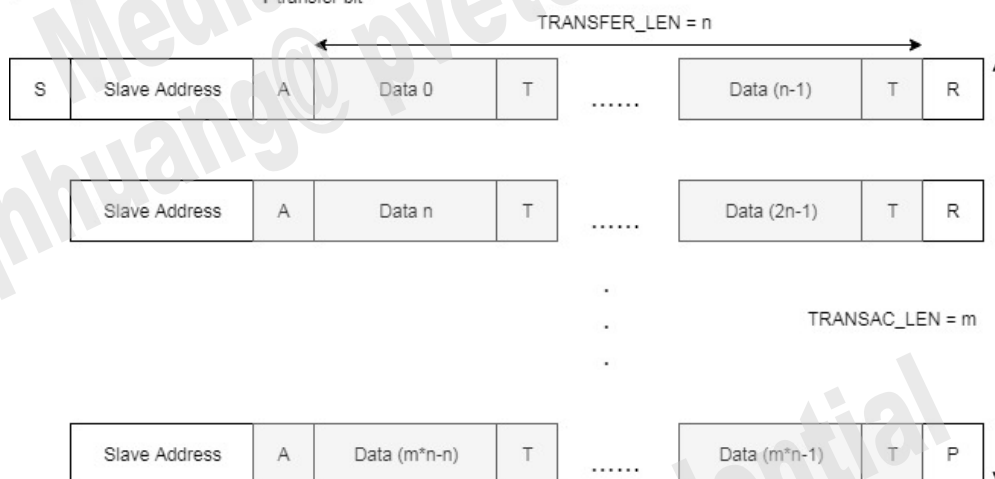


Figure 5-42. I3C transaction (read)

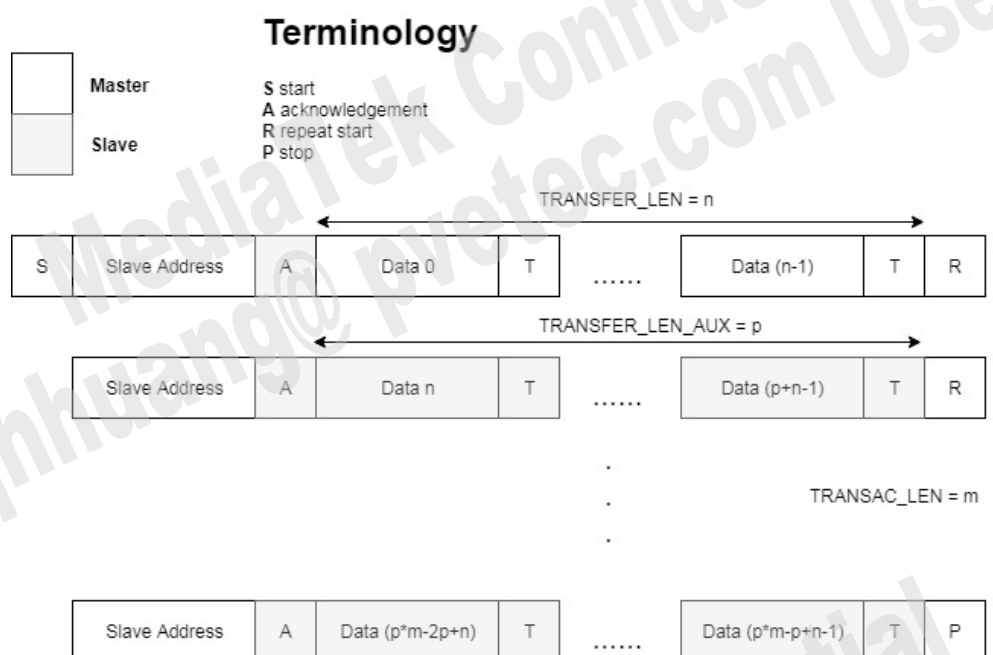


Figure 5-43. I3C transaction (read after write)

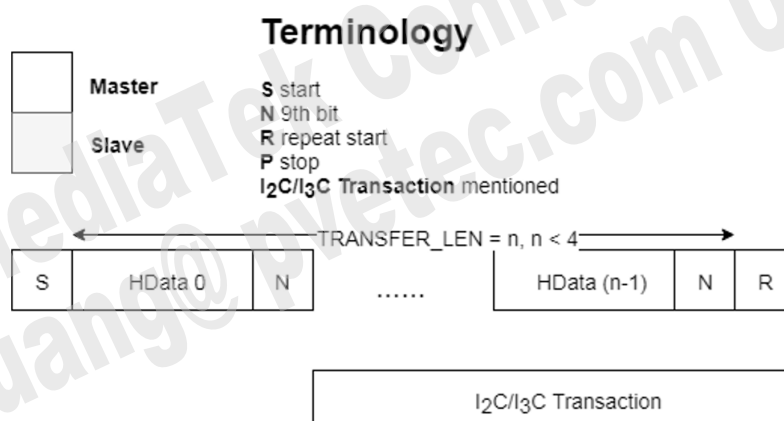


Figure 5-44. I3C transaction (with 7E or CCC)

5.8.2.3 Secure Design

The MediaTek I2C supports secure I2C/I3C slave and non-secure I2C/I3C slave which share an I2C bus.

APB secured channel

As mentioned above, the MediaTek I2C allows many drivers to share an I2C bus. These drivers are parking at different APB channels which are split by memory map from bit8 to bit11, which means every 4 KB memory map for I2C is split into 16 sets of 256B. These channels are named from 0 to 15. In the default design, channel 15 is always in the secure domain which serves the other 256B security-related settings. For example, users can configure channel 15 and make channel 1 a secure channel.

Secured slave address

The most important thing for secure design is the secure slave address. The MediaTek I2C/I3C controller supports four sets of I2C/I3C slave addresses locked in the secure domain. For example, there is a device used for front camera with face ID, and the slave addresses are 0x3 and 0x5; if 0x5 is locked into the secure environment, it is no possible to fetch data from slave address 0x5 which serves the face ID.

Signal secured

For the debugging purpose, the MediaTek I2C/I3C controller supports SCL/SDA snooping, but this feature works if and only if the driver gets the bus authority. For example, if there are two drivers, AP and the camera control unit (CCU) sharing an I2C bus, when AP is transferring data, both SCL/SDA will always be high for the CCU.

Secured common command code

The MediaTek I3C supports three ranges and two exact secured common command codes (CCC) which help the mixed I2C/I3C bus with the secure slave and the non-secure one set by a central driver. For example, when there is dynamic address assignment (DAA) for face ID, which is accessed firstly by the AP driver, the dynamic address (DA) will not be modified by hikers.

5.8.2.4 Arbitration Design

The MediaTek I2C supports secure slave and non-secure slave that shares an I2C bus. This section briefly explains how it works.

Round robin

From APB channel 1 to channel n, the round robin arbitration is adopted. Channel 0 is always executed. For example, in the case of channel 3, the design copies all settings including FIFO data to channel 0 first. Then it resets (finite state machine) FSM and starts the setting. In arbitration, channel 0 is locked and cannot be configured by software.

Interrupt

Every channel has its own MCU_INTR mentioned in the register map. This control register marks the MCU to which the channel issues an interrupt. For example, if two drivers use channel 1 and channel 2, and MCU_INTR are 0x1 and 0x2 respectively, the interrupt for channel 1 will send an interrupt to MCU 1 while channel 2's will be sent to MCU 2. There is a conflict: if MCU_INTR sets 0x1 for channel 1 and 0x3 for channel 2, the design cannot recognize that channel 2 issues an interrupt to the MCU 1 after channel 1 has issued an interrupt to MCU 1. To solve this conflict, the design in this case will hang the arbitration, and wait for software to clear the interrupt issued to the same MCU.

Error handling

If and only if the interrupt status is larger than 0x1, the arbitration will stop and open the authority for channel 0 until MCU interrupted handling error is done. For example, when MCU 1 at channel 2 gets an interrupt status 3, it can then configure channel 0 instead of channel 2 to solve the problem from the slave. When the handling is done, MCU 1 configures channel 0 START mentioned in the register map 0x2 to kick off the arbitration again.

Secure channel

During the arbitration, all information will be copied to channel 0 including the secure information. For example, MCU 1 at channel 2 is a secure channel. In MCU 1's turn, channel 0 and channel 2 are both secure channels.

5.8.3 Block Diagram

Dash line round rectangle, parameter-defined existing/non-existing virtual block
 Dash line rectangle, always existing virtual block
 Rounded rectangle, parameter-defined existing/non-existing block
 Rectangle, always existing block
 Dash line, parameter-defined connection

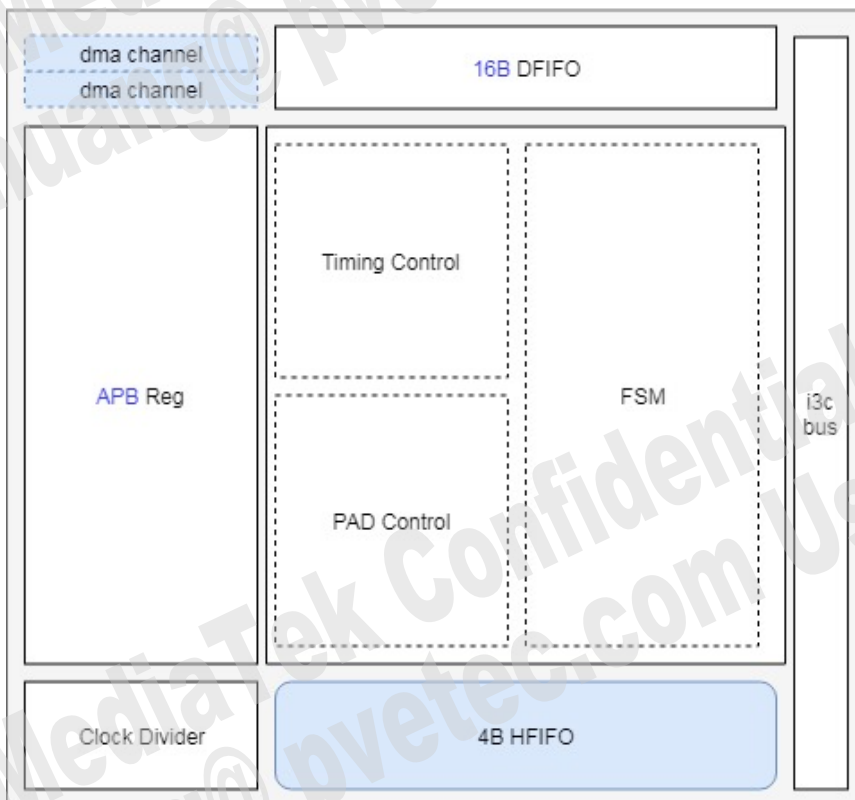


Figure 5-45. Block diagram of I2C/I3C

The MediaTek I2C/I3C is a pre-defined design. All features are well defined for applications. For example, the I2C/I3C now supports secure camera I2C slaves with multi-MCU drivers sharing the same I2C bus, which means the controller supports complex or secure data computing at Arm AP when the camera control unit (CCU) computes other data.

Hardware features discovery

The following content can be found in the register map:

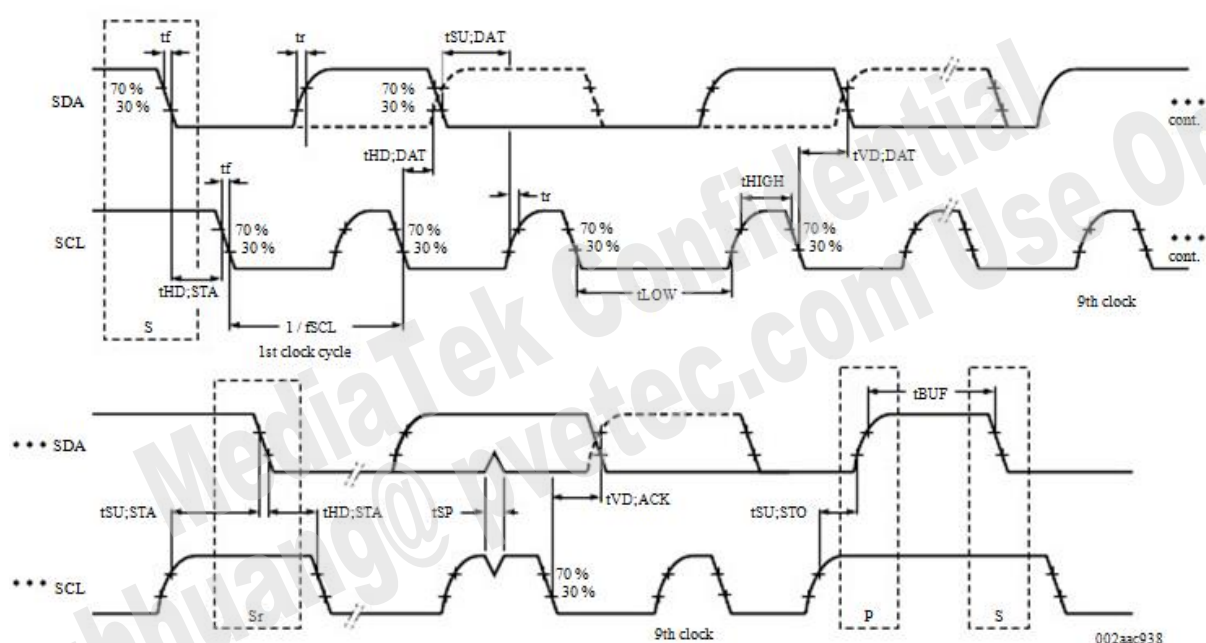
- I3C_EN: This design can be used to transfer I3C protocol with the supported mode mentioned before.
- APB_ID: I2C bus number
- FIFO_SET: The supported number of sets of 8-byte FIFO for I2C and 16-byte FIFO for I3C
- IRQ_SET: The supported number of interrupts for drivers
- DMA_FROM_CHANNEL: Starting channel of APDMA this I2C/I3C uses
- DMA_TO_CHANNEL: Ending channel of APDMA this I2C/I3C uses

For example, an I2C can be got with the following registers.

Field	Value
I3C_EN	1
APB_ID	5
FIFO_SET	4
IRQ_SET	3
DMA_FROM_CHANNEL	6
DMA_TO_CHANNEL	8

The example above indicates that the I3C (I3C_EN) serves I2C bus 5 (APB_ID) with four FIFOs (FIFO_SET) and three interrupts (IRQ_SET) and uses APDMA from channel 6 (DMA_FROM_CHANNEL) to 8 (DMA_TO_CHANNEL).

5.8.4 Timing Characteristics

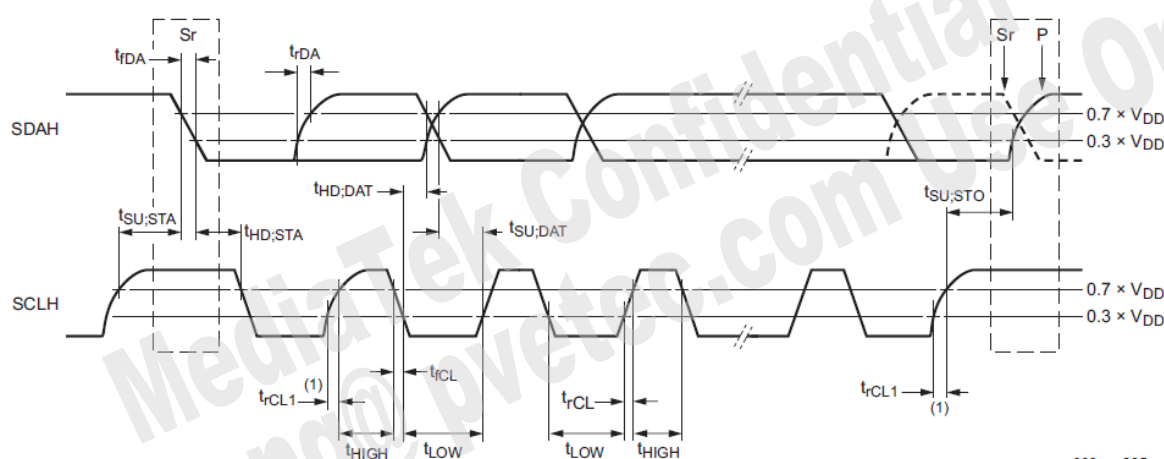


Symbol	Parameter	Standard mode		Fast mode		Fast mode plus		Unit	Note
		Min.	Max.	Min.	Max.	Min.	Max.		
fSCL	SCL clock frequency	0	100	0	400	0	1,000	kHz	-
tHD; STA	Hold time (repeated) START condition	4.0	-	0.6	-	0.26	-	us	-
tLOW	LOW period of the SCL clock	4.7	-	1.3	-	0.5	-	us	-
tHIGH	HIGH period of the SCL clock	4.0	-	0.6	-	0.26	-	us	-
tSU; STA	Set-up time for a repeated START condition	4.7	-	0.6	-	0.26	-	us	-
tHD; DAT	Data hold time	0	-	0	-	0	-	us	I2C-bus devices
tSU; DAT	Data set-up time	250	-	100	-	50	-	ns	-
tr	Rise time of both SDA and SCL signals	-	1000	20	300	-	120	ns	-

Symbol	Parameter	Standard mode		Fast mode		Fast mode plus		Unit	Note
		Min.	Max.	Min.	Max.	Min.	Max.		
t _f	Fall time of both SDA and SCL signals	-	300	20x (V _{DD} /5.5V)	300	20x (V _{DD} /5.5V)	120	ns	V _{DD} is I2C I/O voltage
t _{SU; STO}	Set-up time for STOP condition	4.0	-	0.6	-	0.26	-	ns	-
t _{VDD; DAT}	Data valid time	-	3.45	-	0.9	-	0.45	us	-
t _{VDD; ACK}	Data valid acknowledge time	-	3.45	-	0.9	-	0.45	us	-
C _b	Capacitive load of each bus line	-	400	-	400	-	550	pF	-

Notes:

- V_{DD} is 1.8 V; the minimum value of t_f in the spec: $20 \times (V_{DD}/5.5 V) = 6.5 \text{ ns}$.
- The maximum allowable bus capacitance may vary from this value depending on the actual operating voltage and frequency of the application.



002aag825

Figure 5-46. I2C AC timing diagram of HS mode

Table 5-18. I2C AC timing parameters for HS mode

Symbol	Parameter	C _b = 100 pF (max.)		C _b = 400 pF		Unit	Note
		Min.	Max.	Min.	Max.		
f _{SCL}	SCL clock frequency	0	3.4	0	1.7	MHz	-
t _{SU;STA}	Set-up time (repeated) START condition	160	-	160	-	ns	-
t _{HD;STA}	Hold time (repeated) START condition	160	-	160	-	ns	-
t _{LOW}	LOW period of the SCL clock	160	-	320	-	ns	-
t _{HIGH}	HIGH period of the SCL clock	60	-	120	-	ns	-
t _{HD;DAT}	Data hold time	0	70	0	150	ns	I2C-bus devices
t _{SU;DAT}	Data set-up time	10	-	10	-	ns	-

Symbol	Parameter	Cb = 100 pF (max.)		Cb = 400 pF		Unit	Note
		Min.	Max.	Min.	Max.		
tr	Rise time of SCLH signal	10	40	20	80	ns	-
tf	Fall time of SCLH signal	10	40	20	80	ns	
tSU;STO	Set-up time for STOP condition	160	-	160	-	ns	-

5.8.5 Register Definition

See chapter 3.8 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.9 Pulse-Width Modulators

5.9.1 Introduction

Three generic pulse width modulators (PWMs) are implemented to generate pulse sequences with programmable frequency and duration for LCD backlight, charging or other purposes. Before software enabling PWM, the pulse sequences must be prepared in the memory or registers. Then PWM should read the pulse sequences to generate random waveform to meet all kinds of applications (see [Figure 5-47](#)).

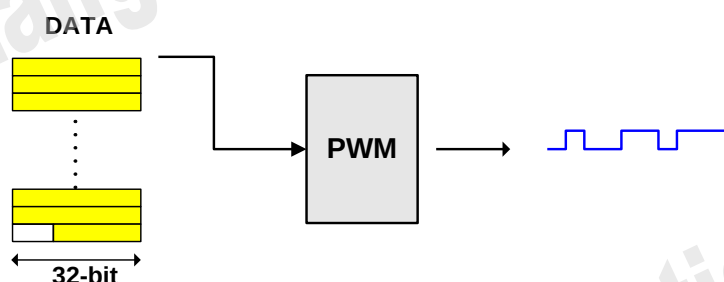


Figure 5-47. Generation procedure of PWM

5.9.2 Features

The features of PWM are as follows:

- Old mode
- FIFO mode
- Periodical memory and random mode
- 3DLCM mode
- Breathing Light mode

5.9.3 Block Diagram

[Figure 5-48](#) is the block diagram of the PWM.

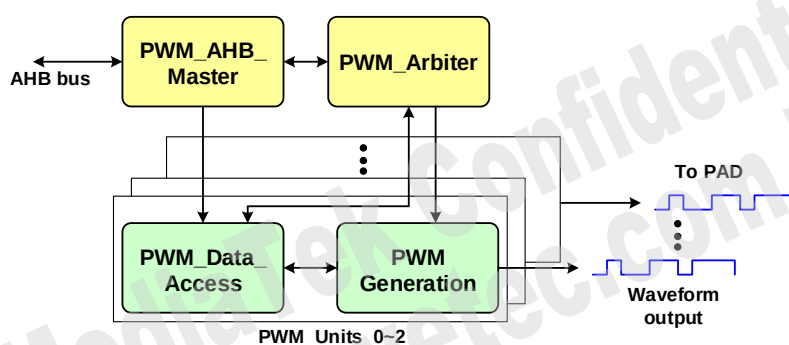


Figure 5-48. Block diagram of PWM

5.9.4 Theory of Operations

5.9.4.1 Old Mode

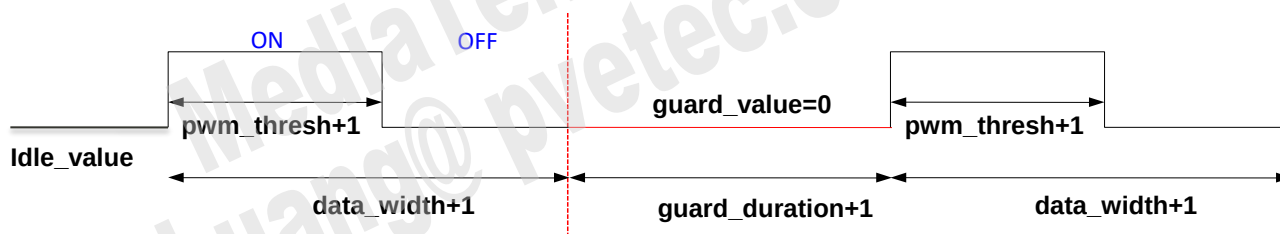


Figure 5-49. Old mode

In the old mode, the waveform generated by PWM is shown in [Figure 5-49](#). The frequency is determined by PWM_DATA_WIDTH (PWM base address + 0x002C)[15:0] and the duty cycle is determined by PWM_THRESH (PWM base address + 0x0030)[15:0].

$$\text{PWM Freq.} = \frac{\text{CLKSRC}}{\text{CLKDIV} * (\text{DATA_WIDTH} + 1)}$$

Without considering guard_duration (PWM base address + 0x000C)[15:0], guard_duration (PWM base address + 0x000C)[15:0] is the time interval between two complete waveforms. When PWM_WAVE_NUM (PWM base address + 0x0028)[15:0] = 0, it means that hardware infinitely outputs the waveform, and the waveform can only be terminated by disabling the PWM.

$$\text{Duty cycle} = \frac{\text{PWM_THRESH} + 1}{\text{DATA_WIDTH} + 1}$$

5.9.4.2 FIFO Mode

If the pulse sequence data is less than or equals to 64-bit, it can be directly set in PWM_SEND_DATA0 (PWM base address + 0x0020)[31:0], PWM_SEND_DATA1 (PWM base address + 0x0024)[31:0] and SRCSEL (PWM base address + 0x0000)[5]=0 to reduce memory bandwidth.

STOP_BITPOS (PWM base address + 0x0000)[14:9] is used to indicate the stop bit position in the total 64-bit data. For example, if STOP_BITPOS (PWM base address + 0x0000)[14:9] is 0, only PWM_SEND_DATA0 (PWM base address + 0x0020)[31:0] is generated.

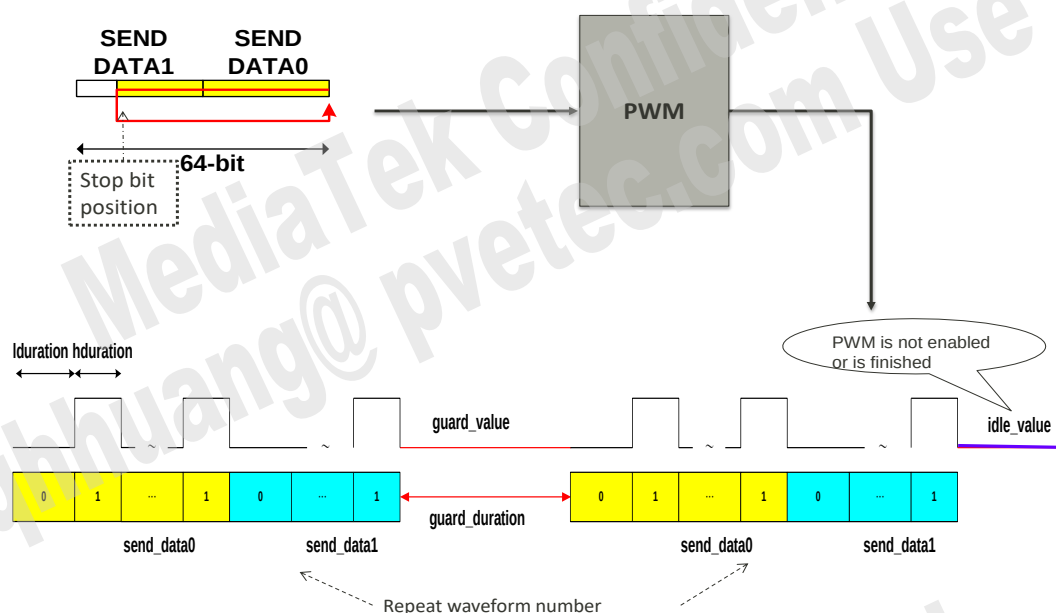


Figure 5-50. FIFO mode

5.9.4.3 Memory Mode

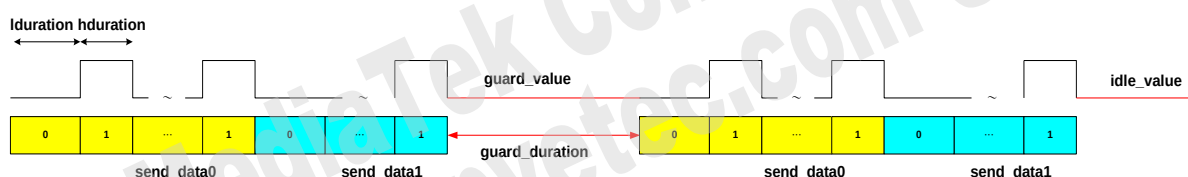


Figure 5-51. Memory mode

In the periodical mode, all pulse sequences are repeatedly generated by the number of PWM_WAVE_NUM (PWM base address + 0x0028)[15:0]. If PWM_WAVE_NUM (PWM base address + 0x0028)[15:0]=0, Hardware infinitely outputs the waveform, and the waveform generation can only be stopped by PWM_ENABLE (PWM global base address + 0x0000)[31:0].

If SRCSEL (PWM base address + 0x0000)[5]=1, PWM is in the memory mode. The pulse sequence data is put in memory with address set by PWM_BUF0_BASE_ADDR (PWM base address + 0x0010)[31:0] & PWM_BUF0_BASE_ADDR2 (PWM base address + 0x003C)[3:0], and the length is PWM_BUF0_SIZE (PWM base address + 0x0014)[15:0]. STOP_BITPOS (PWM base address + 0x0000)[14:9] is to indicate the stop bit position in the last 32-bit data.

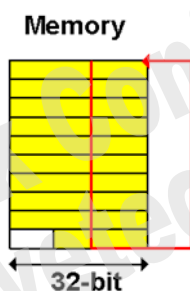


Figure 5-52. Memory mode and stop bit position

Note: Any kind of memory can be used by the PWM (usually SYSRAM) as long as software can read/write the address. Software issues a request, and PWM can locate an unused memory and get the address, so users need to write the address and length at PWM_BUF0_BASE_ADDR.

5.9.4.4 Random Mode

On the other hand, the pulse sequence is stored in dual memory buffers in the random mode. The format of pulse sequences stored in the memory is shown in [Figure 5-53](#). The valid bit is used to indicate that the data is ready in the respective memory buffer. The PWM generation should clear this bit after all data in that buffer are fetched. The memory buffers are set by address PWM_BUF0_BASE_ADDR (PWM base address + 0x0010)[31:0] and PWM_BUF0_SIZE (PWM base address + 0x0014)[15:0] for memory 0, and are set by address PWM_BUF1_BASE_ADDR (PWM base address + 0x0018)[31:0] and PWM_BUF1_SIZE (PWM base address + 0x001C)[15:0] for memory 1.

The program should prepare for the pulse sequence and set the valid bit to 1 in time before all data in other memory buffers are fetched. Otherwise, the hardware should issue the UNDERFLOW interrupt to inform that the pulse generation should be stopped since there is no valid data. If the UNDERFLOW interrupt is received, software needs to disable the PWM to set valid bit again and enable PWM to restart the pulse generation.

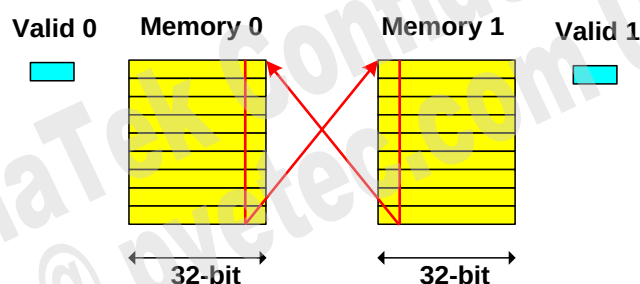


Figure 5-53. Random mode

Note: Any kind of memory can be used by the PWM (usually SYSRAM) as long as software can read / write the address. Software should issue a request, and PWM can locate an un-used memory and get the address, so users need to assign the address and length at PWM_BUF0_BASE_ADDR/PWM_BUF0_SIZE and PWM_BUF1_BASE_ADDR/PWM_BUF1_SIZE.

5.9.4.5 3D_LCM Mode

These PWM channels are interrelated. One PWM channel can be selected as the base channel. Other PWM channels can be selected as the auxiliary channels in the same phase or in the reverse phase. It can be used with old mode and FIFO mode.

For example:

PWM0 acts as the base channel.

PWM1 is selected as auxiliary channel, which is in the reverse phase compared with the base channel.

PWM2 is selected as auxiliary channel, and it is in the same phase compared with the base channel.

The result is shown in [Figure 5-54](#).

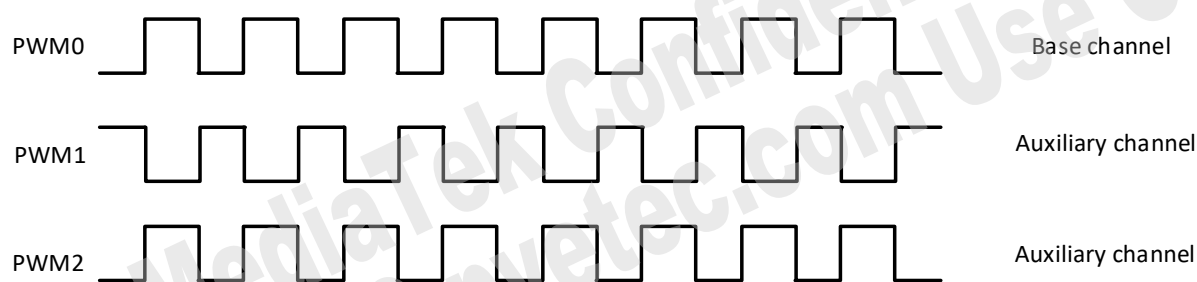


Figure 5-54. 3D_LCM mode

5.9.4.6 Breathing Light Mode

By changing the duty cycle of the PWM, the effective value of the output level is changed, which is reflected in the brightness of the LED. If the duty cycle can be changed from small to large and then small, the effect reflected on the LED is the effect of the breathing light.

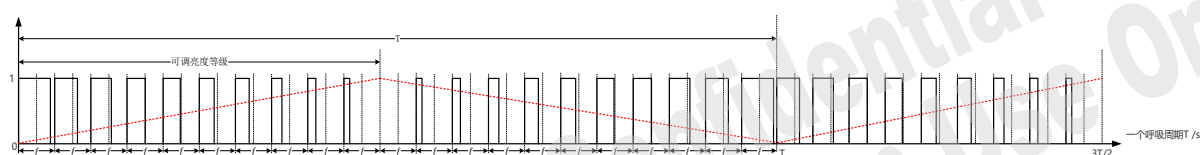


Figure 5-55. Breathing light mode

- Note: 1. The 100% and 0% shown in the figure refer to the duty cycle of the PWM waveform;
 2. f in the figure refers to the period of the PWM waveform;
 3. T in the figure refers to the breathing cycle of the breathing light from bright-dark-bright.

5.9.5 Programming Guide

5.9.5.1 Old Mode

Table 5-19. Old mode setting procedures

PWM setting sequence for old mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[15]	OLD_PWM_MODE	1'b1	
3	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[8]	GUARD_VALUE	USER_DEFINED	
4	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[7]	IDLE_VALUE	USER_DEFINED	
5	Set PWM_WAVE_NUM	W	PWM_BASE + 0x128+ PWM_NUM*0x100	[15:0]	PWM_WAVE_NUM	USER_DEFINED	If WAVE_NUM=0, the waveform generation should

PWM setting sequence for old mode							
							not stop until the PWM is disabled.
6	Set PWM_GDURATION	W	PWM_BASE + 0x10C + PWM_NUM*0x100	[15:0]	PWM_GDURATION	USER_DEFINED	
7	Set PWM_DATA_WIDTH	W	PWM_BASE + 0x12C + PWM_NUM*0x100	[12:0]	PWM_DATA_WIDTH	USER_DEFINED	
8	Set PWM_THRESH	W	PWM_BASE + 0x130 + PWM_NUM*0x100	[12:0]	PWM_THRESH	USER_DEFINED	
9	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b1	Enable PWM[N].

5.9.5.2 FIFO Mode

Table 5-20. FIFO mode setting procedures

PWM setting sequence for FIFO mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[15]	OLD_PWM_MODE	1'b0	
3	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[6]	MODE	1'b0	
4	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[5]	SRCSEL	1'b0	
5	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[8]	GUARD_VALUE	USER_DEFINED	
6	Set IDLE_VALUE	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[7]	IDLE_VALUE	USER_DEFINED	
7	Set PWM_WAVE_NUM	W	PWM_BASE + 0x128 + PWM_NUM*0x100	[15:0]	PWM_WAVE_NUM	USER_DEFINED	If WAVE_NUM=0, the waveform generation should not stop until the PWM is disabled.
8	Set PWM_GDURATION	W	PWM_BASE + 0x10C + PWM_NUM*0x100	[15:0]	PWM_GDURATION	USER_DEFINED	
9	Set PWM_HDURATION	W	PWM_BASE + 0x104 + PWM_NUM*0x100	[15:0]	PWM_HDURATION	USER_DEFINED	
10	Set PWM_LDURATION	W	PWM_BASE + 0x108 + PWM_NUM*0x100	[15:0]	PWM_LDURATION	USER_DEFINED	
11	Set PWM_SEND_DATA0	W	PWM_BASE + 0x120 + PWM_NUM*0x100	[31:0]	PWM_SEND_DATA0	USER_DEFINED	This value should be written only in the periodical FIFO mode. In other modes, this buffer

PWM setting sequence for FIFO mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
							is for internal memory access.
12	Set PWM_SEND_DATA1	W	PWM_BASE + 0x124+ PWM_NUM*0x100	[31:0]	PWM_SEND_DATA1	USER_DEFINED	This value should be written only in the periodical FIFO mode. In other modes, this buffer is for internal memory access.
13	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[14:9]	STOP_BITPOS	USER_DEFINED	Stop bit position for source data in the periodical mode. In the FIFO mode, it is used to indicate the stop bit position in total 64 bits. In the memory mode, it is for the stop bit position of the last 32 bits.
14	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b1	Enable PWM[N].

5.9.5.3 Memory Mode

Table 5-21. Memory mode setting procedures

PWM setting sequence for memory mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[15]	OLD_PWM_MODE	1'b0	
3	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[6]	MODE	1'b0	
4	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[5]	SRCSEL	1'b1	
5	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[8]	GUARD_VALUE	USER_DEFINED	
6	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[7]	IDLE_VALUE	USER_DEFINED	
7	Set PWM_WAVE_NUM	W	PWM_BASE + 0x128+ PWM_NUM*0x100	[15:0]	PWM_WAVE_NUM	USER_DEFINED	If WAVE_NUM=0, the waveform generation should not stop until PWM is disabled.

PWM setting sequence for memory mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
8	Set PWM_GDURATION	W	PWM_BASE + 0x10C + PWM_NUM*0x100	[15:0]	PWM_GDURATION	USER_DEFINED	
9	Set PWM_HDURATION	W	PWM_BASE + 0x104 + PWM_NUM*0x100	[15:0]	PWM_HDURATION	USER_DEFINED	
10	Set PWM_LDURATION	W	PWM_BASE + 0x108 + PWM_NUM*0x100	[15:0]	PWM_LDURATION	USER_DEFINED	
11	Set PWM_BUF0_BASE_ADDR	W	PWM_BASE + 0x110 + PWM_NUM*0x100	[31:0]	PWM_BUF0_BASE_ADDR	USER_DEFINED	Base address of memory buffer0 for the PWM's waveform data.
12	Set PWM_BUF0_BASE_ADDR2	W	PWM_BASE + 0x13C + PWM_NUM*0x100	[3:0]	PWM_BUF0_BASE_ADDR2	USER_DEFINED	Extend base address of memory buffer0 for PWM's waveform data.
13	Set PWM_BUF0_SIZE	W	PWM_BASE + 0x114 + PWM_NUM*0x100	[15:0]	BUF0_SIZE	USER_DEFINED	Length of waveform data in memory buffer0 that the PWM should generate.
14	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[14:9]	STOP_BITPOS	USER_DEFINED	Stop bit position for source data in the periodical mode. In the FIFO mode, STOP_BITPOS is used to indicate the stop bit position in total 64 bits. In the memory mode, it is for the stop bit position of the last 32 bits.
15	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b1	Enable PWM[N].

5.9.5.4 Random Mode

Table 5-22. Random mode setting procedures

PWM setting sequence for random mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[15]	OLD_PWM_MODE	1'b0	
3	Set PWM_CON	W	PWM_BASE + 0x100 + PWM_NUM*0x100	[6]	MODE	1'b0	

PWM setting sequence for random mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
4	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[5]	SRCSEL	1'b1	
5	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[8]	GUARD_VALUE	USER_DEFINED	
6	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[7]	IDLE_VALUE	USER_DEFINED	
7	Set PWM_GDURATION	W	PWM_BASE + 0x10C+ PWM_NUM*0x100	[15:0]	PWM_GDURATION	USER_DEFINED	
8	Set PWM_HDURATION	W	PWM_BASE + 0x104+ PWM_NUM*0x100	[15:0]	PWM_HDURATION	USER_DEFINED	
9	Set PWM_LDURATION	W	PWM_BASE + 0x108+ PWM_NUM*0x100	[15:0]	PWM_LDURATION	USER_DEFINED	
10	Set PWM_BUF0_BASE_ADDR	W	PWM_BASE + 0x110+ PWM_NUM*0x100	[31:0]	PWM_BUF0_BASE_ADDR	USER_DEFINED	Base address of memory buffer0 for the PWM's waveform data.
11	Set PWM_BUF0_SIZE	W	PWM_BASE + 0x114+ PWM_NUM*0x100	[15:0]	BUF0_SIZE	USER_DEFINED	Length of waveform data in memory buffer0 that the PWM should generate.
12	Set PWM_BUF1_BASE_ADDR	W	PWM_BASE + 0x118+ PWM_NUM*0x100	[31:0]	PWM_BUF1_BASE_ADDR	USER_DEFINED	Base address of memory buffer1 for the PWM's waveform data.
13	Set PWM_BUF1_SIZE	W	PWM_BASE + 0x11C+ PWM_NUM*0x100	[2:0]	BUF1_SIZE	USER_DEFINED	Length of waveform data in memory buffer1 that the PWM should generate.
14	Set PWM_VALID	W	PWM_BASE + 0x138+ PWM_NUM*0x100	[1:0]	BUF1_VALID/ BUF0_VALID	2'b11	1: Memory1/0 is not empty. When writing data to memory1 is finished, write 1 to inform the PWM that the data in memory1 is ready.
15	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[14:9]	STOP_BITPOS	USER_DEFINED	Stop the bit position for source data in the periodical mode. In the FIFO mode, STOP_BITPOS is used to indicate the stop bit position in total 64 bits. In the memory mode, it is

PWM setting sequence for random mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
							for the stop bit position of the last 32 bits.
16	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b1	Enable PWM[N].

5.9.5.5 3D_LCM Mode

Table 5-23. 3D_LCM mode setting procedures

PWM setting sequence for 3D_LCM mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_3DLCM	W	PWM_BASE + 0x070	[0]	PWM_3DLCM_EN	1'b1	Enable 3D_LCM for the PWM.
3	Set PWM0 output mode parameters	W				USER_DEFINED	Refer to old/FIFO/memory/random mode procedures.
4	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x1000	[17]	PWM_3DLCM	USER_DEFINED	0: PWM is not selected as the PWM auxiliary channel. 1: PWM is selected as the PWM auxiliary channel.
5	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[18]	PWM_3DLCM_base	USER_DEFINED	0: PWM is not selected as the PWM base channel. 1: PWM is selected as the PWM base channel.
6	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[19]	PWM_3DLCM_INV	USER_DEFINED	0: PWM is the same as the PWM base channel when PWM is selected as the auxiliary channel. 1: PWM is the inversion of the PWM base channel when PWM is selected as the auxiliary channel.
7	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N] N=1~3	PWM_ENABLE	1'b1	Enable the PWM[N] that is selected as the base channel.

5.9.5.6 Breathing Light Mode

Table 5-24. Breathing light mode setting procedures

PWM setting sequence for breathing light mode							
Step	Description	R/W	Address	Bit	MACRO	Value	Note
1	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b0	Disable PWM[N].
2	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[15]	OLD_PWM_MODE	1'b1	
3	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[3]	CLK_SEL	1'b1	
4	Set PWM_CON	W	PWM_BASE + 0x100+ PWM_NUM*0x100	[2:0]	CLKDIV	3'b0	
5	Set PWM_DATA_WIDTH	W	PWM_BASE + 0x12C+ PWM_NUM*0x100	[7:0]	PWM_DATA_WIDTH	USER_DEFINED	
6	Set PWM_BL_CON	W	PWM_BASE + 0x1B8+ PWM_NUM*0x100	[12:8]	BL_THRESH_SEL_MAX	USER_DEFINED	PWM breathing light selects the maximum number of stages control register.
7	Set PWM_BL_CON	W	PWM_BASE + 0x1B8+ PWM_NUM*0x100	[0]	BL_MODE_SEL	USER_DEFINED	
8	set PWM_BL_THRESH_00_03	W	PWM_BASE + 0x1BC+ PWM_NUM*0x100	[31:0]	BL_THRESH_03	USER_DEFINED	PWM breathing light threshold width register 3-0
9	Set PWM_ENABLE	W	PWM_BASE + 0x0000	[N]	PWM_ENABLE	1'b1	Enable PWM[N].

5.9.6 Register Definition

See chapter 3.9 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.10 General-Purpose Timer (GPT)

5.10.1 Introduction

The application processor X general-purpose timer (APXGPT) includes five 32-bit GPTs and one 64-bit GPT. Each GPT supports four operation modes: ONE-SHOT, REPEAT, KEEP-GO, and FREERUN. Each GPT can be operated on one of the two clock sources, real-time clock (RTC) (32.768 kHz) or system clock (13 MHz).

5.10.2 Features

The table below shows the detailed descriptions for the four operation modes for each GPT.

Table 5-25. Operation mode of GPT

Mode	Auto stop	Interrupt supported	Count behavior	When GPTn_COUNT equals to GPTn_COMPARE	Example: Compare is set to 2 (Underline means interrupt asserted)
ONE-SHOT	Yes	Yes	Count stops when GPTn_COUNT equals GPTn_COMPARE.	EN is reset to 0.	0, 1, <u>2</u> , 2, 2, 2, 2, 2, 2, 2, 2,...
REPEAT	No	Yes	Count is reset to 0 when GPTn_COUNT equals GPTn_COMPARE.	Count is reset to 0.	0, 1, <u>2</u> , 0, 1, <u>2</u> , 0, 1, <u>2</u> , 0, 1, <u>2</u> ,...
KEEP-GO	No	Yes	Count is reset to 0 when the count is overflowed.		0, 1, <u>2</u> , 3, 4, 5, 6, 7, 8, 9, 10,...
FREERUN	No	No	Count is reset to 0 when the count is overflowed.		0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,...

Notes:

- GPTn_COUNT (APXGPT Base address+(0x0008+0x20*(n-1))), GPTn_COMPARE (APXGPT Base address + (0x000C+0x20*(n-1))) (n=1,2,3,4,5,6)
- The operation of each timer is independent and the timer can be programmed to select the clock source: RTC (32.768 kHz) or system clock (13 MHz). After the clock source is determined, the division ratio of the selected clock can be programmed. The division ratio can be fine-granulated as 1 to 13 and coarse-granulated as 16, 32 and 64.

5.10.3 Block Diagram

The APXGPT consists of five sets of 32-bit GPTs and one set of 64-bit GPTs.

When the GPT triggers IRQ, it will issue a wake-up signal to "sleep control", then "sleep control" can wake up the MCU if it is in sleep mode.

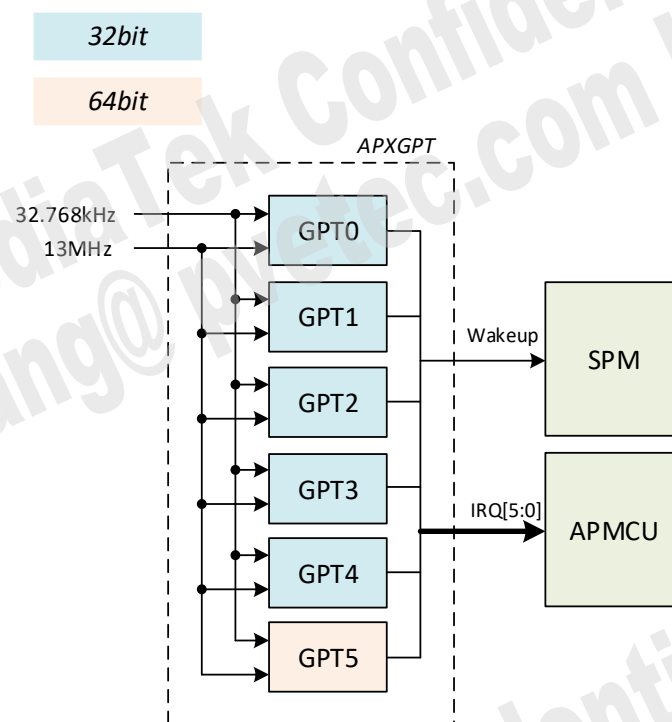


Figure 5-56. Block diagram of APXGPT

5.10.4 Theory of Operations

The read operation value of the GPT6 64-bit timer is separated into two APB reads that each one is of 32-bit width. The read of the value works as the lower word being read first and then the higher word. The read operation of lower word will freeze the “read value” of the higher word but will not freeze the timer counting. This ensures the separated read operation to acquire the correct timer value.

To program and use the GPT, note the following items:

- The counter value can be read at any time when the clock source is the system clock.
- The counter value can be read at any time even when the clock source is the RTC.
- The comparative value can be programmed at any time. When it is rewritten during count operation, the counter would be reset to 0 and restart the count.

5.10.5 Register Definition

See chapter 3.10 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.10.6 Programming Guide

The following sections describe the operating sequence by taking GPT0 as an example of the programming sequence of all the operation modes.

5.10.6.1 APXGPT ONE-SHOT Mode

Table 5-26. APXGPT ONE-SHOT mode setting flow

Step	Address	Register name	Local address	R/W	Value	Description
Set the APXGPT control register.						
1	APXGPT base address + 0x0000	GPT0_CON	CLK	RW	Refer to the description.	Select GPT0 clock source. 00: System clock (13 MHz) 01: RTC (32.768 kHz)
2	APXGPT base address + 0x0000	GPT0_CON	CLKDIV	RW	Refer to the description.	Set the GPT0 clock divider. 0000: Clock source divided by 1 0001: Clock source divided by 2 0010: Clock source divided by 3 0011: Clock source divided by 4 0100: Clock source divided by 5 0101: Clock source divided by 6 0110: Clock source divided by 7 0111: Clock source divided by 8 1000: Clock source divided by 9 1001: Clock source divided by 10 1010: Clock source divided by 11 1011: Clock source divided by 12 1100: Clock source divided by 13 1101: Clock source divided by 16 1110: Clock source divided by 32 1111: Clock source divided by 64
3	APXGPT base address + 0x0000	GPT0_CON	MODE	RW	2'b00	Set the GPT0 control register for ONE-SHOT mode. 00: ONE-SHOT mode 01: REPEAT mode 10: KEEP-GO mode 11: FREERUN mode
4	APXGPT base address + 0x000C	GPT0_COMPARE	COMPARE	RW	Refer to the description.	Set the compare value of GPT0.
5	APXGPT base address + 0x0000	GPT0_CON	IRQEN	RW	1'b1	Set IRQEN = "1" to enable GPT0 interrupt.
Enable GPT0 and wait for GPT0 interrupt.						
6	APXGPT base address + 0x0000	GPT0_CON	EN	RW	1'b1	Set EN = "1" to enable GPT0.
7	APXGPT base address + 0x0000	GPT0_CON	IRQSTA	RO	Refer to the description.	Wait IRQSTA1 = "1". 0: No associated interrupt is generated 1: Associated interrupt is pending and waiting for service.
Clear GPT0 interrupt.						

Step	Address	Register name	Local address	R/W	Value	Description
8	APXGPT base address + 0x0000	GPT0_CON	CLR	WO	1'b1	Set CLR = "1" to clear GPT1 interrupt.

5.10.6.2 APXGPT REPEAT Mode

Table 5-27. APXGPT REPEAT mode setting flow

Step	Address	Register name	Local address	R/W	Value	Description
Set the APXGPT control register.						
1	APXGPT base address + 0x0000	GPT0_CON	CLK	RW	Refer to the description.	Select GPT0 clock source. 00: System clock (13 MHz) 01: RTC (32.768 kHz)
2	APXGPT base address + 0x0000	GPT0_CON	CLKDIV	RW	Refer to the description.	Set the GPT0 clock divider. 0000: Clock source divided by 1 0001: Clock source divided by 2 0010: Clock source divided by 3 0011: Clock source divided by 4 0100: Clock source divided by 5 0101: Clock source divided by 6 0110: Clock source divided by 7 0111: Clock source divided by 8 1000: Clock source divided by 9 1001: Clock source divided by 10 1010: Clock source divided by 11 1011: Clock source divided by 12 1100: Clock source divided by 13 1101: Clock source divided by 16 1110: Clock source divided by 32 1111: Clock source divided by 64
3	APXGPT base address + 0x0000	GPT0_CON	MODE	RW	2'b01	Set the GPT0 control register for REPEAT mode. 00: ONE-SHOT mode 01: REPEAT mode 10: KEEP-GO mode 11: FREERUN mode
4	APXGPT base address + 0x000C	GPT0_COMPARE	COMPARE	RW	Refer to the description.	Set the compare value of GPT0.
5	APXGPT base address + 0x0000	GPT0_CON	IRQEN	RW	1'b1	Set IRQEN = "1" to enable GPT0 interrupt.
Enable GPT0 and wait GPT0 interrupt						
6	APXGPT base	GPT_CON	EN	RW	1'b1	Set EN = "1" to enable GPT0.

Step	Address	Register name	Local address	R/W	Value	Description
	address + 0x0000					
7	APXGPT base address + 0x0000	GPT0_CON	IRQSTA	RO	Refer to the description.	Wait IRQSTA = "1". 0: No associated interrupt is generated 1: Associated interrupt is pending and waiting for service.
Clear GPT0 interrupt						
8	APXGPT base address + 0x0000	GPT0_CON	CLR	WO	1'b1	Set CLR = "1" to clear GPT0 interrupt.

5.10.6.3 APXGPT KEEP-GO Mode

Table 5-28. APXGPT KEEP-GO mode setting flow

Step	Address	Register name	Local address	R/W	Value	Description
Set the APXGPT control register.						
1	APXGPT base address + 0x0000	GPT0_CON	CLK	RW	Refer to the description.	Select the GPT0 clock source. 00: System clock (13 MHz) 01: RTC (32.768 kHz)
2	APXGPT base address + 0x0000	GPT0_CON	CLKDIV	RW	Refer to the description.	Set the GPT0 clock divider. 0000: Clock source divided by 1 0001: Clock source divided by 2 0010: Clock source divided by 3 0011: Clock source divided by 4 0100: Clock source divided by 5 0101: Clock source divided by 6 0110: Clock source divided by 7 0111: Clock source divided by 8 1000: Clock source divided by 9 1001: Clock source divided by 10 1010: Clock source divided by 11 1011: Clock source divided by 12 1100: Clock source divided by 13 1101: Clock source divided by 16 1110: Clock source divided by 32 1111: Clock source divided by 64
3	APXGPT base address + 0x0000	GPT0_CON	MODE	RW	2'b10	Set the GPT0 control register for KEEP-GO mode. 00: ONE-SHOT mode 01: REPEAT mode 10: KEEP-GO mode 11: FREERUN mode
4	APXGPT base	GPT0_COMPARE	COMPARE	RW	Refer to the description.	Set the compare value of GPT0.

Step	Address	Register name	Local address	R/W	Value	Description
	address + 0x000C					
5	APXGPT base address + 0x0000	GPT_IRQEN	IRQEN	RW	1'b1	Set IRQEN = "1" to enable GPT0 interrupt.
Enable GPT0.						
6	APXGPT base address + 0x0000	GPT0_CON	EN	RW	1'b1	Set EN = "1" to enable GPT0.
7	APXGPT base address + 0x0000	GPT0_CON	IRQSTA	RO	Refer to the description.	Wait IRQSTA = "1". 0: No associated interrupt is generated. 1: Associated interrupt is pending and waiting for service.
Read the GPT0 counter value and clear GPT0 interrupt.						
8	APXGPT base address + 0x0008	GPT0_COUNT	COUNTER	RO	Refer to the description.	The timer counter of GPT0
9	APXGPT base address + 0x0000	GPT0_CON	CLR	WO	1'b1	Set CLR = "1" to clear GPT0 interrupt.

5.10.6.4 APXGPT FREERUN Mode

Table 5-29. APXGPT FREERUN mode setting flow

Step	Address	Register name	Local address	R/W	Value	Description
Set the APXGPT control register.						
1	APXGPT base address + 0x0000	GPT0_CON	CLK	RW	Refer to the description.	Select GPT0 clock source. 00: System clock (13 MHz) 01: RTC (32.768 kHz)
2	APXGPT base address + 0x0000	GPT0_CON	CLKDIV	RW	Refer to the description.	Set the GPT0 clock divider. 0000: Clock source divided by 1 0001: Clock source divided by 2 0010: Clock source divided by 3 0011: Clock source divided by 4 0100: Clock source divided by 5 0101: Clock source divided by 6 0110: Clock source divided by 7 0111: Clock source divided by 8 1000: Clock source divided by 9 1001: Clock source divided by 10 1010: Clock source divided by 11 1011: Clock source divided by 12

Step	Address	Register name	Local address	R/W	Value	Description
						1100: Clock source divided by 13 1101: Clock source divided by 16 1110: Clock source divided by 32 1111: Clock source divided by 64
3	APXGPT base address + 0x0000	GPT0_CON	MODE	RW	2'b11	Set the GPT0 control register for KEEP-GO mode. 00: ONE-SHOT mode 01: REPEAT mode 10: KEEP-GO mode 11: FREERUN mode
Enable GPT0.						
4	APXGPT base address + 0x0000	GPT0_CON	EN	RW	1'b1	Set EN = "1" to enable GPT0.
Read the GPT0 counter value.						
5	APXGPT base address + 0x0008	GPT0_COUNT	COUNTER	RO	Refer to the description.	The timer counter of GPT0

5.10.6.5 GPT5 64-Bit Counter Read

Table 5-30. GPT5 64-bit counter read flow

Step	Address	Register name	Local address	R/W	Value	Description
Read the GPT5 counter low 32-bit.						
1	APXGPT base address + 0x00A8	GPT5_COUNT_L		RO	Refer to the description.	The lower word of the timer count of GPT5
Read the GPT5 counter high 32-bit.						
2	APXGPT base address + 0x00B0	GPT5_COUNT_H		RO	Refer to the description.	The higher word of the timer count of GPT5

5.11 Thermal Controller

5.11.1 Introduction

On an SoC platform, thermal management is very fundamental so that an SoC can operate within specific temperature constraints while achieving computing performance requirements. Operations under an over-temperature condition for a long period of time can cause SoC reliability issues.

The thermal management system on the SoC platform includes several thermal sensors embedded in possible hotspots on the die and a thermal controller module for periodic measurement for each hotspot. The measurement results are readable by software. However, in order to minimize software efforts for monitoring temperatures, the thermal controller generates interrupts to a system handler for abnormal conditions.

5.11.2 Features

The thermal management system includes the following features:

- Supports up to 4 thermal sensors per bank
- Programmable periodic temperature measurement
- 2 independent finite state machines (FSM) for temperature monitoring
- Different types of low pass filters for thermal sensor readings

5.11.3 Block Diagram

There are three major building blocks for the thermal control subsystem: the sensing device (thermal sensing micro circuit unit (TSMCU)), the convertor (low voltage thermal sensing (LVTS) convertor) and the controller (LVTS_CTRL), as shown in [Figure 5-57](#).

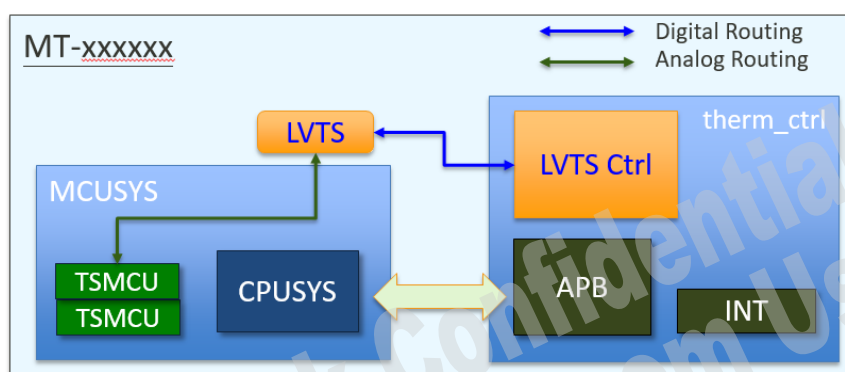


Figure 5-57. Thermal control subsystem

5.11.4 Temperature Measurement Scheme

The thermal controller periodically polls all sensors for the SoC operating within a pre-defined temperature range to avoid function failure and reliability issues. According to the temperature measurement, the system performance can be adjusted for a system design with power dissipation being monitored. The temperature measurement scheme is depicted in [Figure 5-58](#).

When the thermal controller informs the software of an abnormal condition (e.g., over-temperature), the consecutive power reduction methodology should be efficient and with low latency.

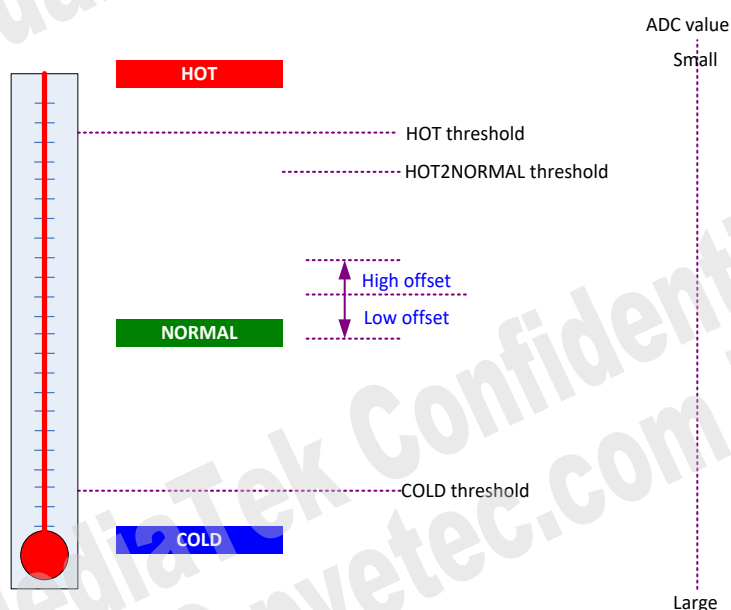


Figure 5-58. Block diagram of system temperature measurement scheme

5.11.5 Theory of Operation

5.11.5.1 Interrupt Control Flow

The interrupt conditions of high and low temperature monitor are shown in [Figure 5-58](#) and [Figure 5-59](#). The software can determine which temperature sensors to be monitored. Once any of the following three interrupt conditions occurs in any of the monitored temperature sensors, interrupts are generated.

- Cold interrupt: asserted when the temperature crosses and falls below the cold threshold.
- Hot interrupt: asserted when the temperature crosses and rises above the hot threshold.
- Hot-to-normal interrupt: asserted when the temperature crosses and falls below the hot-to-normal threshold.

The diagram of FSM diagram is shown in [Figure 5-60](#). The states are described as follows.

- COLD_ST: The temperature is lower than the cold threshold.
- NORMAL_ST: The temperature is within the hot-to-normal threshold and cold threshold.

- HOT1_ST: The temperature is within the hot-to-normal threshold and hot threshold, and the previous state is NORMAL_ST.
- VERY_HOT_ST: The temperature is higher than the hot threshold.
- HOT2_ST: The temperature is within the hot-to-normal threshold and hot threshold, and the previous state is VERY_HOT_ST.

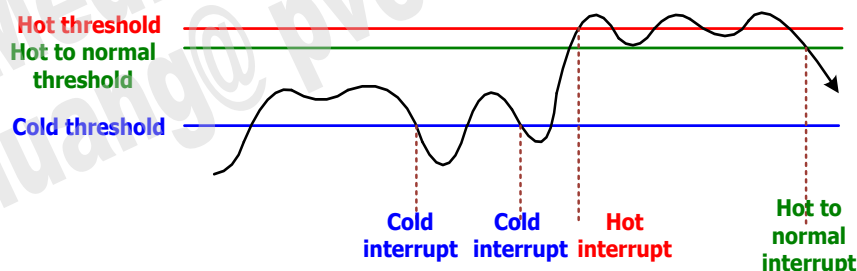


Figure 5-59. Interrupt conditions for high/low temperature monitoring

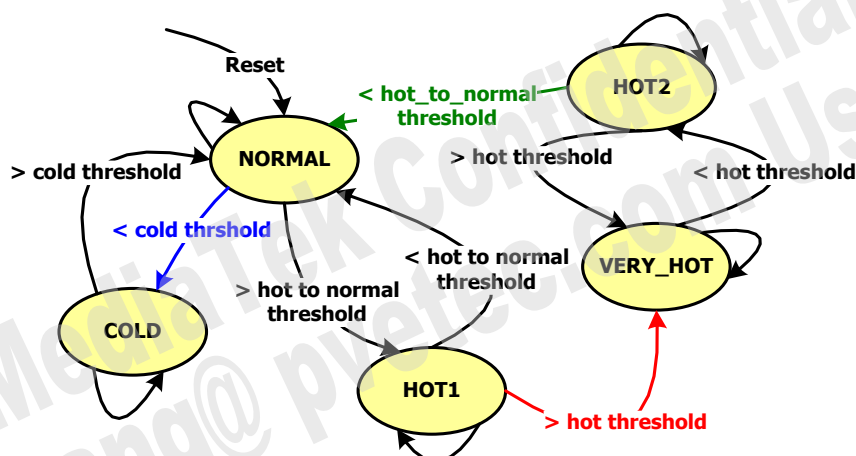


Figure 5-60. Finite state machine for high/low temperature monitoring

The system also provides a feature which can control the junction temperature within the pre-defined HIGH-OFFSET and LOW-OFFSET. [Figure 5-61](#) depicts the concept of the feature, and [Figure 5-62](#) shows the FSM. There are two interrupts and two alarms:

- LOW-OFFSET alarm to RGU: Asserted when the temperature crosses and drops below the low offset.
- HIGH-OFFSET alarm to RGU: Asserted when the temperature crosses and rises above the high offset.
- LOW-OFFSET interrupt: Asserted when the temperature crosses and drops below the low offset. The FSM is transferred from the NORMAL state into the LOW OFFSET state.
- HIGH-OFFSET interrupt: Asserted when the temperature crosses and rises above the high offset. The FSM is transferred from the NORMAL state into the HIGH OFFSET state.

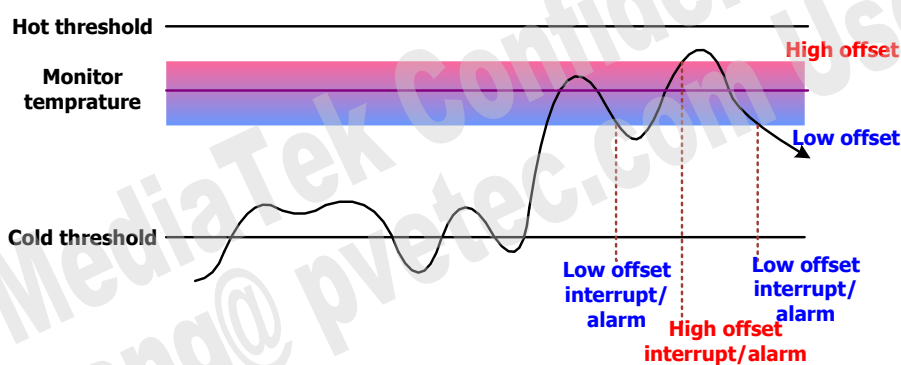


Figure 5-61. Interrupt conditions of high/low OFFSET monitoring

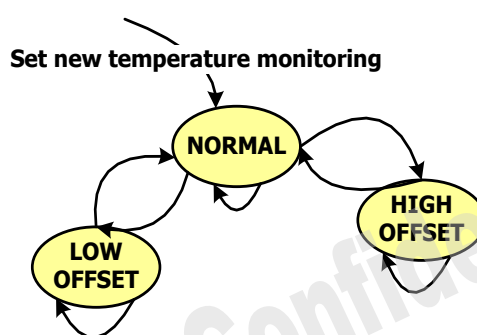


Figure 5-62. Finite state machine for high/low OFFSET monitoring

5.11.6 Thermal Sensor Locations

The thermal sensors are placed at different locations within a die to monitor the junction temperatures of different die locations. All thermal sensor-locations are listed in [Table 5-31](#).

Table 5-31. Temperature sensor monitoring locations

Core	Module	Sensor
#1	BIG CPU	0, 1
#2	Little CPU	0, 1, 2, 3
#3	GPU	0, 1
#4	APU	0, 1
#5	SOC-TOP	0, 1, 2,3
#6	SOC-BOT	0, 1, 2,3
#7	Modem	0, 1, 2,3

5.11.7 Register Definition

See chapter 3.11 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.11.8 Programming Guide

Table 5-32 lists the thermal controller programming sequence.

Table 5-32. Thermal controller programming guide

Step	Sequence	REG_Name	REG_Value	Address offset
1	Enable controller clock.	LVTCLKEN	0x00000001	0x00E4
2	Establish LVTS connection. (Set in order, 2 us delay for each command.)	LVTS_CONFIG	0x8103FFFF 0x8502FC55	0x0050
3	Check LVTS connection ID.	LVTS_ID	Read Only	0x004C
4	Initialize LVTS device. (Set in order, 2 us delay for each command.)	LVTS_CONFIG	0xC1030300	0x0050
5	Prepare calibration data.	LVTSEDA00 LVTSEDA01 LVTSEDA02 LVTSEDA03	0x00xxxxxx 0x00xxxxxx 0x00xxxxxx 0x00xxxxxx	0x0054 0x0058 0x005c 0x0060
6	Configure PNP data to select the monitored sensors.	LVTSTSEL	0x13121110	0x0040
7	Configure sampling method for the thermal sensors.	LVTSMSRCTL0	0x000006DB	0x0038
8	Configure measurement period and cycles.	LVTSMONCTL1 LVTSMONCTL2	0x0000000C 0x000101AD	0x0004 0x0008
9	Trigger event settings (optional).	LVTSMONINT	User-defined	0x000C
10	Enable periodical measurement for the thermal sensors.	LVTSMONCTL0	0x0000020F	0x0000
11	Read thermal sensors measurement results.	LVTMSR0 LVTMSR1 LVTMSR2 LVTMSR3	Read only	0x0090 0x0094 0x0098 0x00B8

5.12 Audio System

5.12.1 Introduction

The audio system provides the audio data exchange ability among the AP, the internal modem and external components. The interfaces are listed as follows.

- 4 master I2S outputs
- 4 master I2S inputs
- 1 master TDM output
- 1 master TDM input
- 1 slave PCM interface for the internal modem
- 1 proprietary audio interface for MediaTek PMIC codec
- 1 slave I2S input interface with ASRC for CONNSYS FM
- 4 PDM interfaces for DMIC

5.12.2 Features

- Audio playing
 - Supports 8/11.025/12/16/22.05/24/32/44.1/48/96/192 kHz sampling rate output
 - Supports playing stereo data
 - Supports 3-channel playing with MT6368
- Audio recording
 - Supports 8/16/32/48/96/192 kHz sampling rate recording
 - Supports recording stereo data
 - Supports 3-channel recording with MT6368 or 8-channel one-wire DMIC on AP
- Speech
 - Supports 4 MICs
 - Supports 8/16/32/48 kHz sampling rate recording
 - Supports side tone filter
 - Supports master PCM interface
- I2S
 - Supports master input mode
 - Supports master output mode
 - Supports 16/24/32-bit stereo data
 - Supports 8/11.025/12/16/22.05/24/32/44.1/48/88.2/96/176.4/192/384 kHz sampling rate in I2S output master mode
 - Supports 8/11.025/12/16/22.05/24/32/44.1/48/88.2/96/176.4/192 kHz sampling rate in I2S input master mode
 - Supports EIAJ/I2S format
- TDM
 - Supports master output mode
 - Supports master input mode
 - Supports 16/24-bit 2/4/8 channel data in master output mode

- Supports 16/24/32-bit 2/4/8 channel data in master input mode
- Supports 8/11.025/12/16/22.05/24/32/44.1/48/96 kHz sampling rate for master output mode based on 8ch configuration
- Supports 8/11.025/12/16/22.05/24/32/44.1/48 kHz sampling rate for master input mode based on 8ch configuration
- Supports 8/11.025/12/16/22.05/24/32/44.1/48/96 kHz sampling rate for master input mode based on 4ch configuration
- Supports EIAJ/I2S format
- Hardware gain function and general ASRC with higher resolution to enhance the audio quality and flexibility of interconnection
- Flexible interconnection system to make data exchange between interfaces without intervention of CPU
- Voice wake-up
- Proximity

5.12.3 Block Diagram

The diagram below shows the flexibility on the interconnection between audio interfaces.

5.12.4 Register Definition

See chapter 5.12 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.13 Digital Signal Processor (DSP)

5.13.1 Introduction

The digital signal processor (DSP) is responsible for running operating system and application programs. It comprises:

- Two-core Cadence HiFi 3 audio engine DSP
- AXI3 bus interface unit (in-house MediaTek bus)
- System power management controller (SPMC) control

The HiFi DSP is a three-slot VLIW machine which can execute up to two 32x32-bit MACs, four 24x24-bit, 32x16-bit and 16x16-bit MACs per cycle. It has an ability to issue two 64-bit loads per cycle, or one load and one store of 32-bit per cycle for the parallel loads and stores of the operand and results. The HiFi 3 DSP offers the additional floating-point precision support for enhanced audio and voice processing through an optional single precision floating point unit (SP FPU) which can perform eight single precision IEEE-754 floating-point MACs per cycle. Since the audio and voice processing at front-end is made in frequency domain, both the floating-point and fixed-point MAC operations in the HiFi 3 DSP are enhanced to operate on complex data types.

5.13.2 Features

Two-core Cadence HiFi 3 DSP operates at 800 MHz (0P75V)/26 MHz (0P55V), including:

- 64 KB L1 I-cache
- 128 KB L1 D-cache
- 36 KB ITCM
- 32 KB DTCM
- Data retention is not supported by pre-fetch buffer, I-cache, D-cache, ITag, or DTag.
- Joint Test Action Group (JTAG)
- Universal asynchronous receiver/transmitter (UART)
- 26 interrupts
- Supports SPM to control the power sequence

Other features:

- UART for audio data transmission with Bluetooth

5.13.2.1 Pipeline Stages

The DSP has a 7-stage pipeline which contains a two-cycle latency for both the instruction-memory fetch and the data-memory fetch.

- Pipeline length 7
- Instruction memory fetch latency 2

- Cycle of execute stage 1
- Cycle of modify stage 3
- Cycle of write-back stage 4

5.13.2.2 Layer 1 Cache Controller

The DSP host processor includes a cache controller with 64 KB instruction cache and 128 KB data cache implemented to improve the code/data fetch performance.

Write-back is supported on the data cache to improve the cache performance. The data cache can be programmatically toggled between write-back and write-through.

Dynamic-cache-way-disable is supported on both instruction cache and data cache. The dynamic-cache-way-disable has the capability of disabling and re-enabling cache ways in both instruction cache and data cache independently to facilitate the power saving. New and modified instructions enable users to clean cache ways before disabling them and initialize cache ways when enabling them. When a cache way is disabled, the cache memory block from service is removed, and therefore total cache capacity will be reduced by 1/(number of ways in service).

- Instruction cache details
 - Instruction cache size in bytes 65,536
 - Instruction cache ways 2
 - Instruction cache line size bytes 128
 - Dynamic way disable Supported
- Data cache details
 - Data cache size in bytes 131,072
 - Data cache ways 4
 - Data cache line size bytes 128
 - Data cache write-back Supported
 - Dynamic way disable Supported

5.13.2.3 Local Memory

Local memory with 36 KB ITCM/32 KB DTCM provides instructions or data for each Xtensa processor core in the shortest time possible.

5.13.2.4 Memory Management

The memory protection unit (MPU) provides memory protection with a smaller footprint than the Linux-compatible memory management unit (MMU), but with much more flexibility and features than the region protection unit (RPU).

The features of MPU are as follows:

- A configurable number (16 or 32) of entries which determine the regions
- A configurable address granularity (minimum: 4 KB)
- 14 choices of access-rights settings per region
- More than 200 different memory-type choices per region, including the ability to specify internal L1 cache behavior which is different from an external (AXI or PIF) cache behavior (attributes)
- Identity map implementation: No address translation
- Runtime modifiable foreground memory map
- Static background memory map
- Unified instruction and data memory maps

5.13.2.5 Interrupt

Each DSP core supports 32 maskable interrupts with four priority levels and a non-maskable interrupt (NMI). The MT8676 DSP is configured with 26 interrupts. The total ADSP interrupt number is 45, which is large than HiFi 3 DSP core capacity of 26 interrupts. There is an INTC (interrupt controller) inside the ADSP to support the remapping and mask function from these 45 interrupts to 16 interrupts to connect each DSP core. The remapping and mask registers of the INTC make the software control more flexible. Each DSP core can receive all 45 interrupts from the ADSP by software setting for any scenario necessary.

Table 5-33. ADSP interrupt function

No.	Type	Note
0	Level	ccu02adsp_irq_b
1	Level	scp2adsp_irq_b
2	Level	sc_spm2adsp_wakeup
3	Level	pcie2adsp_wakeup
4	Level	infrabus_timeout_irq
5	Level	peribus_timeout_irq
6	Level	adsp_mbox_in_irq
7	Level	adsp_mbox_c1_in_irq
8	Level	timer0_irq
9	Level	timer1_irq
10	Level	sw_int0
11	Level	sw_int1
12	Level	sw_int2
13	Level	sw_int32A
14	Level	sw_int32B
15	Level	irq_b_uart
16	Level	uart_bt_wakeup_irq
17	Level	latency_mon_irq
18	Level	bus_dbg_tracker_irq_b
19	Level	xhci_adsp_int_0
20	Level	xhci_adsp_int_1
21	Level	scpvow2adsp_wakeup
22	Level	ccif_adsp_irq_b[0]
23	Level	ccif_adsp_irq_b[1]

No.	Type	Note
24	Level	pwr_ctrl_debug_irq
25	Level	dma_irq_b
26	Level	dma1_irq_b
27	Level	axi_dma0_irqb
28	Level	axi_dma1_irqb
29	Level	audio2hifi3_irq_b
30	Level	audio2hifi3_irq_2_b
31	Level	adsp_wdt_cirq_b
32	Level	adsp_wdt_cirq_b_c1
33	Level	apu2adsp_mbox_irq
34	Level	apu2adsp_mbox_irq2
35	Level	timer2_irq
36	Level	pwr_on_irq_core0
37	Level	pwr_on_irq_core1
38	Level	wakeup_src_irq_c0
39	Level	wakeup_src_irq_c1
40	Level	watchdog_event
41	Level	ocii_conn2adsp_btcsd_irq
42	Level	ocii_conn2adsp_bleiso_irq
43	Level	ocii_bt2dsp_isoch_irq_b
44	Level	ocii_conn2adsp_a2dpo_irq_b

Table 5-34. DSP Core0/1 interrupt connection

Bit	Type	Level	Core 0 BInterrupt pin	Core 1 BInterrupt pin
0	Level	1	0	0
1	Level	1	1	1
2	Level	1	2	2
3	Level	1	3	3
4	Level	1	4	4
5	Level	1	5	5
6	Level	1	6	6
7	Level	2	8	8
8	Level	2	9	9
9	Level	2	10	10
10	Level	2	11	11
11	Level	2	12	12
12	Level	2	13	13
13	Level	2	14	14
14	Level	3	16	16
15	Edge	4	24	-

5.13.3 Block Diagram

The block diagram of the external interrupt controller is shown in [Figure 5-64](#).

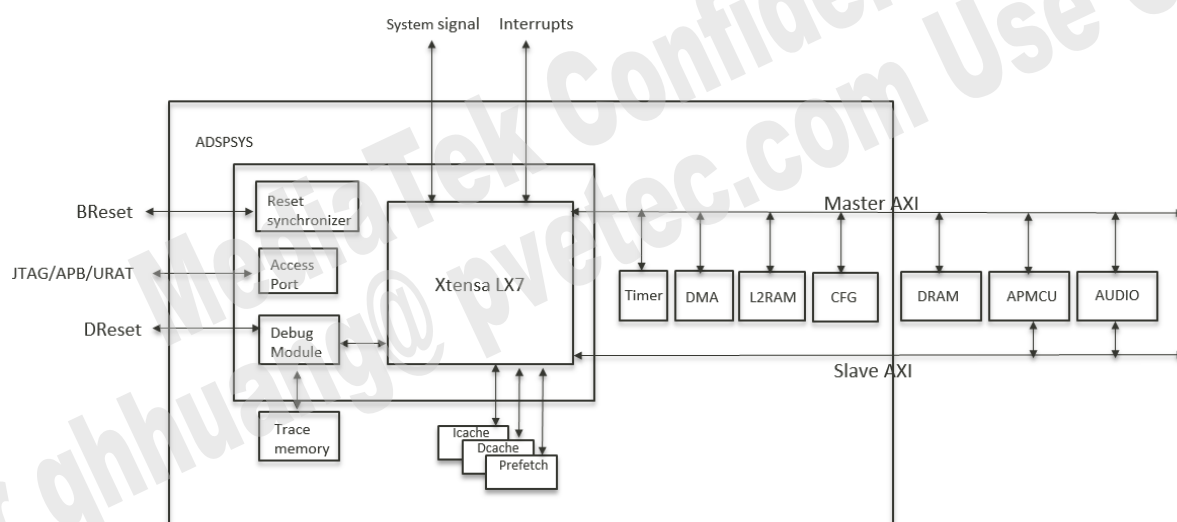


Figure 5-64. Block diagram of audio DSP

5.13.4 Theory of Operations

Refer to HiFi 3 DSP User Guide and Xtensa LX7 Microprocessor Data Book.

5.13.4.1 Constraint to Access DSP Configure Register

The DSP configure power is in the always-on domain.

5.13.4.2 Sleep and Wake Up

When the WAITI instruction is completed, it sets the interrupt level in the processor's PS.INTLEVEL register to the value encoded in the instruction, and then waits for all of the processor memory interfaces to become idle, and causes the processor to assert the DSP_PWaitMode signal. The processor operation is suspended until a non-masked interrupt occurs (refer to Xtensa LX7 Microprocessor Data Book 22.5).

5.13.4.3 Usage for Debug Monitor Register

For each core, DSP has built a set of debug monitor registers which can support the monitoring of DSP core internal signals for the debugging purpose. Refer to Xtensa Debug Guide for details.

Table 5-35. Debug signals

	I/O	Description	Dsp0cfg_reg
PDebugEnable	Input	Enable trace	PDebugBus0[0]
PDebug_latch_en			PDebugBus0[4]
PDebugData[31:0]	Output	Trace debug data	PDebugData

	I/O	Description	Dsp0cfg_reg
PDebugInbPIF[7:0]	Output	Inbound PIF transaction information	PDebugBus1[31:24]
PDebugOutPIF[7:0]	Output	Outbound PIF transaction information	PDebugBus1[23:16]
PDebugInst[31:0]	Output	Instruction information	PDebugInst
PDebugLS0Stat	Output	LSU0 status	PDebugLS0Stat
PDebugLS1Stat	Output	LSU1 status	PDebugLS1Stat
PDebugPC	Output	Trace program counter	PDebugPC
PDebugPrefetchL1Fill[3:0]	Output	Prefetch L1 fill information	PDebugBus0[19:16]
PDebugPrefetchLookup[7:0]	Output	Prefetch lookup information	PDebugBus1[15:8]
PDebugStatus[7:0]	Output	Trace status information	PDebugBus1[7:0]

5.13.5 Register Definition

See chapter 3.13 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.13.6 Programming Guide

5.13.6.1 Basic Setting Flow

Refer to HiFi 3 DSP User Guide and Xtensa LX7 Microprocessor Data Book.

5.13.6.2 Interrupt Setting Flow

For DSP core0:

- CORE0_IRQ_OUT_STA0 (0x1E01_4008) shows the status of core0 interrupt (read only).
- CORE0_IRQ_OUT_EN0 (0x1E01_4010) can mask the IRQ input from other subsystems (1'b0: Disable interrupt).
- CORE0_IRQ_POLO (0x1E01_4018) is used to fix IRQ polarity (1'b0: Low trigger; 1'b1: High trigger).
- CORE0_IRQ_GROUP[n]_0 (0x1E01_4040 + 8n, n is from 0 to 15) is used to set interrupt remapping.
 - For example, to set interrupt no. 5 peribus_timeout_irq (Table 5-33) to DSP core 0 interrupt bit [6] (Level 1), software writes 0x05 to CORE0_IRQ_GROUP6_0 (0x1E01_4070).

For DSP core1:

- CORE1_IRQ_OUT_STA0 (0x1E01_5008) shows the status of CORE1 interrupt (read only).
- CORE1_IRQ_OUT_EN0 (0x1E01_5010) can mask the IRQ input from other subsystems (1'b0: Disable interrupt).
- CORE1_IRQ_POLO (0x1E01_5018) is used to fix IRQ polarity (1'b0: Low trigger; 1'b1: High trigger).
- CORE1_IRQ_GROUP[n]_0 (0x1E01_5040 + 8n, n is from 0 to 15) is used to set interrupt remapping.
 - For example, to set interrupt no. 5 peribus_timeout_irq (Table 5-33) to DSP core 1 interrupt bit [6] (Level 1), software writes 0x05 to CORE1_IRQ_GROUP6_0 (0x1E01_5070).

5.14 Universal Flash Storage (UFS)

5.14.1 Introduction

The universal flash storage (UFS) is popularly used in mobile systems where data is stored in the non-volatile mass storage memory device. The characteristics of the UFS include low power consumption, high data throughput, low electromagnetic interference and optimized mass memory. The UFS host controller is based on JEDEC Standard Universal Flash Storage version 4.0 and composed of three main modules, universal flash storage host controller interface (UFSHCI), UniPro and M-PHY, offering data rate up to 46.592 Gbps with 2-lane configuration and HS-G5B speed gear.

5.14.2 Features

- Supports both 1 and 2 lanes
- Supports PWM-MODE G1 and HS-MODE G1-G5 data rate
- Supports both HS RATE A and RATE B series settings
- Supports UniPro test feature
- Supports auto-hibernate function for power saving
- Supports AES encryption and decryption engine for both XTS and CBC-ESSIV modes
 - XTS mode with the key sizes of 128/256 bits
 - CBC-ESSIV with the key sizes of 128/192/256 bits
- Supports 32 configurable slots for AES
- Supports MCQ function
- Supports extra header segment (EHS)
- Supports queue depth 64
- CPU slave interface: AHB slave design
- DMA master: AXI master design, 36-bit address width supporting 64G memory space
- M-PHY supports 8b/10b line coding.
- M-PHY Tx supports large amplitude only.
- M-PHY supports I2C interface for testing and debugging.

5.14.2.1 References

- JEDEC Standard Universal Flash Storage Version 4.0
- JEDEC UFSHCI Version 4.0
- MIPI UniPro Version 2.0
- MIPI M-PHY Version 5.0

5.14.3 Block Diagram

[Figure 5-65](#) is the block diagram of the UFS host controller which is composed of three main modules, M-PHY, UniPro and UFSHCI.

The M-PHY is a high-bandwidth and power-efficient physical layer, with a low-pin count serial interface. It consists of the analog and digital parts which are shown as “phy A” and “phy D” in the block diagram.

The UniPro is the middle layer between the application layer and physical layer, defining data format, flow control, error handling and power state management.

The UFSHCI provides a uniform interface method of accessing the UFS hardware capabilities, so that a standard/common driver can be provided for the host controller. In order to support cryptographic operations, a secure engine is included in the UFSHCI.

The UFS host controller uses the M-PHY serial interface to connect with the UFS device.

The interface between the M-PHY and the UniPro is called the reference M-PHY module interface (RMMI).

The data interface between the UniPro and the UFSHCI is called the Cport interface.

The control interface between the UniPro and the UFSHCI is called the device management entity (DME) interface. The UFSHCI uses the AXI bus protocol to connect to the system bus and for the CPU to visit the host controller.

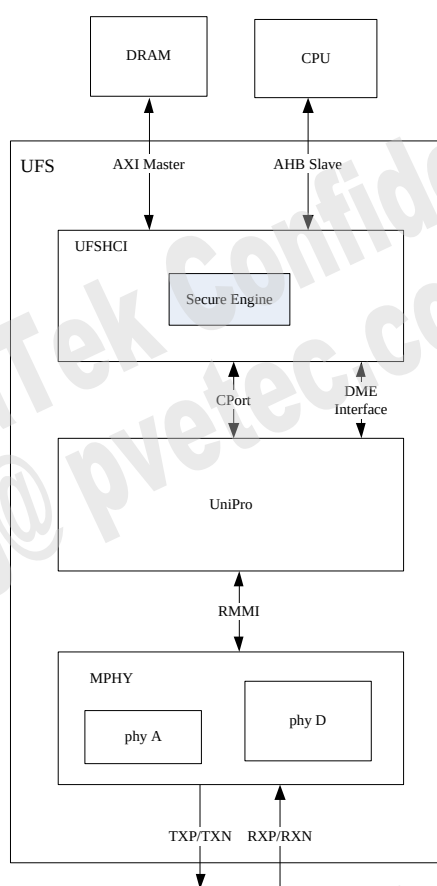


Figure 5-65. Block diagram of UFS host controller

5.14.4 Register Definition

See chapter 3.14 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

5.15 Peripheral Component Interconnect Express (PCIe) Controller

5.15.1 Introduction

PCI express (peripheral component interconnect express, often called PCIe) controller is compliant with the Intel® PIPE (PHY interface for the PCIe) interface, allowing integration with PIPE-compliant PHY and is also compliant with the AMBA® AXI4 specifications.

The chip supports PCIe Gen 1 (2.5 Gbps), PCIe Gen 2 (5.0 Gbps), and PCIe Gen 3 (8.0 Gbps) differential bus speeds.

5.15.2 Features

- PCIe Interface
 - Supports root complex (RC) mode
 - 1-lane PCIe
 - Link rates: 2.5 GT/s, 5.0 GT/s and 8.0 GT/s per lane per PCIe port
 - PCIe Base Specification Revision 3.0 compliant
 - PIPE 4.0 compliant PHY
 - Memory, I/O, configuration, and message transactions specified in the following sections
 - Maximum payload size: 128 to 256 bytes, depending on hardware setting
 - Maximum read request size: 128 to 256 bytes, depending on hardware setting
 - 1 VC (virtual channel)
 - 1 physical function
 - AER (advanced error reporting)
 - ECRC generation and check
 - Lane reversal
 - Polarity inverse
 - Supports legacy PCI power management
 - Native active state power management (ASPM) L0s and L1 states
 - L1 power management substates (L1PMSS) with CLKREQ#
 - Latency tolerance reporting (LTR)
 - Message signaled interrupt (MSI) per function up to 32 and INT x (legacy interrupts)
- APB and AXI
 - 1 APB slave interface for bridge configuration
 - Up to 1 AXI master interface, up to 16/8 outstanding write/read requests
 - Up to 1 AXI slave interface, up to 8/4 outstanding write/read requests
 - 128-bit data support for AXI master, 128-bit data support for slave interfaces

5.15.3 Block Diagram

The PCIe consists of 4 different layers, PCIe layer, bridge layer, AXI layer and physical layer.

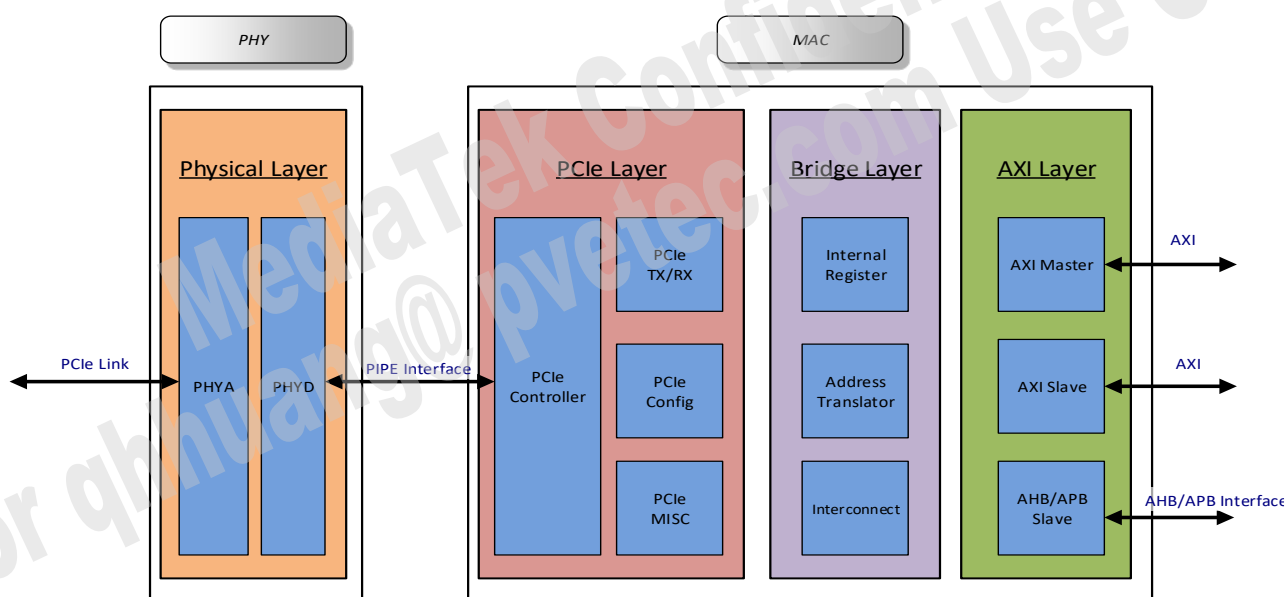


Figure 5-66. Block diagram of PCIe controller

5.15.3.1 PCIe Layer

The PCIe layer consists of:

- PCIe controller, which is the PCIe core IP
- PCIe Tx/Rx interface between the bridge layer and the PCIe controller
- PCIe configuration interface to give the bridge layer access to the PCIe configuration space
- PCIe misc. interface to allow the bridge layer to manage low-power, interrupts, etc.

5.15.3.2 Bridge Layer

The bridge layer consists of:

- Internal registers of the PCIe controller
- Address translator modules to convert between the AXI and PCIe interfaces
- When transferring PCIe receive requests to the AXI master, the address translator adds the corresponding AXI base address and forwards the request to the desired AXI master interface.
- The address translation method is similar when transferring AXI receive requests to the PCIe interface.
- An interconnect module to interconnect and arbitrate between input and output flow

5.15.3.3 AXI Layer

The AXI layer consists of the following three parts:

- An AXI master interface, which manages requests and completions from bridge layer to AXI
- An AXI slave interface, which manages requests and completions from AXI to bridge layer
- An AHB or APB slave interface for bridge configuration

5.15.3.4 Physical Layer

Refer to section 5.15.13.

5.15.4 PCIe Controller Initial Sequence

For the RC mode, refer to the following steps.

Step	Address	Register name	Local address	R/W	Value	Description
PCIe Controller Initialization						
1	PCIE_MAC Base address +0x0080	port_type	GEN_SETTINGS[0]	W	1'b1	Port type must be chosen as Rootport. Set bit 0 of GEN_SETTINGS register (PCIE_MAC Base Address +0x0080) if the default value is not as expected.
Reset Setting						
2	PCIE_MAC Base address +0x0148	ResetMAC ResetPHY ResetBRG	PCIE_SW [0] PCIE_SW [1] PCIE_SW [2]	W	1'b1 1'b1 1'b1	Software reset PCIE_SW[2:0](PCIE_MAC Base Address +0x148) may be asserted and de-asserted for safety purposes.
3	PCIE_MAC Base address +0x0148	ResetMAC ResetPHY ResetBRG	PCIE_SW [0] PCIE_SW [1] PCIE_SW [2]	W	1'b0 1'b0 1'b0	
4	PCIE_MAC Base address +0x0148	ResetPE	PCIE_SW [3]	W	1'b0	
Misc. Setting						
5	PCIE_MAC Base address +0x0098	DeviceID VendorID	PCIE_PCI_IDS_0[31:16] PCIE_PCI_IDS_0[15:0]	W	Appropriate value	The Vendor ID, Device ID, Revision ID, Classcode can be set by filling the new value to the PCIE_PCI_IDS_0 register (PCIE_MAC Base Address +0x0098), PCIE_PCI_IDS_1 register (PCIE_MAC Base Address +0x009C) if the default value is not as expected.
6	PCIE_MAC Base address +0x009C	Classcode RevisionID	PCIE_PCI_IDS_1[31:8] PCIE_PCI_IDS_1[7:0]	W	Appropriate value	
Interrupt Enable Setting						
7	PCIE_MAC Base address +0x0180		IMASK_LOCAL[31:0]	W	Appropriate value	Enable the corresponded interrupt by setting IMASK_LOCAL register (PCIE_MAC Base address + 0x0180)
Address Translation Setting						
8	PCIE_MAC Base address	SRC_ADDR_LSB SRC_ADDR_MSB	ATR_PCIE_WIN0 ATR_PCIE_WIN1	W	Appropriate value	Set PCIe address translation window for I/O memory,

Step	Address	Register name	Local address	R/W	Value	Description
	+0x0600~0x07FF PCIE_MAC Base address +0x0800~0x0BFF	TRSL_ADDR_LSB TRSL_ADDR_MSB TRSL_PARAM	ATR_AXI_SLV0 ATR_AXI_SLV1 ATR_AXI_SLV2 ATR_AXI_SLV3			prefetchable and non-prefetchable memory resource. Set the address transaction window to remap PCIe bus address if there is demand from users.
9	PCIE_MAC Base address +0x00FC	PFWIN_64B PFWIN_IMPL IOWIN_32B IOWIN_IMPL	PCIE_BAR_WIN[3:0]	W	Appropriate value	Set 3'b11 to bit 3:2 of PCIE_BAR_WIN (PCIE_MAC Base address + 0x00FC) if prefetchable memory resource is needed. Set 3'b11 to bit 1:0 of PCIE_BAR_WIN (PCIE_MAC Base address + 0x00FC) if I/O resource is needed.
Configuration Space Setting						
10	PCIE_MAC Base address +0x1004	IO_Enable MEM_ENABLE Bus_master_enable	PCIE_CONF_HDR1[0] PCIE_CONF_HDR1[1] PCIE_CONF_HDR1[2]	W	1'b1 1'b1 1'b1	Bus_Master_Enable and MEM_ENABLE should be asserted by writing 3'b11 to bit 2:1 of the PCIE_CONF_HDR1 register (PCIE_MAC Base address + 0x1004). Set I/O enable by writing 1 to bit 0 of the PCIE_CONF_HDR1 register (PCIE_MAC Base address + 0x1004) if I/O transaction is needed.
11	PCIE_MAC Base address +0x1010~0x1017	Bar0~Bar1	PCIE_CONF_HDR4 PCIE_CONF_HDR5	W	Appropriate value	The field of BARx register (PCIE_MAC Base address + 0x1010~) for RC internal register accessing via PCIe host should be set to a proper value. It is set by native driver during host boot-up.
12	PCIE_MAC Base address +0x1018	Prim_Bus_Num Sec_Bus_Num Sub_Bus_Num	PCIE_CONF_HDR6_TYPE 1[7:0] PCIE_CONF_HDR6_TYPE 1[15:8] PCIE_CONF_HDR6_TYPE 1[23:16]	W	Appropriate value	It is recommended that you give an effective subordinate bus number, secondary bus number, and primary bus number by writing appropriate values to bits 23 to 16, bits 15 to 8, and bits 7 to 0 of the PCIE_CONF_HDR6_TYPE1 register (PCIE_MAC Base address + 0x1018). It is set by native driver during host boot-up.
13	PCIE_MAC Base address +0x101C	IO_Base IO_Limit IO_Base_Up IO_Limit_Up	PCIE_CONF_HDR7_TYPE 1[7:4] PCIE_CONF_HDR7_TYPE 1[15:12]	W	Appropriate value	IO_Base and IO_Limit should be specified according to the downstream device I/O range to bits 15 to 12 and bits 7 to 4

Step	Address	Register name	Local address	R/W	Value	Description
	PCIE_MAC Base address +0x1030		PCIE_CONF_HDR12_TYP E1[15:0] PCIE_CONF_HDR12_TYP E1[31:16]			of the PCIE_CONF_HDR7_TYPE1 register (PCIE_MAC Base address + 0x0101C). IO_Base_Up and IO_Limit_Up should also be specified if IOWIN_32B (bit 1 of PCIE_BAR_WIN (PCIE_MAC Base address + 0x00FC)) is set. It is set by native driver during host boot-up.
14	PCIE_MAC Base address +0x1020~0x10 2F	Mem_Base Mem_Limit Pref_Mem_Base Pref_Mem_Limit Pref_Base_Up_32b Pref_Limit_Up_32b	PCIE_CONF_HDR8_TYPE 1[15:4] PCIE_CONF_HDR8_TYPE 1[31:20] PCIE_CONF_HDR9_TYPE 1[15:4] PCIE_CONF_HDR9_TYPE 1[31:20] PCIE_CONF_HDR10_TYP E1 PCIE_CONF_HDR11_TYP E1	W	Appropriate value	Mem_Limit and Mem_Base should be specified according to the downstream device memory range to bits 31 to 20 and bits 15 to 4 of the PCIE_CONF_HDR8_TYPE1 register (PCIE_MAC Base address + 0x1020). Pref_Mem_Limit and Pref_Mem_Base should also be specified according to the downstream device memory range to bits 31 to 20 and bits 15 to 4 of the PCIE_CONF_HDR9_TYPE1 register (PCIE_MAC Base address + 0x1024) if PFWIN_IMPL(bit 2 of PCIE_BAR_WIN (PCIE_MAC Base address + 0x00FC)) is set. Pref_Base_Up_32 and Pref_Limit_Up_32 should also be specified according to the downstream device memory range to PCIE_CONF_HDR10_TYPE1 register (PCIE_MAC Base address + 0x1028) and PCIE_CONF_HDR11_TYPE1 register (PCIE_MAC Base address + 0x102C) if PFWIN_64B (bit 3 of PCIE_BAR_WIN (PCIE_MAC Base address + 0x00FC)) is set. It is set by native driver during host boot-up.

5.15.5 PCIe Reset Scenarios

PCIe supports the following 4 reset scenarios:

- PERST# asserted and de-asserted: It can be reached by writing 1'b1 to bit 3 of the PCIE_SW register (PCIE_MAC Base address + 0x0148). PERST# deasserts at least 100 ms (TPVPERL) after the clock and power are stable. Then, clear bit 3 of the PCIE_SW register (PCIE_MAC Base address + 0x0148) back to zero to complete PERST# sequence.
- Reset PCIe itself: The power-on reset (POR) is the active-low reset used to clear all PCIe controller registers and sticky registers.
- Reset PCIe itself via software control: It can be achieved by setting 3'b111 to bit 2:0 of the PCIE_SW register (PCIE_MAC Base address + 0x0148). Wait for an appropriate amount of time, for example, 100 ms, and then clear them to finish software reset sequence.
- Reset secondary bus: It can be reached by writing 1'b1 to bit 22 of PCIE_CONF_HDR15_TYPE1 register (PCIE_MAC Base address + 0x103C), which triggers a hot reset on the corresponding PCIe port. Hot reset uses bit 0 (hot reset) in the training control field within the TS1 ordered sets. Wait for a minimum reset duration (at least 1 ms) defined in PCI local bus specification and then clear bit 26 back to 1'b0 to complete secondary bus reset sequence.

5.15.6 Memory Transactions

Memory transactions are memory space read and write requests, MRd and MWr TLPs, which are generated and sent over the PCIe link, and the respective completion TLPs received in return.

Features supported by master memory requester include:

- Maximum memory read request: 128 to 256 bytes, depending on hardware configuration
- Maximum memory write request: 128 to 256 bytes, depending on hardware configuration
- 32-bit or 64-bit addressing

For initiating memory read or write requests, memory space enable should be asserted by writing 1'b1 to bit 1 of the PCIE_CONF_HDR1 register. Then, you can initiate memory access based on memory-mapped address. The related TLP should be passed to target PCIe component based on address-routing rule.

5.15.7 I/O Transactions

I/O transactions are I/O space read and write requests, IORd and IOWr TLPs, which are generated and sent over the PCIe link, and the respective completion TLPs received in return.

Features supported by master I/O requester include:

- Maximum I/O read request: 4 bytes
- Maximum I/O write request: 4 bytes
- 32-bit addressing

For initiating I/O read or write requests, I/O space enable should be asserted by writing 1'b1 to bit 0 of the PCIE_CONF_HDR1 register (PCIE_MAC Base address + 0x1004). Then, you can initiate I/O access based on I/O-mapped address. The related TLP should be passed to target PCIe component based on address-routing rule.

5.15.8 Configuration Transactions

Configuration transactions are configuration space read and write requests, CfgRd0, CfgWr0, CfgRd1 and CfgWr1 TLPs, generated and sent over the PCIe link, and the respective completion TLPs received in return.

Features supported by master configuration requester include:

- Maximum configuration read request: 4 bytes
- Maximum configuration write request: 4 bytes
- Extended register number: 4 KB extended PCIe configuration header space

For initiating configuration read or write request to target endpoint, specify the bus number, device number and function number of corresponding endpoint to the PCIE_CFGNUM register (PCIE_MAC Base address + 0x0140). Then, you can trigger the desired configuration request by directly accessing memory from (PCIE_MAC Base address + 0x1000) to (PCIE_MAC Base address + 0x1FFF). The related TLP should be passed to target PCIe component based on ID-routing rule.

5.15.9 Message Transactions

Message transactions use message space to support all prior sideband signals, such as interrupts and power-management requests. They are used to support in-band communication of events between devices.

The following groups of messages are NOT supported:

- Locked transaction
- OBFF (optimized buffer flush/fill) messages

The following groups of messages are supported:

- INTx interrupt signaling
- Power management
- Error signaling
- Slot power limit support
- Vendor-defined messages
- LTR messages

5.15.10 Interrupt Handling

5.15.10.1 Interrupt to Local CPU

If one of the following events occurs, the source of the interrupt is reported in bits 31 to 0 of the ISTATUS_LOCAL (PCIE_MAC Base address + 0x0184) register. The host processor can mask or enable each interrupt source independently by setting or clearing the corresponding bit in the IMASK_LOCAL (PCIE_MAC Base address + 0x0180) register.

PCIe can issue interrupts to the local processor for the following events:

- Bit [31]: System error signaled.

- Bit [30]: PM/LTR/Hotplug event.
- Bit [29]: AER event for RC.
- Bit [28]: Message TLP received except LTR and PTM.
- Bit [27]: Asserted when PCI interrupt line D is asserted.
- Bit [26]: Asserted when PCI interrupt line C is asserted.
- Bit [25]: Asserted when PCI interrupt line B is asserted.
- Bit [24]: Asserted when PCI interrupt line A is asserted.
- Bit [23]: L2 wake-up: Asserted when L2 wake-up event happened.
- Bit [22]: PCIe discard error: Asserted to signal a completion timeout on a PCIe read request.
- Bit [21]: PCIe fetch error: Asserted to indicate that an error occurred on a PCIe read request.
- Bit [20]: PCIe post error: Asserted to indicate that an error occurred on a PCIe write request.
- Bit [19]: PCIe PTM message received: Asserted to indicate that a precise time message was received.
- Bit [18]: AXI discard error: Asserted to signal a completion timeout on an AXI read request.
- Bit [17]: AXI fetch error: Asserted to indicate that an error occurred on an AXI read request.
- Bit [16]: AXI post error: Asserted to indicate that an error occurred on an AXI write request.
- Bit [15:8]: Multiple MSI received: Report MSI events to the local processor. Bit number *i* corresponds to function number *i*.
- Bit [7:0]: Reserved

5.15.11 Error Handling Function

PCIe supports hardware-controlled autonomous error reporting and handling to improve fault isolation and recovery solution. The following description defines these functions.

- Error recovery: The data link layer handles error recovery. It is based on 32-bit CRC (cyclic redundancy check) error detection, TLP sequence number, replay buffer and ACK/NAK DLLP exchange.
- Replay buffer: The replay buffer, located in the data link layer, stores a copy of a transmitted TLP until the transmitted packet is acknowledged by the receiving side of the link.
- Completion timeout: PCIe controller uses a design-specific timeout mechanism for failed read request transaction in order to report error messages and to free completion resources.

5.15.12 PCIe Module Register Definition

The PCIe module register consists of:

- Internal registers of the PCIe controller
- PCIe configuration space and PCIe extended configuration space, accessible through the PCIe configuration interface
- Extended internal register of the PCIe controller

Table 5-36. PCIe module register mapping

Offset	Description
0x0000 - 0x0FFF	Internal registers of PCIe controller

Offset	Description
0x1000 - 0x1FFF	PCIe configuration space and PCIe extended configuration space, accessible through the PCIe configuration interface
0x2000 - 0x2FFF	Reserved
0x3000 - 0x3FFF	Extended internal registers of PCIe controller

5.15.12.1 Internal Register of PCIe Controller

The internal register of PCIe controller can control PCIe controller. It includes control, status, interrupt, and event registers and maps to PCIe module register offset 0x0000 (PCIE_MAC Base address + 0x0000).

Table 5-37. PCIe internal register

Offset	Description
0x0000 - 0x017F	Control and status registers
0x0180 - 0x01FF	Interrupt and event registers
0x0200 - 0x02FF	Routing, arbitration, and priority rules
0x0300 - 0x03FF	Interface optional feature definitions
0x0400 - 0x05FF	Reserved
0x0600 - 0x0BFF	Address translation registers
0x0C00 - 0x0E7F	Optional features register
0x0E80 - 0x0FFF	Reserved
0x1000 - 0x1FFF	Configuration space registers
0x2000 - 0x2FFF	Reserved
0x3000 - 0x3FFF	Reserved

5.15.12.2 PCIe Configuration Space and PCIe Extended Configuration Space

The PCIe configuration space and PCIe extended configuration space is accessible through PCIe configuration space interface. The configuration space register maps to PCIe module register offset 0x1000 (PCIE_MAC Base address + 0x1000).

Table 5-38. PCIe configuration space and PCIe extended configuration space

Offset	Description
0x0000 - 0x003F	Type 0/1 standard PCI configuration header
0x0040 - 0x007F	Reserved
0x0080 - 0x00BB	PCIe capability
0x00BC - 0x00CF	Reserved
0x00D0 - 0x00DB	MSI-X capability
0x00DC - 0x00DF	Reserved
0x00E0 - 0x00F7	MSI capability
0x00F8 - 0x00FF	PCI power management capability
0x0100 - 0x0107	Vendor-specific capability
0x0108 - 0x010F	LTR capability

Offset	Description
0x0110 - 0x011F	L1 PM substates capability
0x0120 - 0x01CF	Reserved
0x01D0 - 0x01DB	PTM capability
0x01DC - 0x01FF	Reserved
0x0200 - 0x0247	AER capability
0x0248 - 0x02FF	Reserved
0x0300 - 0x032B	Secondary PCIe extended capability
0x032C - 0x0FFF	Reserved

5.15.13 PCIe Physical Layer Overview

5.15.13.1 Overview

The PCIe PHY (physical layer) handles low-level PCIe protocol and signaling, including data serialization and de-serialization, 8b/10b encoding/decoding, 128b/130b encoding/decoding (8 GT/s), analog buffers, elastic buffers and receiver detection. The primary focus of this block is to shift the clock domain of the data from the PCIe rate to one that is compatible with the general logic.

5.15.13.2 Features

- Standard PHY interface enables multiple IP sources for PCIe logical layer and provides a target interface for PCIe PHY vendors
- Utilizes 16-bit parallel interface for Gen 1 and Gen 2
- Utilizes 32-bit parallel interface for Gen 3
- Allows integration of high-speed components into a single functional block
- Supports direct disparity control for use in transmitting compliance pattern(s)
- Data and clock recovery from serial stream on the PCIe bus
- Holding registers to stage transmit and receive data
- 8b/10b encoding/decoding and error indication
- 128b/130b encoding/decoding and error indication
- Receiver detection
- Selectable Tx margining, Tx de-emphasis and signal swing values
- Electrical sub-blocks that support 2.5, 5.0 and 8.0 GT/s operation compliant with PCIe Gen 3 electrical specification

5.15.13.3 Block Diagram

[Figure 5-67](#) shows PCIe PHY block diagram. There are 2 main sub-modules. One is PHYA block, including Tx driver, serializer, Rx front-end, CDR and de-serializer. The other is PHYD block, including 8b/10b encoder and decoder, 128b/130b encoder/decoder (8 GT/s), and elastic buffers.

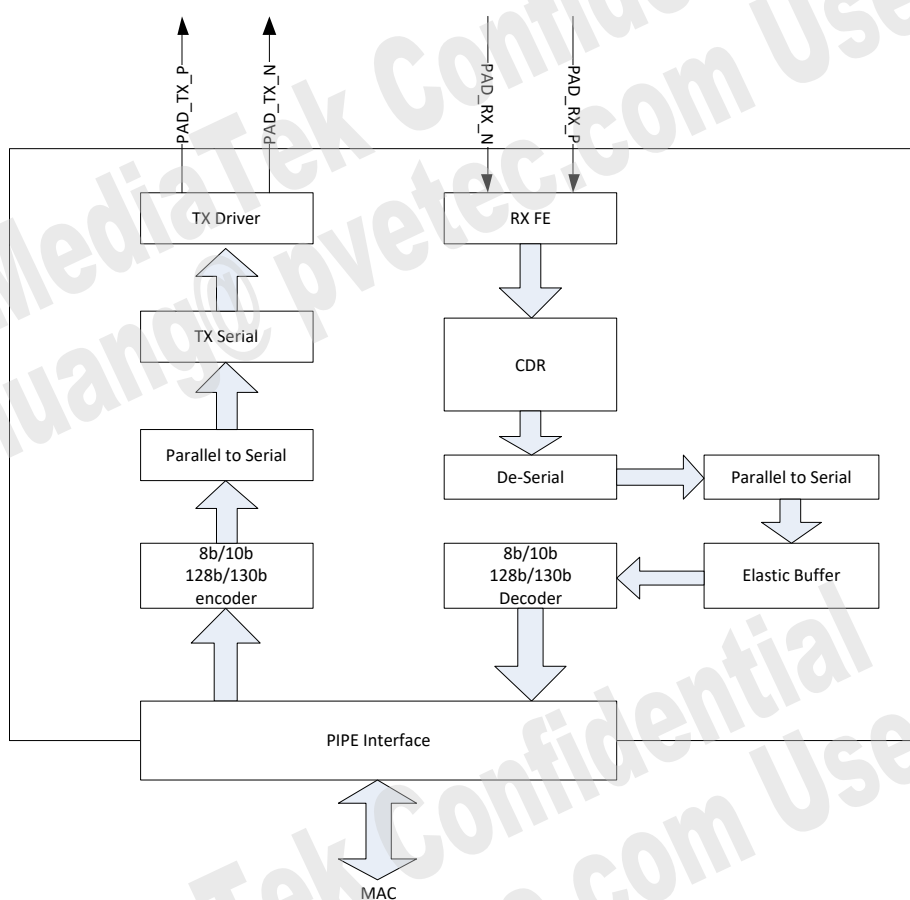


Figure 5-67. Block diagram of PCIe PHY

5.15.14 Register Definition

Refer to chapter 3.15 of "MT8676 5G Connected Cockpit Application Processor Software Register Table".

6 Multimedia

6.1 Display Controller

6.1.1 Introduction

The display controller contains the multimedia layer pipeline and the display pipeline. The multimedia layer pipeline is the pipeline data flow controller to process picture quality enhancement, resizing and rotation with memory access. The display pipeline sends the pixels to either the MIPI DSI or the DisplayPort interface. It handles the function of overlay, color enhancement, adaptive ambient light processing, color correction and gamma correction. It also contains the function of sub-pixel rendering, over-drive and demura.

6.1.2 Features

- Supports MIPI DSI transmitter with D-PHY 4+4 lanes, or C-PHY 3+3 trios, or C-PHY 3+(2 or 1) trios
- Supports DisplayPort 1.4, 4 lanes
- Supports VESA DSC compression
- Supports Arm frame buffer decompression (AFBC)
- Supports 8 overlay layers with per-pixel alpha channel
- Supports MiraVision™ for picture quality enhancement (color enhancement, adaptive contrast enhancement, sharpness enhancement, and dynamic backlight scaling)
- Supports hardware wide-color-gamut
- Supports gamma correction and panel rounded corner
- Supports panel overdrive
- Supports sub-pixel rendering for OLED panel
- Supports demura for OLED panel

6.1.3 Block Diagram

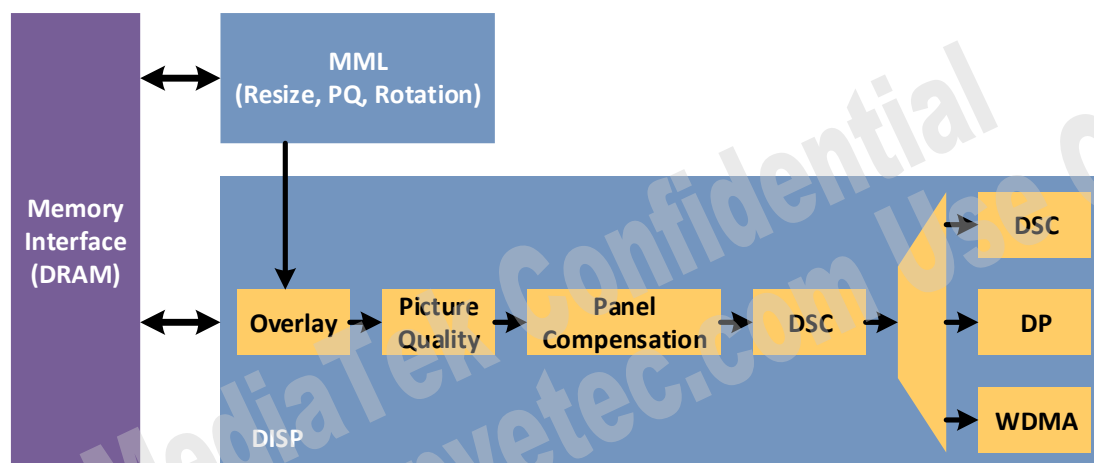


Figure 6-1. Block diagram of display controller

6.1.4 Register Definition

See chapter 4.1 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.2 Camera Image Signal Processor (ISP)

6.2.1 Introduction

The camera image signal processor (ISP) subsystem contains two types of data path, cam (pass 1) and dip (pass 2).

6.2.2 Features

- Integrated image signal processor supports 108 MP@30 fps, 4-cell 320 MP@7.5 fps
- Video stabilization
- Preference color adjustment
- Noise reduction
- Multiple frame noise reduction for image capture
- Multi-scale noise reduction for preview, video record and image capture
- Temporal noise reduction for video recording
- Anti-blooming correction
- Lens shading correction
- Auto sensor defect pixel correction
- Supports auto exposure (AE), auto white balance (AWB), and auto focus (AF)
- Edge enhancement (sharpness)
- Deep learning face detection
- Zero shutter delay image capture
- Captures full size image when preview (up to 108M sensors)
- 6 MIPI CSI-2 high-speed camera serial interfaces; max. 4 data lanes of D-DHY or max. 3 trios of C-PHY for each interface
- Supports C-PHY 12 MP @ 240 fps fast readout
- Tri-CAM, [32 MP + 32 MP + 32 MP] @ 30 fps
- Supports video HDR for stagger HDR sensor (16M60 fps 2 exp)
- Supports video HDR for 4-cell HDR sensor
- Multiple-frame HDR
- PD hardware accelerator
- Throughput can afford SMVR HD480 sensor read out
- Bayer/YUV frame buffer compression to reduce the DRAM bandwidth requirement
- Dual-camera real time depth
- Video face beautification
- Electronic image stabilization

6.2.3 Register Definition

Refer to chapter 4.2 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.3 Camera Serial Interface (CSI)

6.3.1 Introduction

The camera serial interface (CSI) is based on the MIPI Alliance specification CSI2 v2.1. It supports high-speed serial data transfer between the host processor and peripheral devices such as camera modules. According to the data type, the data transferred from physical layer can be sent by different virtual channels of image signal processor (ISP) and transmitted via this high-speed serial interface. The physical layer (CD-PHY) is configurable to meet different throughput requirements of camera modules.

6.3.2 Features

The CSI engine has the following features:

- Compliant with electrical characteristics of C-PHY Revision 2.0
- Compliant with electrical characteristics of D-PHY Revision 2.1
- Supports throughput up to 6 Gbps per C-PHY trio. (CSI PHY & MAC only, actual data rate reference to ISP speed)
- Supports throughput up to 6.5 Gbps per D-PHY data lane. (CSI PHY & MAC only, actual data rate reference to ISP speed)
- Support one 4-data lane D-PHY sensor or 2-lane + 1-lane D-PHY sensor
- Support one 3-trio C-PHY sensor or 2-trio + 1-trio CPHY sensor
- Compliant with MIPI CSI2 v2.1
- Supports CSI-2 long and short packet decoding
- Supports pixel format of RAW8/RAW10/RAW12/RAW14/RAW16/yuv420 10 bits/yuv422 10 bits/RGB565
- Escape mode is not supported.
- Alternate low-power mode (ALP) is not supported.

6.3.3 Block Diagram

The block diagram of CSI engine is shown in [Figure 6-2](#). The csirx_mac_top module receives the signal from cdphy_top module which includes D-PHY & C-PHY modules. When users connect a 4-lane sensor (4D1C), the signals of 4 lanes will be sent to module Mac_csi2_top_1 (refer to [Figure 6-2](#)). When users connect one 2-lane sensor and one 1-lane sensor (2D1D), the lane0 and lane1 signals will be sent to module Mac_csi2_top_1, and the lane2 signals will be sent to module Mac_csi2_top_2.

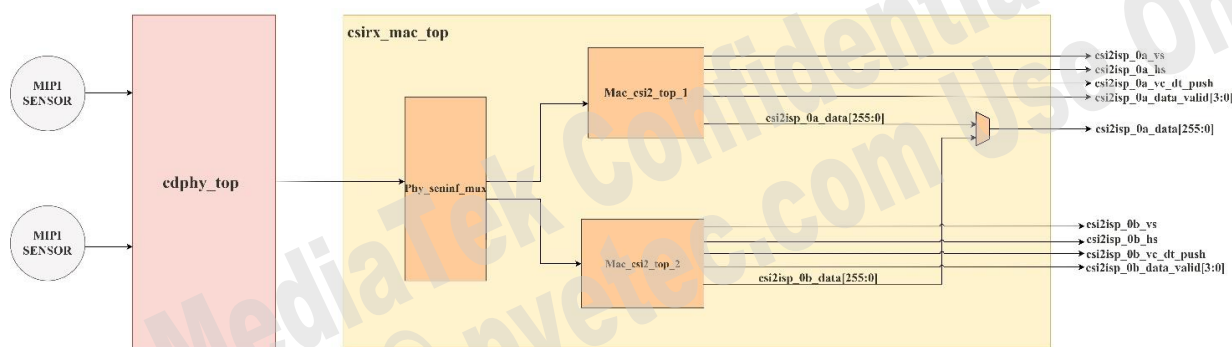


Figure 6-2. Block diagram of CSI

6.3.4 Lanes Mapping

Table 6-1. CDPHY_TOP PAD interface

Pad name	I/O type	Description
PAD_CSIOA_L0P_T0A	I	D-PHY lane 0 P and CPHY trio 0 A
PAD_CSIOA_L0N_T0B	I	D-PHY lane 0 N and CPHY trio 0 B
PAD_CSIOA_L1P_T0C	I	D-PHY lane 1 P and CPHY trio 0 C
PAD_CSIOA_L1N_T1A	I	D-PHY lane 1 N and CPHY trio 1 A
PAD_CSIOA_L2P_T1B	I	D-PHY lane 2 P and CPHY trio 1 B
PAD_CSIOA_L2N_T1C	I	D-PHY lane 2 N and CPHY trio 1 C
PAD_CSIOB_L0P_T0A	I	D-PHY lane 3 P and CPHY trio 2 A
PAD_CSIOB_L0N_T0B	I	D-PHY lane 3 N and CPHY trio 2 B
PAD_CSIOB_L1P_T0C	I	D-PHY lane 4 P and CPHY trio 2 C
PAD_CSIOB_L1N_T1A	I	D-PHY lane 4 N
AVDD12_CSI	Power	CSI analog power 1.2V
AVDD15_CSI	Power	CSI analog power 1.5V

6.3.5 Electrical Characteristics

The operation voltage requirement for CSI PHY is shown in the following table.

Table 6-2. Operation requirements for CSI PHY

Symbol	Parameter	Minimum	Typical	Maximum	Unit
VA12	Analog power 1.2V	1.14	1.2	1.26	V
VIO18	Analog power 1.8V	1.7	1.8	1.9	V

The MIPI Rx electrical characteristics are compatible with MIPI C-PHY Specification Revision 2.0, and MIPI D-PHY Specification Revision 2.1.

Table 6-3. C-PHY Rx high speed electrical characteristics

Description	Minimum	Normal	Maximum	Unit
High speed symbol rate	0.08	-	6	Gbps
High speed common-point voltage	95	-	390	mV
High speed differential input impedance	80	-	120	Ω
High speed common-point interference (> 450 MHz)	-	-	50	mV
High speed common-point interference (50~450 MHz)	-25	-	25	mV
High speed differential input high voltage	-	-	40	mV
High speed differential input low voltage	-40	-	-	mV
High speed single-ended input high voltage	-	-	535	mV
High speed single-ended input low voltage	-40	-	-	mV
High speed common-point termination	-	-	90	pF

Table 6-4. D-PHY Rx high speed electrical characteristics

Description	Minimum	Normal	Maximum	Unit
High speed data rate	0.08	-	6.5	Gbps
High speed common-mode voltage	70	-	330	mV
High speed differential input impedance	80	100	125	Ω
High speed common-mode interference (> 450 MHz)	-	-	50	mV
High speed common-mode interference (50~450 MHz)	-25	-	25	mV
High speed differential input high voltage	-	-	40	mV
High speed differential input low voltage	-40	-	-	mV
High speed single-ended input high voltage	-	-	460	mV
High speed single-ended input low voltage	-40	-	-	mV
High speed common-mode termination	-	-	60	pF

Table 6-5. CD-PHY Rx low power electrical characteristics

Description	Minimum	Normal	Maximum	Unit
Low power logic 1 input voltage	740	-	-	mV
Low power logic 0 input voltage	-	-	550	mV
Low power input hysteresis	25	-	-	mV
Low power input pulse rejection	-	-	300	V·ps
Low power minimum pulse width response	20	-	-	ns
Low power peak interference amplitude	-	-	200	mV
Low power interference frequency	450	-	-	MHz

6.3.6 Register Definition

See chapter 4.3 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.4 Display Power Width Module (DISP_PWM) Generator

6.4.1 Introduction

The display power width module (DISP_PWM) generator provides pulse-width modulation (PWM) signals for the LED driver of a mobile liquid crystal module (LCM).

6.4.2 Features

Operating clocks: 26 MHz, 104 MHz, 124.8 MHz, 130 MHz (default), 136.5 MHz

6.4.3 Register Definition

See chapter 4.4 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.4.4 Programming Guide

1. Turn on the DISP_PWM operating clock.
2. Get the MMSYS mutex.
3. Set up DISP_PWM_CON_0.
4. Write DISP_PWM_EN = 1.
5. Release the MMSYS mutex.

6.5 Display Serial Interface (DSI)

6.5.1 Introduction

The display serial interface (DSI) is a MIPI CD-PHY combo IP, based on MIPI Alliance Specification, supporting high-speed serial data transfer between the host processor and peripheral devices such as display modules. The DSI supports both video mode and command mode data transfer defined in MIPI spec, and it also provides bi-directional transmission with low-power mode to receive messages from the peripheral. The DSI should work with the MIPI_TX_Config module to obtain its engine clock from analog macro, and it should work with DMA engines in the previous stage of DISP path to read frame pixels from the memory.

6.5.2 Features

The DSI engine has the following features for the DSI:

- MIPI DSI D-PHY interface (4-lane/1-port)
 - Throughput up to 2.5 Gbps for 1 data lane at the highest voltage
 - Bi-directional data transmission in low-power mode in data lane 0
 - Uni-directional data transmission in high-speed mode in data lane 0~3
 - Supports non-continuous high-speed transmission in both clock and data lanes
 - Supports DSI-2 for D-option version 2.1
 - Supports D-PHY version 1.1
 - 2 DSI ports
- MIPI DSI C-PHY interface (3-trio/1-port)
 - Up to 3 trios; each trio is composed of 3 wires.
 - Throughput up to 2.187 Gbps for 1 trio at the highest voltage
 - Low-power mode is identical as D-PHY mode
 - Supports continuous high-speed transmission in all lanes.
 - Supports DSI-2 for C-option version 2.1
 - Supports C-PHY version 1.0
 - Two DSI ports
- 128-entry command queue for command transmission
- Supports 3 types of video mode: sync-event, sync-pulse, and burst mode
- Pixel format of RGB565/RGB666/loosely RGB666/RGB888/RGB101010/sub-pixel/compression stream
- Supports command mode frame transmission freerun
- Supports peripheral TE and external TE signal detection
- Supports limited high-speed residual packet transmission during video mode blanking period
- Supports ultra-low power mode control
- Supports low frame rate (LFR) technique

6.5.3 Register Definition

See chapter 4.5 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.6 Video Decoder (VDEC)

6.6.1 Introduction

The video decoder (VDEC) which supports multi-standard video compression formats, is capable of greatly reducing the CPU load and achieving high performance video decompression.

The video standards supported by the VDEC include:

- HEVC decoder 4K2K@60 fps/160 Mbps, 8/10 bits
- VP9 decoder 4K2K@60 fps/120 Mbps, 8/10 bits
- AV1 decoder 4K2K@60 fps/120 Mbps, 8/10 bits
- H.264 decoder: Constrained baseline 4K2K@60 fps/160 Mbps, 8 bits
- H.264 decoder: Main/High profile 4K2K@60 fps/160 Mbps, 8 bits
- H.264 decoder: High 10 profile 4K2K@60 fps/160 Mbps, 10 bits
- MPEG-4 SP/ASP decoder: 1,080p@60 fps/60 Mbps, 8 bits
- HEIF decoder: Maximum resolution 16,383 x 16,383 (grid mode up to 20,656 x 15,496 for 320M pixels capture), 8/10 bits

6.6.2 Block Diagram

[Figure 6-3](#) shows the architecture and core blocks of the VDEC, including entropy decoder, IS/IQ/IT, MV calculation, intra prediction, motion compensation and de-blocking filter. The input to VDEC is a compressed video bitstream. After the decoding processes, the reconstructed video is sent to the display stage.

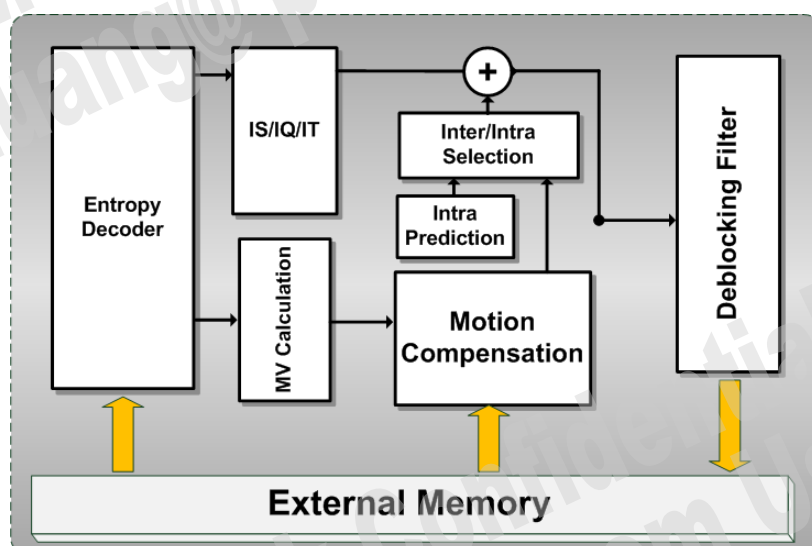


Figure 6-3. Block diagram of video decoder

6.6.3 Theory of Operation

The VDEC hardware architecture is shown in [Figure 6-4](#). Firstly, the VDEC parses bitstream with entropy decoding; and secondly, inverse scan, inverse quant and inverse transformation are used to generate residual samples. The decoding pictures use intra/inter prediction to generate prediction sample, depending on the prediction block type. Thirdly, the reconstruction module adds the residual samples and prediction samples. Lastly, the deblocking filter filters the boundary distortion of the reconstruction samples and sends the result to the DRAM.

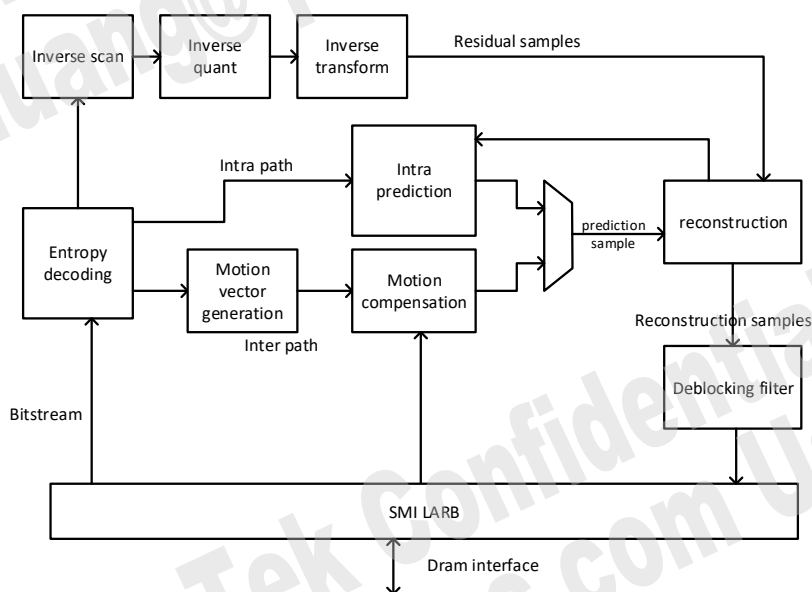


Figure 6-4. VDEC hardware architecture

6.6.4 Register Definition

See chapter 4.6 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.7 Video Encoder (VENC)

6.7.1 Introduction

The MT8676 main-stream video encoder (VENC) consists of two video encoders: H.264 and HEVC. The VENC is capable of encoding up to 2160P video at 60 frames per second (FPS) with promising superior video quality for H.264 and HEVC. It also supports various encoding methods that satisfy basic software controllability and considers the usage of portable devices with multiple power saving capabilities.

With advanced encoding technology, the MT8676 VENC brings astonishing high quality and low memory bandwidth requirements.

6.7.2 Features

The supported video codecs and the corresponding capabilities are listed in the table below.

Table 6-6. Features of VENC

H.264 encoder	Profile	High (10-bit)
	Level	5.2
	Speed	3,840 x 2,176p@60 fps
HEVC encoder	Profile	Main 10 (10-bit)
	Level	5.1
	Speed	3,840 x 2,176p@60 fps

The MT8676 VENC takes the DRAM as input, output, and working buffer. It reads input frame buffers, executes video encoding, and then writes encoded bitstream to the output buffer. See [Figure 3-18](#) for the working procedure of it. The driver software maintains all buffers and assigns proper values to the VENC to ensure the hardware to work correctly. YUV420 two-plane scan-line (NV12/NV21), YUV420 three-plane scan-line (YV12/I420) and MediaTek block formats are supported.

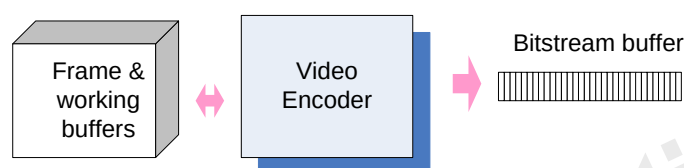


Figure 6-5. Procedure of VENC

6.7.3 Block Diagram

[Figure 6-6](#) is the brief architectures of the MT8676 VENC and the local on-chip-bus. The interface for controlling it consists of Arm APB and MediaTek proprietary SMI bus. It reports hardware events through interrupt or software polling. In addition, it adopts several SMI ports and one APB port.

The MT8676 VENC is configured by software through the APB interface. As the register is configured, the sequencer sends the corresponding control signals to trigger submodules. DMA acquires and stores back the image data and bitstream from and to the memory based on the configured address. ME conducts motion estimation to decide the motion vector for later encoding. MC conducts motion compensation to give predicted pixel values. TQ conducts the transformation and quantization operations, and writes reconstructed pixels to DB and quantized transformed coefficient to EC. DB conducts the de-blocking operation and allows DMA to store back the processed frame as the reference for the next frame. EC conducts entropy encoding, and the coding can be variable-length code, context-based arithmetic code, or context-based variable-length code. The encoded bitstream will be written to the memory by DMA.

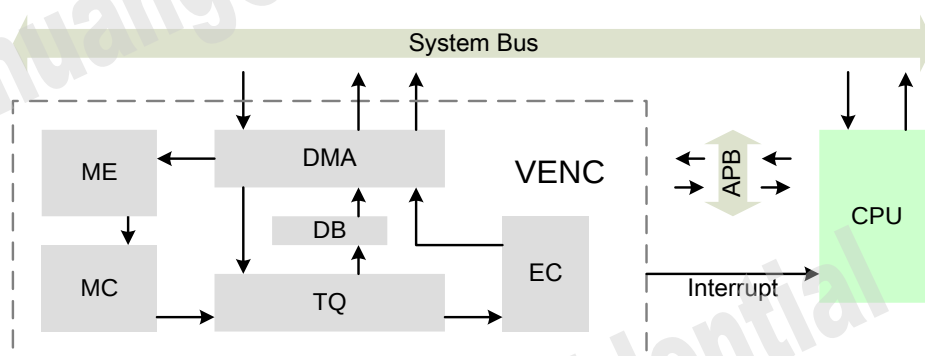


Figure 6-6. Block diagram of VENC

6.7.4 Register Definition

See chapter 4.7 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

6.8 MFlexGraphics (MFG)

6.8.1 Introduction

The MFlexGraphics (MFG) contains the Mali-Turso GPU and the clock/reset control logic. The Mali-Turso series of GPUs process extremely complicated graphics and perform general processing tasks assigned by the main application processor. The diagram below shows the main components and the interface of MFG.

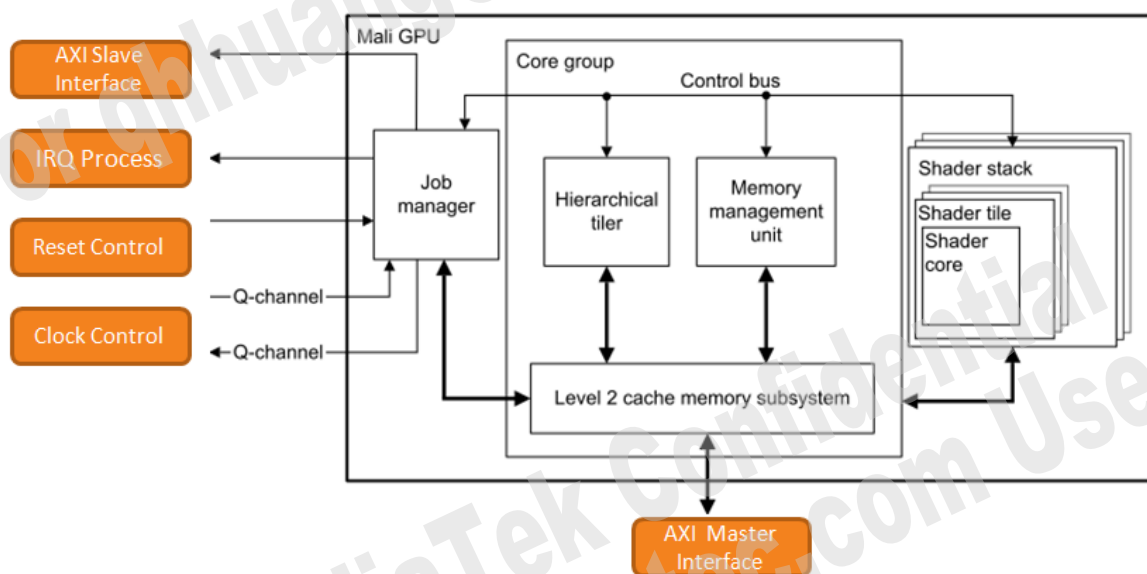


Figure 6-7. Block diagram and interface of MFG

6.8.2 Features

The Mali-Turso GPU includes the following features:

- Supports multiple clock and voltage domains which allow users to maximize performance within a given power budget
- An API feature set which supports shader-based and fixed-function graphics APIs
- Anti-aliasing capabilities
- High memory bandwidth and low power consumption for 3D graphics content
- Scalability for products from Connected Cockpits to high-end mobile computing
- Arm AMBA® AXI-Lite slave interface for GPU configuration
- L2 cache
 - 128-bit AMBA 4 ACE master interface for external memory access
 - Input/output cache coherency support for all L1 load/store caches and external coherent masters
 - Cache maintenance operations using the job control network
 - Performance-counter outputs
 - Cache organization, 8-way set associative
 - Error response generation for non-supported transfers

- Compressed texture formats
- Configurable per-core power management for enabling the optimal power and performance combination for each application
- Coherency aware interconnects for system memory and resource sharing
- Arm® fixed rate compression (AFRC)
- Arm® frame buffer compression (AFBC) 1.3
- 8-bit, 10-bit, and 16-bit YUV input and output formats
- Secure processing of DRM-protected content
- Supports the computing API standards of OpenCL up to 2.1.
- Supports the graphics API standards of OpenGL ES 1.1, 2.0, 3.2 and Vulkan 1.2.

6.8.3 Register Definition

See chapter 4.8 of “MT8676 5G Connected Cockpit Application Processor Software Register Table”.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.